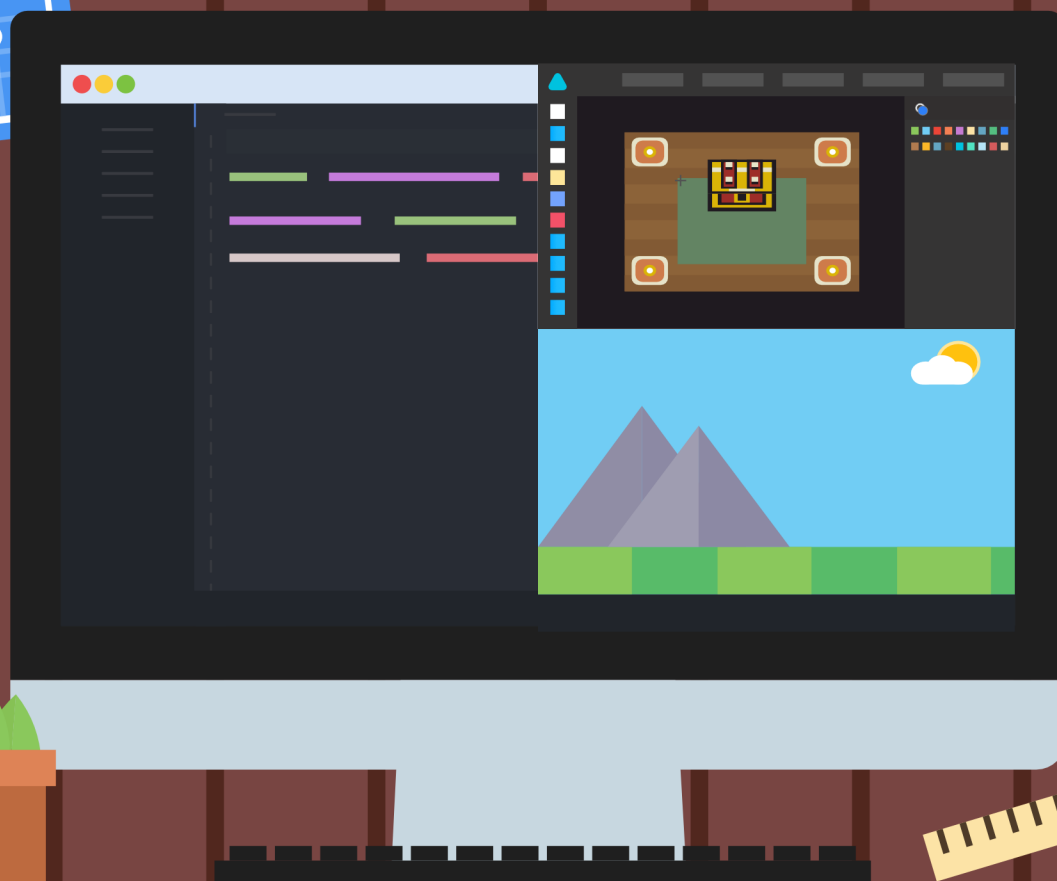


4:00 PM

REACT.JS FOR THE VISUAL LEARNER

Mike Mangialardi



from the founder of  coding artist

React.js for the Visual Learner

Mike Mangialardi

This book is for sale at <http://leanpub.com/reactjsforthevisuallearner>

This version was published on 2017-06-17



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2017 Mike Mangialardi

Tweet This Book!

Please help Mike Mangialardi by spreading the word about this book on [Twitter](#)!

The suggested hashtag for this book is [#reactjsvisuallearner](#).

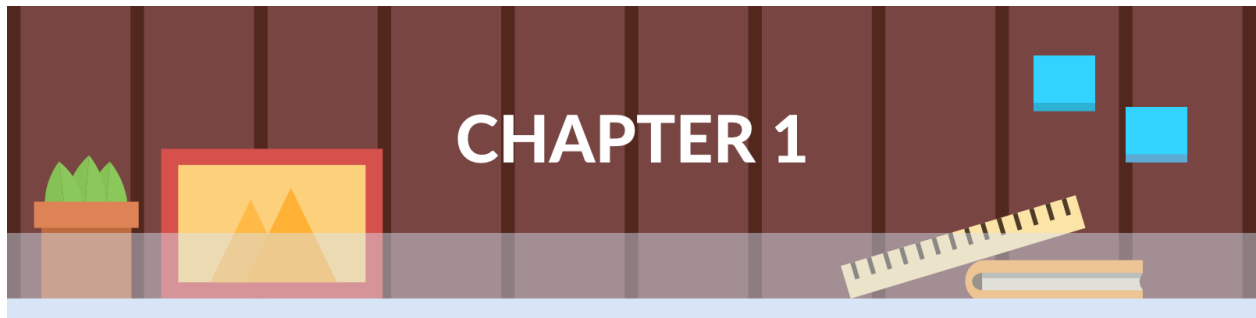
Find out what other people are saying about the book by clicking on this link to search for this hashtag on Twitter:

<https://twitter.com/search?q=#reactjsvisuallearner>

Contents

Chapter 1 What is This All About?	1
Chapter 2 What is React and Why is it Cool?	6
Chapter 3 Building Our Houses	33
Chapter 4 Our First Village	57
Chapter 5 Additions to Our Houses	81
Chapter 6 Building Our First Project	134
Chapter 7 Building Our First User Interface	173
Chapter 8 Implementing React Router	226
Chapter 9 Chat Bot Demo Using Firebase	247
Chapter 10 Reacting to What We've Learned	302

Chapter 1 | What is This All About?



Before we dive into coding in React, I am going to do a lot of explanation. In this chapter, I am a going to be focusing on addressing what this project is all about. If you are tempted to skip over this and look for code samples, you *especially* need to read this.

...

There's one quick thing to address before we officially begin.

This book was written in Markdown and self-published in my spare time. I've done my best to make sure all the code works, all typos/formatting issues are corrected, and all instructions are super clear. Nevertheless, I know it won't be perfect.

To address this, I've created a Discord server where you can talk to me (or others) about questions, comments, suggestions, or anything else in regards to this book.

[Join the Discord server¹](https://discord.gg/ehSNtfD)

The Compact Explanation

I have written a series of blog posts that teaches the absolute basics of React.js to an absolute beginner through design principles, visual-friendly examples, and a super practical teaching methodology. This ebook is the compilation of the blog post series originally written on Medium.

What Does “Visual-Friendly” Mean?

Since this is intended for an absolute beginner to React.js, there is a lot of concepts and terminology to unpack.

¹<https://discord.gg/ehSNtfD>

Personally, I had a hard time “getting things to stick” when learning React myself. I got so frustrated trying to learn React that I fell into a season of impostor syndrome and almost decided to give up on being a web developer all-together.

I had a bachelor’s degree in Computer Science while working a day job as an IT Support Specialist. On one hand, I was frustrated that I wasn’t really using my degree. On the other hand, I was frustrated that I was not taught modern web development despite my degree and falling short whenever I tried to get caught up.

I honestly can’t remember exactly how this happened (I think maybe I needed to make a flyer for a broken printer or something), but I was introduced to Inkscape which is a free program for making vector graphics. I started messing around with it during my lunch breaks as stress-relief after a morning of trying to figure out why Microsoft Access Runtime would not install on some the PCs (*and other similar IT support stuff*).

Over time, I got better and better at making vector graphics and eventually upgraded to a more robust tool called Affinity Designer.

I started to realize that awesome vector graphics could be made by combining simple shapes, bold colors, and an intentional layout. It was a matter of breaking down a final graphic into smaller components.

After months of practicing vector graphics, I got really good and developed a lot of confidence. I decided to take that confidence to try and learn web development again.

Here was the roadmap of my journey back into web development:

Vector graphics lead to pure CSS images

Pure CSS images lead to visual learning of Vue.js

Pure CSS images lead to scalable vector graphics (SVGs)

SVGs lead to UI design

Vue.js lead to re-attempting React.js

All of the above lead to an increase in creativity which then lead to writing, launching email challenges, creating video courses, and starting my own brand to encapsulate everything

Unintentionally, I learned three major things:

Visuals are awesome for explaining web development concepts and terms.

There is a strong connection between general design principles and web development principles.

Coding graphics to learn web development was much more exciting than *yet another* To-Do app.

With these three things in mind, you can guess how I will use visuals to teach React.js to beginners:

I will use visuals to explain concepts and terms when possible.

I will also explain concepts and terms by making connections with design principles.

We will code graphics together as we learn React.

My hope is that this method of teaching will be a helpful and refreshing experience to finally conquering React.js. In addition to visuals, there are some other unique characteristics of the teaching methodology that I will use in this book that is worth mentioning.

My Super Practical Teaching Methodology

As you may have already noticed, my writing style always includes stories and personal experiences. I do this because I want to be as genuine and practical as possible.

As someone who has been writing technical blog posts for quite some time, I can tell you that tutorials are often the polished end result of a very tedious process. Some of my most popular articles may be only a 15-minute read, but they usually take between 10–20 hours. In those 10–20 hours, I often am in the middle of teaching myself the very thing the article is about.

I have often been on the opposite side of things where I was going through some tutorial and the following thoughts would pop up into my mind:

“Wow! This seems so easy for the instructor to teach...sure wish this came as easily to me.”

“How in the world did the instructor figure this out? Am I really that far behind?”

“I have no idea what that term means. Is that obvious or did they overlook something not so obvious?”

The list can go on and on. The point is that there can often seem to be such a wide gap between the instructor and the learner. I try my best to bridge that gap by letting you know that I've been there. Heck, I almost threw away web development and here I am writing this. Despite that, I am still often learning as I write and I still often get confused over the things that are often left vague.

In addition to being empathetic through sharing stories and personal experiences, I tend to avoid making assumptions that you previously know something that isn't listed as a prerequisite and using buzz words without explanations. Rather, I try to explain things in a very beginner-friendly fashion using "this is how of I thought of it" type of stuff.

With all this in mind, here are a few fair warnings:

I will go into great detail to explain things, even when things that might seem trivial to you. I do this because I rather explain too much than too little.

If I err, it is because I write in a way that may not be technical. This may happen as I am often trying to avoid coming off like I am being overly technical and confusing.

Despite these warnings, I hope that this book is an easy and engaging read.

How to Read This Book

In addition to finding the connection between visual learning and web development, there was another major thing I learned that helped me conquer impostor syndrome.

The best thing about modern development is that there is a tremendous amount of available resources. The worst thing about modern development is that there is a tremendous amount of available resources.

It's 12:35 AM right now and Grammarly is probably going to tell me a bunch of typos and mistakes that I need to fix shortly. However, the statement above is not a typo.

You're on Medium and learning about web development...there's a good chance that you've been animated with excitement from all the posts on frameworks, startups, tutorials, courses, etc.

There's also a good chance that your mind might be spinning with things you want to learn. *Angular, React, Vue.js, Elm, Elixir, Firebase, Mongo DB, Meteor, Redux, Babel, TypeScript, ES6, Node.js, Electron, Ember, Webpack, Gulp, NPM, GitHub, GitLab, artificial intelligence, machine learning, Python, Three.js, AJAX, Scala, Rust, Bootstrap, Express, Socket.io, Docker, Anime.js, GreenSock...* just to name a few.

In my childhood, I experienced the sensation of money burning a hole in my pocket (as my mother would tell me). Someone would get me a gift card to a store. I went to the store. Let's say Best Buy. I would walk around for an hour (maybe more) tossing back and forth on how to spend my gift

card. A lot of times, there was nothing I really wanted, and if I had waited longer, I could have gotten something better. Nevertheless, I felt the urge to buy something immediately and was often disappointed.

This same sensation can be created when trying to learn web development. There are so many great resources to learn and topics to explore, but there's only so much time in a day. If not handled properly, this can easily lead to fatigue.

You will learn much faster if you slow down. I know that sounds backward but I guarantee that if you concentrate on one topic or skill at a time, you will learn at a faster rate than if you try to rush through your huge backlog too rapidly.

Want me to let you in on a secret?

I have purposely structured this first chapter to contain no visuals (other than the cover), no code samples, and no mention of anything about React.

Why? Because I want to drive the point of this section.

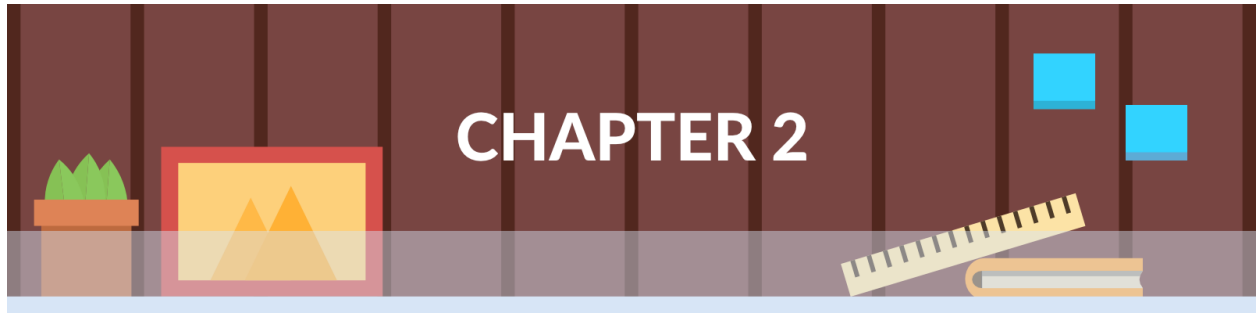
Slow down. Don't rush. Concentrate on this book.

If you can do this, you will learn a ton!

What is Next?

In the next chapter, I am going to give an overview of React. I will then progressively go into more and more detail as we go over new concepts.

Chapter 2 | What is React and Why is it Cool?



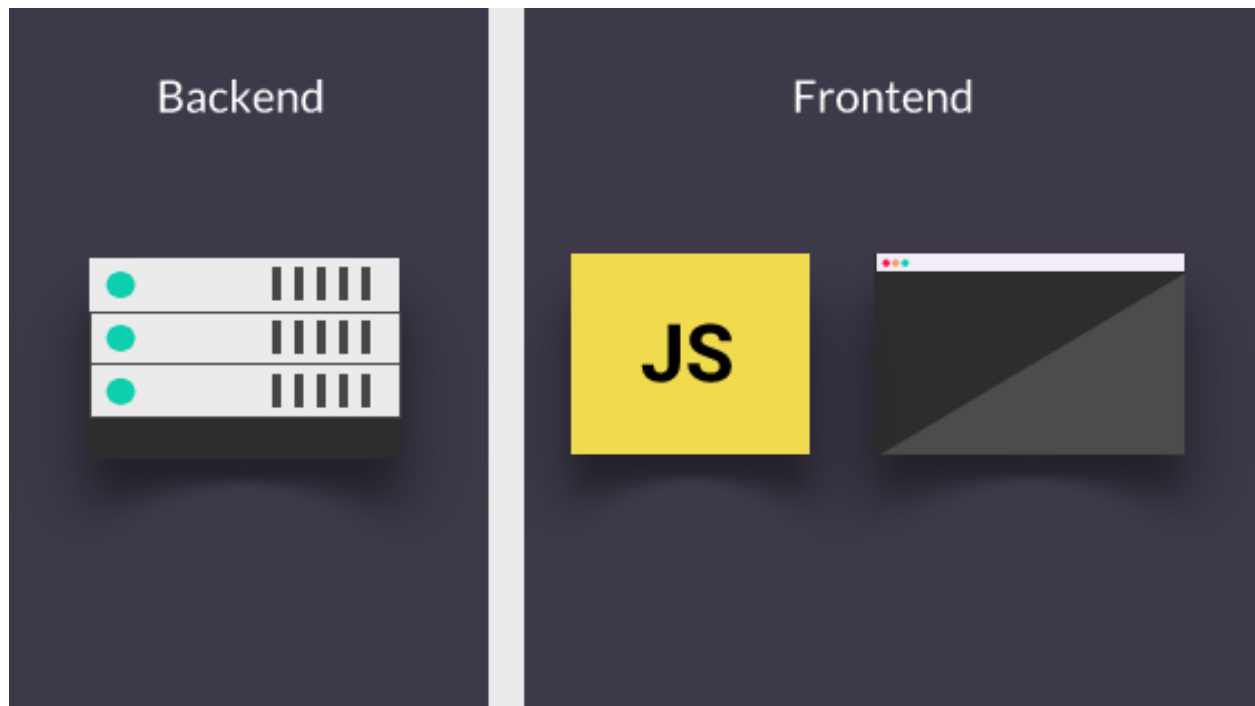
Overview of Modern JavaScript Web Development

If you are coming here as a beginner looking to learn React.js, chances are that you have heard “React” buzzing around the internet.

This might have you asking: “What is React and why do I keep hearing about it?”

We will be answering this thoroughly in this chapter. However, React is only a slice of the pie. We need to first ensure that we have a high-level understanding of the big picture of modern web development before we go on to discussing React in depth.

Brief Overview of Modern JavaScript Web Applications



I don't want to intimidate anyone with too much terminology at the start so I am purposefully going to keep this very brief.

A web application consists of a *backend* and a *frontend*. There is a *server* side and a *client* side.

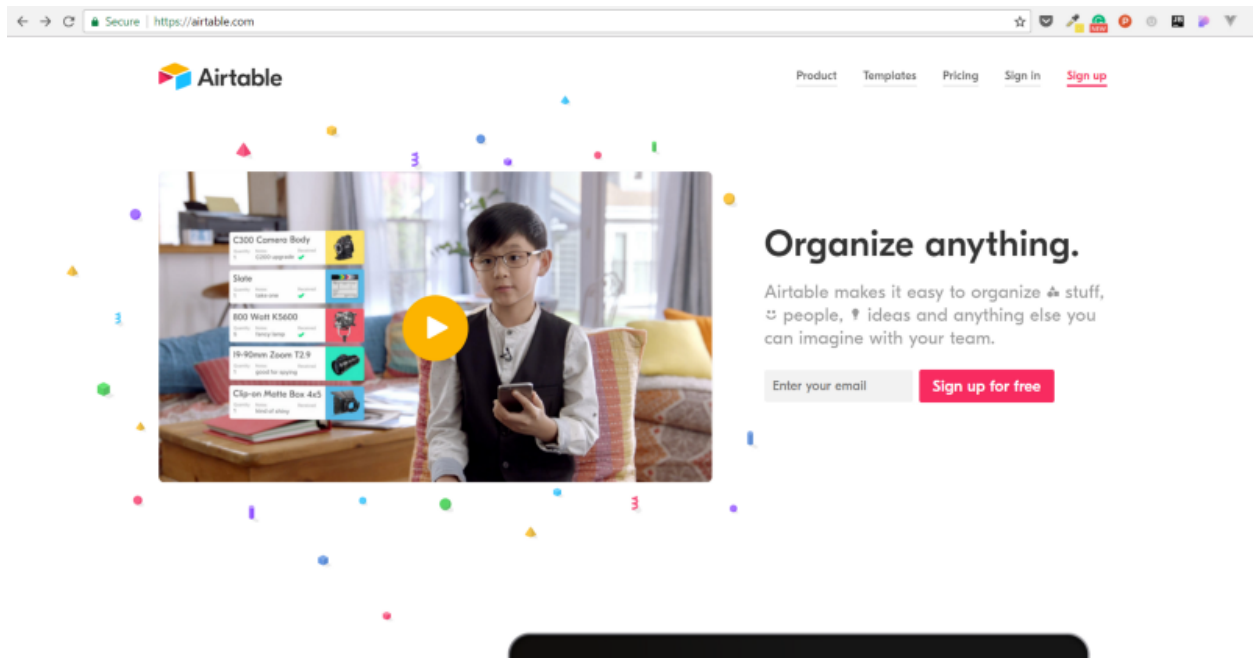
The backend consists of a server, an application, and a database. The server hosts our application. The server also communicates with a database to get data.

The backend passes *data* to the frontend. In modern web development, JavaScript on the frontend takes the data from the backend and *converts* it to *HTML*.

Both the frontend and a backend compose a web application. However, in the frontend, specifically the browser, the web application is manifested visibly through a user interface. The JavaScript, again, converts data from the backend and outputs HTML which you can think of as instructions for what to have displayed as part of the user interface.

Single Page Applications

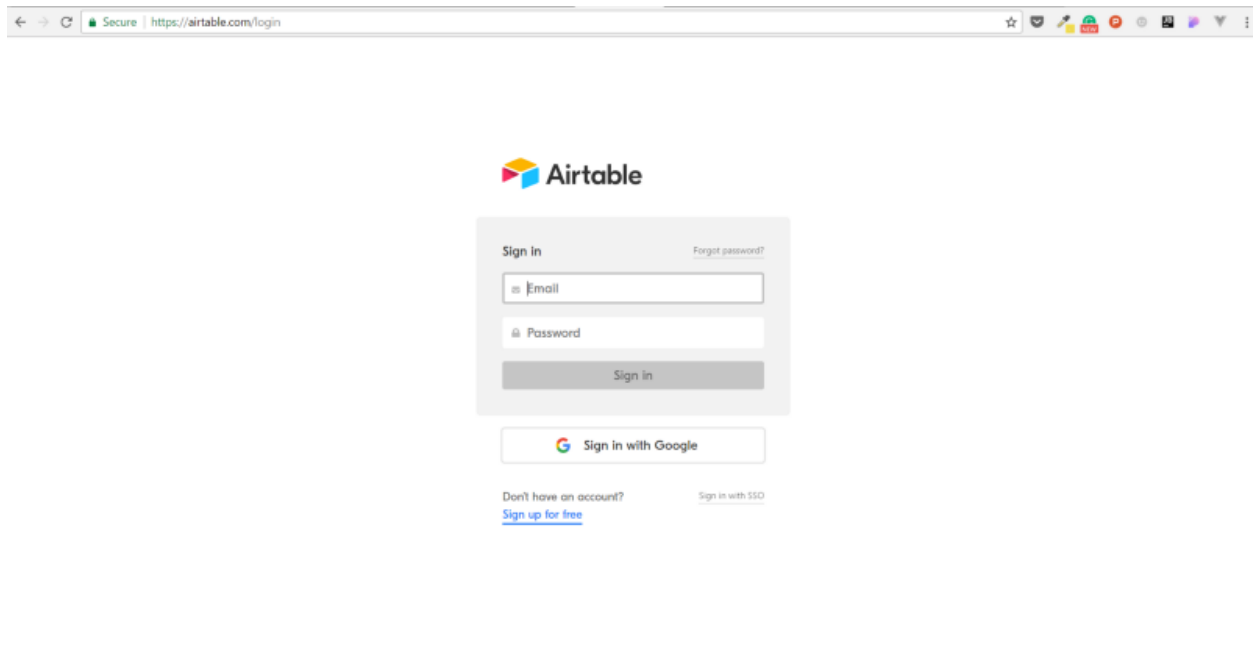
On any web application, there are multiple *views*. Let's look at an example.



Airtable Home Page

In the image above, we see the home page for a web application called Airtable. This is one *view* of the web application.

If I went ahead and clicked on “Sign In”, I would be brought to another view for the login:



Airtable Login Page

Unless you were aware of the modern trends in web development, most of us would associate the change of views with the change of *pages*. Meaning, when a new *view* comes up in a web *page*, a new page has been loaded.

Traditionally, this was the case. A web application would be composed of multiple different pages.



In order to create a new view, an entirely separate page containing that view had to be loaded.

However, a more modern approach is to create **single page applications**.

A single page application refers to an application that is composed of multiple views but only a single page.

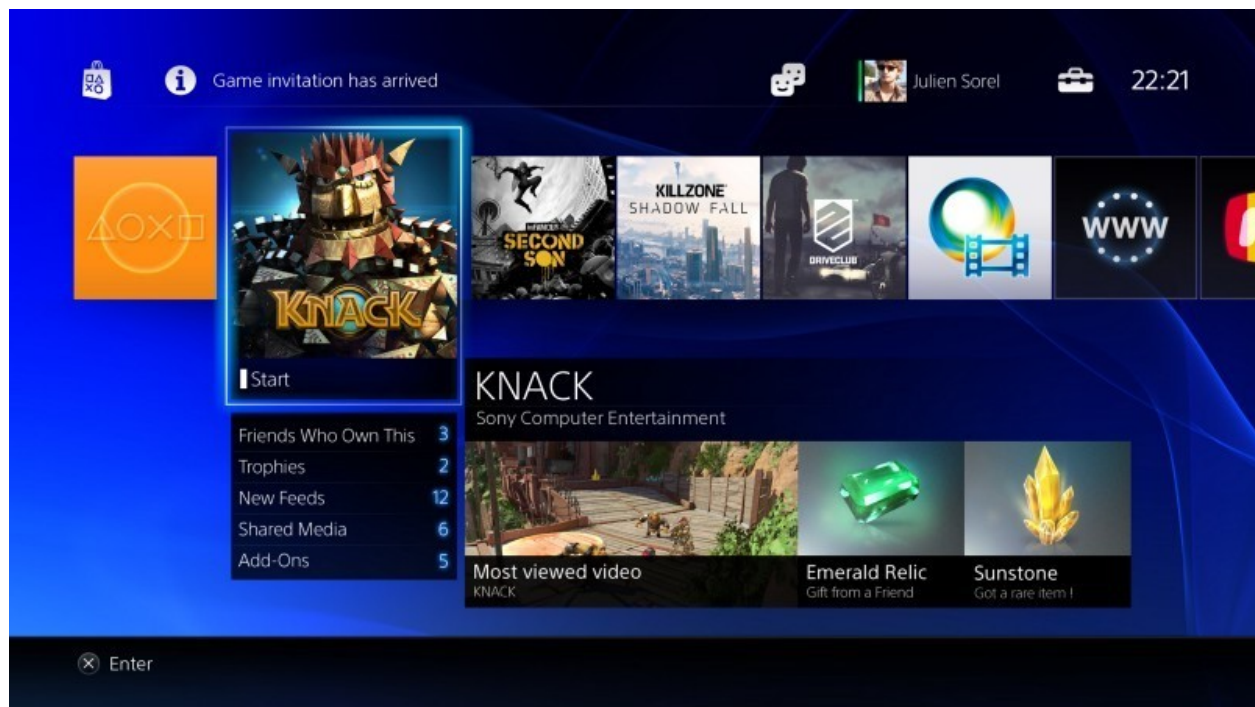


Instead of multiple pages being available to load separately and a new page being loaded each time a new view is needed, a single page containing all possible views is loaded up-front and the active view can be dynamically toggled.

To give an illustration, think about old video game consoles like the Nintendo 64.



If you wanted to load a new game, you would have to load a new cartridge.
Compare this to the PS4 where you can have games digitally downloaded on your console.

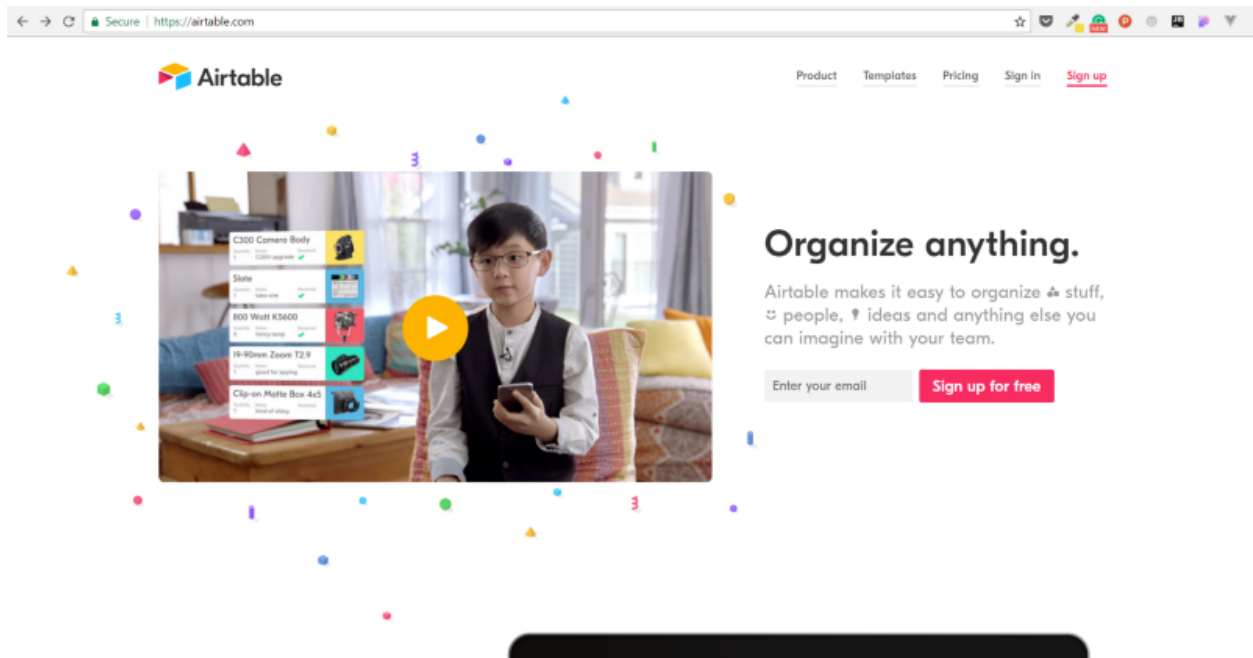


©2013 Sony Computer Entertainment Inc. All rights reserved. Design and specifications are subject to change without notice.

These digital games are already downloaded and available to play. If you want to switch games, you don't need to load in a new cartridge like the N64. Instead, you can turn off the current game and load a game that is already available on the console.

With this illustration in mind, let's return again to the example of Airtable.

Here is Airtable's home page:



Airtable Home Page

In a multi-page application, the product, templates, pricing, sign in, and sign up pages are all views on separate pages. If a user clicks the sign in link on the navigation bar, the sign in view needs to be loaded. In order for the sign in view to be loaded, an independent sign in page would have to be loaded.

In a single page application, a single page would be loaded and all the possible views in the navigation bar would be available. Although all of the views are available, there is only one active view for the home page. If a user clicks on the sign in link, the home page view would be toggled off as the active view and the sign in view would be toggled on. There is no need for a separate page to be loaded.

With all this in mind, let's discuss how React fits in these modern concepts of web development.

What is React.js?



The official and broadest answer is that it is a JavaScript library created by Facebook for building user interfaces.

I will now unpack the specifics of what this means and how it is fitting for modern web development.

JavaScript Library

Saying that React is a JavaScript library is a fitting term. Meaning, the term encapsulates a really nice visual of what it means.

At a library, there is a collection of books. These books are already gathered and sorted for you. The heavy lifting is already done for you. If you want a book from the library, you can just go ahead and locate a book by referencing the Dewey Decimal System and grab a copy.



With a JavaScript library, there is code that is already created and made available for us. Specifically for React, there is prepackaged code available for us that we can use to make views for user interfaces. In the chapters that follow, we will look into much detail at the *syntax* for being able to use the code available in the library.

Declarative Syntax



We will also see that React has nice *declarative* syntax.

Declarative syntax means that we will write more along the lines of being direct with *what we want* instead of writing out all the steps of *how we want to get there* (imperative syntax).

Without getting technical, let's look at an example in plain English:

Declarative: "I need milk."

Imperative: "Get in the car. Drive down the store. Find the milk aisle. Grab milk."

An advantage of declarative syntax is that it is easier to see what the code wants to do which clears some confusion when debugging your code.

Single Page Applications

We previously discussed the modern approach of single page applications. With React, we can create single page applications by creating views for each state in our application (home page, login page, etc.) and dynamically toggle on and off the views depending on the state.

Because an application only has to load a single page at the beginning, navigating through different views in a React web application is really fast and requires no page loading.

Component-Based



We have discussed how single page applications are made up of views which we can create using React.

One of the main features of React is that it is component-based.

Components are the building blocks of views.** We will go into the finer details of components when we actually start coding them. However, the basic gist is that we can create reusable, predefined chunks of code that can be put together to create a view. A basic example of a component would be a button on the user interface.

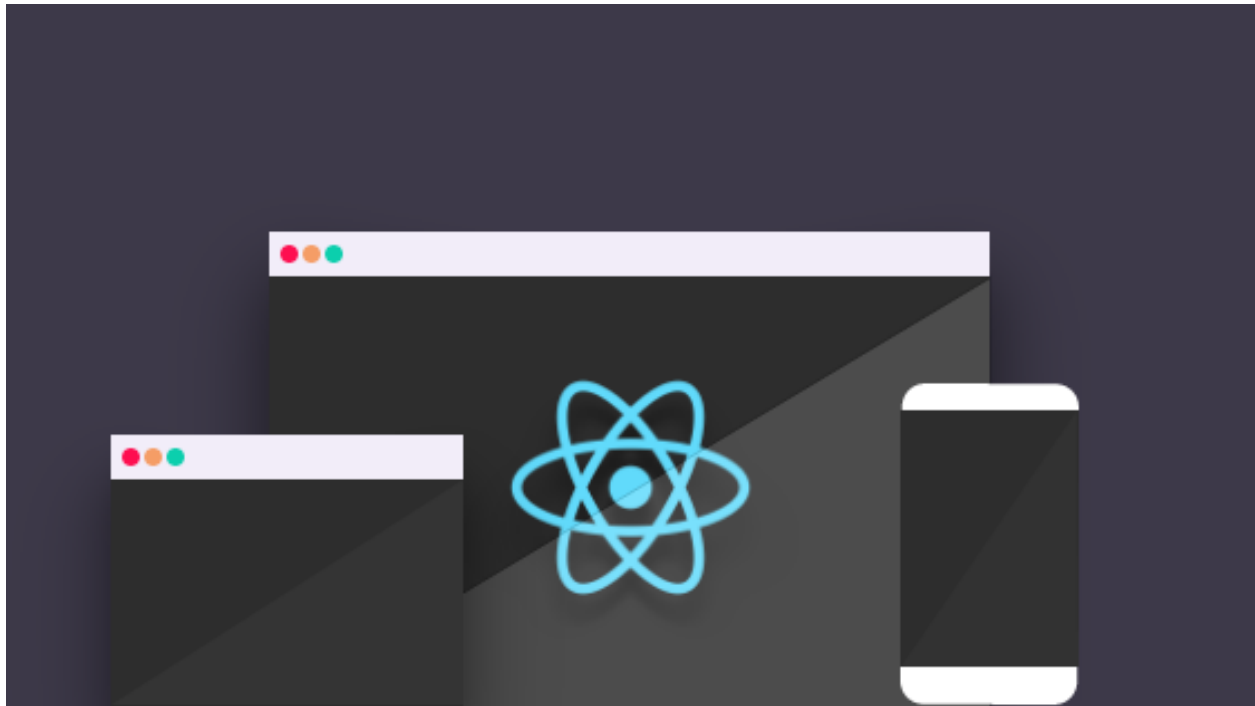
To provide a non-technical illustration, imagine we wanted to build a log cabin. If the final log cabin is equivalent to a view, then all the logs that make up the log cabin are the components.



If 20 logs of the same size need to be made, a single blueprint can be referenced to cut the 20 logs into the proper size before assembly. In a similar way, we can write code to create a blueprint for a component that can be used multiple times in a view.

Components are also reusable across views as well. For example, a button component can be used on a home page and a login page.

Coding for All Platforms



Nowadays, JavaScript code can be used to write not only web applications but desktop and mobile applications as well.

Cross-platform desktop applications can be written using React and other web technologies using Electron.

React also has its own framework called React Native for writing mobile applications. It uses the same design as React.

Both Electron and React Native are outside the scope of this book. However, it is important to mention that this is a really nice benefit to learning React.

Not only is it exciting from a developer perspective, but also from a business one. Businesses can now hire just one React developer who theoretically could handle desktop, mobile, and web applications without learning separate languages and technologies. This can potentially save a business money from having to find a developer for each specific platform.

This should also serve as motivation to continue on with this book and learn React.

The React Village Design



Alright. So far, we have discussed the concept of *views* and *components* that make up single page applications on a high level.

Views refer to the *content* that would traditionally appear on a separate page on multi-page applications.

For example, the entire content, or view, of a home page or a login page.

An illustration that I gave is thinking of a view as a log cabin and with the components being the logs that make up the cabin. I alluded to the fact that React allows us to create blueprints for different types of logs that we can use in multiple places in the construction of a log cabin.

To be honest, I think that this illustration, while good for getting the basic gist, is too simplistic. Reason being, the concept of components is *too vague*.

This may be one of the main reasons that React was so difficult for me to wrap my head around. I couldn't visualize the concept of components that making up views.

After learning vector graphics, I soon realized that the concept of components made perfect sense. Why? Because you are forced to look at graphics in terms of their components. You are also forced to develop a terminology to more clearly define the components.

Let me show you what I mean by this. Take a look at this vector graphic of a koala that I made:



Now, if I asked you to tell me what *components* make up this koala, there are two possible answers I suspect you would give.

On one hand, you could say that the koala's components are ears, eyes, a nose, a head, and hair.

On the other hand, you could be even more specific and say that the koala's components are 3 light gray circles, 2 dark gray circles, 2 white circles, 2 navy blue circles, a brown rounded rectangle, and two light gray triangles.

Both of these answers aren't wrong. The former explanation is broader and categorized while the latter explanation is very precise and particular.

What I am trying to get at is that there is a hierarchy when it comes to describing components. For this reason, there needs to be more clear terminology to understand how we can describe a React application in terms of components.

I have developed terminology to do just that. I call it **The React Village Design**.

There are 4 major levels of a village hierarchy (going from broadest to most specific):

Village
Neighborhoods

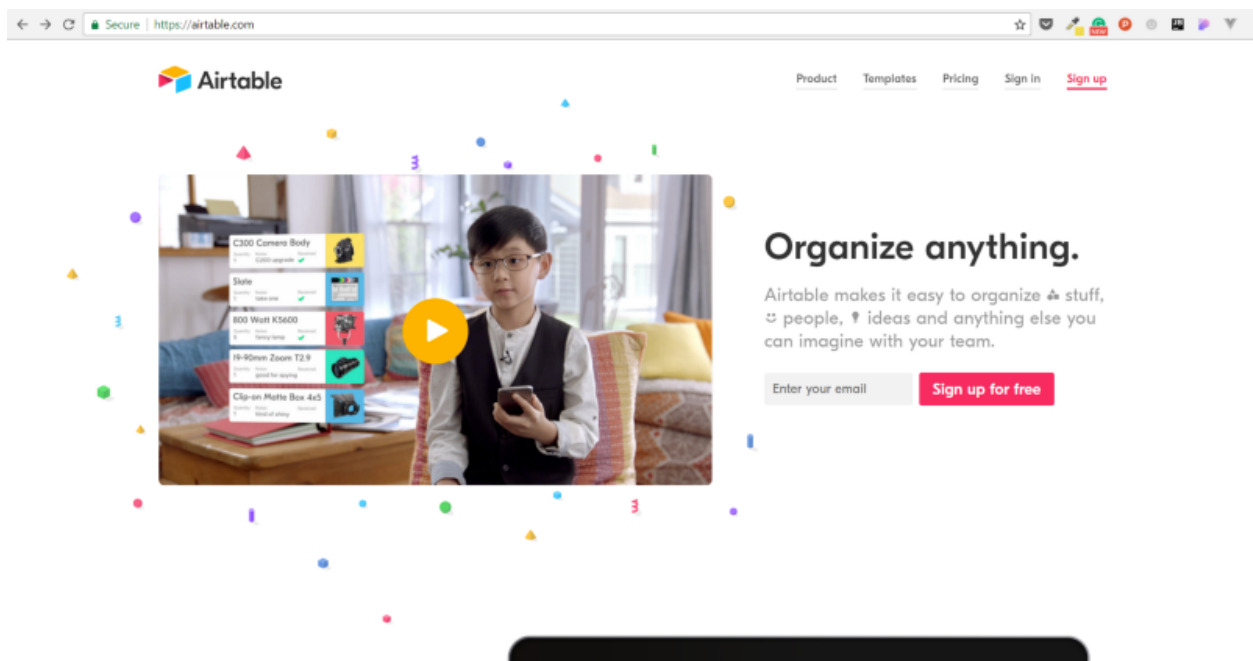
Blocks
Houses

Village



Let's start with village since it is the easiest to grasp.

A village is the equivalent of a view such as a home page.



Airtable Home Page

This is the broadest part of a village hierarchy.

Now, what if I asked you to tell me the components that make up a village. Just like our koala, there are different ways in which we could describe this.

Houses

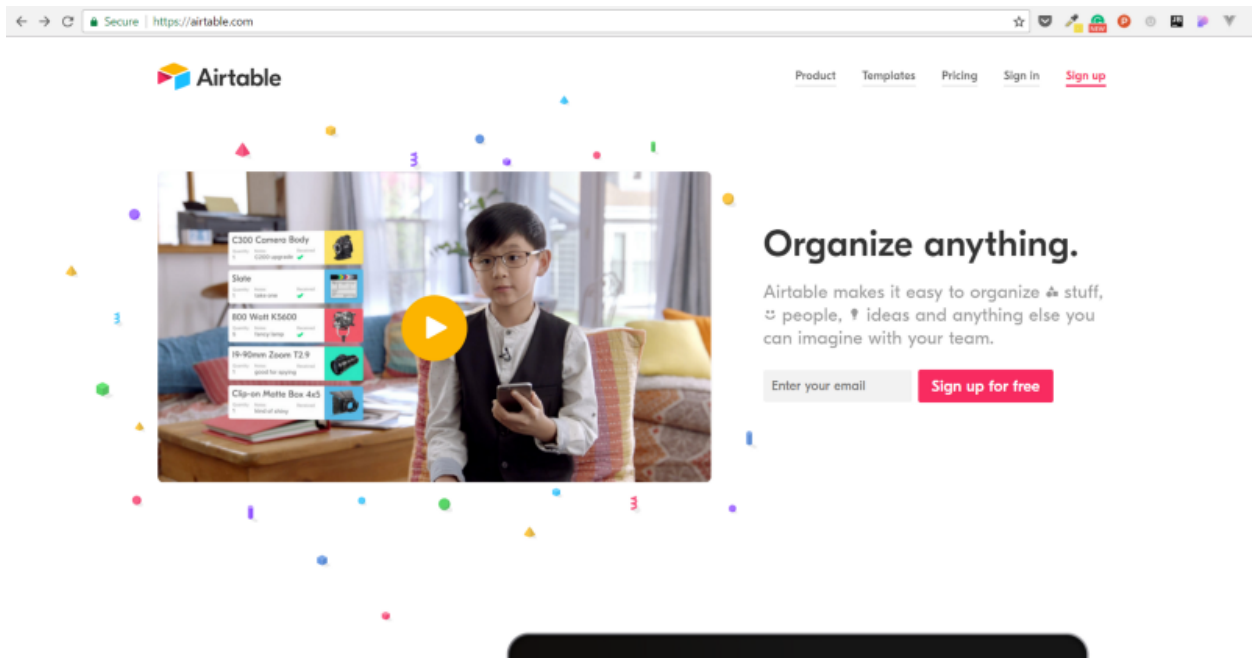


Let's start at the most specific level, the lowest level of the hierarchy. At the most basic level, we could say that a village is a collection of houses.

Houses are the building blocks of a village.

In a similar way, *houses can be used to describe the building blocks of a view*. They cannot be broken down any further.

In our Koala example, a dark gray circle is a house. In the example of the Airtable home page, the title, paragraph, sign up button, etc. are all individual houses that make up the view.



Airtable Home Page

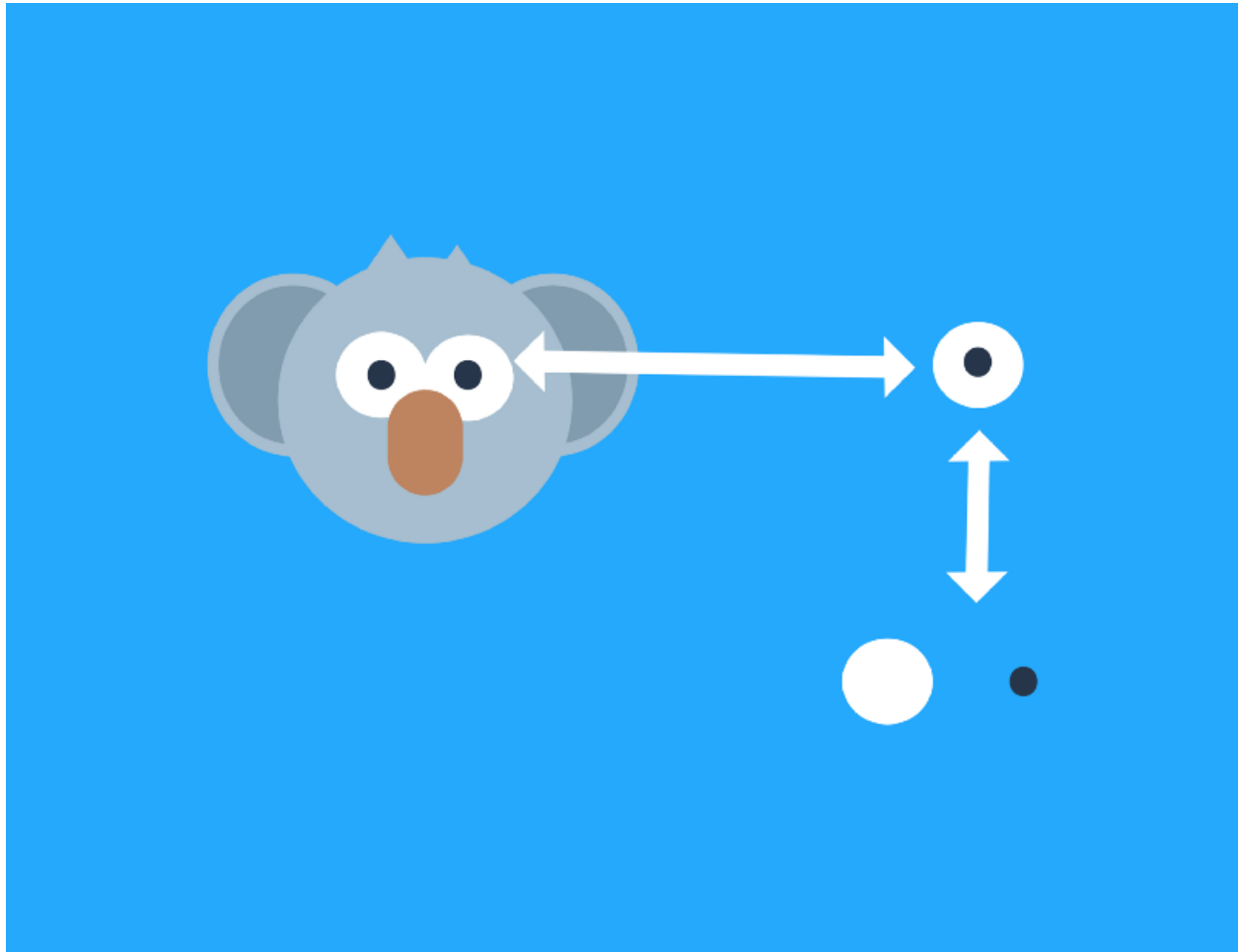
Blocks



At the next level of a village hierarchy is blocks.

Blocks are a collection of houses. So in a broader sense than houses, we could say a village is a collection of blocks.

In our koala example, we could say that the eyes are an example of a block:



The eyes are the next level up above the houses which contain a white circle and a navy blue circle. While we can categorize the eyes into a block, they do not form a final image on their own.

Blocks in a user interface are collections of houses that work together to form a specific functionality/purpose to the view.

Like our eyes in the Koala, they can't stand on their own.

For example, the following title, paragraph, input form, and button in the Airtable home page are working together to get a visitor to "opt-in".

Organize anything.

Airtable makes it easy to organize 📁 stuff,
😊 people, 💡 ideas and anything else you
can imagine with your team.

[Sign up for free](#)

For this reason, we can call it a block made up of individual houses. Or, an opt-in section made up of a title, paragraph, input form, and button. Again, it cannot stand on its own as a view.

Neighborhoods



In our village hierarchy, we could go another level up and describe a village as a collection of neighborhoods.

A neighborhood is a collection of blocks. In terms of a user interface, I like to think of neighborhoods as the sections of the interface.

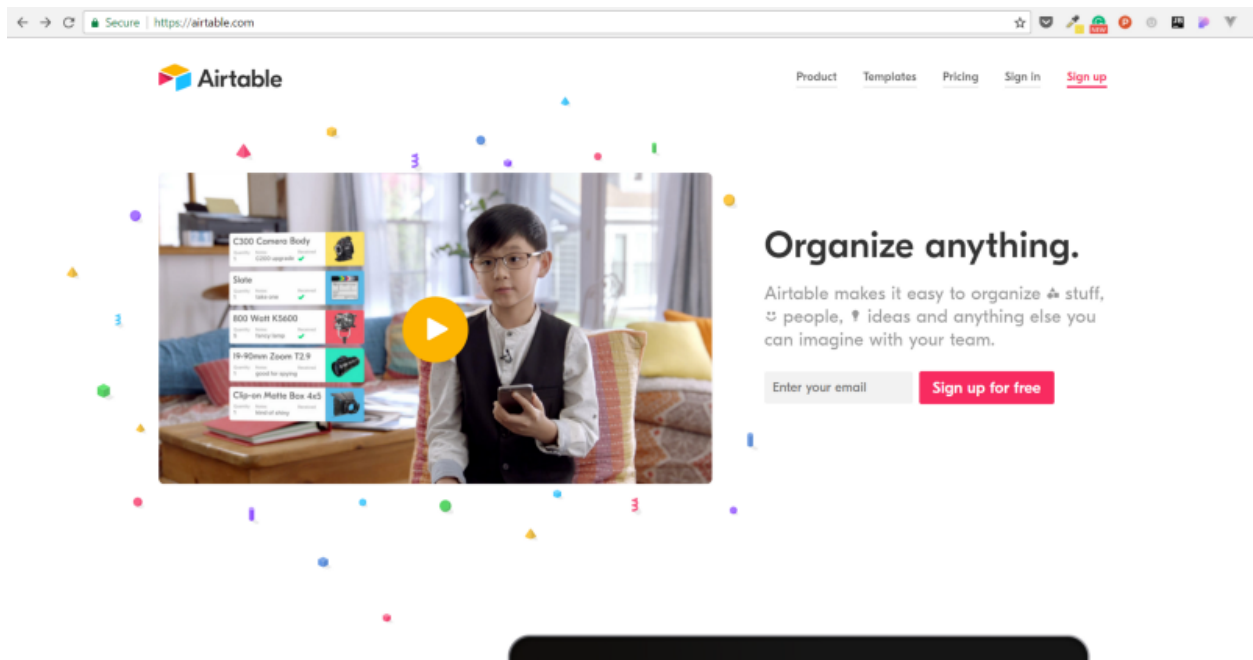
For example, this is the header of the Airtable home page which is a neighborhood:

The image shows the header of the Airtable website. On the left, there is a video player with a yellow play button. The video shows a young boy with glasses sitting at a desk, looking at a smartphone. On the desk, there is a tablet displaying a list of items: C300 Camera Body, Slate, 800 Watt K5600, 19-90mm Zoom T2.9, and Clip-on Matte Box 4x5. To the right of the video player, the text "Organize anything." is displayed in a large, bold font. Below this, a smaller line of text says "Airtable makes it easy to organize stuff, people, ideas and anything else you can imagine with your team." At the bottom right, there is a sign-up form with a text input field labeled "Enter your email" and a red button labeled "Sign up for free". The entire header is decorated with colorful confetti.

This header neighborhood is a collection of two blocks. We have the video and the surrounding pattern forming one block on the left. On the right, we have the opt-in block which we previously described. These two blocks form our header section, or again, our header neighborhood.

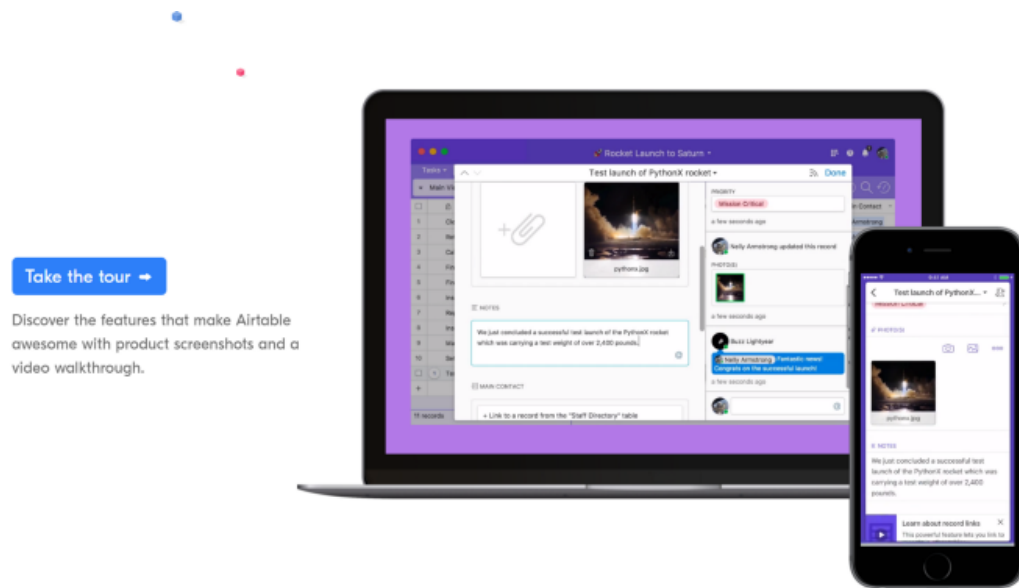
Let's go from the top of the Airtable site and point out the different neighborhoods.

At the top, we have the *header* neighborhood:

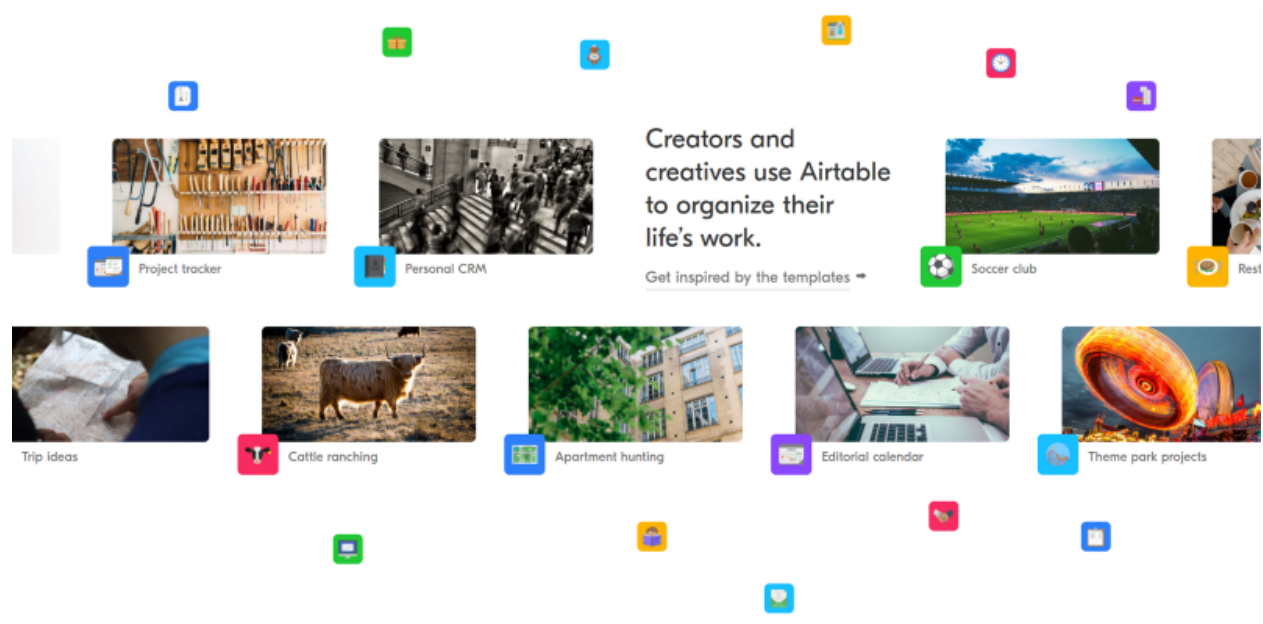


Airtable Home Page

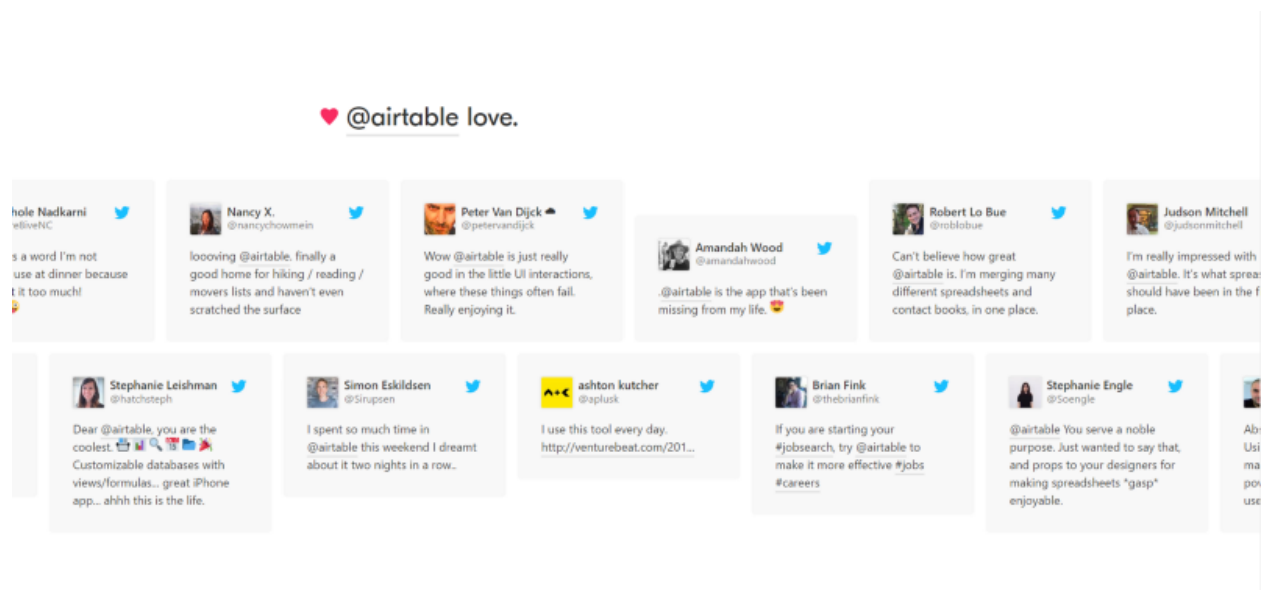
Next, we have a content section neighborhood. Let's call this *Section A*:



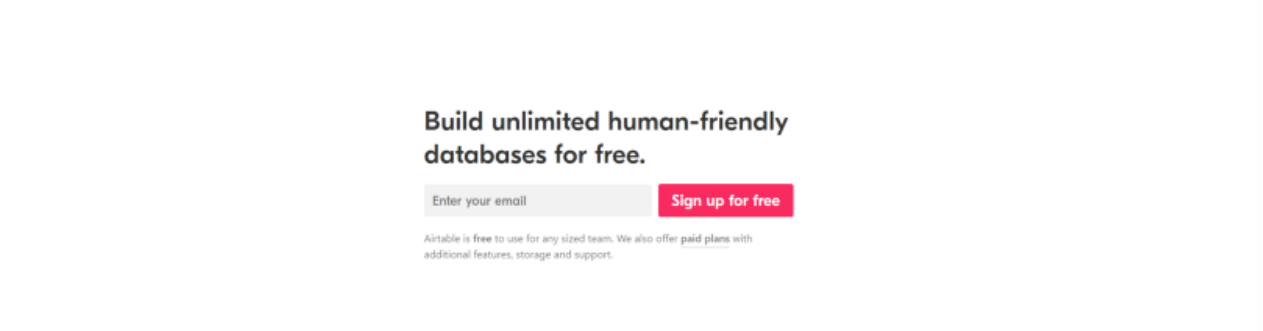
There is another content section neighborhood which we can call *Section B*:



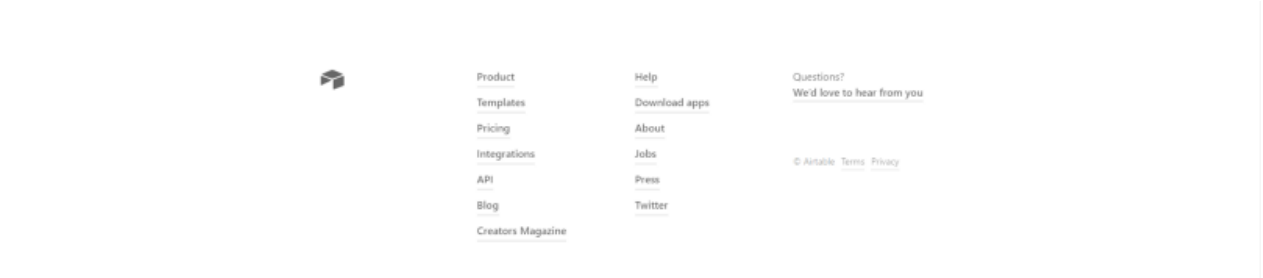
We then have a section with testimonials which we can call *Section C*:



Following that, there is another neighborhood called *Section D*:



Lastly, there is a Footer neighborhood:



If you put all these neighborhoods together, we have a village (or a view).

I acknowledge that The React Village Design is not always going to be a perfect analogy. However, I think it serves its purpose to communicate the idea that with React we can create single page web applications consisting of views that contain a hierarchy of components.

As we go deeper into React in this book, we will begin to see the benefits of having a component-based approach to creating user interfaces.

Concluding Thoughts

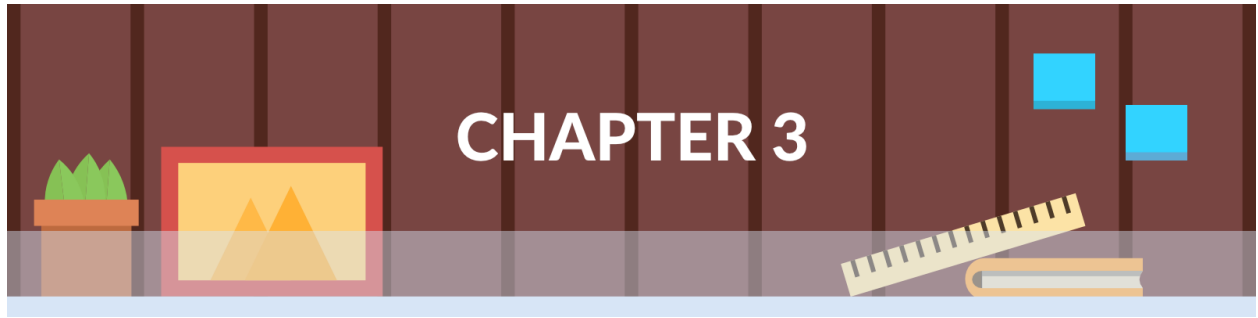
In this chapter, I have attempted to provide an overview of how React fits in the scope of modern web development and its benefits.

Particularly, I discussed the concepts of single page applications and component-based user interfaces. I made sure to go into extra detail to the about the latter as it is crucial to understanding how we can use React to create web applications.

To summarize the answer to “What is React and why is it cool?”, I would say the following:

React is a JavaScript library that allows us to construct user interfaces that are component-based and that can be dynamically updated depending on the state of an application. This allows us to create intentional, complex and yet organized user interfaces that fit nicely into the current state of modern web development.

Chapter 3 | Building Our Houses

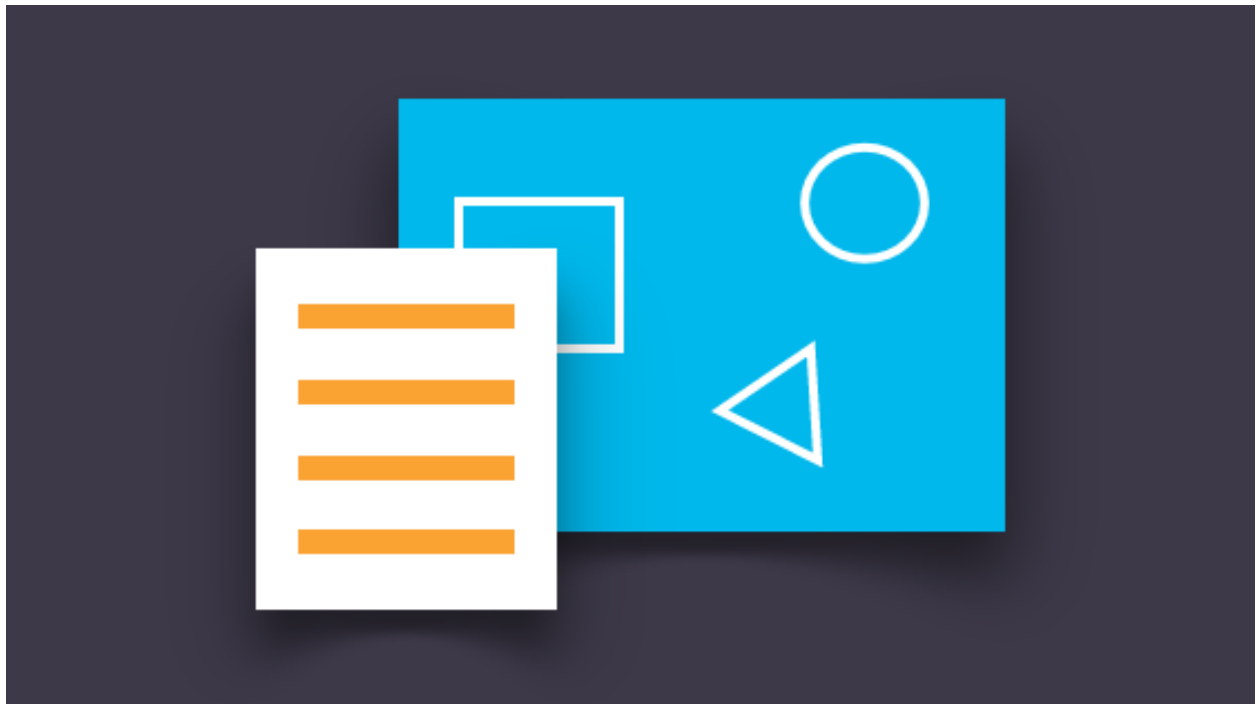


Good news! We will finally get to coding with React after dedicating 2 chapters as an introduction to what we are working with.

Hopefully, you have been able to gather the basic concepts of React and not been too intimidated.

With that being said, let's begin.

Defining Our Approach



In this chapter, we are to start by going to go over the core concepts of building houses. In other words, we are going to go over how to get started with making basic React components that can be used in a user interface.

As promised, we are going to be using visual code examples to understand these core concepts.

Before we begin, however, I want to make note of how I am going to approach teaching these core concepts.

React is notoriously confusing for a beginner because there is a lot of setup that is usually needed to get started. Many tutorials will acknowledge this but still go on ahead and try tackling the setup from the onset.

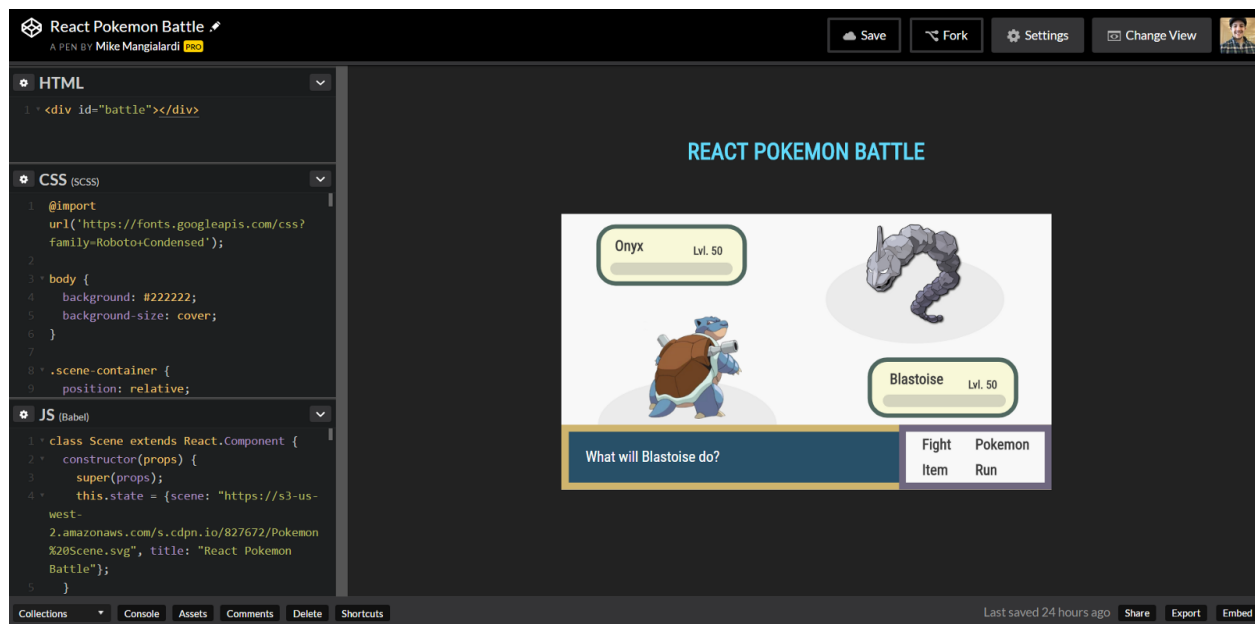
We will eventually be discussing the setup for React in a complete project, but I don't want to introduce that when we just trying to get started with learning the core concepts.

Codepen



As an alternative, we will start off by doing our development in [Codepen](https://codepen.io/)². Codepen will allow us to write out our code and instantly see the results on the same page.

²<https://codepen.io/>



It also allows us to easily add the React library while doing all the setup automatically for us “behind-the-scenes”.

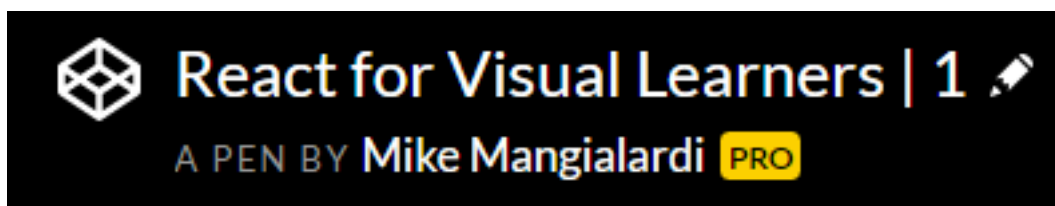
There will still be some setup, however, it won’t require nearly the amount of effort as if we were creating our own local project.

With that said, go ahead and create a Codepen account if you do not have one already.

After that, create a new pen and we will go over the rest together.

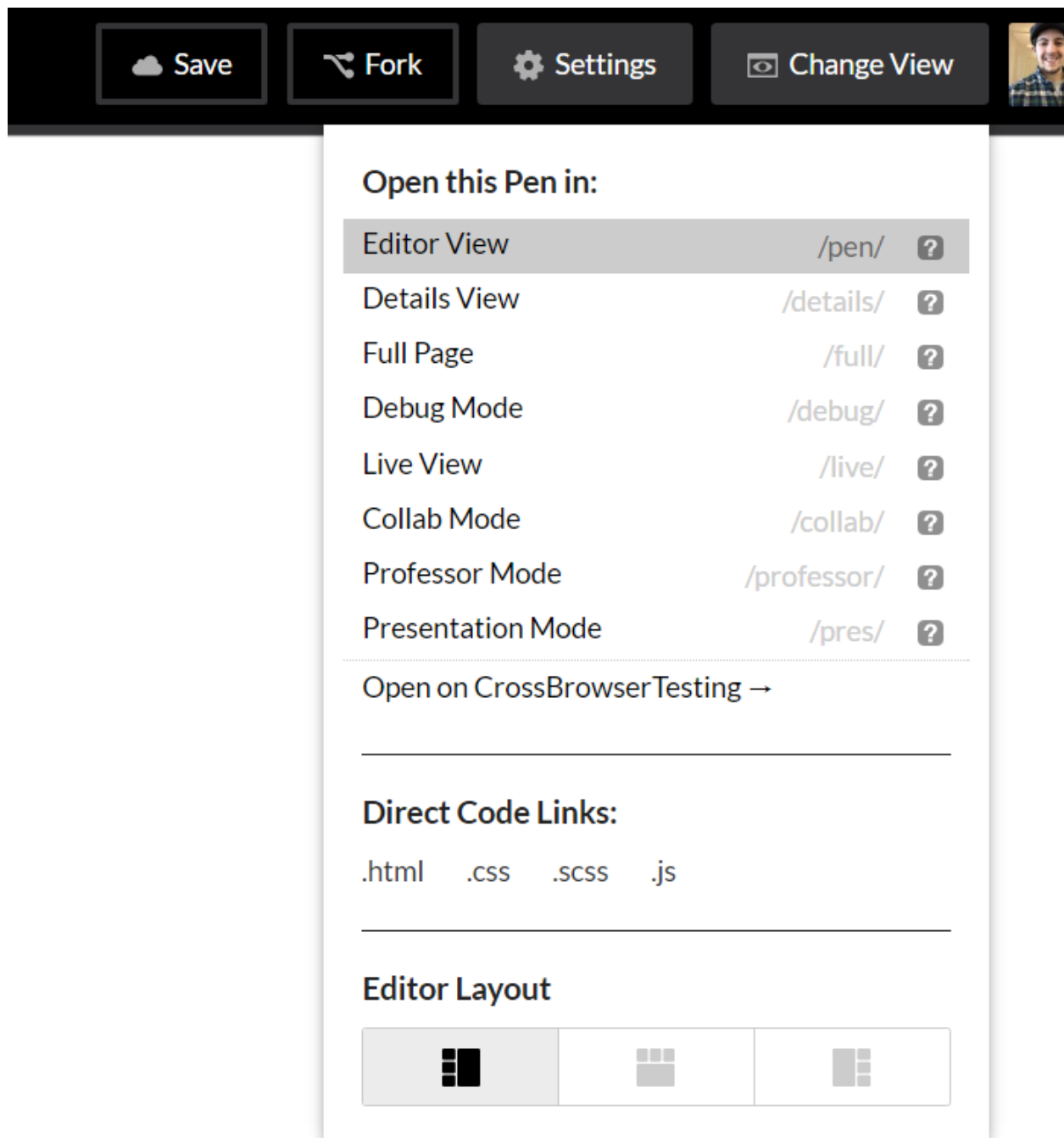
Setting Up Our Pen

Once you have your new pen open, let’s start by naming it React for Visual Learners | 1.

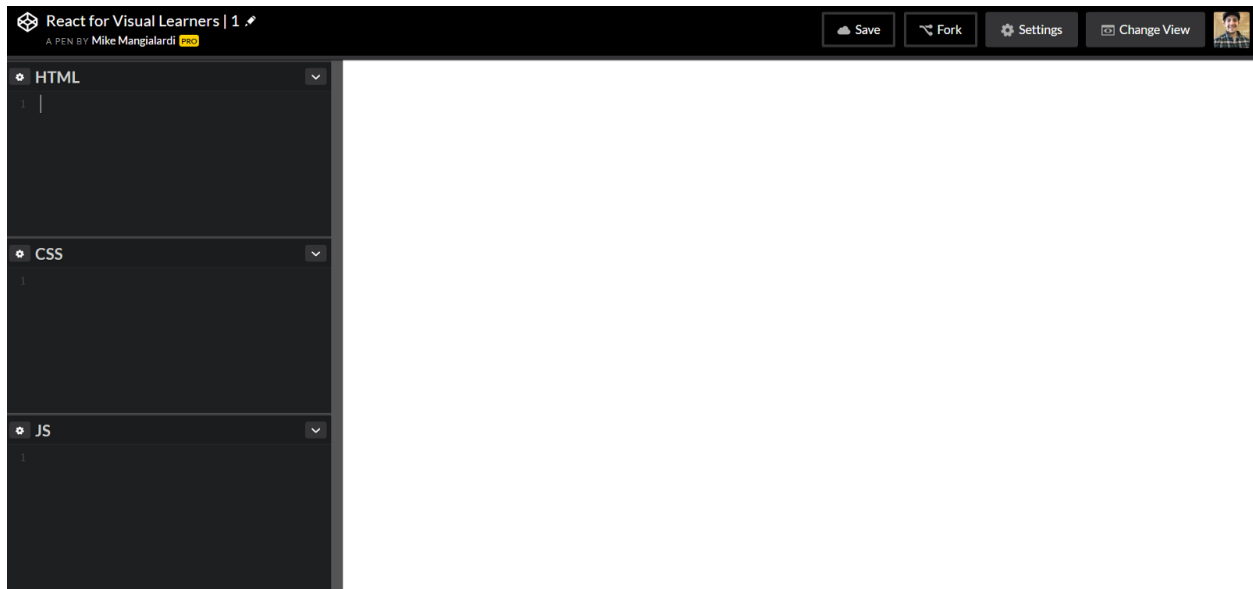


Next, let’s change the *editor layout*.

Click on **Change View** and select the far left icon under **Editor Layout**:



Now, we will have the editor (where we can put HTML, CSS, and JS) on the left-hand side of the screen and the preview of our code will be on the right-hand side of the screen:



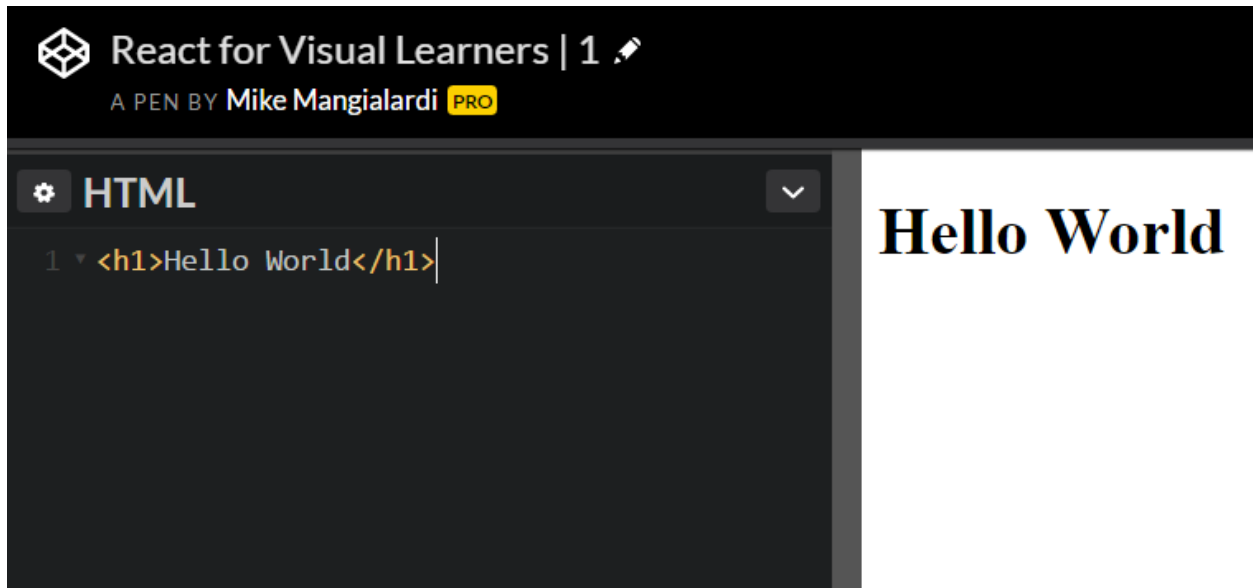
Hello World

Let's make sure this is working.

Type the following in the **HTML** column:

```
<h1>Hello World</h1>
```

Now, we should see the following:



Cool! We can see the output of our code instantly.

Go ahead and remove this Hello World line.

Adding Babel

Next, we will tell Codepen to add the React library and it will do the work behind-the-scenes.

Click on *Settings* and then click *JavaScript*.

Under *JavaScript Preprocessor*, select *Babel*:

Pen Settings

HTML CSS **JavaScript** Behavior Screenshot

JavaScript Preprocessor

Babel

Add External JavaScript

These scripts will run in this order and before the code in the JavaScript editor. You can also link to another Pen here, and it will run the JavaScript from it. Also try typing the name of any popular library.

https://codepen.io/username/pen/aBcDef

https://codepen.io/username/pen/aBcDef

Quick-add: ---

+ add another resource

Public Private PRO

PEN TITLE

React for Visual Learners | 1

PEN DESCRIPTION

Explain what's going on in your Pen here. This text is searchable, so it can also help others find your work. Remember to credit others where credit is due. Markdown supported.

TAGS

COMMA SEPARATED, MAX OF FIVE

TEMPLATE?

Don't close just yet, we need to add update more settings. But before we do that, let's explain why we just added a "JavaScript preprocessor" called Babel.

ECMAScript, ES5, ES6, and Babel Explained



What is Babel? Before we answer that, it is important to define some terminology of ES5 vs. ES6.

First off, there is ECMAScript which is a language standardized by ECMA International and overseen by the TC39 committee.

Basically, there are people who are standardizing a coding language. Just as book publishers might publish new additions of books with updated features, so there are people working on adding new features to this coding language.

JavaScript is merely implementations of the standardized ECMAScript language. JavaScript is not tied to any one particular edition of the language. As new editions with new updates and features of ECMAScript as released, JavaScript can implement them.

The problem is that it takes time for browsers to be compatible with the new editions. Therefore, there may be new editions the standardized ECMAScript language available but supported well in the browsers.

ECMAScript 5 (ES5) was standardized in 2009 and most contemporary browsers can process it.

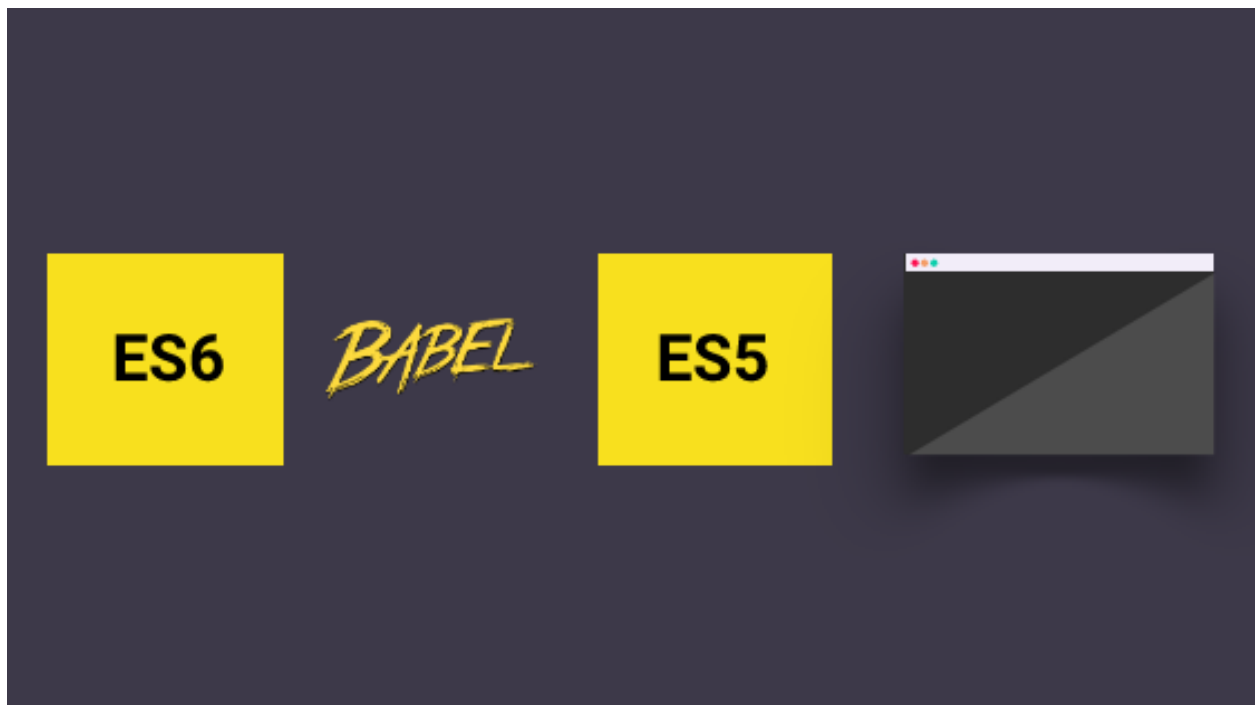
ECMAScript 6 (ES6) was standardized more recently in 2015. Therefore, browsers have not yet caught up to process it.

To write code in React, we want to latest features of ES6. This begs the question, how can we write React code using the ES6 standard if browsers have not yet caught up? The answer is Babel.

Babel is a JavaScript preprocessor.

Essentially, Babel says this: “Hey you want to write code using ES6? Yeah, I know...those browsers are taking forever! No worries, though! I got you covered. Just add me to your project and write away in ES6. I’ll straighten things out so the browser can still process you.”

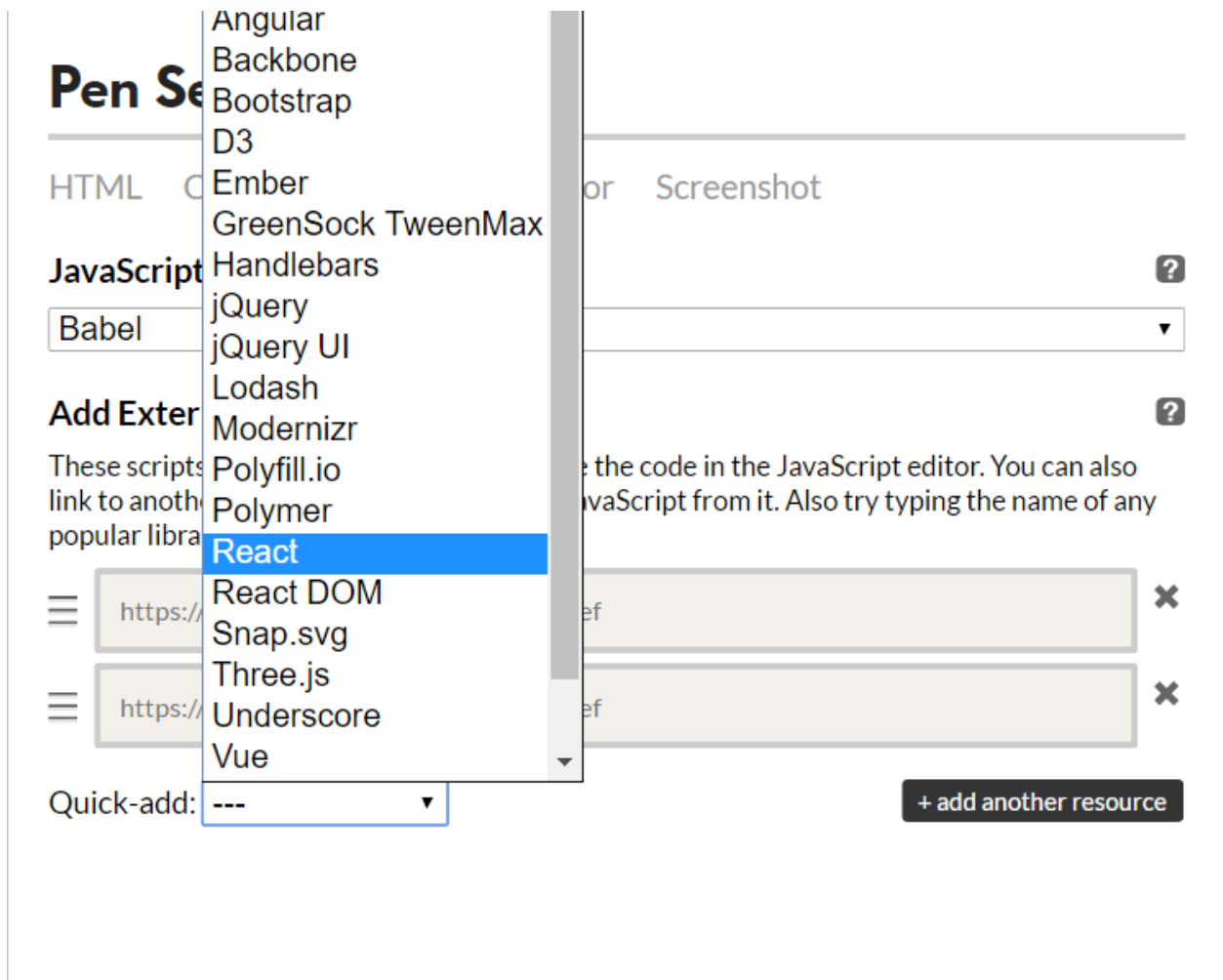
In short, Babel takes ES6 code and converts it to ES5 so the browser can process it.



Adding React

Now that we have added and explained Babel, we will need to add React as an external library to our pen.

In the same **JavaScript Settings** window you have open, select **React** from the **Quick-add** dropdown:

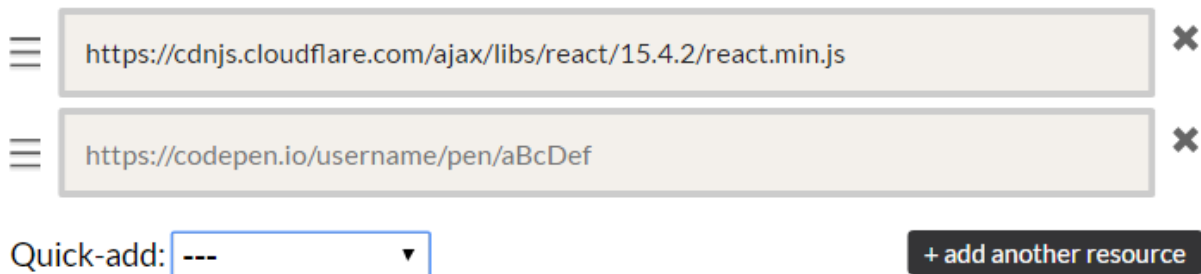


We can now see that a URL was inserted under *Add External JavaScript*:

Add External JavaScript



These scripts will run in this order and before the code in the JavaScript editor. You can also link to another Pen here, and it will run the JavaScript from it. Also try typing the name of any popular library.



To use an illustration, adding this URL is like getting a physical address for your local library. Having an address to your library would allow you to then be able to go the library and pick up books. In a similar sense, we have specified the address to the React JavaScript library so we can make use of it.

Before you save and close, we also need to add ReactDOM which can be added from the same *Quick-add* drop-down:

Pen Settings

HTML CSS **JavaScript** Behavior Screenshot

JavaScript Preprocessor ?

Babel ▼

Add External JavaScript ?

These scripts will run in this order and before the code in the JavaScript editor. You can also link to another Pen here, and it will run the JavaScript from it. Also try typing the name of any popular library.



<https://cdnjs.cloudflare.com/ajax/libs/react/15.4.2/react.min.js>



<https://cdnjs.cloudflare.com/ajax/libs/react/15.4.2/react-dom.min.js>



Quick-add:

--- ▼

+ add another resource

Wait a second, Mike! Hold up!

Are you pulling a fast one on me? This entire time you have been talking about using a React library not React libraries.

React and ReactDOM might be confusing at first but they are both subsets of the entire React library. However, both are technically separate libraries. The short answer is that React is the core library and ReactDOM is the bridge between React and the DOM.

If that definition still seems confusing, don't fret!

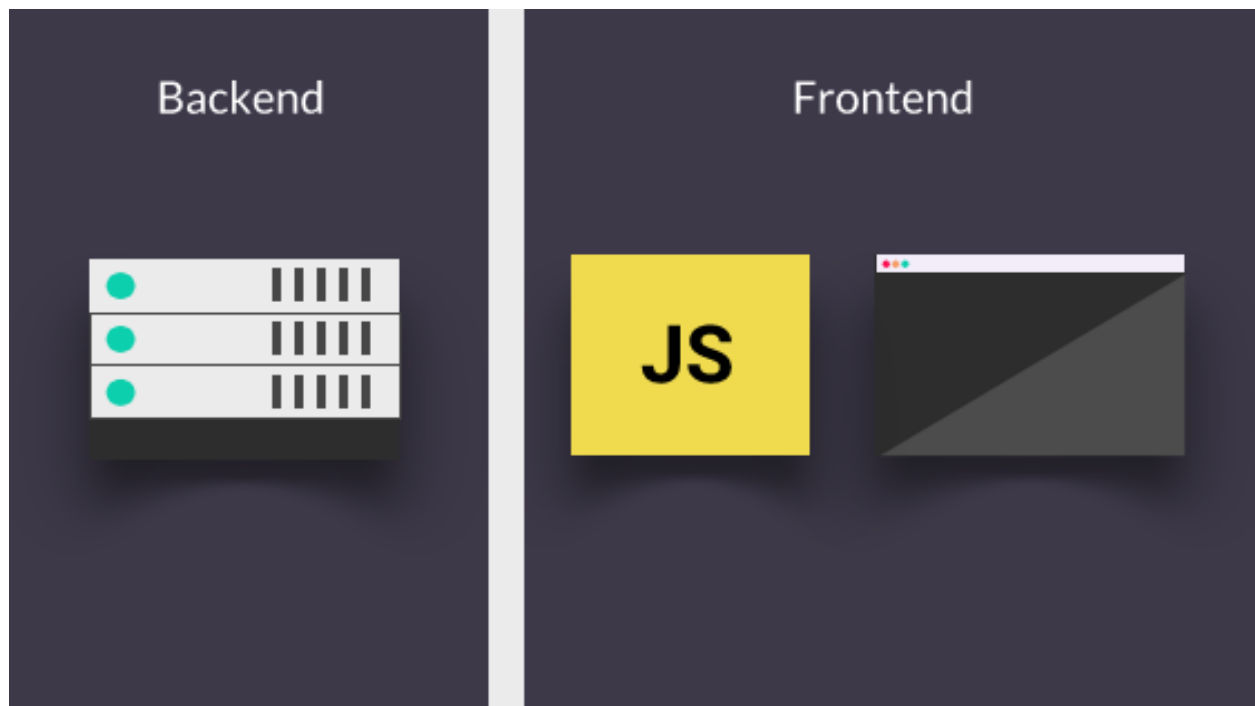
For now, you can think of ReactDOM as an assistant to the core React library.

Understanding React Core and ReactDOM

So far, we have distinguished that ReactDOM is an assistant to the core library. For this reason, I will distinguish between the React (describing the entire React library as a whole) and React Core (describing the core library used in making React components as distinguished from ReactDOM).

Each one serves a particular purpose. To better understand their purposes, we need to explain what exactly a component does on a bit more technical level.

In the previous chapter, we briefly mentioned that modern web development, such as in React, JavaScript *outputs* HTML.



With React Core, we create components that have encapsulated instructions for what to display in the user interface.

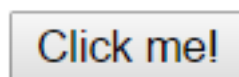
For example, we could create the following React component:

```
<Button/>
```

This component could be an encapsulation the following:

```
<button>Click Me!</button>
```

We would ultimately have the following displayed in our user interface:



React Core is used to define the encapsulation of a React component.

ReactDOM is the bridge between taking an encapsulated component and having it render as HTML.

In the button component example above, ReactDOM would be used to render the `<Button />` component in the HTML so the button could be displayed in the user interface as shown above.

I know we have used a lot of terminologies so far, so let's use another illustration to summarize.

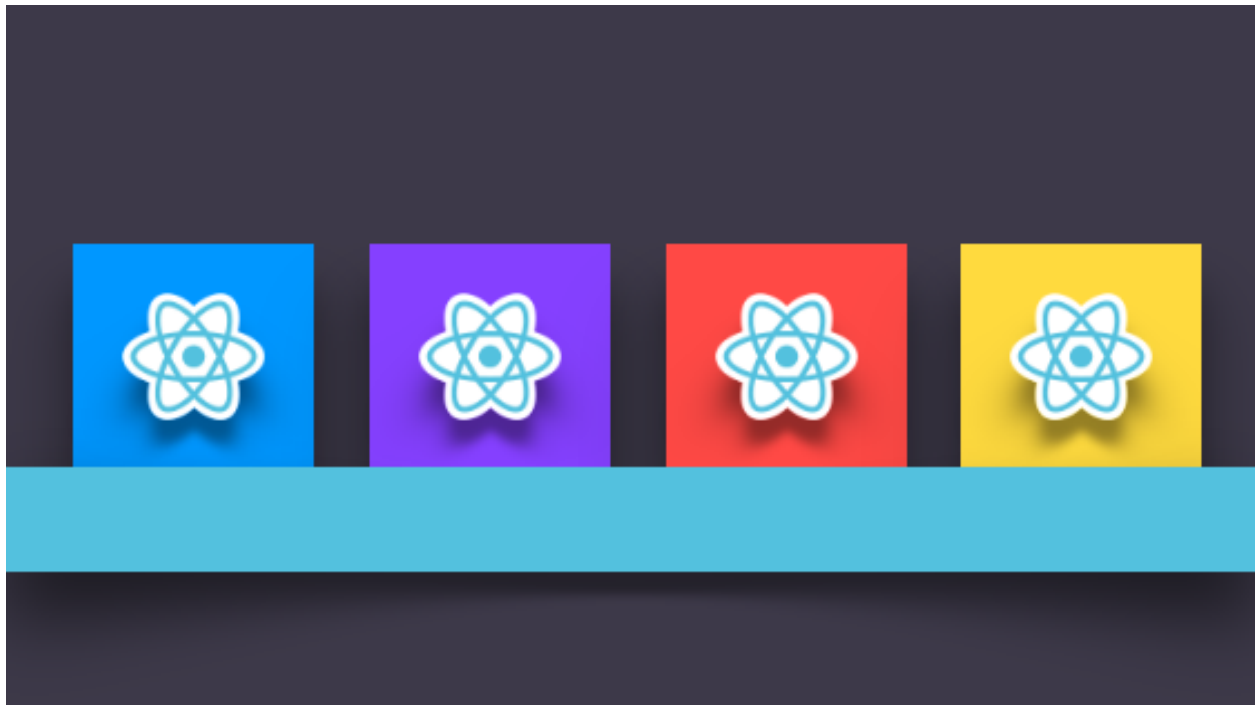
Imagine an online store that sells stickers. If someone places an order, they have to assemble the requested stickers that need to be sent out to the user into a package. This is essentially what we do when using *React Core*.



In order for the package to get to the user, the postal service has to transport the package. This is essentially what we do when using *ReactDOM*.

Now that we have our pen all set up, it's time to dive into creating our first React component. In terms of The React Village, we will be building our first house.

Creating Our First Component



Insert imaginary sound of drum roll here

We finally have some code to look at!

Here's code to add to the JS column in Codepen:

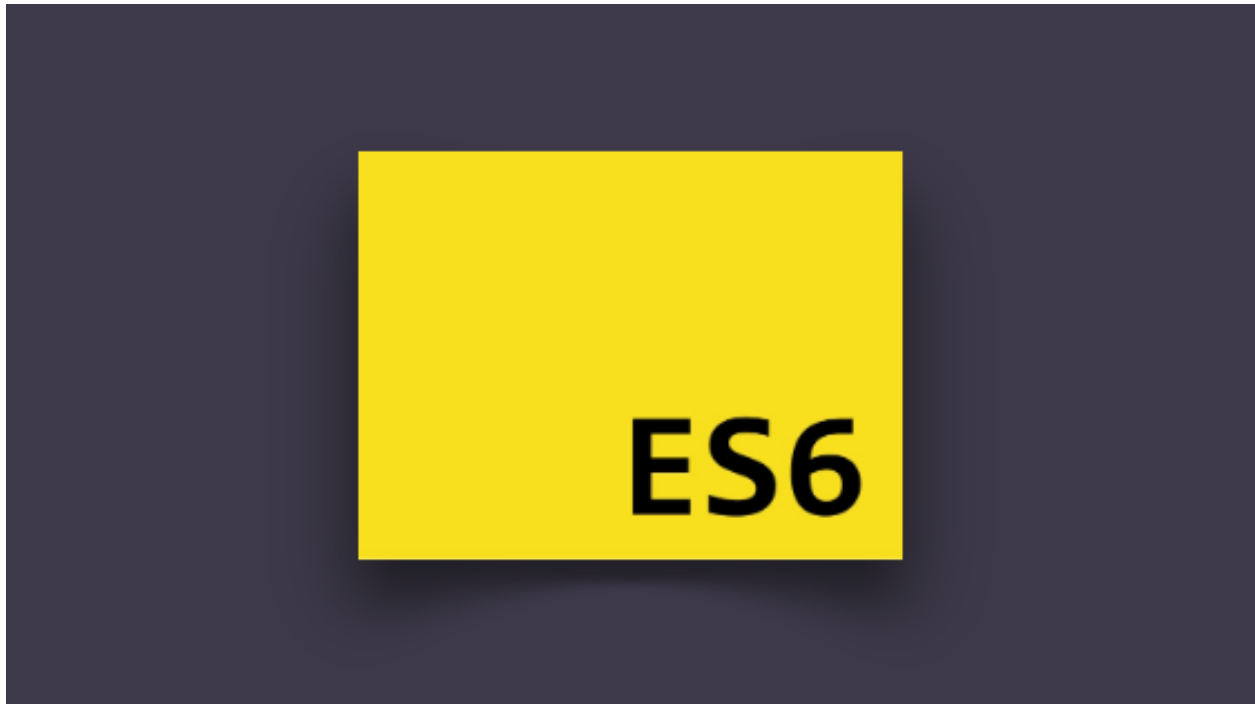
```
1 class Hello extends React.Component {  
2   render() {  
3     return <h1>Hello World</h1>;  
4   }  
5 }
```

In this code snippet, we won't see anything on the screen. Why?

This is only the *React Core* portion where we create an encapsulated component. We will later use *ReactDOM* so the encapsulated component can be rendered in the HTML.

Before we move on to *ReactDOM*, however, let's explain what's going on in the code above by re-creating it piece by piece.

ES6 Class



First off, we create a *child* class called *Hello* from a *parent* class called *React.Component* (hence *extends* *React.Component*):

```
1 class Hello extends React.Component {  
2  
3 }
```

The use of classes like is shown above was introduced with ES6.

Quick reminder, using ES6 is totally cool because we used the Babel as the JavaScript preprocessor.

For now, you can think of classes are “special functions”. We don’t need to use any of the special features of classes (in comparison to functions) at the moment. However, we will use them in the chapters to follow.

extends *React.Component* might seem confusing at first so we will explain it some more.

In Chapter 2, we mentioned that React is a JavaScript library. A JavaScript library has code that is already created that can be accessed and used for a particular purpose. With the React library, we can use existing code that does some heavy lifting for us to help with defining encapsulated components and rendering them (using React Core and ReactDOM accordingly).

Specifically, the *React.Component* class contains existing code that allows us to make components. Adding *extends React.Component* after class Hello basically says this: “I see you have class Hello here. You want to create a new class called Hello? Cool! What’s this? You want to extend Hello from *React.Component*? Ok. No problem. You now have access to all the code within *React.Component*.”

Render Function

Here’s the shell of a render function in a React component:

```
1 class Hello extends React.Component {  
2   render() {  
3  
4   }  
5 }
```

Remember, we are trying to create a basic component that can be rendered as HTML. In the code above, we have added a *render function*. In this render function, we are going to return what we want our component to render as when ReactDOM does its thing.

We will add the return statement in the next step.

JSX



JSX?! Another term?!

Don’t worry. One, JSX is really simple. Also, I’ve added a glossary at the end of this chapter.

```
1 class Hello extends React.Component {  
2   render() {  
3     return <h1>Hello World</h1>;  
4   }  
5 }
```

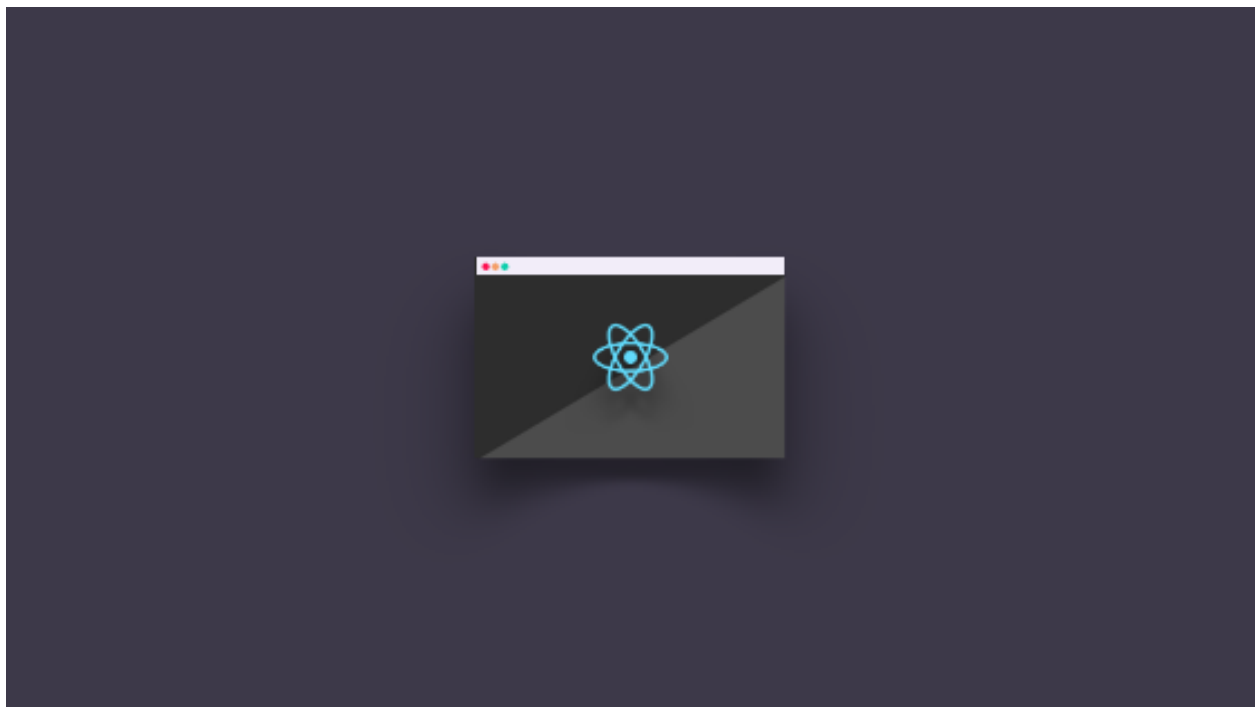
JSX is just extended JavaScript syntax that allows us to write out what we want our component to render as when the component is used.

The bold code above is the JSX. When we use this Hello component, a Hello World will be rendered as if we had written out the following in HTML:

```
<h1>Hello World</h1>
```

This completes the creation of our component. Now, we need to have it render as HTML using ReactDOM.

Rendering Our First Component With ReactDOM



Alright. We have created our first component using React Core. Now, we need to render the component using ReactDOM.

Add the following code at the end of your JS column:


```
1 ReactDOM.render(  
2   <Hello/>,  
3   document.getElementById( 'root' )  
4 );
```

We still should see nothing output. There's one final piece which we will get to in just a moment. Again, using React does heavy lifting for us.

We can use ReactDOM to do the heavy lifting to access the render function from all the components we have:

```
1 ReactDOM.render(  
2  
3 );
```

Next, we just need to use the components that we have defined.

The syntax is as follows:

```
1 //if class Example extends React.Component is up here  
2  
3 <Example />
```

In our case, we add the following for our Hello component:

```
1 ReactDOM.render(  
2   <Hello/>  
3 );
```

Remember, our Hello component's render function returns this:

```
<h1>Hello World</h1>
```

We have our component and we want to have it render as HTML. Previously, I used an illustration of ReactDOM as being like the postal service shipping a package. We need to “ship” this component to our HTML where it can then be rendered as the code snipped above.

With physical shipping, you need to define an address. In the same way, we need an address for an element in our HTML to render our Hello component. The proper terminology is to call say we need to specify an element in our HTML where we can *mount* our component.

This is why we add this last line of code:

```
1 ReactDOM.render(  
2   <Hello/>,  
3   document.getElementById( 'root' )  
4 );
```

Here, we are saying that we want to mount our component where we have an element in our HTML with an id of root.

Time for the final piece!

Let's add the following in our HTML column:

```
<div id="root"></div>
```

We now see the following:

Hello World

Cool beans! We have created and rendered our very first React component!

Because this is a very basic component, we can say that we have just built our very first house.

Final Code:

[Hello World Pen³](https://codepen.io/mikemang/pen/118c21ac4d801aed42199b552e35023f)

³<https://codepen.io/mikemang/pen/118c21ac4d801aed42199b552e35023f>

A More Visual Component



Now that we have built our first house, let's begin a more visual example.

Eventually, we will be creating this Koala graphic:



If we described this Koala in terms of *The React Village Design* detailed in Chapter 2, then any of these shapes would be the houses.

Since this chapter is all about building houses, we are going to create a simple component called *Head* that will render as the head of the koala.

Grasping Pure CSS Images

The koala which we will create using React is going to be a *pure CSS image*.

Pure CSS images are a topic that I have discussed extensively.

Therefore, I am just going to punt to [an existing video](#)⁴ I have which explains pure CSS images. It also walks you through the process of making the koala graphic which we will eventually be making using React components.

[Here is the article that accompanies the video.](#)⁵

Once you complete this video, we can continue on making the head of our pure CSS koala using a React component.

⁴https://youtu.be/ULhv96NdF_Q

⁵<https://medium.com/coding-artist/a-beginners-guide-to-pure-css-images-ef9a5d069dd2>

Creating a Head Component

Go to your current pen and click **Fork**

This is going to create a new pen with the same code and settings

Rename it to React for Visual Learners

2

Replace the JS code with the following code shown below:

```
1 class Head extends React.Component {  
2   render() {  
3     return <div className="head"></div>;  
4   }  
5 }  
6 ReactDOM.render(  
7   <Head/>,  
8   document.getElementById( 'head-target' )  
9 );
```

Update the HTML column to the following:

```
1 <div class="box">  
2   <div id="head-target"></div>  
3 </div>
```

Add the following to the CSS column:

```
1 body { background: #25A9FC }  
2 .box{  
3   position: relative;  
4   margin: auto;  
5   display: block;  
6   margin-top: 8%;  
7   width: 600px;  
8   height: 420px;  
9   background: none;  
10 }
```

```
11 .head{
12   position: absolute;
13   top: 16.5%;
14   left: 25%;
15   width: 50%;
16   height: 67%;
17   background: #A6BECF;
18   border-radius: 50%;
19 }
```



This has already been a lengthy chapter so let's do a short summary of what we just did. First off, we added our invisible box class straight into our HTML:

```
1 <div class="box">
2   <div id="head-target"></div>
3 </div>
```

We also added the element where our Head component will *mount*.

In our JS, we have the following:

```
1 class Head extends React.Component {
2   render() {
3     return <div className="head"></div>;
4   }
5 }
6 ReactDOM.render(
7   <Head/>,
8   document.getElementById('head-target')
9 );
```

The changes for this Head component are found in bold. This should be straight forward.

The only thing to note is that the JSX (what's returned in our render function) has *className* instead of class.

This is something specific to JSX. You always replace class with className. I don't want to go into the technical details explaining why but just remember going forward.

Just like that, we have created a head for our koala using React.

We still have a lot more to add to our koala and features of React components to showcase. We will continue this in the next chapter.

Final code:

[React for Visual Learners | 2⁶](#)

Glossary

Codepen: CodePen is a playground for the front end side of the web. We can use it to write React code without dealing with the “behind the scenes” work.

DOM: The Document Object Model (DOM) is a programming interface for HTML and XML documents. It provides a structured representation of the document and it defines a way that the structure can be accessed from programs so that they can change the document structure, style and content. The DOM provides a representation of the document as a structured group of nodes and objects that have properties and methods. Essentially, it connects web pages to scripts or programming languages.

⁶<https://codepen.io/mikemang/pen/bcd77fc27e8fe3ec249ebb69e198004e>

React: A JavaScript library for building user interfaces.

React Village Design: Terminology to describe the hierarchy of React components.

ECMAScript: ECMAScript is a language standardized by ECMA International and overseen by the TC39 committee that is implemented in JavaScript.

ES5: The 5th edition of ECMAScript, standardized in 2009. This standard has been implemented fairly completely in all modern browsers.

ECMAScript 6 (ES6)/ ECMAScript 2015 (ES2015): The 6th edition of ECMAScript, standardized in 2015. This standard has been partially implemented in most modern browsers.

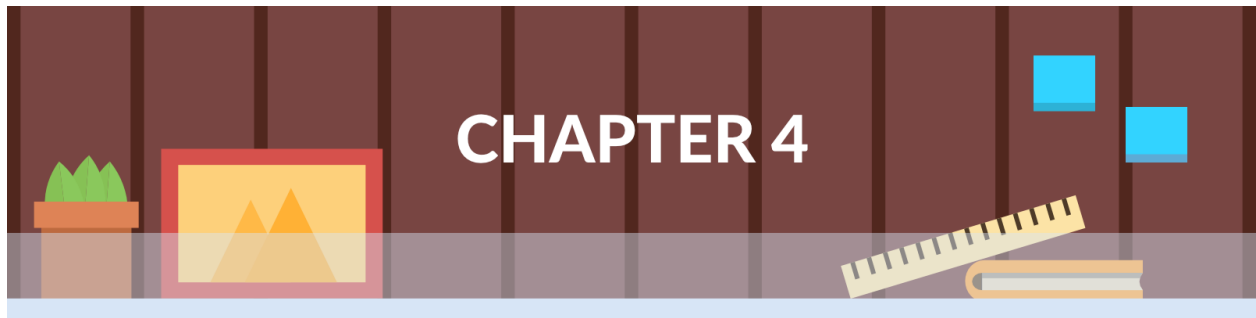
Babel: Compiles ES6 code to ES5 so it can be processed by browsers. It can be selected as a JavaScript preprocessor in the JavaScript settings in Codepen.

JSX: A JavaScript syntax extension used to specify what we want our component to render as.

React Core: The entire React library is divided into two separate libraries, React and ReactDOM. I find this to be poor naming convention so I refer to the separate core React library (as distinguished from ReactDOM) as React Core.

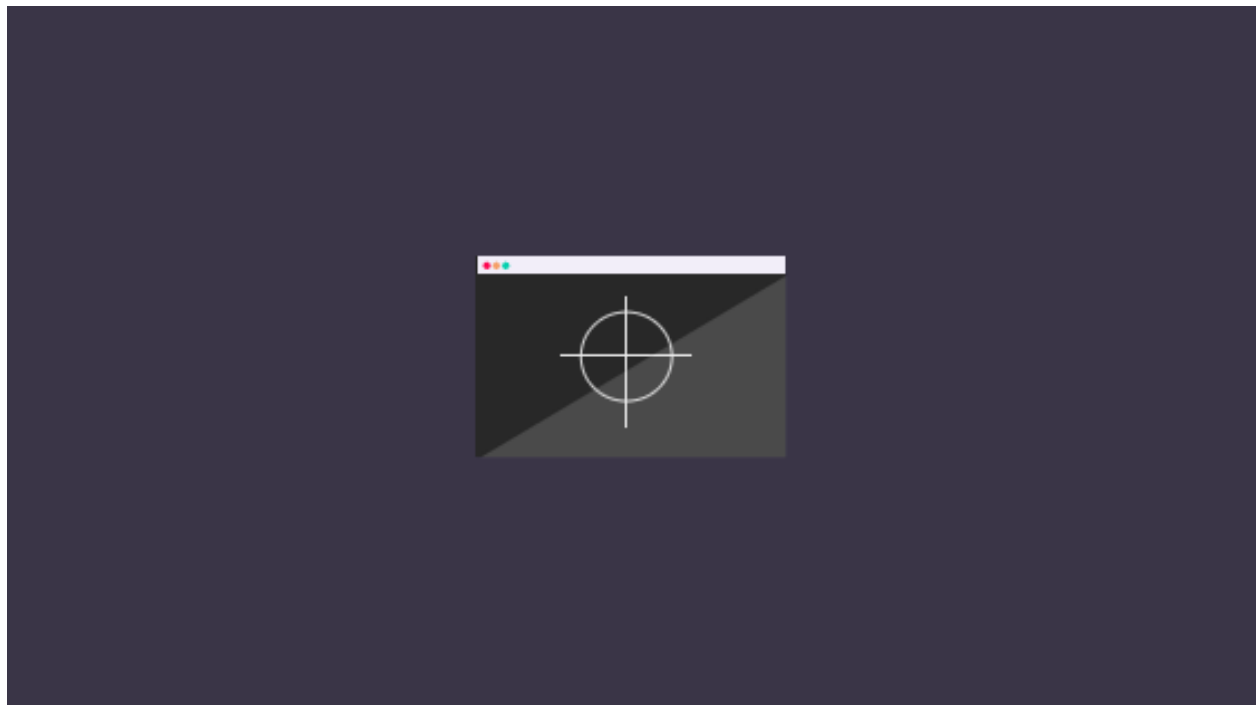
ReactDOM: The entire React library is divided into two separate libraries, React and ReactDOM. ReactDOM is the glue between React components and the DOM. It allows you to mount React components to HTML elements.

Chapter 4 | Our First Village



Before we begin, make sure you are using Chrome or Firefox Developer.

Scope of This Chapter



Alright! In the previous chapter, we were able to create two simple components using React.js. First, we did a simple Hello React.js app. Following that, we were able to create a head component for our koala. In between, we learned about pure CSS images and created a koala together without React.

This might not seem like significant progress for some people, but given how overwhelming React can be, this deserves a pat on the back (plus learning pure CSS images is a cool bonus).

In this chapter, we are going to finish our koala pure CSS image using basic React components.

...

As you noticed when we made our head component, we just specified a simple div tag via a React class for ReactDOM to render to the DOM. The forming of the shape was handled with purely CSS.

[Look at it again just to get a refresher.](#)⁷

Constructing the components for the rest of the shapes for our koala will be just like our head. The only new concept that we have to tackle is nesting components in a hierarchy by applying the concepts of The React Village Design discussed in Chapter 2 with React.

The scope of this chapter, therefore, is to take houses (the basic building blocks of our koala) and create a village (the completed koala).

Breaking Down the Koala Using The React Village Design



⁷<https://codepen.io/mikemang/pen/bcd77fc27e8fe3ec249ebb69e198004e>

Houses

Houses are the basic building blocks of a user interface. In terms of a user interface, an example of a house is a button.

In terms of our koala, the houses are the basic building blocks of the graphic, which are the components that only return one shape.

Here is a complete list of the houses for our koala graphic:

- Head
- Nose
- Hair Left
- Hair Right
- Outer Ear Left
- Outer Ear Right
- Inner Ear Left
- Inner Ear Right
- Outer Eye Left
- Outer Eye Right
- Pupil Left
- Pupil Right

Cool. We have defined the first level of our component hierarchy.

Blocks

Next, we go up in the component hierarchy in terms of the village design, blocks.

Blocks in a user interface are collections of houses that work together to form a specific functionality/purpose to the view. An example is a navigation menu.

With our koala, the blocks are going to be the body parts that have multiple shapes such as:

- Eye Left
- Eye Right
- Ear Left
- Ear Right

Neighborhoods

The next level up on the component hierarchy of the village design is Neighborhoods.

Neighborhoods in a user interface are collections of blocks that work together to form a complete section of the view. An example is a header.

With our koala, the neighborhood is going to be the invisible box which is the container for the pure CSS image:

Box

Village

A village is the final user interface (synonymous with a view of a single page application).

The complete graphic of the koala is the village in our current example. The village will also be its own component where all other components are nested.

Given how we have broken things down, our koala is going to be made up of the following React components (indentation notes hierarchy):

```
1  //React components to make our koala
2  - Koala (village)
3    - Box (neighborhood)
4      - Head (house)
5      - Nose (house)
6      - Hair Left (house)
7      - Hair Right (house)
8      - Eye Left (block)
9        - Outer Eye Left (house)
10       - Pupil Left (house)
11     - Eye Right (block)
12       - Outer Eye Right (house)
13     - Pupil Right (house)
14     - Ear Left (block)
15       - Outer Ear Left (house)
16       - Inner Ear Left (house)
17     - Ear Right (block)
18       - Outer Ear Right (house)
19       - Inner Ear Right (house)
```

Quick Clarification

One thing I do want to clarify is that The React Village Design is more suiting for a user interface. For that reason, the application of the village design to our koala graphic is not meant to be a *perfect* comparison.

However, I still think there are benefits of starting with the creation of our koala before diving into a user interface:

It gets us familiar with the concept of component hierarchy in a more underwhelming fashion.

It doesn't require data, event handlers, etc.

Setting Up Our Village



Codepen Setup

Fork your previous pen ([React for Visual Learners | 2⁸](https://codepen.io/mikemang/pen/bcd77fc27e8fe3ec249ebb69e198004e)).

Rename it React for Visual Learners

Koala.

Replace all your CSS with the following:

⁸<http://codepen.io/mikemang/pen/bcd77fc27e8fe3ec249ebb69e198004e>

[Koala CSS via Github](#)⁹

Rendering One Component to the DOM

Now that we have broken down our koala in terms of components in the village design, it's time to start implementation.

Even though we break things down from house all the way up to village, we will construct things in the opposite direction.

Therefore, we will begin with setting up our village.

...

In our previous pen, we created a component called *Head* and rendered it via ReactDOM like so:

```
1 ReactDOM.render(  
2   <Head/>,  
3   document.getElementById('head-target')  
4 );
```

This won't work. We still want to use ReactDOM to render just one component. We want to render a Koala component to an element in our HTML called koala.

Replace your JS with the following:

```
1 //village  
2 class Koala extends React.Component {  
3   render() {  
4     return (  
5     )  
6   }  
7 }  
8  
9 ReactDOM.render(  
10  <Koala/>,  
11  document.getElementById('koala')  
12 );
```

Replace your HTML with the following:

```
<div id="koala"></div>
```

With these changes, we are now going to render a component called Koala to the DOM.

Instead of rendering multiple components like so:

⁹<https://gist.github.com/michaelmang/7f5f706ec05966ee75de36ee69968353>

```
1 ReactDOM.render(  
2   <Village/>,  
3   <Neighborhood/>  
4 //and so on  
5 document.getElementById('village')  
6 );
```

We have to render all the components in their hierarchical order through *nesting*.

For example...

```
1 class Village extends React.Component {  
2   render() {  
3     return (  
4  
5       <Neighborhood />  
6     )  
7   }  
8 }  
9 class Neighborhood extends React.Component {  
10   render() {  
11     return (  
12  
13       <Blocks/>  
14     )  
15   }  
16 }  
17 class Blocks extends React.Component {  
18   render() {  
19     return (  
20  
21       <HouseOne/>  
22       <HouseTwo/>  
23     )  
24   }  
25 }
```

We will render the village which nests the neighborhood. The neighborhood will render the blocks which nests houses.



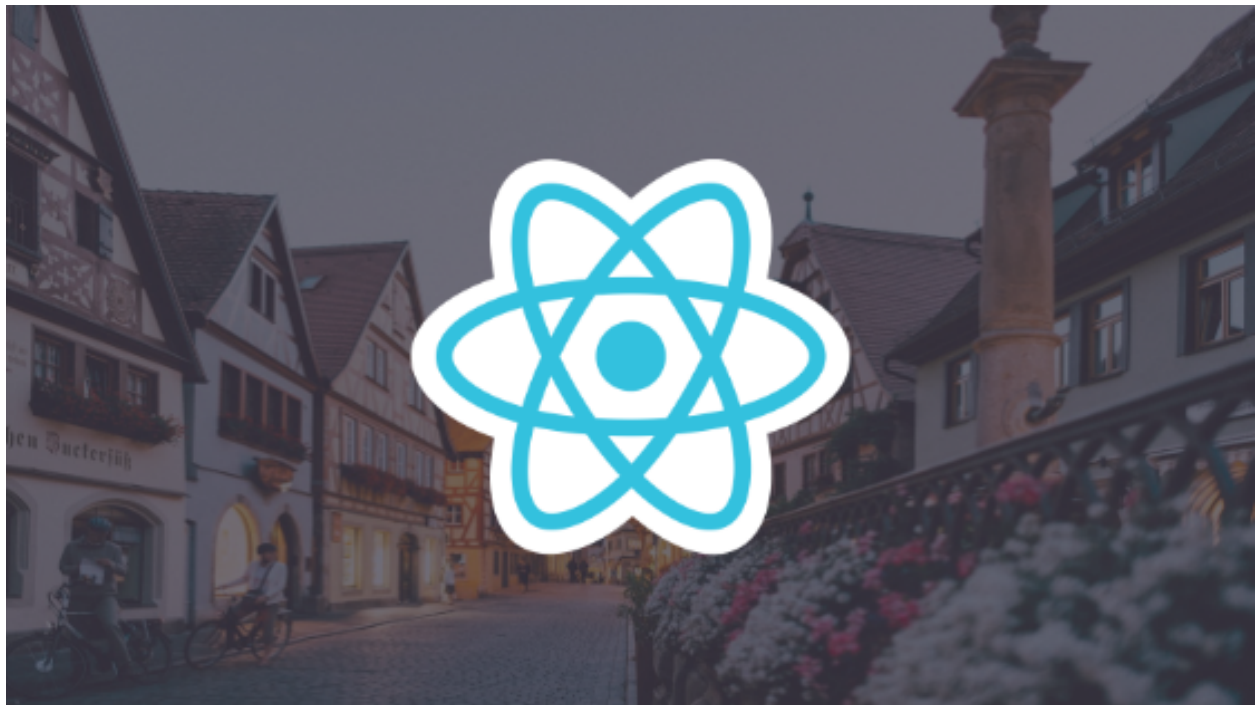
Again, this is why React gets confusing for beginners. It becomes component inception.

Note:

We can use `render ( ... )` when rendering multiple components like so:

```
1  class Blocks extends React.Component {  
2    render() {  
3      return (  
4  
5        <HouseOne/>  
6        <HouseTwo/>  
7      )  
8    }  
9  }
```


Creating Our Neighborhood



Recall, our neighborhood is going to be the box which is the container for the pure CSS image.

Box Component

We will now create the Box component which we will nest underneath the Koala component we just created.

First, we can create the Box component:

```
1 //neighborhood
2 class Box extends React.Component {
3   render() {
4     return (
5       <div className="box">
6         </div>
7     )
8   }
9 }
```

This is going to return a simple div element with a class named box.

Nesting Underneath Koala

Next, we nest it underneath our Koala component like so:

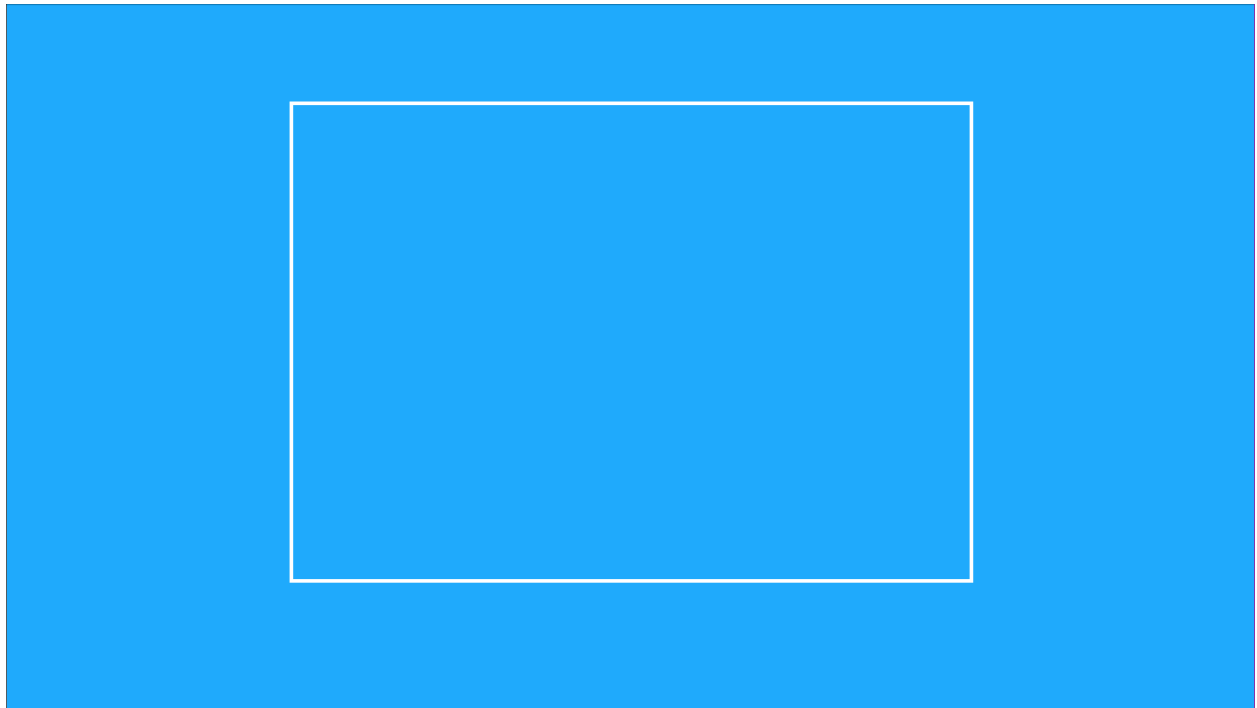
```
1 //village
2 class Koala extends React.Component {
3   render() {
4     return (
5       <Box/>
6     )
7   }
8 }
```

Putting It All Together

In total, we should have the following in our JS:

```
1 //village
2 class Koala extends React.Component {
3   render() {
4     return (
5       <Box/>
6     )
7   }
8 }
9 //neighborhood
10 class Box extends React.Component {
11   render() {
12     return (
13       <div className="box">
14         </div>
15     )
16   }
17 }
18 ReactDOM.render(
19   <Koala/>,
20   document.getElementById('koala')
21 );
```

Because of how I styled the box class, we should see the following:



Defining Our Blocks



Working our way down the village design hierarchy, it's time to implement the blocks.

The blocks are going to be the body parts that have multiple shapes such as:

Eye Left
Eye Right
Ear Left
Ear Right

These blocks will be nested within our *Box* component which is nested underneath our *Koala* component, I know...



Am I right?

Block Components

First, let's create the shell of the blocks:

```
1  //blocks
2  class EarLeft extends React.Component {
3    render() {
4      return (
5      )
6    }
7  }
8  class EarRight extends React.Component {
9    render() {
10     return (
11     )
12   }
```

```
13 }
14 class EyeLeft extends React.Component {
15   render() {
16     return (
17
18     )
19   }
20 }
21 class EyeRight extends React.Component {
22   render() {
23     return (
24
25     )
26 }
```

These are *shells* because they don't have anything in the returns. In the next section of this chapter, we are going to create the houses which will eventually be nested here.

Nesting Underneath Box

Next, we can nest these blocks under our neighborhood component, Box:

```
1 //neighborhood
2 class Box extends React.Component {
3   render() {
4     return (
5       <div className="box">
6         <EarLeft/>
7         <EarRight/>
8         <EyeLeft/>
9         <EyeRight/>
10      </div>
11    )
12  }
13 }
```

Putting It All Together

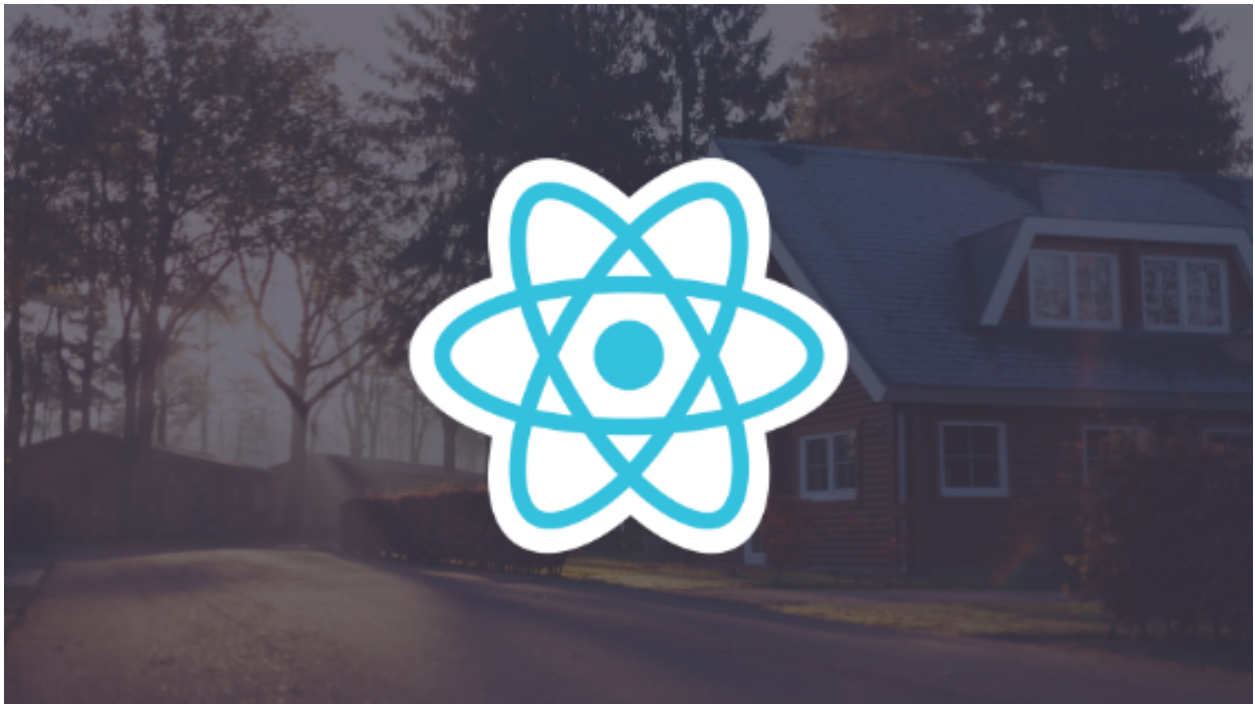
In total, we should now have the following in our JS:

```
1  //village
2  class Koala extends React.Component {
3    render() {
4      return (
5        <Box/>
6      )
7    }
8  }
9  //neighborhood
10 class Box extends React.Component {
11   render() {
12     return (
13       <div className="box">
14         <EarLeft/>
15         <EarRight/>
16         <EyeLeft/>
17         <EyeRight/>
18       </div>
19     )
20   }
21 }
22 //blocks
23 class EarLeft extends React.Component {
24   render() {
25     return (
26     )
27   }
28 }
29 class EarRight extends React.Component {
30   render() {
31     return (
32     )
33   }
34 }
35 class EyeLeft extends React.Component {
36   render() {
37     return (
38
39     )
40   }
41 }
42 class EyeRight extends React.Component {
```

```
43   render() {  
44     return (  
45     )  
46   }  
47 }  
48 ReactDOM.render(  
49   <Koala/>,  
50   document.getElementById('koala')  
51 );
```

Nothing should have changed except an error for the fact that there is nothing being returned. Don't worry. We will fix that next.

Constructing Our Houses



Stand Alone vs. Block Houses

Alright. It's time to knock out all the components that make up our houses. Some of these houses will stand alone and be nested directly under the Box component. Others will be nested within the blocks we got started with.

Here are the stand alone houses:

Head
Nose
Hair Left
Hair Right

Again, these will be directly nested underneath the Box component.

Here are the houses that will be nested underneath the blocks we set up in the previous step:

Outer Ear Left
Outer Ear Right
Inner Ear Left
Inner Ear Right
Outer Eye Left
Outer Eye Right
Pupil Left
Pupil Right

We will do one example of a stand alone house and one example of creating the houses to complete a block.

Once we do that, I will provide the rest of the code all at once.

Head Component

We have done this before, so it should be pretty straight forward.

First, we define the component:

```
1 class Head extends React.Component {  
2   render() {  
3     return (  
4       <div className="head"></div>  
5     )  
6   }  
7 }
```

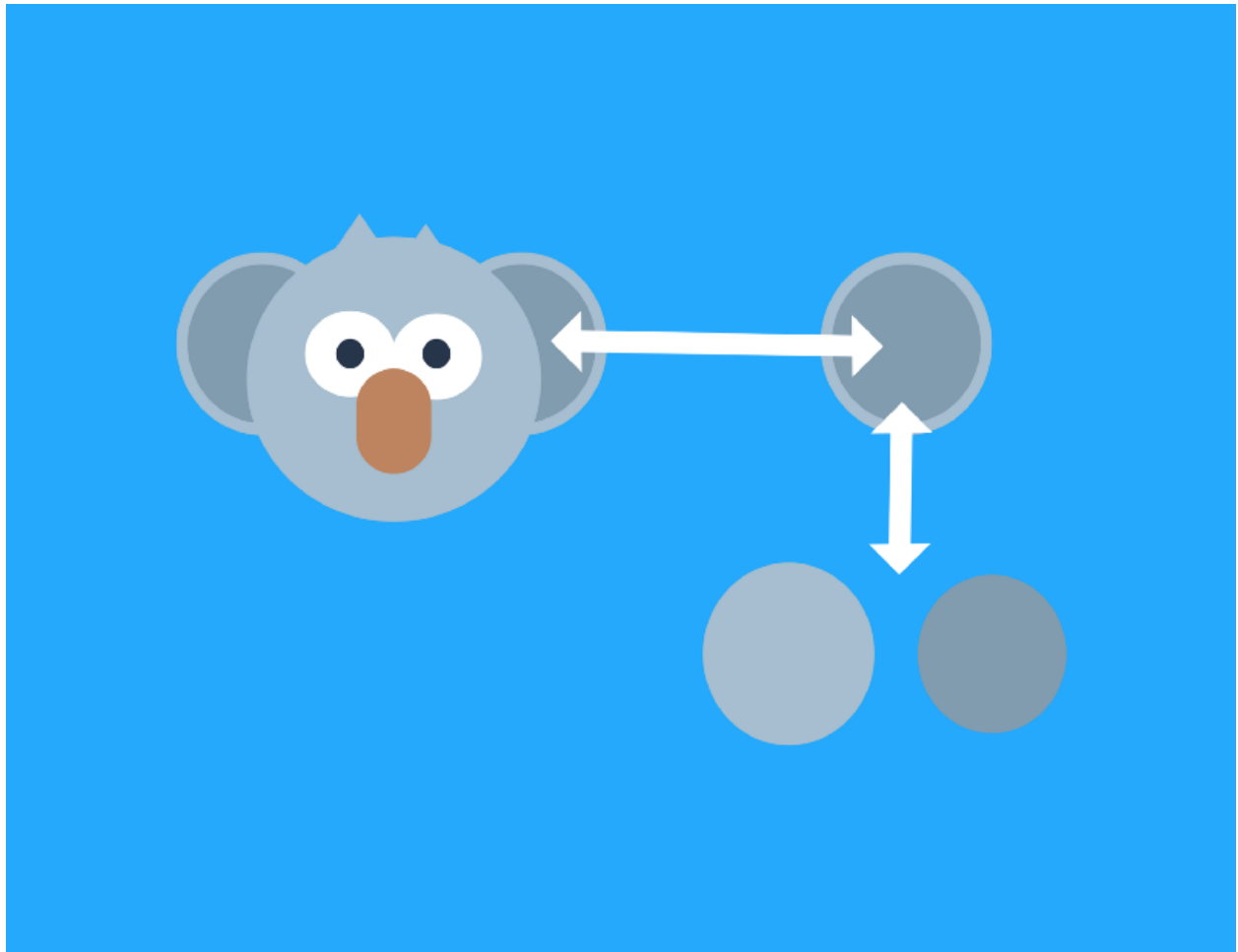
Then, we will nest this underneath the Box component:


```
1  //neighborhood
2  class Box extends React.Component {
3    render() {
4      return (
5        <div className="box">
6          <EarLeft/>
7          <EarRight/>
8          <EyeLeft/>
9          <EyeRight/>
10
11         <Head/>
12       </div>
13     )
14   }
15 }
```

Ear Right Component

That was pretty easy, hopefully. Now, let's go ahead and create some houses for Ear Right and nest them. That way, we will have accomplished an entire block.

We have to first make two houses, Outer Ear Right and Inner Ear Right since there are two shapes for the ear on each side.



Again, we start by defining the components:

```
1  //houses
2  class Head extends React.Component {
3    render() {
4      return (
5        <div className="head"></div>
6      )
7    }
8  }
9  class OuterEarRight extends React.Component {
10   render() {
11     return (
12       <div className="outer-ear-right"></div>
13     )
14   }
15 }
```

```

16 class InnerEarRight extends React.Component {
17   render() {
18     return (
19       <div className="inner-ear-right"></div>
20     )
21   }
22 }

```

Then, we have to nest these components underneath the Ear Right component (block):

```

1 class EarRight extends React.Component {
2   render() {
3     return (
4       <div>
5         <OuterEarRight/>
6         <InnerEarRight/>
7       </div>
8     )
9   }
10 }

```

Note: There is a div tag surrounding the nested components. This is required.

Checking Our Progress

Before I provide the rest of the code, let's make sure this is working correctly.

The empty blocks are going to cause an error, so let's comment them out for now:

```

1  //blocks
2  /*
3  class EarLeft extends React.Component {
4    render() {
5      return (
6    )
7    }
8  }
9  */
10 class EarRight extends React.Component {
11   render() {
12     return (
13       <OuterEarRight/>,

```

```

14     <InnerEarRight/>
15   )
16 }
17 }
18 /*
19 class EyeLeft extends React.Component {
20   render() {
21     return (
22
23     )
24   }
25 }
26 class EyeRight extends React.Component {
27   render() {
28     return (
29
30     )
31   }
32 }
33 */

```

Let's also update our neighborhood so that it only has Ear Right and Head as nested components:

```

1  //neighborhood
2  class Box extends React.Component {
3    render() {
4      return (
5        <div className="box">
6          <EarRight/>
7          <Head/>
8        </div>
9      )
10   }
11 }

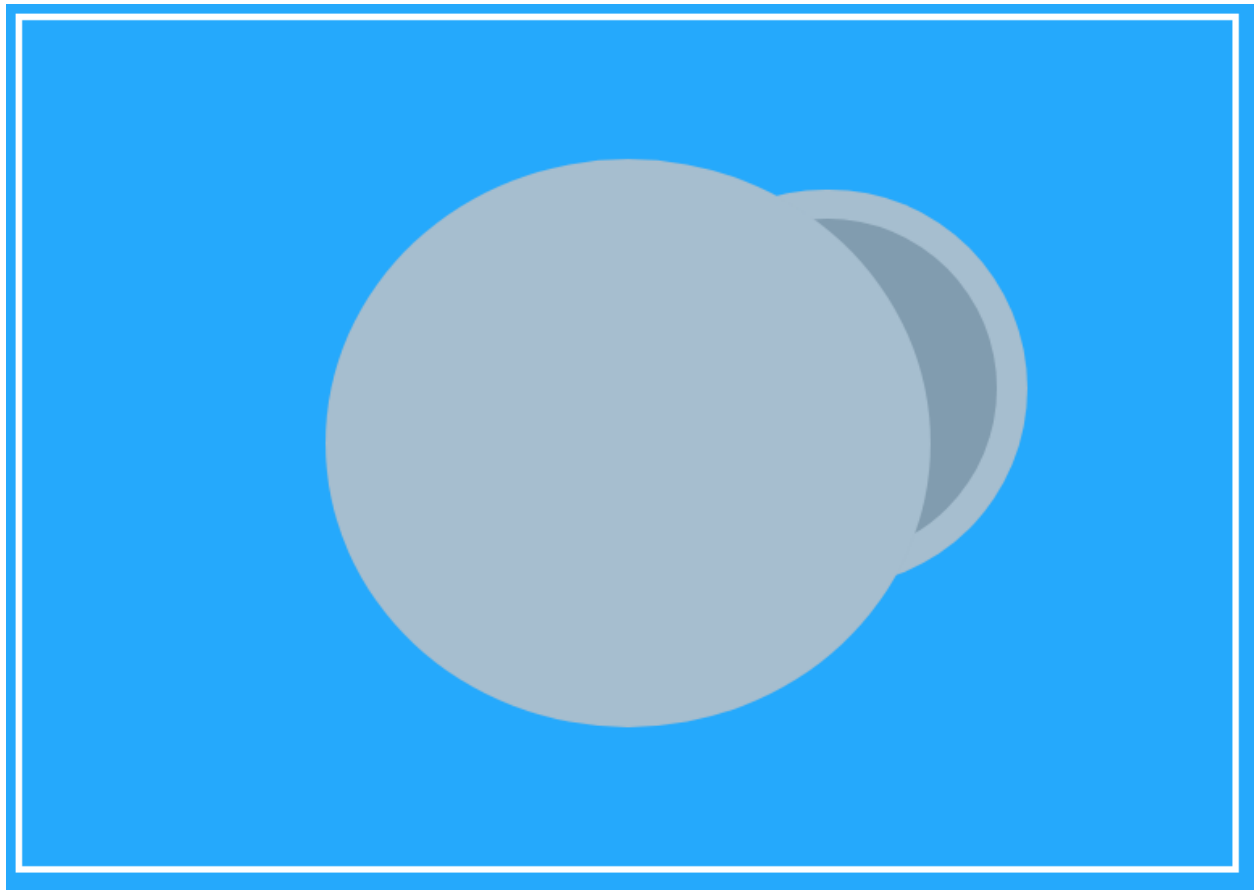
```

In total, we should have the following in our JS:

```
1  //village
2  class Koala extends React.Component {
3    render() {
4      return (
5        <Box/>
6      )
7    }
8  }
9  //neighborhood
10 class Box extends React.Component {
11   render() {
12     return (
13       <div className="box">
14         <EarRight/>
15         <Head/>
16       </div>
17     )
18   }
19 }
20 //blocks
21 /*
22 class EarLeft extends React.Component {
23   render() {
24     return (
25   )
26   }
27 }
28 */
29 class EarRight extends React.Component {
30   render() {
31     return (
32       <div>
33         <OuterEarRight/>
34         <InnerEarRight/>
35       </div>
36     )
37   }
38 }
39 /*
40 class EyeLeft extends React.Component {
41   render() {
42     return (
```

```
43
44     )
45   }
46 }
47 class EyeRight extends React.Component {
48   render() {
49     return (
50
51     )
52   }
53 }
54 */
55 //houses
56 class Head extends React.Component {
57   render() {
58     return (
59       <div className="head"></div>
60     )
61   }
62 }
63 class OuterEarRight extends React.Component {
64   render() {
65     return (
66       <div className="outer-ear-right"></div>
67     )
68   }
69 }
70 class InnerEarRight extends React.Component {
71   render() {
72     return (
73       <div className="inner-ear-right"></div>
74     )
75   }
76 }
77 ReactDOM.render(
78   <Koala/>,
79   document.getElementById('koala')
80 );
```

We should now see the following:



Awesome, we can see 3 houses: Outer Ear Right, Inner Ear Right, and Head.

However, we have set up a hierarchy of components so that Outer Ear Right and Inner Ear Right are combined into a block component called Ear Right.

Ear Right (block) and Head (house) are nested directly under our neighborhood component called Box.

Box (neighborhood) is nested under our village component called Koala which is injected (so to speak) into the DOM using ReactDOM:

```
1 ReactDOM.render(  
2   <Koala/>,  
3   document.getElementById('koala')  
4 );
```

By doing all of this, we can get a taste of how hierarchy of components in Reacts work. I'll provide a more in-depth conclusion in just a moment, but for now, take a look at the final code that fills in the koala with the rest of the components:

React for Visual Learners: Koala¹⁰

Concluding Thoughts

As I mentioned before, The React Village Design is well suited for explaining the hierarchy of user interfaces. While the koala pure CSS image doesn't fit perfectly with the village design, it has provided for us an underwhelming example of how to nest components in a hierarchy using React.

Not only were we able to learn how to implement nested components in a hierarchy, we also learned the order of how to approach it.

We first started by breaking down an existing koala graphic into our component hierarchy starting with houses and working our way up. When actually creating the classes, we went from a village and worked our way down. While the order of things may be more of a gray area, I still think our example has practical, real-world applications.

In the real-world, you might be given a mock-up of a user interface. You will have to break that user interface down into components and construct those components in a hierarchy using React. This is where The React Village Design is really beneficial and practical. We will get to the actual creation of a user interface in a few chapters, but again, what we have done is get a taste of that in a more underwhelming fashion.

When I was learning React, the hierarchy of components was really confusing. It was also taught to me after going through all the ins and outs of other features of components like managing the state (data), event handlers, conditional rendering, etc. In the koala example, we have been able to explain the big picture of understanding React components before getting lost in the features of components.

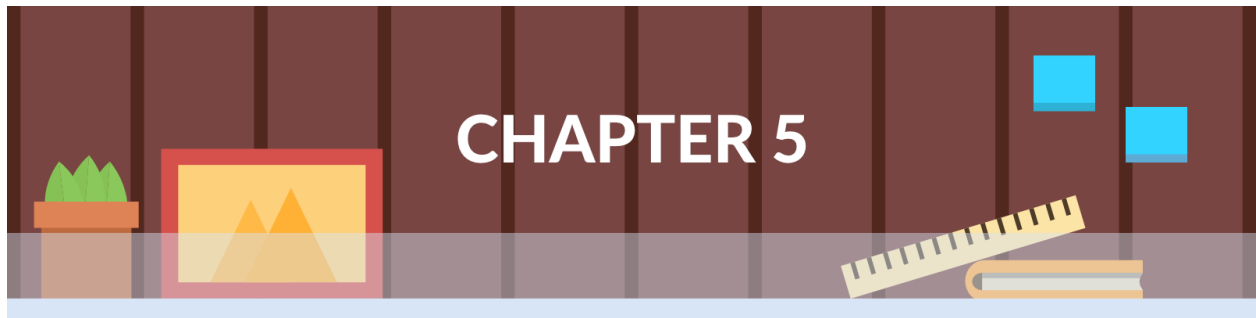
One last thing that I wanted to mention is that I understand that having all the components in one JavaScript column in Codepen might seem a bit jumbled. However, I still think this is the right move to start here.

We can organize things a lot better as we will see when we setup our project directory and do development locally. However, we didn't have to get "lost in the sauce" (as a friend of mine would say) about setup because we used Codepen. Having all the components in one column also helps see how they are related, something that is often hard to grasp when you just start with an existing React project template.

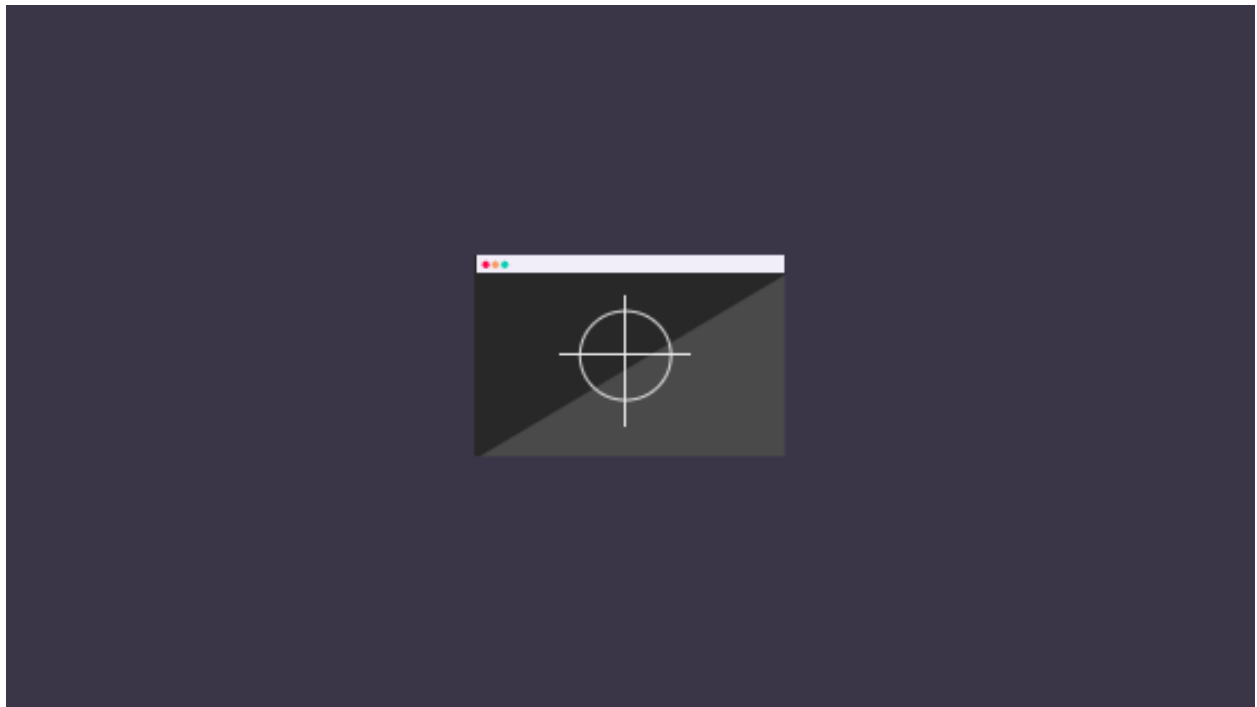
We will be eventually switching to local development and dedicate an entire chapter just on the setup. Before that, however, we are going to use our React-fueled koala for visual learning of the ins and outs of React components.

¹⁰<https://codepen.io/mikemang/pen/58f339afac397fbb2f2ae678d6334030>

Chapter 5 | Additions to Our Houses



Scope of This Chapter



Cool beans! In the previous chapter, we were able to create a pure CSS koala image using React components. Moreover, we constructed these components in a hierarchy which have frequently referenced in this book as The React Village Design (village design for short).

Now, we should have a taste at the big picture of how component hierarchy works in React.

With this as the foundation, we are now ready to dive deeper into React components.

...

Up to this point, we have just been creating simple React components that just have JSX, the JavaScript syntax extension used to specify what we want our component to render as. In other words, we have just been creating React components that ultimately render as HTML.

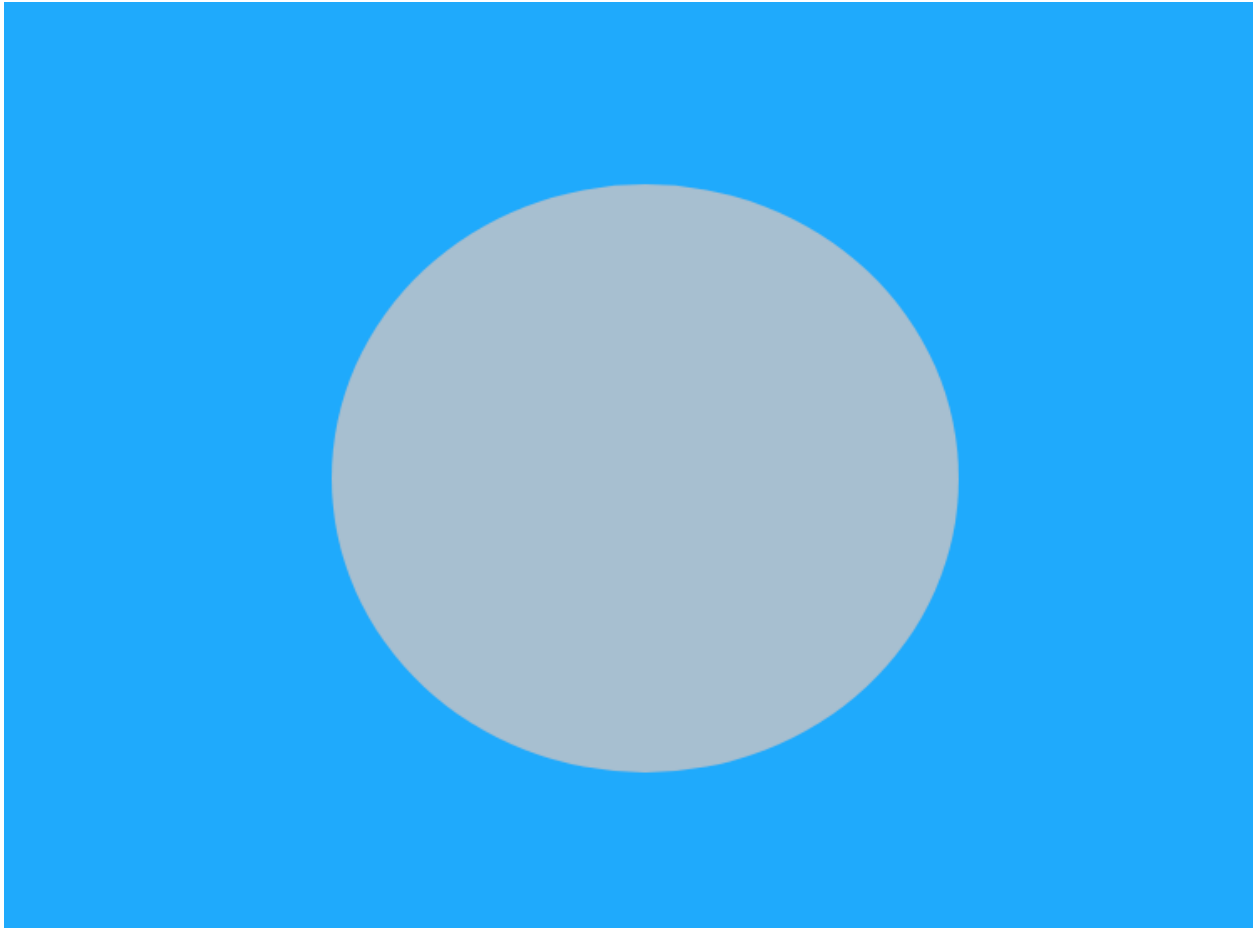
Here's an example:

```
1 class Head extends React.Component {  
2   render() {  
3     return (  
4       <div className="head"></div>  
5     )  
6   }  
7 }
```

In the code above, the Head component is ultimately render as:

```
<div class="head"></div>
```

By adding CSS magic, we can create a shape.



In a sense, we have *encapsulated* HTML into a React component.

While this is certainly cool, there's a lot more awesome stuff that we can encapsulate into a component.

In the broadest sense, we can also encapsulate the following into React components:

- Data
- Methods
- Lifecycles

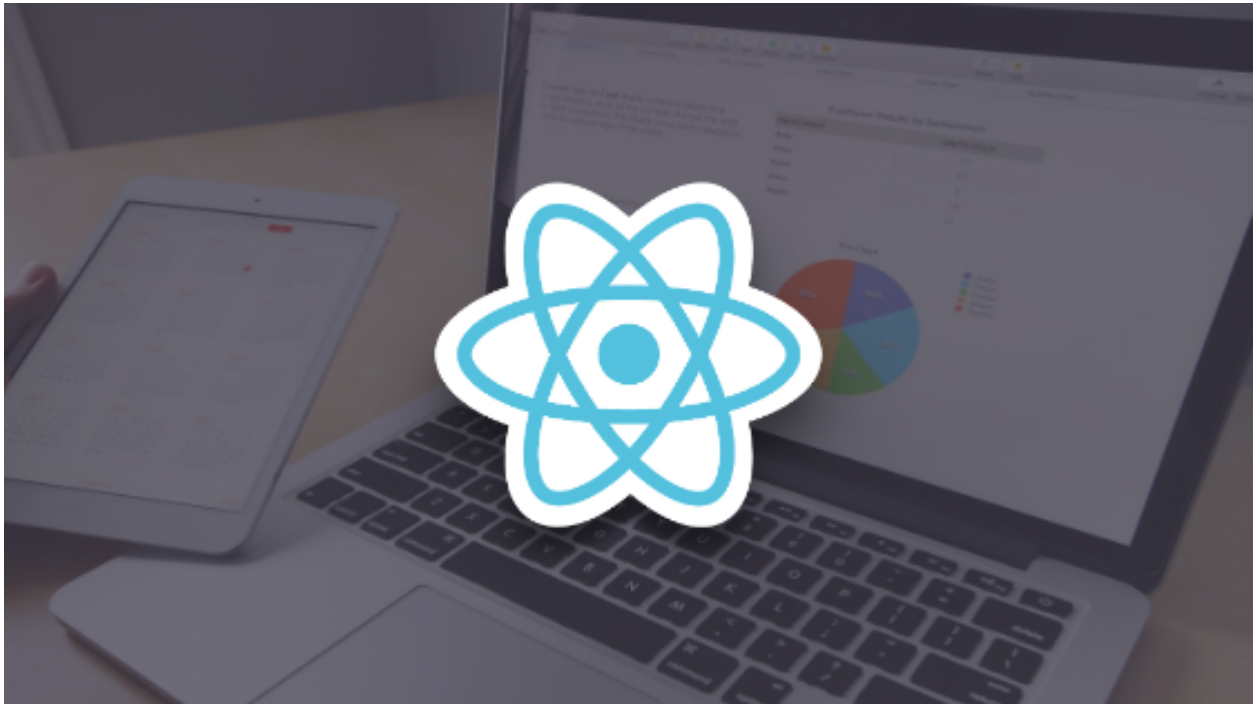
The scope of this chapter is to cover those additional things which we can encapsulate in a component.

To be more precise, we will be covering the following:

- Handling *props*
- Managing *state*
- Specifying *event handlers*

Using inline *JSX styling*
Implementing *conditional rendering* of components
Setting up *lifecycle hooks*

Data



Before we distinguish between ways to encapsulate data into a component, let's do some explaining of what data is.

Data Types

In all programming languages, there are *data types*. In ES6 (the 6th edition of the standardized language used in JavaScript), there are seven JavaScript data types.

The seven data types are:

- Boolean
- Null
- Undefined
- Number
- String
- Symbol (new in ES6)
- Object

Mozilla Developer Network has a good explanation for all of these [here](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Data_structures)¹¹. I am only going to be focusing on Boolean, Number, String, and Object.

Boolean simply represents true or false. Depending if something is true or false, we can do something for that condition.

Usage in Plain English: “If sale is true, then buy. Else if sale is false, don’t buy.”

Usage in JavaScript:

```
1 //let is same as var for ES6
2 let flag = true
3 if(flag === false) {
4   //do something here
5 }
```

Without getting technical, a number is pretty straightforward. Unlike other programming languages, there aren’t distinguished into separate types (i.e. double and int).

```
1 let whole = 2
2 let decimal = 2.2
```

String refers to textual data.

```
1 let string = "Hey there! I am a string!"
```

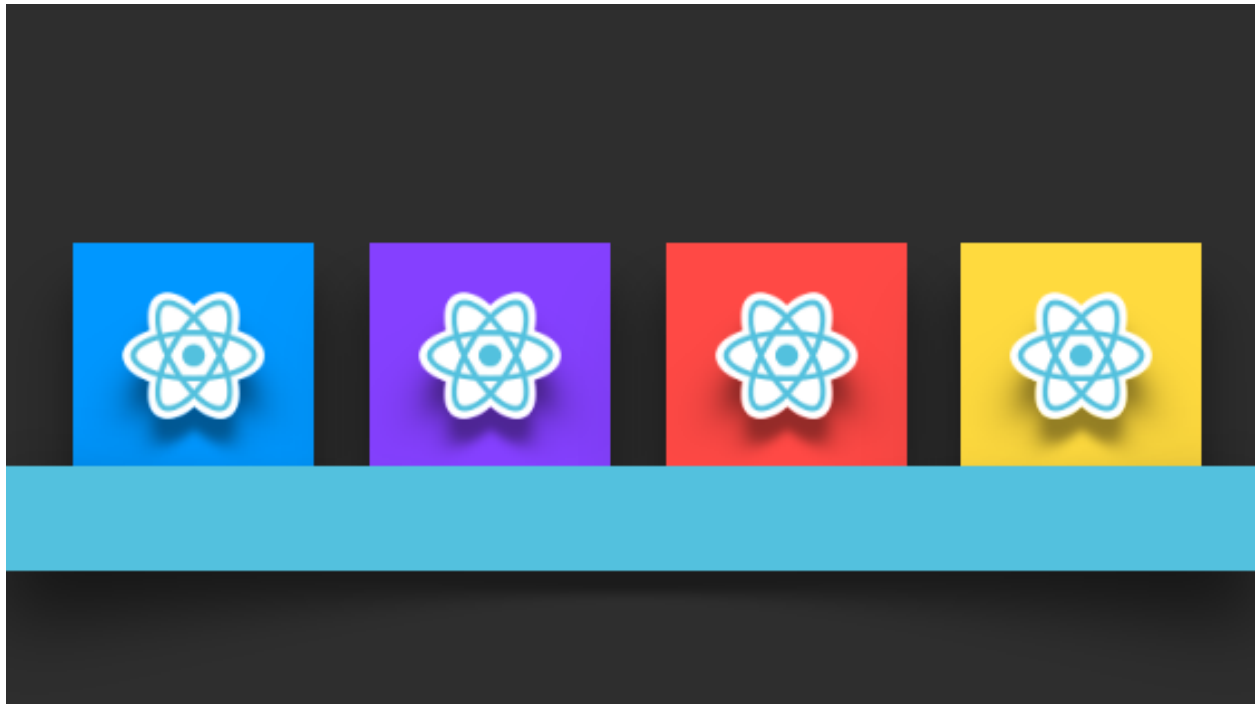
Object refers to a collection of properties.

```
1 let myHonda = {color: 'red', wheels: 4}
2 //myHonda.color will equal 'red'
```

These data types can be encapsulated into React components. They can be encapsulated in two different ways, props and state.

¹¹https://developer.mozilla.org/en-US/docs/Web/JavaScript/Data_structures

Props



Let's create an example of a component that renders two buttons and has *props* that specify the text for each button. As we create this example, we will explain how props work.

To start, go ahead and [fork the pen from the previous chapter](#)¹².

Go ahead and remove all the HTML, CSS, and JS so that we have an empty project.

Name it **React for Visual Learners: Props**.

First, let's add the following HTML that will be the target for our component:

```
<div id="buttons"></div>
```

Next, let's define a new React component called Buttons with some JSX that will render two buttons with breaks in between:

¹²<http://codepen.io/mikemang/pen/58f339afac397fbb2f2ae678d6334030>

```
1 class Buttons extends React.Component {
2   render() {
3     return (
4       <div>
5         <button></button>
6         <br/><br/>
7         <button></button>
8       </div>
9     )
10  }
11 }
```

Now, we can add the ReactDOM magic to inject our Buttons component into the HTML target:

```
1 // "inject" to DOM
2 ReactDOM.render(
3   <Buttons/>,
4   document.getElementById('buttons')
5 );
```

So far, this is nothing new to what we have done in previous chapters. We have a component that is just ultimately rendering HTML.

We are going to want to add some data in our example to specify the text for each button. This will be data of type *string*.

As previously mentioned, we can use *props* to do this.

It's tempting to want to move on and complete this example since it is so trivial. However, we have to question that isn't as obvious as it is often portrayed.

What in the world is props?

Props is short for *properties*.

If I asked you to look in the mirror and list some of your properties, one of them could be that you have brown hair.

Props is used to encapsulate properties of a certain data type.

If we had a component called *Person*, we could encapsulate a property called *hair* with a string value of *brown*.

So, how and where do we define props? We can define props like so:

```
<Person hair={"brown"}/>
```

In the code above, the hair prop is specified and assigned a value of brown within the component.

We assign a value just like any variable except we always wrap it around {...}.

This component could be found nested within another component within the hierarchy:

```
1 class People extends React.Component {
2   render() {
3     return (
4       <div>
5         <Person hair="brown" />
6       </div>
7     )
8   }
9 }
```

Or, it could be found in ReactDOM.render like so:

```
1 ReactDOM.render(
2   <Person hair="brown" />,
3   document.getElementById( 'person' )
4 );
```

In short, anywhere the component is used, *props* could be specified.

If we wanted to do more props, we could so like this:

```
<Person name={"Mike"} hair={"brown"} age={17}/>
```

Again, props are forms of data. Therefore, we can use any of the data types we previously listed. In this example, we have string, string, and int (from left to right).

In order to use (or inject) our props in a component, we do the following:

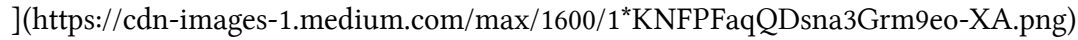
```
1 class Person extends React.Component {
2   render() {
3     return (
4       <div>
5         <h1>Person Info</h1>
6         <ul>
7           <li>{this.props.name}</li>
8           <li>{this.props.hair}</li>
9           <li>{this.props.age}</li>
10        </ul>
11      </div>
12    )
13  }
14 }
```


We use the syntax of `{this.props.insert prop name here}` in order to use our prop.

If we had `<li>{this.props.name}</li>`, it will ultimately render as the following:

```
<li>Mike</li>
```

This example would then display the following:



Let's take some time to explain why we have `this.props.____`.

We previously had an object as an example data type. There was this example:

```
1 let myHonda = {color: 'red', wheels: 4}
```

The object is myHonda which has properties, color and wheels.

If we wanted to access the value of wheels, we would do the following:

```
let four = myHonda.wheels //four now equals 4
```

It's the same thing when we want to access the value of a property encapsulated in a React component.

That explains the `props.____` part of it.

However, what's up with this in `this.props.____`?

`this` is a keyword used to specify that we are accessing a prop defined in context of the current component.

Think of it this way, props are like items in a cabinet that belong to a component.



If I was asking someone to grab chips from a cabinet that was within my reach, I would say: “Hey man, can you grab me chips from *this* cabinet?” As I *put my hand on the cabinet*

If I was asking someone to grab chips from a cabinet out of my reach, I would say: “Hey man, can you grab me chips from *that* cabinet?” As I *point at the cabinet*

In a similar way, *this* in *this.props.** **is like saying: “Hey React, I’d like to use **this** property I have defined called ***.”** *this* notes that it is within our reach to access in our component class.

Someone might then ask: “If our props are defined outside of the class like so:

```
<Example name={"Example 1"}/>
```

How come I can say from within a class ‘*this* prop is in my reach’?”

The answer is that React does behind the scenes work. Even though props are defined within a class, it goes ahead and automatically says: “Hey class! They defined some props for this component. Please let them be within the reach of this class so they can use *this.props*___ and make things easier.”

Putting it all together, we have something like this:

```
1 <li>{this.props.name}</li>
2 //<Person name={"Mike"}/>
```

...

While the code for defining the props may look simple, it can be a bit difficult to conceptualize in your mind.

Why do I say that?

Well, when we define a component, we see code like this:

```
1  class Example extends React.Component {
2    render() {
3      return (
4        <div>
5          <button>Example Button</button>
6        </div>
7      )
8    }
9  }
```

We think of this as a *definition* of our component.

We then *use* our component elsewhere like so:

```
<Example/>
```

With props, the definition is in the *use* of the component:

```
<Example name={"Example 1"}/>
```

The use of the prop(s) is in the *definition* of the component:

```
1  class Example extends React.Component {
2    render() {
3      return (
4        <div>
5          <button>{this.props.name}</button>
6        </div>
7      )
8    }
9  }
```

This can be confusing as is often just assumed without explanation.

So, how do we understand this?

Let's look at this example:

```

1 ReactDOM.render(
2   <Example name={"Example 1"}/>
3   document.getElementById( 'app' )
4 );

```

What happens here when we use the Example component?

Eventually, ReactDOM will go “I see they’ve put this Example component here. What exactly do they want to render when using this component? Let’s check what they specified to return in the render function. Oh, I see they’ve specified the following:

```

1 class Example extends React.Component {
2   render() {
3     return (
4       <div>
5         <button>{this.props.name}</button>
6       </div>
7     )
8   }
9 }

```

Cool! I’ll render this to the DOM!”

My point is this. While you might think of the flow of the process as:

>Defining the component

Using the component

It’s helpful to think of the flow of the process as:

Defining the component

Using the component

ReactDOM goes about checking what was defined in the component

ReactDOM is the bridge to render the component to the DOM

Why is this important?

It is important because what is *defined* in a component is ultimately checked *after* the component is *used*.

Therefore, we can think of the ReactDOM.render process where props are in use as: “I see they’ve put this Example component here. What exactly do they want to render when using this component? Let’s check what they specified to return in the render function. Oh, I see they’ve specified the following:

```

1  class Example extends React.Component {
2    render() {
3      return (
4        <div>
5          <button>{this.props.name}</button>
6        </div>
7      )
8    }
9  }

```

Hmm... it's saying there is a prop called name specified in this component they render. Let me see:

```
<Example name={"Example 1"}/>
```

Ah! Yup, there is a prop here called name with a value of "Example 1".

Cool! I'll render this to the DOM as:

```
<button>Example 1</button>
```

Awesome, all set!

Hopefully, working through the thought process like this is helpful.

...

Ok. I know we went off on some tangents, but let's finally return to the pen we had started off with that had two buttons with text controlled by props.

So far, we have the following in our JavaScript:

```

1  class Buttons extends React.Component {
2    render() {
3      return (
4        <div>
5          <button></button>
6          <br/><br/>
7          <button></button>
8        </div>
9      )
10   }
11 }
12 ReactDOM.render(
13   <Buttons/>,
14   document.getElementById('buttons')
15 );

```

Next, let's define the props like so. We will have a prop that will contain the text for each button accordingly:

```
1 ReactDOM.render(  
2   <Buttons text1="Button 1" text2="Button 2"/>,  
3   document.getElementById('buttons')  
4 );
```

Then, we can use the props in the component definition:

```
1 class Buttons extends React.Component {  
2   render() {  
3     return (  
4       <div>  
5         <button>{this.props.text1}</button>  
6         <br/><br/>  
7         <button>{this.props.text2}</button>  
8       </div>  
9     )  
10  }  
11 }
```

We now have a completed example where we have two buttons containing different texts.

[React for the Visual Learner: Props¹³](#)

...

Two more things to also mention about props as we wrap up.

The first thing I wanted to mention is that we can use props in a component hierarchy. I will simply provide an example so you can see this in action:

[React for the Visual Learner: Props Hierarchy¹⁴](#)

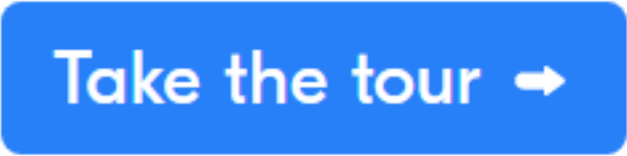
The next thing I wanted to mention is important for distinguishing using props for data and using state.

Props is only used for data that is never updated.

For instance, let's say we had a button component that ultimately displays this:

¹³<https://codepen.io/mikemang/pen/4939b836c47734e2848b7f841cb959f8>

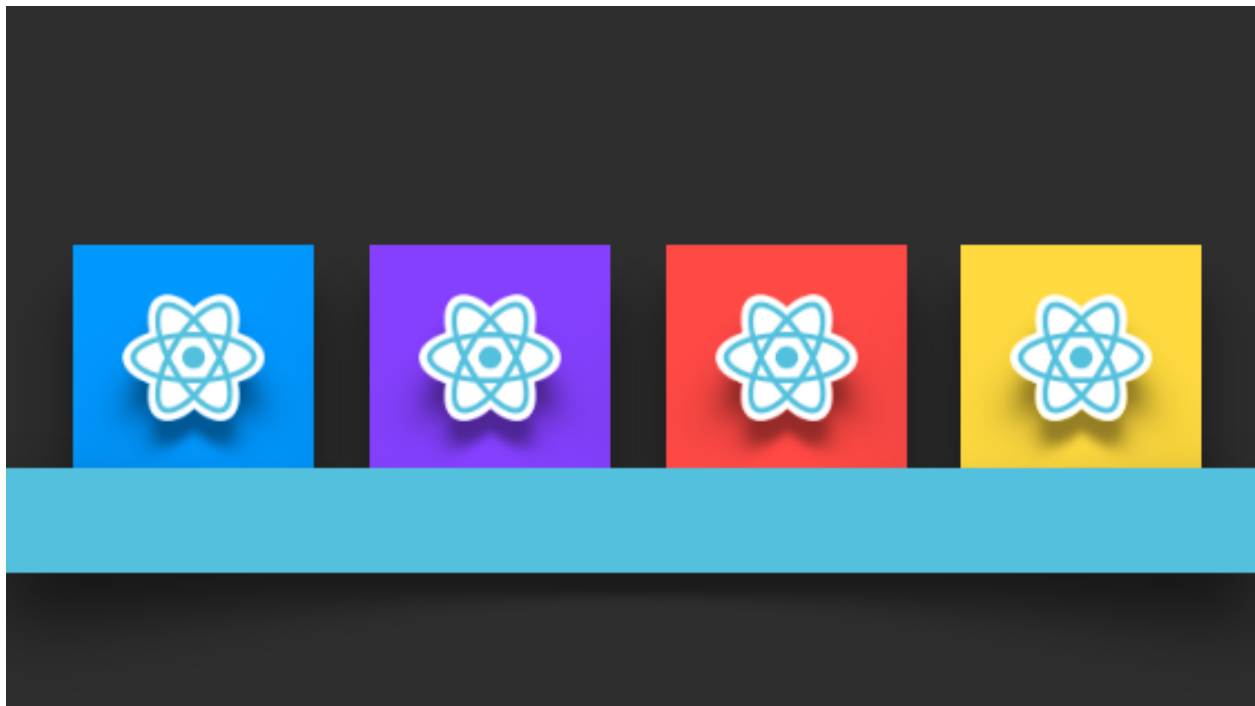
¹⁴<https://codepen.io/mikemang/pen/488668e19837cf12e41cd0a4ebf5215b>



Let's also say that this text is never changed dynamically. Meaning, it will always say "Take the tour" no matter what event we input as a user.

Since it never changes, we would use the text for this button as a prop.

State



No, state is not part of the village design where I'm going to come here and add to component inception and say a state is a collection of villages.

State is another way encapsulate data in a React component.

Unlike props, state is used for data that is updated.

For example, think of a weather app.



The current temperature is going to change over time. Therefore, we use state to control the data instead of props.

Also, state controls data fully within the class where the component is defined. The meaning of this will make more sense when we see the code.

Go ahead and [fork the previous pen of our two buttons](#)¹⁵.

Name it **React for Visual Learners: State**.

Don't erase the code as we will just make some edits from our props implementation.

First, go ahead and replace all the *this.props.** **with** **this.state..*

¹⁵<http://codepen.io/mikemang/pen/4939b836c47734e2848b7f841cb959f8?editors=1010>


```

1  class Buttons extends React.Component {
2    render() {
3      return (
4        <div>
5          <button>{this.state.text1}</button>
6          <br/><br/>
7          <button>{this.state.text2}</button>
8        </div>
9      )
10   }
11 }

```

Nothing too difficult here.

Next, remove the defined props when the Button component is used:

```

1  ReactDOM.render(
2    <Buttons/>,
3    document.getElementById('buttons')
4  );

```

Again, the data in a state implementation will be defined within the component class.

Now, we have to add an object called *state* within our class:

```

1  class Buttons extends React.Component {
2    state = {text1: "Button 1", text2: "Button 2"};
3
4    render() {
5      return (
6        <div>
7          <button>{this.state.text1}</button>
8          <br/><br/>
9          <button>{this.state.text2}</button>
10         </div>
11       )
12     }
13 }

```

state is an object that contains the data that we want to encapsulate in our component.

This is why we were able to start off by switching wherever there was a *this.props.** **with** **this.state..*

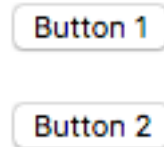
{this.state.text1} and {this.state.text2} are being used in our render function within the class and will ultimately render as:

```

1  <button>Button 1</button>
2  <br/><br/>
3  <button>Button 2</button>

```

We should now have the same display as before except we are using state:



Now that we are using state, the data is being handled completely within the class.

Here, I want to interject with a possible question: “If state is an object defined within our class, it makes sense why I can do *this.state.**. **However, why does **this.props***. work for our props if it is defined outside our class?”

As I noted in the section covering props, React does behind the scenes work so props defined where a component is used are accessible as if they were an object within our class.

For this reason, both props and state can be accessed using *this.props* and *this.state* accordingly even though the state is manually defined and controlled within a class.

...

Let me also answer a few more possible questions.

If you were curious, you might have asked: “Why can’t we just do props as an object in the class just like we do with state?”

Well, it won’t work. Props is a reserved thing in React. If you want proof, you can test out this code which won’t render the text:

```

1  class Buttons extends React.Component {
2    props = {text1: "Button 1", text2: "Button 2"};
3
4    render() {
5      return (
6        <div>
7          <button>{this.props.text1}</button>
8          <br/><br/>
9          <button>{this.props.text2}</button>
10         </div>
11       )
12     }
13   }

```

The display of the code above is:

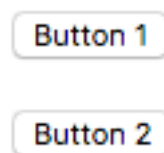


Going further in that line of thought, some of you may have also asked: “Ok. I get we can’t do props as an object since it is reserved, but what about something totally random?”

If we replace state with *unicorn*, everything will work. You can test this code out as well for proof:

```
1 class Buttons extends React.Component {
2   unicorn = {text1: "Button 1", text2: "Button 2"};
3
4   render() {
5     return (
6       <div>
7         <button>{this.unicorn.text1}</button>
8         <br/><br/>
9         <button>{this.unicorn.text2}</button>
10      </div>
11    )
12  }
13 }
```

The above code displays as:



If that is the case, then why are we using state?

The simple answer is because it is the official, standard naming convention for React and we should stick to it.

More importantly, while there is no predefined magic for initializing and using state, there will be predefined magic for updating our state.

Therefore, it is not negotiable to use anything other than state if we are using data in a component that needs to be updated over time.

For that reason, our final JavaScript code for our pen should be:

[React for the Visual Learner: State](#)¹⁶

...

At this point, our example should be working just like props except we are using state. Earlier, I mentioned that state is for data that can change dynamically. Let's give that a shot as we discuss event handlers.

Event Handlers



Event handlers are methods that we can use to update the state of a component.

The event handlers will be defined within the class for our components.

Setup

First things first, go ahead and [fork the previous pen](#)¹⁷ where we controlled the text of our two buttons using state.

Name it **React for Visual Learners: State w/ Event Handlers**.

¹⁶<https://codepen.io/mikemang/pen/ff262bf1876ccf5d70620e1ff7811a0f>

¹⁷<http://codepen.io/mikemang/pen/ff262bf1876ccf5d70620e1ff7811a0f?editors=1010>

Setting Up Event Handlers

We will make some modifications to this app so that the button text will be a count of the number of clicks on each button.

Let's start by changing up the names, values, and uses of the properties in our state:

```
1 class Buttons extends React.Component {
2   state = {clicks1: 0, clicks2: 0};
3
4   render() {
5     return (
6       <div>
7         <button>{this.state.clicks1}</button>
8         <br/><br/>
9         <button>{this.state.clicks2}</button>
10      </div>
11    )
12  }
13 }
```

We should now see the following:

A square button with a light gray background and a thin border, containing the number '0' in a dark font.A square button with a light gray background and a thin border, containing the number '0' in a dark font.

Next, let's add the shell of two methods that will handle our events:

```
1 class Buttons extends React.Component {
2   state = {clicks1: 0, clicks2: 0};
3   updateButton1() {
4   }
5
6   updateButton2() {
7   }
8   render() {
9     return (
10      <div>
11        <button>{this.state.clicks1}</button>
```

```

12         <br/><br/>
13         <button>{this.state.clicks2}</button>
14     </div>
15     )
16   }
17 }
18 // "inject" to DOM
19 ReactDOM.render(
20   <Buttons/>,
21   document.getElementById('buttons')
22 );

```

Within these methods, we are going to set a new state so we can increment *clicks1* and *clicks2* accordingly.

Next, we need to specify that we want these methods to be called on the event of a click:

```

1  render() {
2    return (
3      <div>
4        <button onClick={this.updateButton1}>           {this.state.clicks1}
5        </button>
6        <br/><br/>
7        <button onClick={this.updateButton2}>           {this.state.clicks2}
8        </button>
9      </div>
10    )
11  }

```

We are essentially saying: “Just a heads up, I want to call a method in *this* class when the buttons are clicked.”

Note that we are using camelCase for our event (onClick not onclick).

The *{this.updateButton1}* isn’t anything too new since we have already discussed doing *{this.props}** and **{this.state}*.

However, we have to do a bit more work to get the *this.____* part to work when it comes to event handling.

We have to do some extra work to confirm that the method is within our reach and that we can use *this*.

The reason for the extra work is that ES6 classes do not automatically confirm that the methods we want to handle an event and the element (i.e. button) where the method to handle an event is specified is within the same context.

There are a [few different approaches](#)¹⁸ to doing this. Each approach has tradeoffs of sacrificing readability for performance.

Because everything that we will do in this book is on a smaller scale than large, real-world applications, I prefer the most readable approach:

```
1 render() {
2   return (
3     <div>
4       <button onClick={this.updateButton1.bind(this)}>{this.state.clicks1}</button>
5     </button>
6     <br/><br/>
7     <button onClick={this.updateButton2.bind(this)}>{this.state.clicks2}</button>
8   </button>
9   </div>
10 )
11 }
```

In the code above, the extra work is adding `.bind(this)`. While it looks goofy and *this* is notoriously confusing in JavaScript, it simply is saying this: “Alright. On the click of this element (which is a button in our case), I want to call a method to handle the click that is within this class. Since you want confirmation that it is a method within this class, I’ve added `.bind(this)`. Now, you have no choice but to allow me to access this method using *this*.”

Completing the Methods to Update Our State on an Event

With this explanation out of the way, we just have to fill in the methods to update our state.

You might be thinking: “This is going to be so simple! I’ll just change the state like I would any object:”

```
1 updateButton1() {
2   this.state.clicks1++
3 }
4 updateButton2() {
5   this.state.clicks2++
6 }
```

Not so fast! This is React after all, did you expect it to be that easy!

¹⁸<https://medium.freecodecamp.com/react-binding-patterns-5-approaches-for-handling-this-92c651b5af56>

With React, we do not modify our state directly. Instead, we modify our state using a predefined `this.setState()`:

```
this.setState();
```

Let's go ahead and update the state of *clicks1* and *clicks2* accordingly:

```
1  updateButton1() {  
2    this.setState({clicks1: 1});  
3  }  
4  updateButton2() {  
5    this.setState({clicks2: 1});  
6  }
```

In the code above, you can think about it as if we are resetting our state. Inside `this.setState()`, we have object notation just like we have when we define our initial state.

In `updateButton1`, we are setting the value of `clicks1` to 1. In `updateButton2`, we are setting the value of `clicks2` to 1.

If we click each button once, we will now see the following:



Cool! However, we want this to be a counter app. Meaning, we want `clicks1` and `clicks2` to increment on each click and not just updated one time to a new value.

If we are writing updating the state using object notation within `this.setState()`, then how do we do this?

Could we just do this?

```
1  updateButton1() {  
2    this.setState({clicks1: clicks1++});  
3  }  
4  updateButton2() {  
5    this.setState({clicks2: clicks2++});  
6  }
```

Nope, that won't work.

The missing piece to this puzzle is the concept of `prevState` which is also some more React magic.

Within `this.setState()`, we are going to have a function:


```
1  updateButton1() {
2    this.setState(
3      function(){
4
5      }
6    );
7  }
8  updateButton2() {
9    this.setState(
10     function(){
11
12     }
13   );
14 }
```

We can also pass in *prevState* to this function. *prevState* can be thought of as a copy of the state before it gets updated:

```
1  updateButton1() {
2    this.setState(
3      function(prevState){
4
5      }
6    );
7  }
8  updateButton2() {
9    this.setState(
10     function(prevState){
11
12     }
13   );
14 }
```

Now, we can use *prevState* within this function as a normal object to update our state:

```
1  updateButton1() {
2      this.setState(function(prevState){
3          prevState.clicks1 += 1
4      });
5  }
6  updateButton2() {
7      this.setState(function(prevState){
8          prevState.clicks2 += 1
9      });
10 }
```

This code is saying: “I want to take the previous state of clicks1 and clicks2 and do something with it. Let’s go ahead and increment them by 1.”

If clicks1 is currently 0 before being update, then `prevState.clicks1` equals 0. If we do `prevState.clicks1 += 1`, then we have the same thing as the following:

```
1  function(prevState) {
2      0 + 1
3  }
```

What’s missing? We have to reassign this value to clicks1 (and clicks2).

Therefore, we can complete these methods with the following:

```
1  updateButton1() {
2      this.setState(function(prevState){
3          clicks1: prevState.clicks1 += 1
4      });
5  }
6  updateButton2() {
7      this.setState(function(prevState){
8          clicks2: prevState.clicks2 += 1
9      });
10 }
```

We are still using object notation to reassign our state. However, we have to wrap it around a function that is passed *prevState* if we need what will soon be the previous state (can be thought of more as the current) in order to reassign the state.

With ES6, we can use a fat arrow function:

```
1  updateButton1() {
2      this.setState((prevState) => {
3          clicks1: prevState.clicks1 += 1
4      });
5  }
6  updateButton2() {
7      this.setState((prevState) => {
8          clicks2: prevState.clicks2 += 1
9      });
10 }
```

Here's a comparison between the syntax between a fat arrow function and a traditional function:

```
1  //traditional
2  function(*parameters go here*) {
3  }
4  //fat arrow
5  (*parameters go here*) => {
6  }
```

This makes things less wordy and is a nice feature of ES6 so let's keep it with the fat arrow functions.

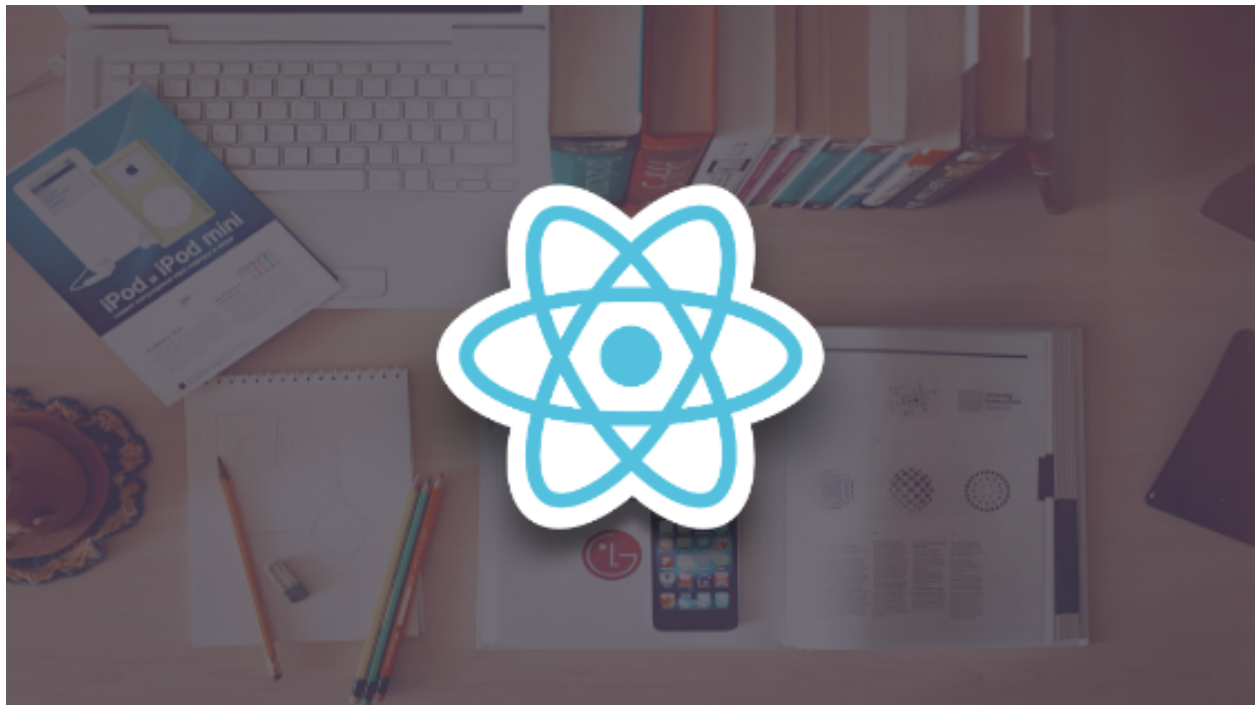
Now, we have functioning counters:

[React for the Visual Learner: State w/ Event Handling¹⁹](#)

Perfect! While we only covered one type of event, we have learned the core principles behind event handlers in React.

¹⁹<https://codepen.io/mikemang/pen/0c4623b471e2509d57fc7adf3e1cf2a3>

Study Break



So far, we have covered the following in this chapter:

- Handling props
- Managing state
- Specifying event handlers

We have yet to cover:

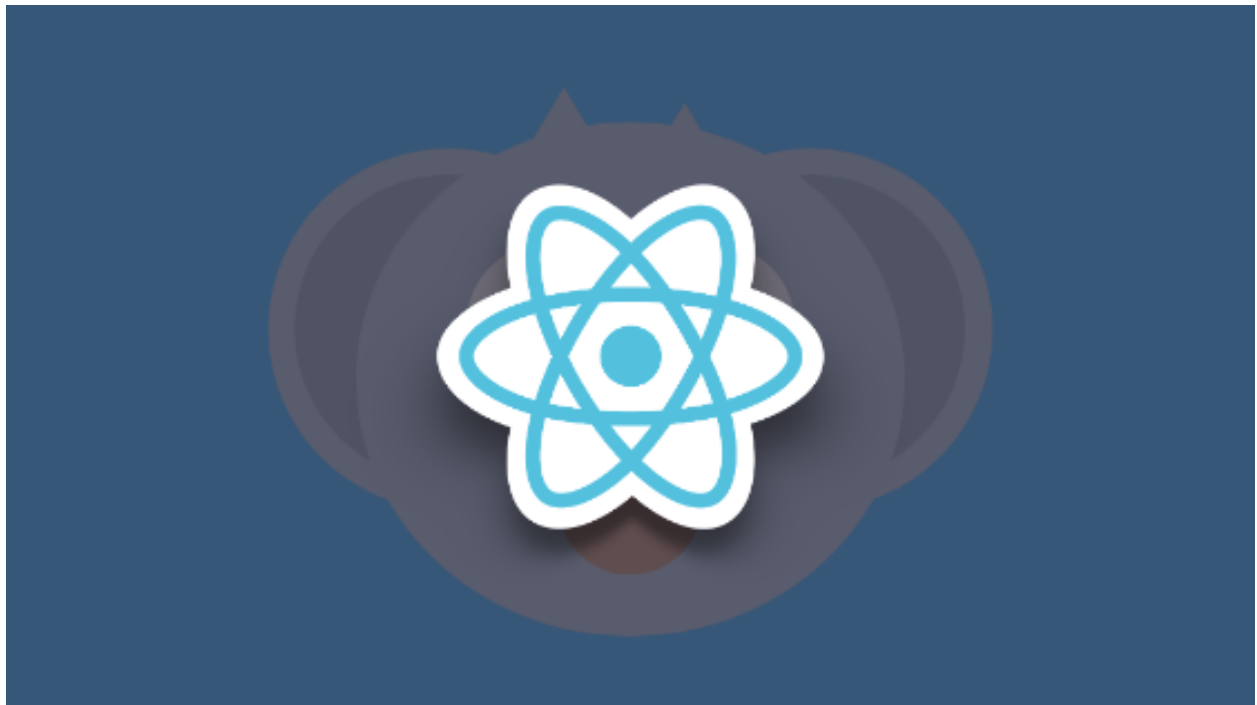
- Using inline JSX styling
- Implementing conditional rendering of components
- Setting up lifecycle hooks

This chapter is already long enough despite just hitting the halfway mark. Feel free to take a break or save this for later.

Once you are ready, we are going to cover those last three topics.

Before that, I want to now switch over to using our Koala that we made with React in the previous chapter to help explain these concepts.

Switching to Our Koala



Initial Setup

Fork the React Koala pen we previously made²⁰.

Name the new pen React for Visual Learners

7 Koala With Color Props.

Adding Props

We talked about props being properties that never get updated. With our koala, we are going to add props to define the background colors (i.e. `background: #000000`) of our koala.

This begs the following question: “We can define styles in JavaScript?”

Yes, we will be able to override our CSS and have some styling done in React.

By doing this, we are going to learn about *inline JSX styling*. We will also get practice with how to use props in a component hierarchy.

Passing Down Props

Let’s start off by adding a constant (immutable) variable called `colorsObject` which will have a value of a JavaScript object. The JavaScript object will contain props for all the different colors used in our koala graphic:

²⁰<http://codepen.io/mikemang/pen/58f339afac397fbb2f2ae678d6334030>

```
1 //add this just above ReactDOM code
2 //props objects
3 const colorsObject = {
4   white: "#FFFFFF",
5   lightGray: "#A6BECF",
6   darkGray: "#819CAF",
7   brown: "#BE845F",
8   navy: "#27354A"
9 }
```

This object can be used in place of writing out all of our props within a component. Let's add this object (like we would a prop) in our koala component which is used in the ReactDOM.render:

```
1 // "inject" to DOM
2 ReactDOM.render(
3   <Koala colors={colorsObject}/>,
4   document.getElementById('koala')
5 );
```

Alright. We have given our koala component props that contain all the colors used in our graphic. Now, we can pass these props down from our *village* to all the *houses*. The houses will then be able to use these color props for inline JSX styling.

How do we pass down our props?

Let's answer this by working through the logic as we go from top to bottom of the component hierarchy.

If we needed to use the color props for our Koala component, how would we do that?

In the class for the Koala component, we could do:

```
1 this.props.colors.___
2 //___ can be filled in with any color
3 //we want to access from the colors object
```

Well, what is in our Koala component's class? We have:

```
1 //village
2 class Koala extends React.Component {
3   render() {
4     return (
5       <Box/>
6     )
7   }
8 }
```

All the houses are *children* of the Box component as well. Remember, we define props wherever a component is being used.

Therefore, we can pass the colors props like so:

```
1 //village
2 class Koala extends React.Component {
3   render() {
4     return (
5       <Box colors={this.props.colors}/>
6     )
7   }
8 }
```

Note we do *this.props.colors* because we want to pass down all the colors.

Now, our Box component has the colors props defined and can be used within its class.

Let's take a look at what we have in the Box component:

```
1 //neighborhood
2 class Box extends React.Component {
3   render() {
4     return (
5       <div className="box">
6         <EarLeft/>
7         <EarRight/>
8         <EyeLeft/>
9         <EyeRight/>
10        <Head/>
11        <Nose/>
12        <HairLeft/>
13        <HairRight/>
14      </div>
```

```

15     )
16   }
17 }

```

In the Box component, we have some house components that are being used directly (Ears and Eyes) and some block components that are parents to house components (Head, Nose, Hair).

We want to pass all the colors until we get to our houses.

Therefore, the block components will be passed down the entire colors props. The house components will be passed down the specific color that they will need for styling (we will call this prop color instead of colors).

We can then do the following:

```

1  //neighborhood
2  class Box extends React.Component {
3    render() {
4      return (
5        <div className="box">
6          <EarLeft colors={this.props.colors}/>
7          <EarRight colors={this.props.colors}/>
8          <EyeLeft colors={this.props.colors}/>
9          <EyeRight colors={this.props.colors}/>
10         <Head color={this.props.colors.lightGray}/>
11         <Nose color={this.props.colors.brown}/>
12         <HairLeft color={this.props.colors.lightGray}/>
13         <HairRight color={this.props.colors.lightGray}/>
14       </div>
15     )
16   }
17 }

```

Let's pass down a color to the houses within the blocks. Then, we will demonstrate how to do inline JSX styling using our color prop for each house.

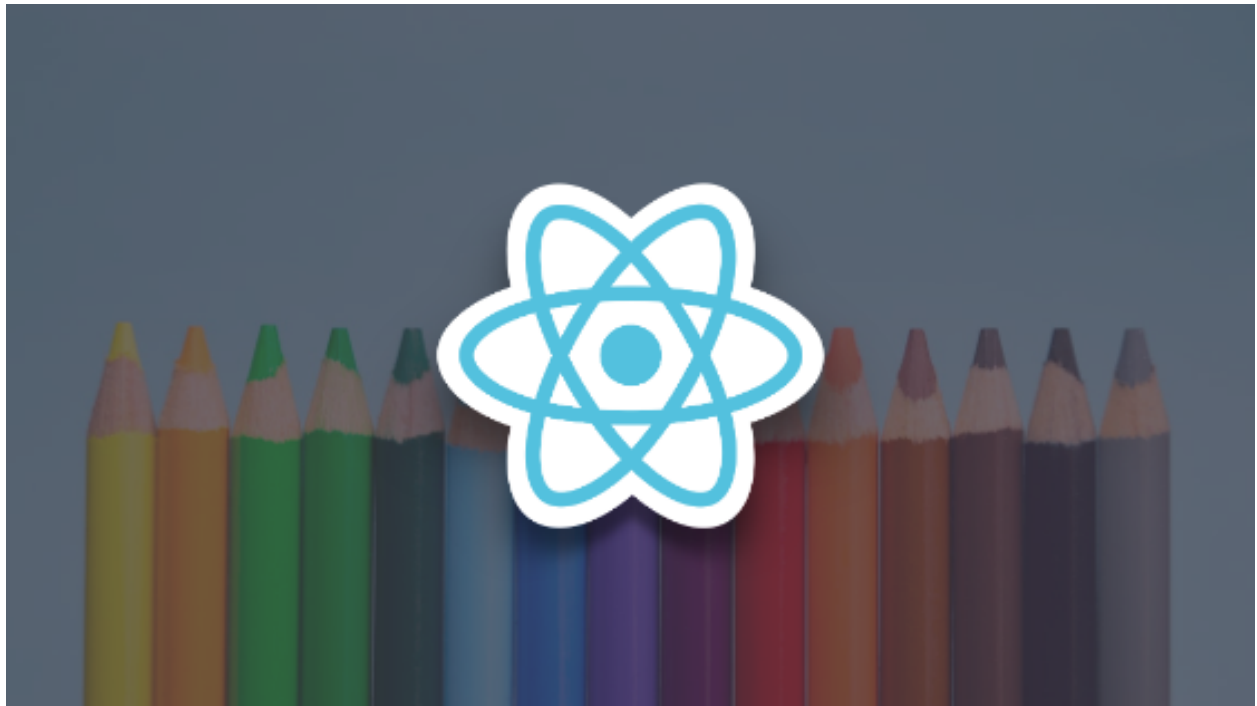
We can pass down a color to the houses within the blocks just like we did with the houses that were directly under the Box component.

Therefore, we will end up with the following code for our blocks:


```
1  //blocks
2  class EarLeft extends React.Component {
3    render() {
4      return (
5        <div>
6          <OuterEarLeft color={this.props.colors.lightGray}/>
7          <InnerEarLeft color={this.props.colors.darkGray}/>
8        </div>
9      )
10   }
11 }
12 class EarRight extends React.Component {
13   render() {
14     return (
15       <div>
16         <OuterEarRight color={this.props.colors.lightGray}/>
17         <InnerEarRight color={this.props.colors.darkGray}/>
18       </div>
19     )
20   }
21 }
22 class EyeLeft extends React.Component {
23   render() {
24     return (
25       <div>
26         <OuterEyeLeft color={this.props.colors.white}/>
27         <PupilLeft color={this.props.colors.navy}/>
28       </div>
29     )
30   }
31 }
32 class EyeRight extends React.Component {
33   render() {
34     return (
35       <div>
36         <OuterEyeRight color={this.props.colors.white}/>
37         <PupilRight color={this.props.colors.navy}/>
38       </div>
39     )
40   }
41 }
```

Cool! All of our houses have been passed down the colors they will need to do inline JSX styling.

Inline JSX Styling



Let's demonstrate using the `color` prop to do inline styling with our `Head` component.

To do inline JSX styling, we just have to add `style={{...}}` within the element(s) in the JSX. For example:

```
1  class Example extends React.Component {
2    render() {
3      return (
4        <div
5          style={{background: "#000000"}}
6          className="example"
7        >
8        </div>
9      )
10   }
11 }
```

In the example above, we are setting the style for an element using JavaScript. The naming conventions for the [attributes](https://css-tricks.com/almanac/selectors/a/attribute/)²¹ we want to style will be the same as CSS. The difference is that it is using object notation.

²¹<https://css-tricks.com/almanac/selectors/a/attribute/>

Also, note that we have double braces for the inline styling (`style={{...}}`).

We could do multiple inline JSX styles like this:

```
1 class Example extends React.Component {
2   render() {
3     return (
4       <div style={{ background: "#000000", color: "#FFFFFF" }}
5         className="example">
6       </div>
7     )
8   }
9 }
```

Note that we use a comma in between the styling because it is object notation and not inline CSS.

With all this being said, we can style our Head component using its color prop like so:

```
1 class Head extends React.Component {
2   render() {
3     return (
4       <div style={{background: this.props.color}}
5         className="head">
6       </div>
7     )
8   }
9 }
```

In the code above, we use *this.props.color* to use the color prop that was passed down for the background. Note that it does not have to be surrounded with curly braces since it is already within double braces.

Sweet, we now have the koala's head color being controlled using JavaScript while learning how to pass down props!



To answer the possible curious reader, we can know that this is overriding our CSS. To prove it, you can use the following code:

```
1 class Head extends React.Component {
2   render() {
3     return (
4       <div style={{background: "white"}}
5         className="head">
6         </div>
7     )
8   }
9 }
```

We see the following which proves it is overriding our JavaScript:



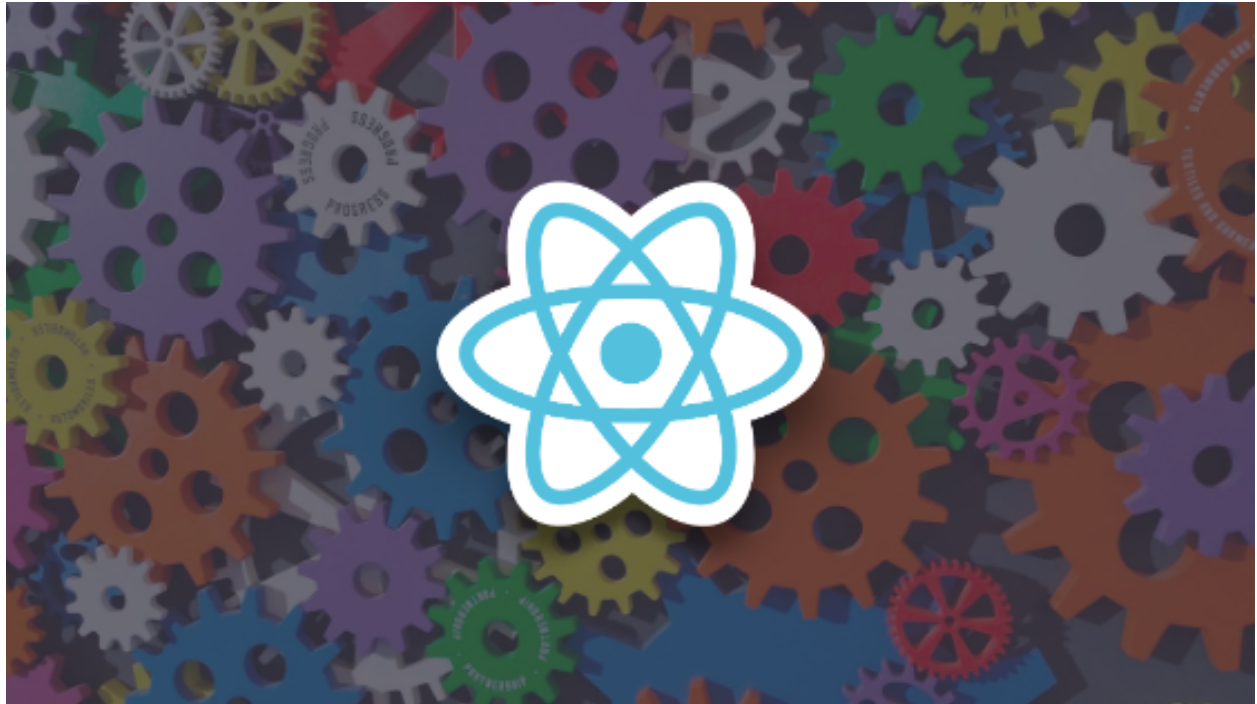
It's also worth mentioning that our koala's head has maintained the same size, shape, and position. Meaning, inline JSX styling will only override what is specified and not the rest of the styling done in CSS. Also, if we had styling in JSX that was missing in our CSS, the JSX styling would add to our CSS styling.

If we repeated this same step to all of our houses, we get this for the final code:

[React for Visual Learners: Koala With Color Props²²](https://codepen.io/mikemang/pen/fbdae2153f3b70ce1964fad689d24cf6)

²²<https://codepen.io/mikemang/pen/fbdae2153f3b70ce1964fad689d24cf6>

Dynamic JSX Styling



I'm sure there are some who may be wondering: "Why would I use inline JSX styling if I have CSS?" Well, you could do all your styling using JSX styling. Here's an example:

```
1 let Letter = React.createClass({
2   render: function() {
3     let letterStyle = {
4       padding: 10,
5       margin: 10,
6       backgroundColor: "#ffde00",
7       color: "#333",
8       display: "inline-block",
9       fontFamily: "monospace",
10      fontSize: "32",
11      textAlign: "center"
12    };
13
14    return (
15      <div style={letterStyle}>
16        {this.props.children}
17      </div>
18    );
```

```

19     }
20   });

```

In the example above, a JavaScript object is created within the render function (but above the return) containing all the styling:

```

1  let letterStyle = {
2    padding: 10,
3    margin: 10,
4    backgroundColor: "#ffde00",
5    color: "#333",
6    display: "inline-block",
7    fontFamily: "monospace",
8    fontSize: "32",
9    textAlign: "center"
10  };

```

Then, the inline style can be that object within single braces:

```

1  <div style={letterStyle}>
2    {this.props.children}
3  </div>

```

The nice part about this approach is that the styling is now encapsulated within a component. This makes organization better than a CSS file that can sometimes get out of hand.

To be honest, this is more of a gray area and a matter of preference. I like to keep my styling out of JavaScript to remove clutter and use [SCSS](#)²³.

With that being said, there is one exception. If you style in JavaScript, you have the ability to make your styling dynamic.

We will do a quick example of this together so you can see what I mean.

[Fork the previous pen](#)²⁴ and name the new pen 8 Koala With Dynamic Styles.
React for Visual Learners

In this pen, we are just going to have the color of our head change on the click of it from light gray to dark gray.

²³http://sass-lang.com/documentation/file.SCSS_FOR_SASS_USERS.html

²⁴<http://codepen.io/mikemang/pen/fbdae2153f3b70ce1964fad689d24cf6>

First, let's change up the props for our Head component so that we pass it all the colors as props:

```
1 //neighborhood
2 class Box extends React.Component {
3   render() {
4     return (
5       <div className="box">
6         <EarLeft colors={this.props.colors}/>
7         <EarRight colors={this.props.colors}/>
8         <EyeLeft colors={this.props.colors}/>
9         <EyeRight colors={this.props.colors}/>
10        <Head colors={this.props.colors}/>
11        <Nose color={this.props.colors.brown}/>
12        <HairLeft color={this.props.colors.lightGray}/>
13        <HairRight color={this.props.colors.lightGray}/>
14      </div>
15    )
16  }
17 }
```

Now, we are going to have to control the color using state since we are now going to have data that is changing over time.

Therefore, we can add a state to our Head component that has a property called *color* and is set to the light gray color from the passed down *colors* props. We also update *this.props.color* to *this.state.color* for the inline JSX styling:

```
1 class Head extends React.Component {
2   state = {color: this.props.colors.lightGray};
3   render() {
4     return (
5       <div style={{background: this.state.color}}
6         className="head">
7       </div>
8     )
9   }
10 }
```

Our Head component is now being controlled by the local state.

Using what we learned previously in this chapter, we can add a method that will update the state so that *color* is changed to dark gray:


```
1 class Head extends React.Component {
2   state = {color: this.props.colors.lightGray};
3   updateColor() {
4     this.setState({color: this.props.colors.darkGray});
5   }
6   render() {
7     return (
8       <div style={{background: this.state.color}}
9         className="head">
10      </div>
11    )
12  }
13 }
```

Next, let's have this method trigger on the click of the head element:

```
1 class Head extends React.Component {
2   state = {color: this.props.colors.lightGray};
3   updateColor() {
4     this.setState({color: this.props.colors.darkGray});
5   }
6   render() {
7     return (
8       <div onClick={this.updateColor.bind(this)}
9         style={{background: this.state.color}}
10        className="head">
11      </div>
12    )
13  }
14 }
```

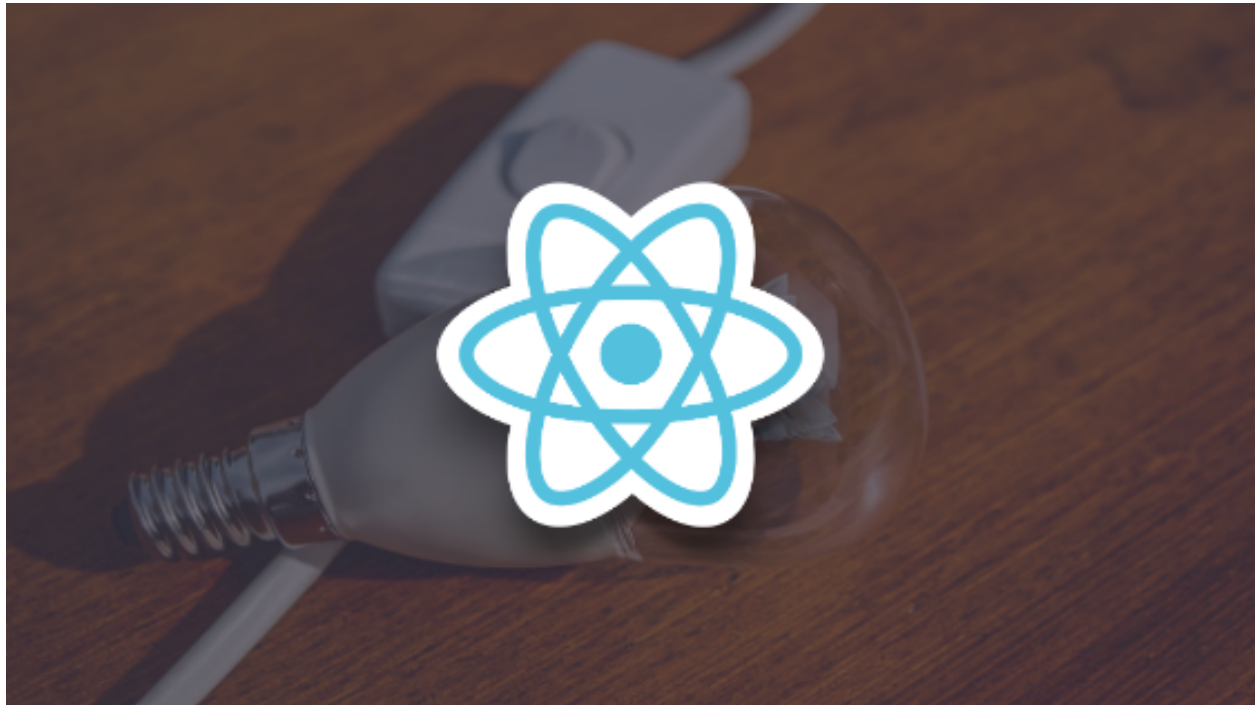
Now, our koala's head changes to dark gray on the click of it:

[React for the Visual Learner: Koala With Dynamic Styles](https://codepen.io/mikemang/pen/1a9444f1d770863803b0ce0ec1d56df5)²⁵

While a simple example, we could use our creativity to take advantage of dynamic styling in React.

²⁵<https://codepen.io/mikemang/pen/1a9444f1d770863803b0ce0ec1d56df5>

Conditional Rendering



Cool beans! We have been able to learn about props, state, event handlers, and inline JSX styling. We also were just able to apply these things with our koala app to get a better visual.

The next thing to learn in this chapter is conditional rendering.

Conditional rendering refers to being able to toggle on and toggle off what components and elements to render using a Boolean flag.

We will learn how to do this and why we would do this in the next example.

Initial Setup

Fork the previous pen²⁶ and name the new pen React for Visual Learners | 9 | Koala With Conditional Rendering.

Toggling Our Head

We are going to test out conditional rendering by doing an example where our Head component's JSX will render a head element or not render a head element depending on a boolean flag in the local state.

²⁶<http://codepen.io/mikemang/pen/1a9444f1d770863803b0ce0ec1d56df5?editors=0010>

First things first, let's change our Head component to being styled with a color passed as a prop and not via state.

Go ahead and update the props definition for our Head component within the Box class:

```
1  class Box extends React.Component {
2    render() {
3      return (
4        <div className="box">
5          <EarLeft colors={this.props.colors}/>
6          <EarRight colors={this.props.colors}/>
7          <EyeLeft colors={this.props.colors}/>
8          <EyeRight colors={this.props.colors}/>
9          <Head color={this.props.colors.lightGray}/>
10         <Nose color={this.props.colors.brown}/>
11         <HairLeft color={this.props.colors.lightGray}/>
12         <HairRight color={this.props.colors.lightGray}/>
13       </div>
14     )
15   }
16 }
```

Then, we can update our Head class to the following:

```
1  class Head extends React.Component {
2    render() {
3      return (
4        <div style={{background: this.props.color}}
5          className="head">
6        </div>
7      )
8    }
9  }
```

In the code above, we have removed the local state and all event handling as well as update our inline JSX styling to be done using a passed down prop.

Next, let's add a local state that will have a boolean flag (true or false) as the data called *showHead*:

```
1 class Head extends React.Component {
2   state = {showHead: true};
3   render() {
4     return (
5       <div style={{background: this.props.color}}
6         className="head">
7         </div>
8     )
9   }
10 }
```

Underneath the render function, let's store *this.state.showHead* in a variable called *showHead*:

Now, we need to add *if/else* logic to return a different element in our JSX depending on whether *showHead* is true or false.

If *showHead* is true, we want our head to return as is. If it is false, we want nothing to return.

In order to add *if/else* logic in JSX, we have to wrap everything around an outermost *div* element. We also have to add braces around what we want to be rendered for our component like so:

```
1 class Head extends React.Component {
2   state = {showHead: true};
3   render() {
4     const showHead = this.state.showHead;
5     return (
6       <div>
7         {
8           <div style={{background: this.props.color}}
9             className="head">
10             </div>
11         }
12       </div>
13     )
14   }
15 }
```

Finally, we can add *if/else* logic to have our head element only render if *showHead* is true:

```
1  class Head extends React.Component {
2    state = {showHead: true};
3    render() {
4      const showHead = this.state.showHead;
5      return (
6        <div>
7          {showHead ? (
8            <div style={{background: this.props.color}}
9              className="head">
10             </div>
11          ) : (
12            <div></div>
13          )}
14        </div>
15      )
16    }
17  }
```

Note we have to use ES6 syntax for our if/else statement. ? is shorthand for if and : is shorthand for else.

If showHead is true, our code will return:

```
1  <div style={{background: this.props.color}}
2  className="head"></div>
```

Else, our code will return an empty div:

```
<div></div>
```

Currently, we can still see our koala.

However, try changing showHead to false in the state:

```
1  state = {showHead: false};
```



Now, our head component is not rendering.

Here's the final code:

[React for the Visual Learner: Koala With Conditional Rendering²⁷](https://codepen.io/mikemang/pen/751899348492a81c9fb5334130ca1dae)

Cool! Not only can we hide/show an element like in our example, but we use conditional rendering to just display something different:

```
1 class Head extends React.Component {
2   state = {grayHead: false};
3   render() {
4     const grayHead = this.state.showHead;
5     return (
6       <div>
7         {grayHead ? (
8           <div style={{background: this.props.color}} className="head">
9             </div>
10        ) : (
11          <div style={{background: "yellow"}} className="head">
```

²⁷<https://codepen.io/mikemang/pen/751899348492a81c9fb5334130ca1dae>

```
12         </div>
13     }}
14 </div>
15 )
16 }
17 }
```

In the code above, we could return a gray or yellow head depending on a boolean flag in our state.

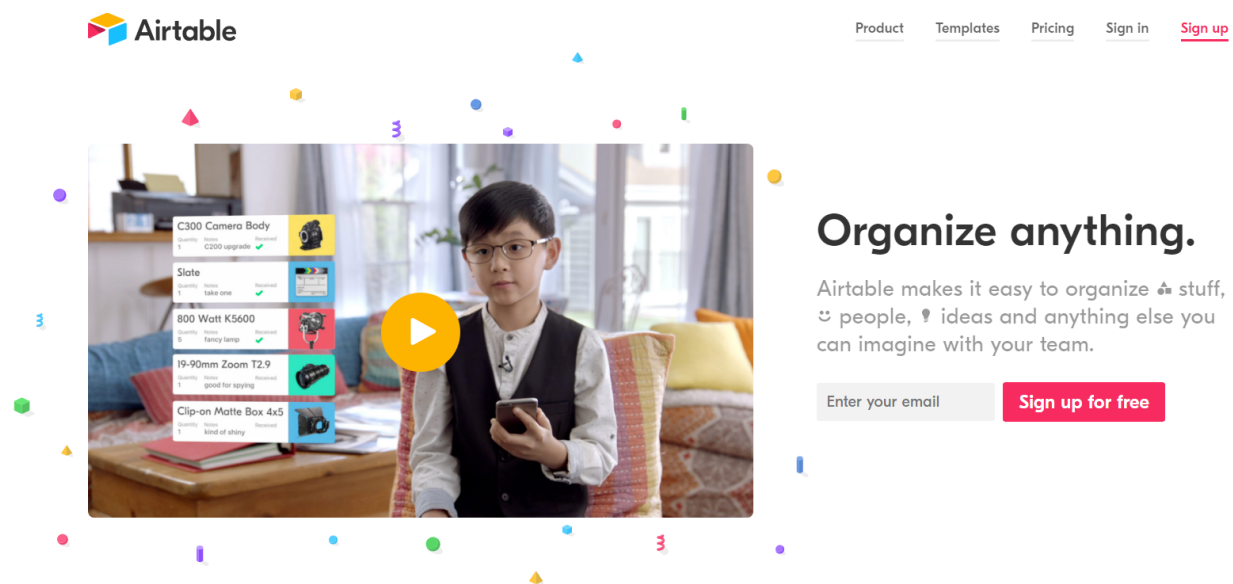
If we were working with a component that is higher up than a house, we could do something like this to conditionally render different components based on the boolean flag:

```
1 render() {
2   const isLoggedIn = this.state.isLoggedIn;
3   return (
4     <div>
5       {isLoggedIn ? (
6         <LogoutButton onClick={this.handleLogoutClick} />
7       ) : (
8         <LoginButton onClick={this.handleLoginClick} />
9       )}
10    </div>
11  );
12 }
```

Application in the Real-World

Conditional rendering has a lot of use cases where you just have to learn to get creative.

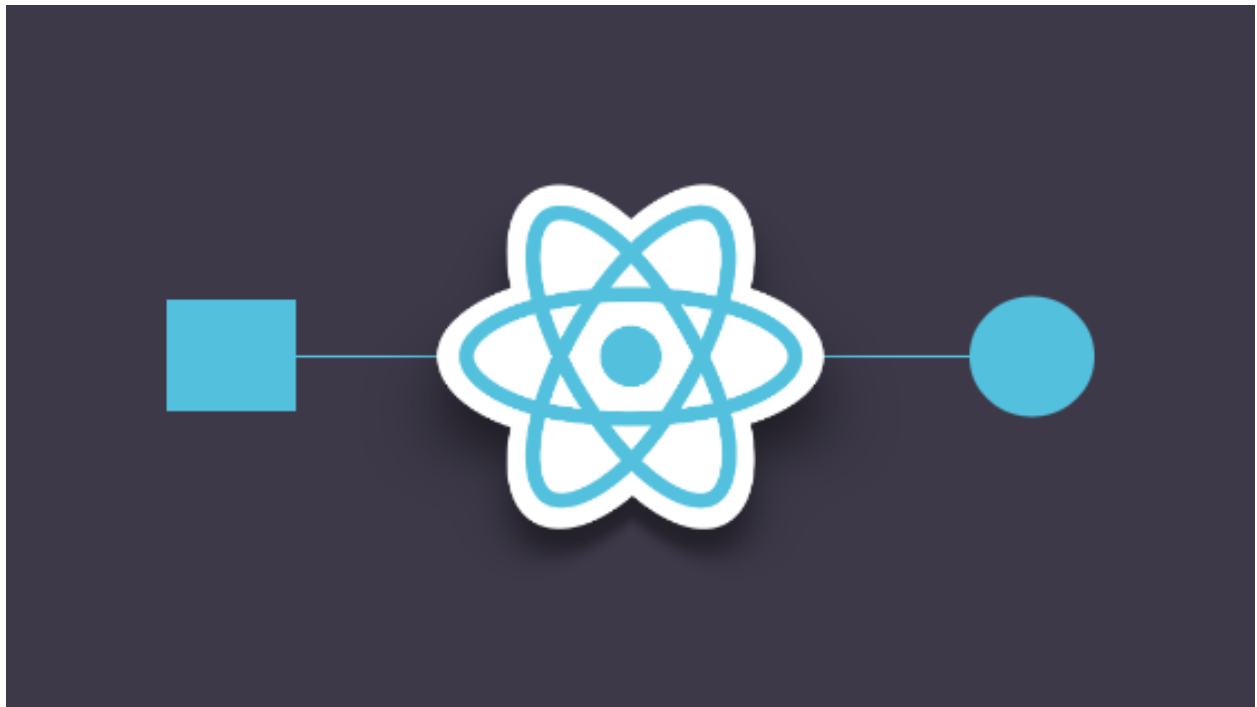
One possible example is with a navigation menu on a home page:



In the example above, there is an underlined and highlighted *Sign up* link in the navigation menu. We could have a flag in our state called *isSignedUp*.

If *isSignedUp* is true, we could not render the Sign Up link and have Sign In be the underlined and highlighted link. If it was false, we could have like we can see in the picture.

Lifecycle Hooks



Lifecycle hooks refer to methods where we can set up that do certain things within our React components depending on the lifecycle of the component.

There are 3 main lifecycles of a component:

- Mounting
- Updating
- Unmounting

Mounting refers to the process of a component being inserted into the DOM. You can think of this as the magic that happens as a result of ReactDOM.

Unmounting, therefore, refers to a component being removed from the DOM. For instance, we toggled off a component using conditional rendering.

Updating refers to changes caused by props or state.

Mounting Lifecycle Hooks

There are 2 lifecycle hooks worth mentioning that fall within the mounting lifecycle:

- `componentWillMount()`
- `componentDidMount()`

The former can be used to do something *before* our component mounts. The latter can be used to do something *after* our component mounts.

Updating Lifecycle Hooks

There are 2 lifecycle hooks worth mentioning that fall within the updating lifecycle:

```
componentWillUpdate()  
componentDidUpdate()
```

The former can be used to do something *before* our component updates. The latter can be used to do something *after* our component updates.

Unmounting Lifecycle Hooks

There is 1 lifecycle hook that falls within the updating lifecycle:

```
componentWillUnmount()
```

This can be used to do something before our component unmounts.

Quick Example

Lifecycle hooks are another feature for React components. They have more use cases with large-scale, dynamic web applications. For now, we can simply use the two mounting lifecycle hooks to change data without a user-triggered event (i.e. click).

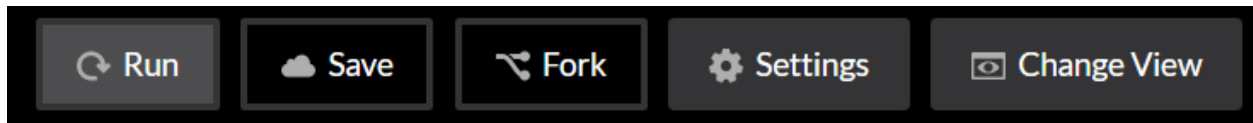
In our example, we are going to have a local state controlling our inline JSX styling for our head like we did before. This time, we will have the local state change between the two lifecycle hooks. More specifically, we are going to have two different styles for our Head component within these two lifecycle hooks.

[Fork our 8th pen²⁸](#) and name the new pen 10 Koala With Lifecycle Hooks.
React for Visual Learners

In your pen, click **Settings** then **Behavior** then disable **Auto-Updating Preview**.

Now, we can manually update our preview:

²⁸<http://codepen.io/mikemang/pen/54ed7f63e52e60d7173f79574f3c06a7>



In our Head component, let's add the shell of these two lifecycle hooks:

```
1 class Head extends React.Component {
2   state = {color: this.props.colors.lightGray};
3   componentDidMount() {
4
5   }
6   componentWillMount() {
7
8   }
9   updateColor() {
10    this.setState({color: this.props.colors.darkGray});
11  }
12  render() {
13    return (
14      <div onClick={this.updateColor.bind(this)}
15        style={{background: this.state.color}}
16        className="head"></div>
17    )
18  }
19 }
```

Next, we can copy the line from the updateColor method and paste it into both lifecycle hooks:

```
1 componentDidMount() {
2   this.setState({color: this.props.colors.darkGray});
3 }
4 componentWillMount() {
5   this.setState({color: this.props.colors.darkGray});
6 }
```

Then, let's update the *componentWillUnmount* so the a white color is applied in the setState:

```
1  componentDidMount() {
2    this.setState({color: this.props.colors.darkGray});
3  }
4  componentWillMount() {
5    this.setState({color: this.props.colors.white});
6  }
```

If you hit run, you probably still can't see anything. Reason being, the component mounts so quickly that we can't even see the styling for *before* it mounted.

To get around this, we can throw in a manual delay using `setTimeout`²⁹:

```
1  componentDidMount() {
2    setTimeout( () => {
3      this.setState({color: this.props.colors.darkGray});
4    }, 1000)
5  }
```

In the code above, we use a fat arrow function that is wrapped within:

```
setTimeout( , 1000)
```

1000 refers to 1000ms (1s). `setTimeout` will then run the code within our fat arrow function (`this.setState...`) with a 1 second delay.

Hit run and you will now be able to see the change of color for the head from white to dark gray:

[React for the Visual Learner: Koala with Lifecycle Hooks](#)³⁰

Awesome! We have gotten a taste of lifecycle hooks with React.

Concluding Thoughts

This chapter has been quite lengthy so I will keep this short.

Essentially, we have been able to go from understanding components on a very high level to diving into the main features they are capable of.

While all that we have learned in this chapter might seem trivial, learning those features really wraps up the major functionality of React.

I know that when it comes to learning web development, there always is this high amount of anticipation to finally get to the point where you feel comfortable saying: "I finally learned ____"

²⁹ https://www.w3schools.com/jsref/met_win_settimeout.asp

³⁰ <https://codepen.io/mikemang/pen/54ed7f63e52e60d7173f79574f3c06a7>

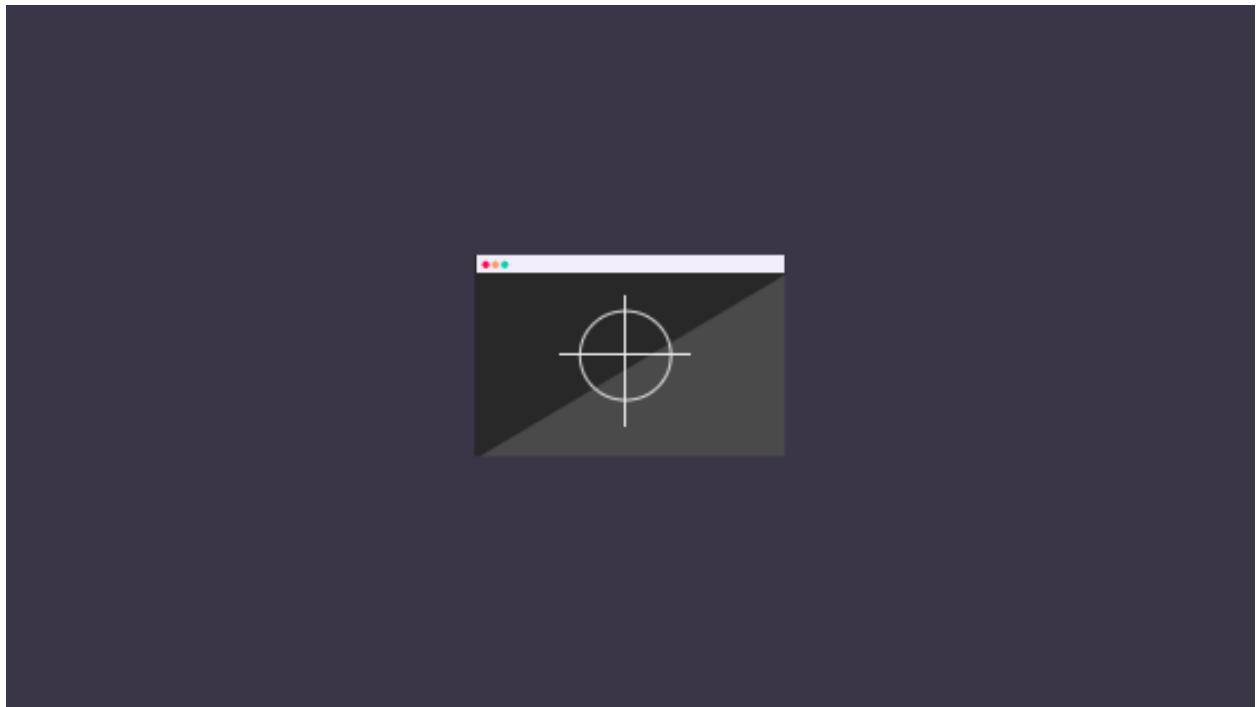
To relive this anticipation, we are going to take what we have learned so far and do a real-world application. In the next few chapters, we are going to work towards creating a user interface for a landing page using React. We will then add a chat bot demo on the landing page.

After we do all that, you can finally breathe and say: “I have learned React.”

Chapter 6 | Building Our First Project



The Scope of This Chapter



Wow! We are making some serious progress. By now, we truly have covered the core concepts and features of using React. The only thing left to do is to start applying what we have learned. To get a taste of a real-world example of using React, we are going to be creating a landing page user interface from an already-existing mockup. We will then make things more dynamic and create a working chat bot demo on that landing page.

Before we get to all of that, we have to do some transitioning. Currently, we have been doing all of our development using Codepen. I like this method because it really removes some of the overwhelming parts of setting up a React project.

We have been able to solely focus on writing React code and didn't really care about the behind the scenes setup that Codepen handles for us. While this is great for bite-sized code examples, we want to dive into creating an entire React project from scratch.

First and foremost, we will learn about the frontend *tools* that are needed in a React project. We will explain why all these tools are needed and how to use them.

Additionally, we will organize our code within a *project directory*. Meaning, we can organize all our code using folders and individual files.

In summary, we are going to be creating a React project from scratch in this chapter as we move away from Codepen. To be specific, we will discuss all the frontend tools that are needed and how to organize our project. We will approach this using modern, real-world solutions.

There is much to discuss and a load of terminologies to unpack. For that reason, it will take up the entire chapter. While this may not seem to be the most exciting part of this book (since we aren't doing any coding with React), it is hands-down one of the most important.

Once we finish this chapter, we can move on to creating our very first user interface using React.

Understanding the Tasks in the JavaScript Ecosystem

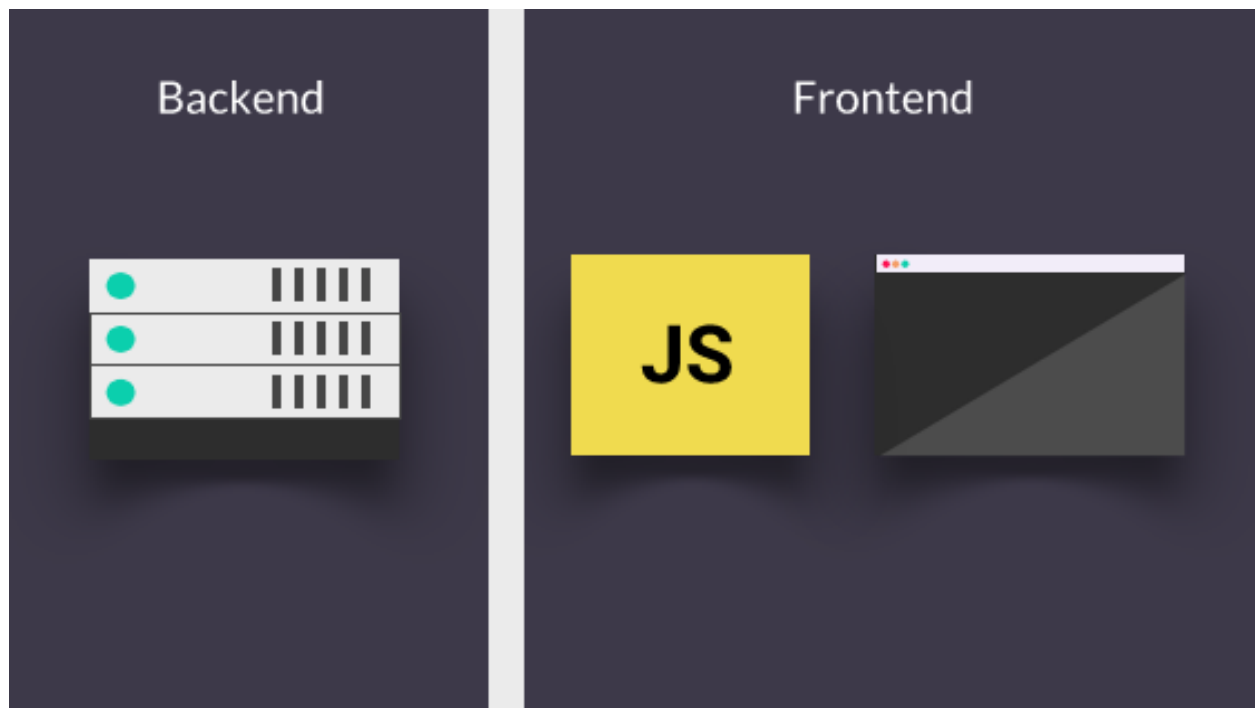


In order to understand the tools that will be needed for our React project, we have to understand tasks within the JavaScript ecosystem for modern web development.

In this section, we will look at modern web development on a high level and dive into the specific tasks that fall under JavaScript.

A Brief Overview of Modern JavaScript Web Applications (Refresher)

The following description was provided in Chapter 2. However, I'm sure there's been a lot of other information to digest between then and now so I'm including it once again.



A web application consists of a *backend* and a *frontend*. There is a *client* side and a *server* side.

The backend consists of a server, an application, and a database. The server *hosts* our application. The server also communicates with a database to get data.

The backend passes *data* to the frontend. In modern web development, JavaScript on the frontend takes the data from the backend and *converts it to HTML*.

Both the frontend and a backend compose a web application. However, in the frontend, specifically the browser, the web application is manifested visibly through a user interface. The JavaScript, again, converts data from the backend and outputs HTML which you can think of as instructions for what to have displayed as part of the user interface.

This is the role JavaScript plays in modern web applications at a high level. With this in mind, let's dive deeper into what tasks occur in the JavaScript ecosystem.

Managing Packages



As we have discussed before, React is a JavaScript library for making user interfaces out of encapsulated components. As we mentioned before, React makes use of two libraries: React Core (the core library of React) and ReactDOM (the bridge between the React code and displaying the user interface).

Using Codepen, we added these *external JavaScript resources* in the JavaScript settings:

Pen Settings

HTML CSS **JavaScript** Behavior Screenshot

JavaScript Preprocessor



Babel

Add External JavaScript



These scripts will run in this order and before the code in the JavaScript editor. You can also link to another Pen here, and it will run the JavaScript from it. Also try typing the name of any popular library.

https://cdnjs.cloudflare.com/ajax/libs/react/15.4.2/react.min.js

✕
👁

https://cdnjs.cloudflare.com/ajax/libs/react/15.4.2/react-dom.min.js

✕
👁

Quick-add: ---

+ add another resource

Specifying the URLs is like specifying the physical address of a library (the real-world library) so you can go and grab the books and resources you need. Meaning, the URLs specify the *address* of existing JavaScript library that you want to use.

Although we are specifying this as a setting and just worrying about the HTML, CSS, and JS columns, Codepen does the following for us behind the scenes:

```

1 <body>
2
3 <script src="https://cdnjs.cloudflare.com/ajax/libs/react/15.4.2/react.min.js"><\
4 /script>
5
6 <script src="https://cdnjs.cloudflare.com/ajax/libs/react/15.4.2/react-dom.min.j\
7 s"></script>
8
9 </body>

```

In the code above, the URL is wrapped in a script tag at the end of the body so it can be used in the JavaScript files.

React is not the only external JavaScript resource that may need to be added. Let's say I wanted to make use of D3, a data visualization library for JavaScript. I would have to add a script tag like so (or Codepen would do this behind the scenes):

```
1 <script src="https://cdnjs.cloudflare.com/ajax/libs/react/15.4.2/react-dom.min.js"></script>
2
3
4 //other script tags here
```

Adding all of these script tags isn't very difficult to get wrap your mind around. However, there is one major issue.

You might be thinking that the issue is how one gets the URLs for the script tags. That's not the issue and can be easily done using [cdnjs](https://cdnjs.com)³¹. In addition, the documentation for most JavaScript libraries provide the URLs for you:

A code snippet showing a script tag for Vue.js: `<script src="https://unpkg.com/vue"></script>`. The text is displayed in a light blue monospace font on a light gray background. In the top right corner of the background, the word "HTML" is written in a small, gray, sans-serif font.

The issue is that you have to manually update the URLs every time a new version is released. In small projects, this isn't too terrible but in a large project, it becomes a nightmare.

Fortunately, there is a solution that will help manage all of our external JavaScript resources without worrying about constantly updating script tags.

The solution is a *package manager*.

> Package refers to a directory with files in it that contain the code for the JavaScript libraries.

You will often hear the word *modules* tossed around. Modules is a bit of a broader term than packages (more on that later).

For instance, if you went to the Github page for React, you will see that the library itself has a directory for all the code:

³¹<https://cdnjs.com/libraries>

The screenshot shows the GitHub repository for 'facebook/react'. At the top, it displays the repository name, a description: 'A declarative, efficient, and flexible JavaScript library for building user interfaces.', and a link to the React website. Below this, statistics are shown: 8,617 commits, 38 branches, 61 releases, 1,008 contributors, and the BSD-3-Clause license. A navigation bar includes links for Code, Issues (592), Pull requests (128), Projects (0), Wiki, and Insights. A yellow bar highlights the repository's activity. Below the bar, there are buttons for 'Branch: master', 'New pull request', 'Find file', and 'Clone or download'. The main content area shows a list of files and directories, each with a commit message and a timestamp. The files listed include .github, docs, eslint-rules, fixtures, flow, mocks, packages, scripts, src, .babelrc, .editorconfig, .eslintignore, .eslintrc.js, and .flowconfig.

File/Directory	Commit Message	Time Ago
.github	Update JSFiddle for issue template (#9289)	2 months ago
docs	Add ReactiveConf (#9723)	5 hours ago
eslint-rules	Make nodeType into constant for readability and reuse (#9113)	a month ago
fixtures	Remove invalid css from DOMFixtures (#9600)	5 days ago
flow	Show more meaningful stack trace for ReactNative errors (#9291)	2 months ago
mocks	Update tests to use plain JavaScript classes	a month ago
packages	16.0.0-alpha.12	20 days ago
scripts	Add test for key warning at top level (#9473)	21 hours ago
src	Expose ReactFiberTreeReflection in FB build (#9741)	20 hours ago
.babelrc	Convert current build system to Rollup and adopt flat bundles (#9327)	2 months ago
.editorconfig	Add insert_final_newline to editorconfig	2 years ago
.eslintignore	Add React benchmarking infrastructure (#9465)	13 days ago
.eslintrc.js	Fix lint rules	2 months ago
.flowconfig	Add React benchmarking infrastructure (#9465)	13 days ago

React Directory on GitHub

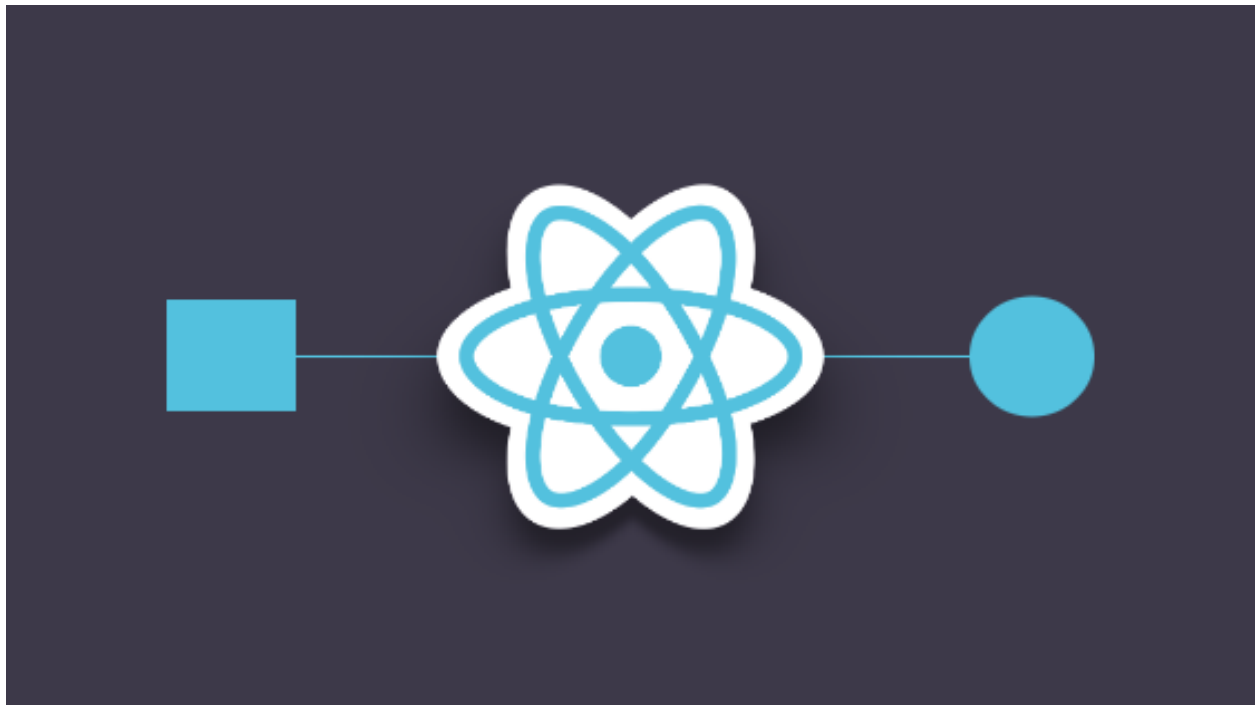
React Directory on GitHub

A *package manager* will help us install and maintain packages (containing a directory like shown above) in a project.

We will be using a package manager called *npm* (abbreviation for Node Package Manager).

We will dive more into npm and how to use it later. For now, you just need to understand that managing packages is one task that has to be handled for a JavaScript project in modern web development.

Module System



We just discussed the concept of modules (aka packages) and package managers.

In place of script tags, we are going to have to specify that we want to use the React module installed with our package manager. We do this by adding code for the *importing* of a module.

Later on, we are going to cover how we can break down our component hierarchy into folders and individual files within a project directory like so (Two dashes refer to a folder and one dash refers to a file):

```
1  -- Components
2      -- Villages
3          - village.js
4      -- Neighborhoods
5          - neighborhood.js
6      -- Blocks
7          - block.js
8      -- Houses
9          - house.js
```

Let's say for a house component file (house.js using the example above), we imported React and made a component like so:

```

1      class House extends React.Component {
2          state = {color: this.props.colors.lightGray};
3
4      render() {
5          return (
6              <div style={{background: this.state.color}} className="house">      </div>
7      v>
8          )
9      }
10     }

```

Given the component hierarchy according to The React Village Design, we will eventually need to nest this house component like so:

```

1      class Block extends React.Component {
2          render() {
3              return (
4                  <div className="box">
5                      <House colors={this.props.colors}/>
6                  </div>
7              )
8          }
9      }

```

If we've broken down the code of the House and Block components into separate files, then the Block class won't recognize the House component by default. Therefore, we have to write code for* exporting* the House component so then code can be written to* import* it wherever it needs to be used.

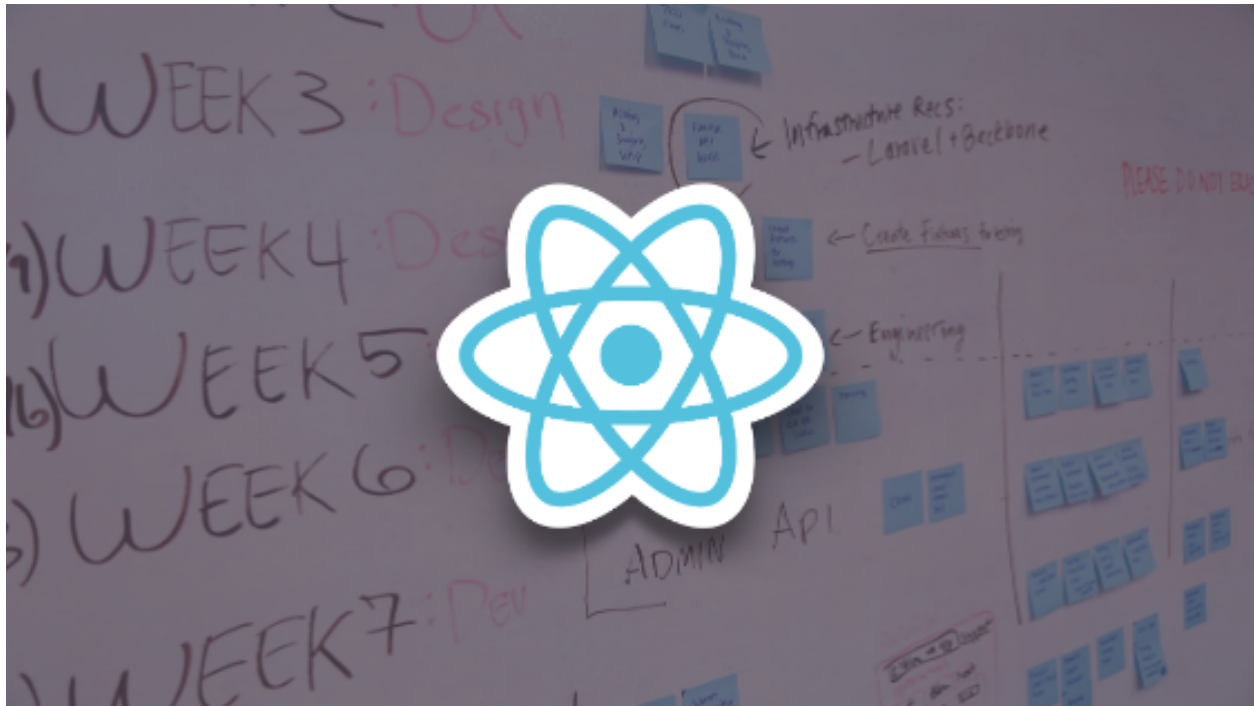
For this reason, modules is a broader term than packages. Modules can be thought of code that can be imported and exported in a project.

Package refers specifically to a directory with files in it that contain the code for the JavaScript libraries that we want to use in a project. Since packages can be installed with a package manager and imported (i.e. importing React), they are a type of modules. Modules also include code that we write that has to be imported and exported within a project (i.e. our React components).

In short, we need some sort of *module system* that allows us to write code that will import *modules* installed with our package manager (i.e. React). We also need to use the module system to write code to make the components *exportable* so that they can be used elsewhere (i.e. making a house exportable so it can be imported and nested in a block's class).

Using a module system is another task to be handled in a JavaScript project in modern development. ES6 has its own module system that we will use in our project. Like our package manager, we will get into the specifics later.

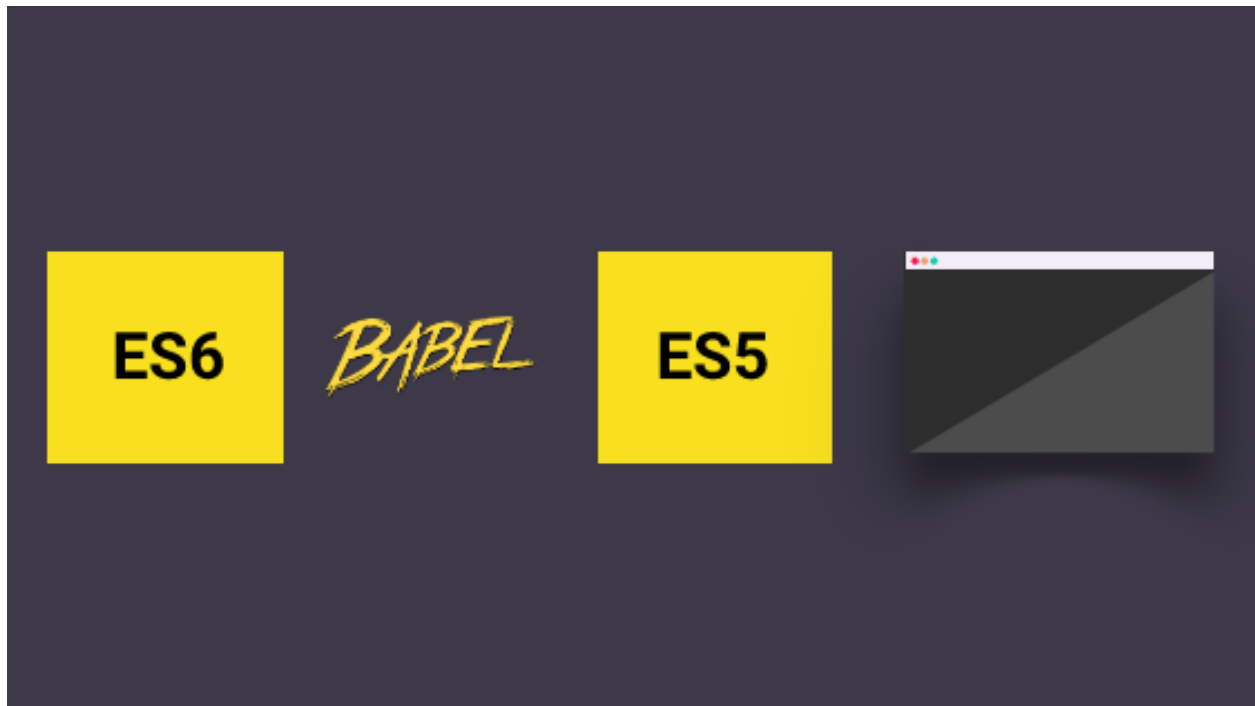
Preprocessing



Preprocessing sounds like a scary concept, but we have already discussed this.

In Chapter 3, we discussed the difference between ES6 and ES5. We mentioned that ES6 is a more updated syntax of the ECMAScript language which is the standardized language implemented into JavaScript than ES5.

ES6 provides cool features that are needed to write good React code, such as our ES6 component classes. Babel also will allow us to use ES6 as a module system. However, browser support isn't quite there like it is for ES5. Therefore, we brought Babel into the discussion.



In short, Babel takes ES6 code and converts it to ES5 so the browser can process it. This is an example of another task called *preprocessing*.

Now, we were using Babel in the context of Codepen. Because of that, setup was very simple:

Pen Settings

HTML CSS **JavaScript** Behavior Screenshot

JavaScript Preprocessor

Babel 

Add External JavaScript

These scripts will run in this order and before the code in the JavaScript editor. You can also link to another Pen here, and it will run the JavaScript from it. Also try typing the name of any popular library.

- 

https://cdnjs.cloudflare.com/ajax/libs/react/15.4.2/react.min.js


- 

https://cdnjs.cloudflare.com/ajax/libs/react/15.4.2/react-dom.min.js



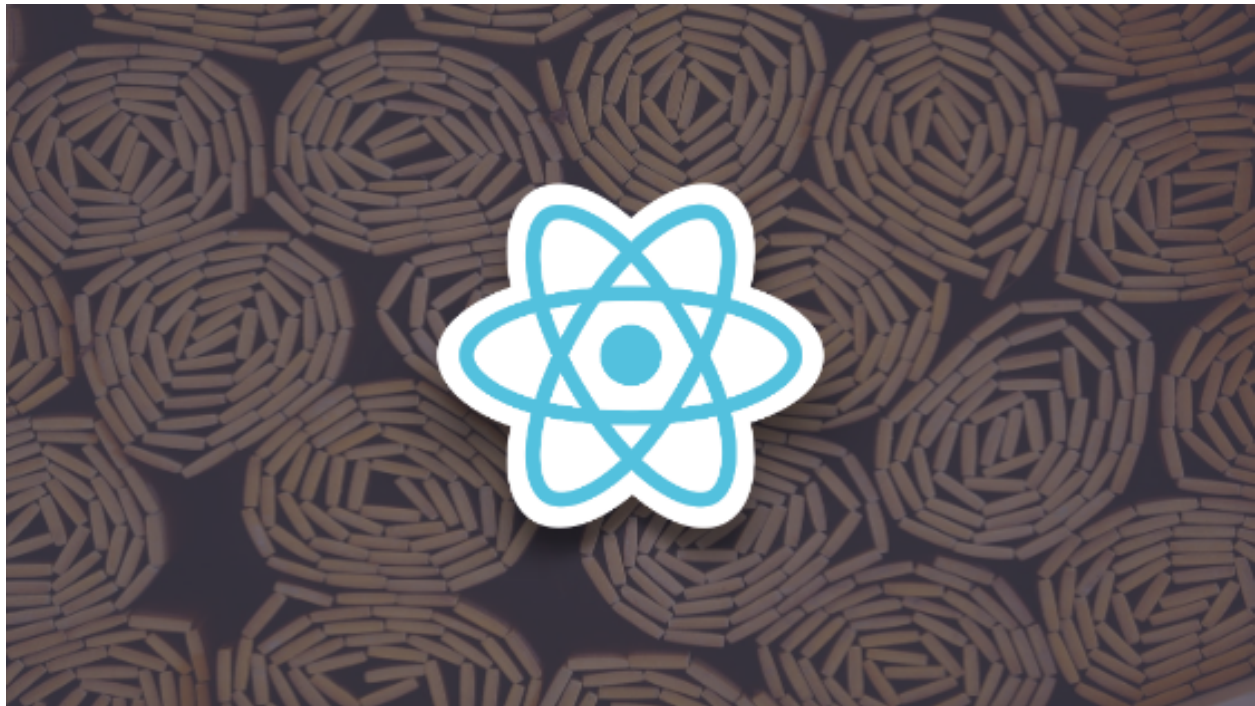
Quick-add: 

[+ add another resource](#)

We just specified Babel as the JavaScript preprocessor from a drop down menu and Codepen did the work for us.

Later on, we will discuss using Babel in the context of our own project.

Bundling



I have already mentioned that we are going to be breaking down our React project into individual folders and files. For example, we will organize our component hierarchy in a project directory like so:

```
1 -- Components
2   -- Villages
3     - village.js
4   -- Neighborhoods
5     - neighborhood.js
6   -- Blocks
7     - block.js
8   -- Houses
9     - house.js
```

We also mentioned that we will use a *module system* so we can export a component in one file, import it into another, and then make use of it.

For example, we could create a house component and make it exportable. Then, we could import it into a file containing our block component and nest it.

Working our way up the component hierarchy, we will have neighborhoods imported into a file for our village. The village will then be made exportable and imported into a file called *index.js*.

index.js will contain all of the villages. Remember, there are many villages in a web application. For instance, we could have a login page, sign up page, landing page, etc. We will then be able to toggle on and off which village is currently displaying (rendered) in our single page application.

index.js contains all the villages, the villages contain their neighborhoods, the neighborhoods contain their blocks, and blocks contain their houses. Point being, index.js will contain all of our code even if all of the code isn't visible in that file since it's broken down into modules (individual files with code that can be imported and exported).

All of this code is using ES6, so we mentioned that we need to use Babel in the preprocessing task. Well, we need to carry out another task called bundling.

Bundling refers to taking our index.js (the top of the hierarchy for our code) and specifying the preprocessing that we want to be done (Babel in our case). It then spits out a single file with the final code that can be processed in browsers.

We will be using a tool called Webpack to handle the bundling task.

Study Break



To summarize the previous section, here are the tasks in the JavaScript ecosystem:

Managing of packages

Using a module system

Preprocessing our JavaScript code

Bundling a single file

Additionally, we mentioned the tools needed to complete these tasks:

npm

ES6

Babel

Webpack

With this out of the way, we can actually get started building our project. As we do this, we will see how we to organize our project directory and make use of all our tools so the tasks get accomplished.

At this point, I have tried to explain things as best as I possibly could. With that being said, there is no way to really avoid the overwhelming nature of all these new terminologies. If you are feeling confused, this is totally normal. That doesn't mean you are a pseudo-developer who can't make it in the real world like I thought to myself. It just means you're figuring out: "Wow, no wonder people have a hard time learning React!".

Take a deep breath or a break if you want, then we can dive into things. I will continue to try to be as practical as possible as we go through things.

Installations

Make sure to install the following (if you haven't already):

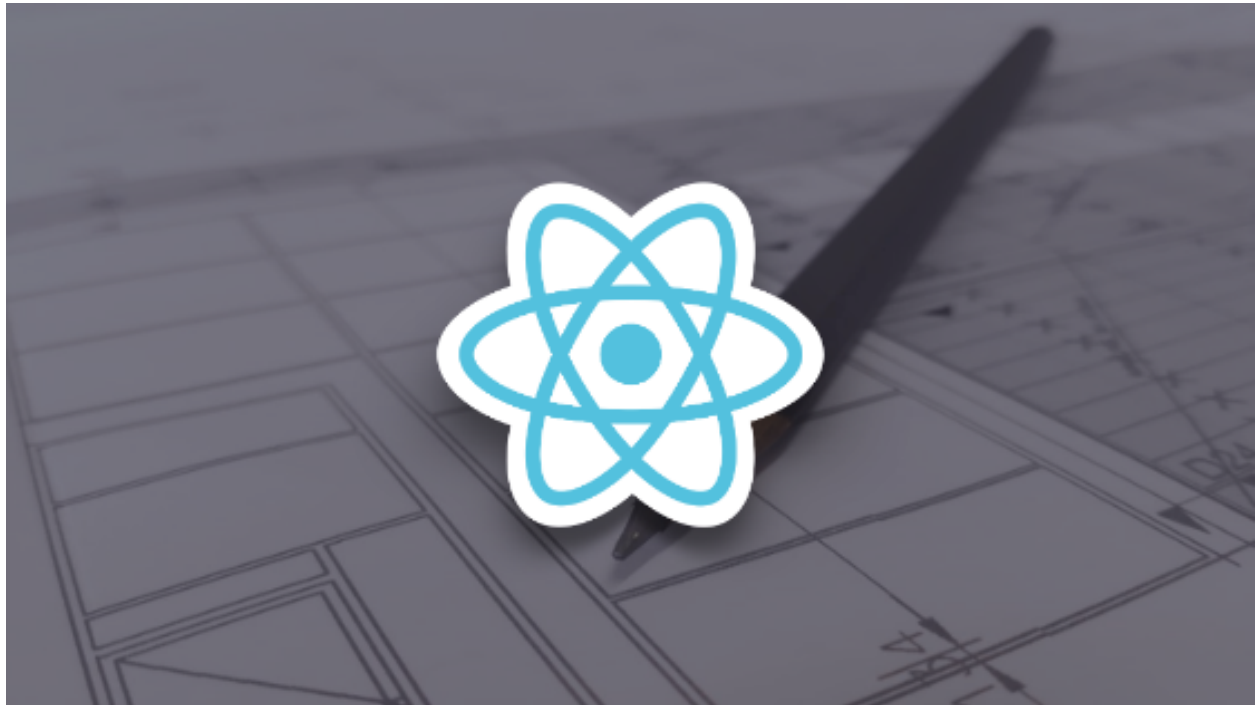
Node.js³²

Atom³³

³²<https://nodejs.org/en/>

³³<https://atom.io/>

Building Our First Project



Creating Our Project Folder

First things first, we need to make a new project folder called **react-landing-page** and have it as the current directory in command line.

If you are not familiar with command line, here are the steps:

1. Open command prompt (Windows) or terminal (Mac)
2. Type “**cd Desktop**”. This will *change directory* (cd) to your Desktop. Essentially, we are entering into your Desktop so we can run commands that will apply there.
3. Type “**mkdir react-landing-page**”. This will **make directory* *(mkdir) on your Desktop. In other words, it will create a new folder called **react-landing-page** on your Desktop.
4. Next, type “**cd react-landing-page**”. We have now entered into the newly created directory where we can run commands.

We have now created our project folder for our project called react-landing-page and entered into it from command line. We will now run commands to set up our package manager.

Initializing Our Package Manager

Once we have done all that, we can type the following:

```
npm init
```

Let's break this down.

Remember, npm stands for Node Package Manager. It is the tool for completing the task of managing our packages.

npm is a command line tool. Therefore, we can run npm commands by doing “npm ____”.

In the example above, we do *npm init*.

npm specifies that we are going to run a npm command.

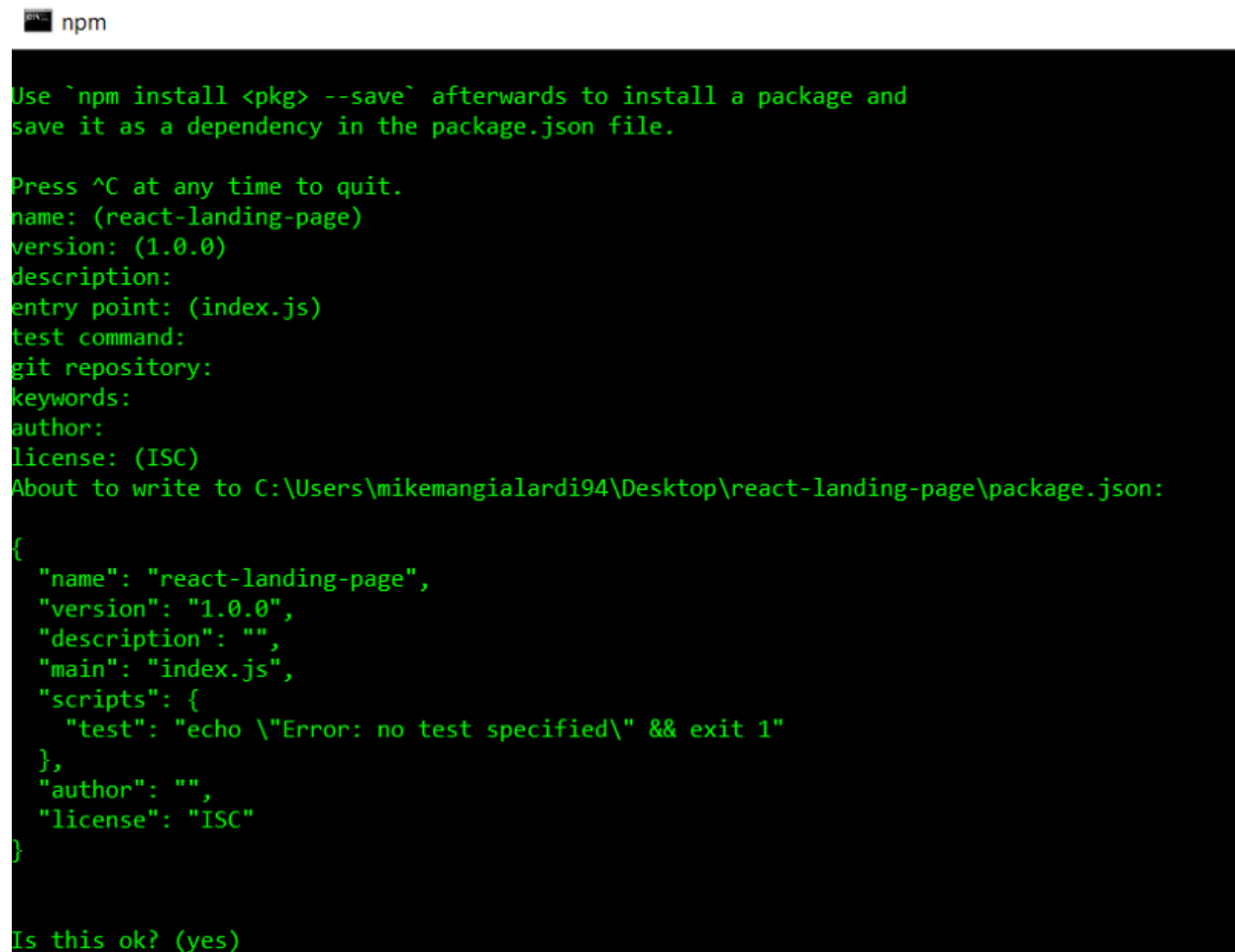
init is a particular npm command that will initialize the package manager for our project.

Looking at your command line, we should now see the following prompt:

```
name: (react-landing-page) _
```

This is the first prompt of several that npm init causes. These prompts just allow us to update the meta data. Meta data just refers to basic information about the project.

Go ahead and hit enter until you get prompted with the following:



```
npm

Use `npm install <pkg> --save` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
name: (react-landing-page)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
About to write to C:\Users\mikemangialardi94\Desktop\react-landing-page\package.json:

{
  "name": "react-landing-page",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}

Is this ok? (yes)
```

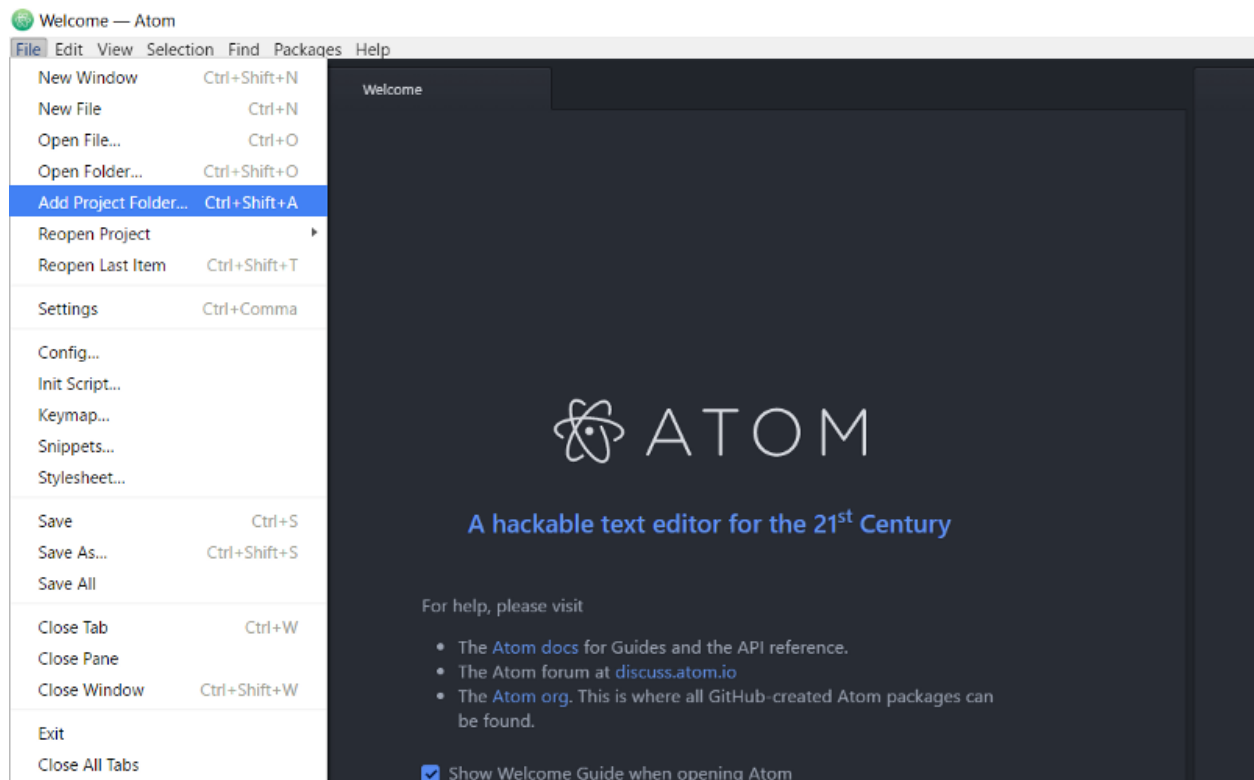
Go ahead and type “yes”.

After you do so, the command will finish.

So, what just happened?

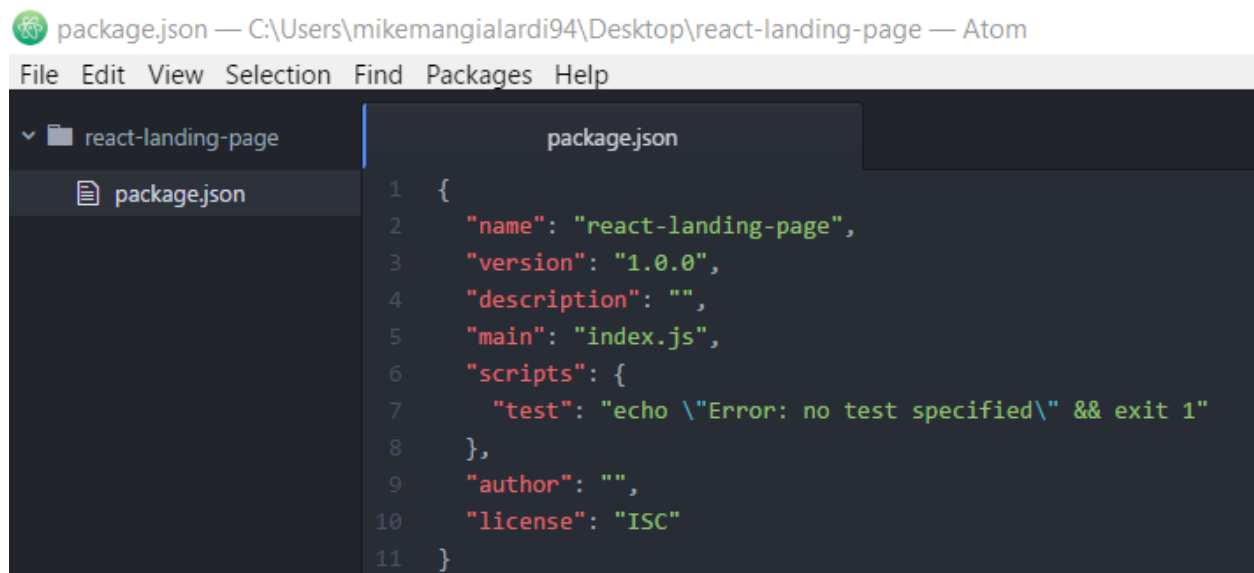
Let’s go ahead and see for ourselves.

Open Atom (a text editor that was a prerequisite to install) and select **File** → **Add Project Folder**:



Browse to your Desktop and select **react-landing-page** and click **Select Folder**.

We should now see a new file that was created by npm init called *package.json*:



Here, we have the meta data that was populated as we hit enter through the prompts caused by npm init represented in JSON format.

JSON stands for JavaScript Object Notation and it meant to be a readable way for structuring data.

The meta data is pretty straightforward with two exceptions, *main* and *scripts*.

main is the entry point of the application. Our entry point is specified as `index.js` which we mentioned before. In terms of our React single page applications, this is going to be the file with the code at the top of the hierarchy. Meaning, it is the single page that contains all of the villages (views) that make up the application. Again, examples of villages include a landing page, sign in page, etc. In our project, `index.js` will just include our landing page village.

scripts refer to scripts that you can have run at various times in the lifecycle of your package. This will be used when we get to our Webpack configuration.

Installing Packages as Dependencies

Cool! We've gotten started with initializing our package manager.

Next, we need to install the packages that will be used for our application. To start, let's install React (we have called it React Core but is officially called React) and React DOM.

To install a package that is necessary for our application to run, we can use the syntax:

```
npm install --save *insert package name here*
```

For React, we can do the following:

```
npm install --save react
```

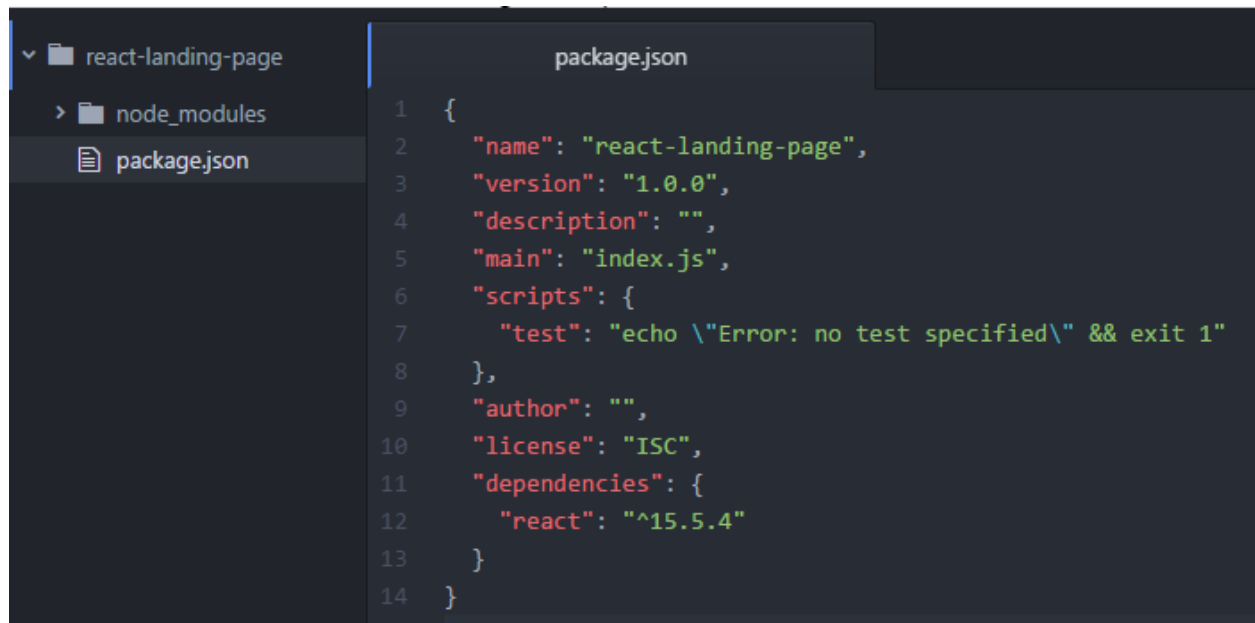
We should see the following once this completes:

```
~ -- react@15.5.4
  +-- fbjs@0.8.12
    | +-- core-js@1.2.7
    | +-- isomorphic-fetch@2.2.1
    | | +-- node-fetch@1.6.3
    | | | +-- encoding@0.1.12
    | | | | `-- iconv-lite@0.4.17
    | | | `-- is-stream@1.1.0
    | | `-- whatwg-fetch@2.0.3
    | +-- promise@7.1.1
    | | `-- asap@2.0.5
    | +-- setimmediate@1.0.5
    | `-- ua-parser-js@0.7.12
  +-- loose-envify@1.3.1
  | `-- js-tokens@3.0.1
  +-- object-assign@4.1.1
  `-- prop-types@15.5.10

npm WARN react-landing-page@1.0.0 No description
npm WARN react-landing-page@1.0.0 No repository field.
```

So...what is all of this?

Well, let's take a look at our project in Atom again.

A screenshot of a code editor interface. On the left, a file explorer shows a project named 'react-landing-page' with subfolders 'node_modules' and 'package.json'. The 'package.json' file is selected. On the right, the content of 'package.json' is displayed in a dark-themed editor with syntax highlighting. The JSON object includes fields for name, version, description, main, scripts, author, license, and dependencies. The 'scripts' field has a 'test' script. The 'dependencies' field lists 'react' as a dependency.

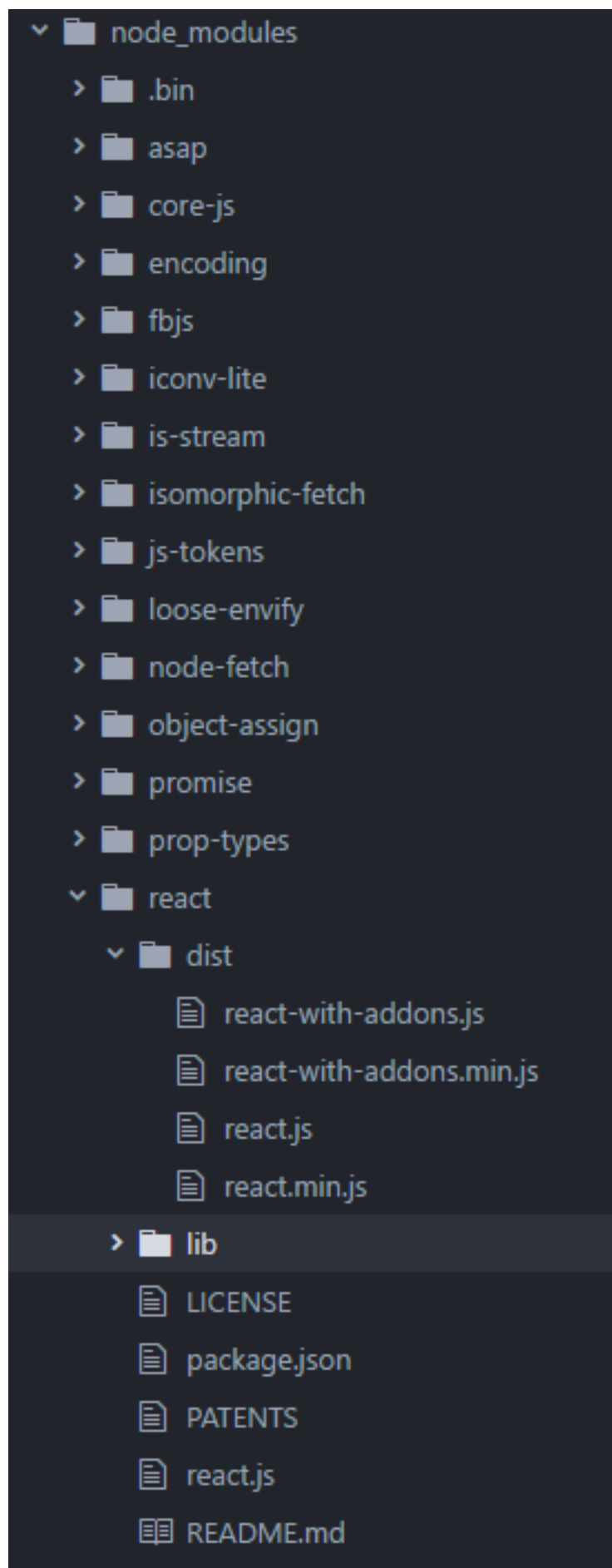
```
1  {
2    "name": "react-landing-page",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "author": "",
10   "license": "ISC",
11   "dependencies": {
12     "react": "^15.5.4"
13   }
14 }
```

Woah! There's a new folder called *node_modules* and the following in our *package.json* file:

```
1   "dependencies": {
2     "react": "^15.5.4"
3   }
```

Our *package.json* has added an object called *dependencies* which contains the name and version of all of the packages that have been installed in our application that our project is dependent on to run.

Let's expand the new *node_modules* folder.



Here, we can see all the packages that have been installed.

You might be thinking: “Wait a second, we only installed one package for React. Why are there so many packages?”

When we install React, it also installs all of the dependencies that are required for React to run. Therefore, we have all the packages also under *node_modules*. If you are curious, you can check the *package.json* file under the *react* folder in *node_modules* to see what each dependency is.

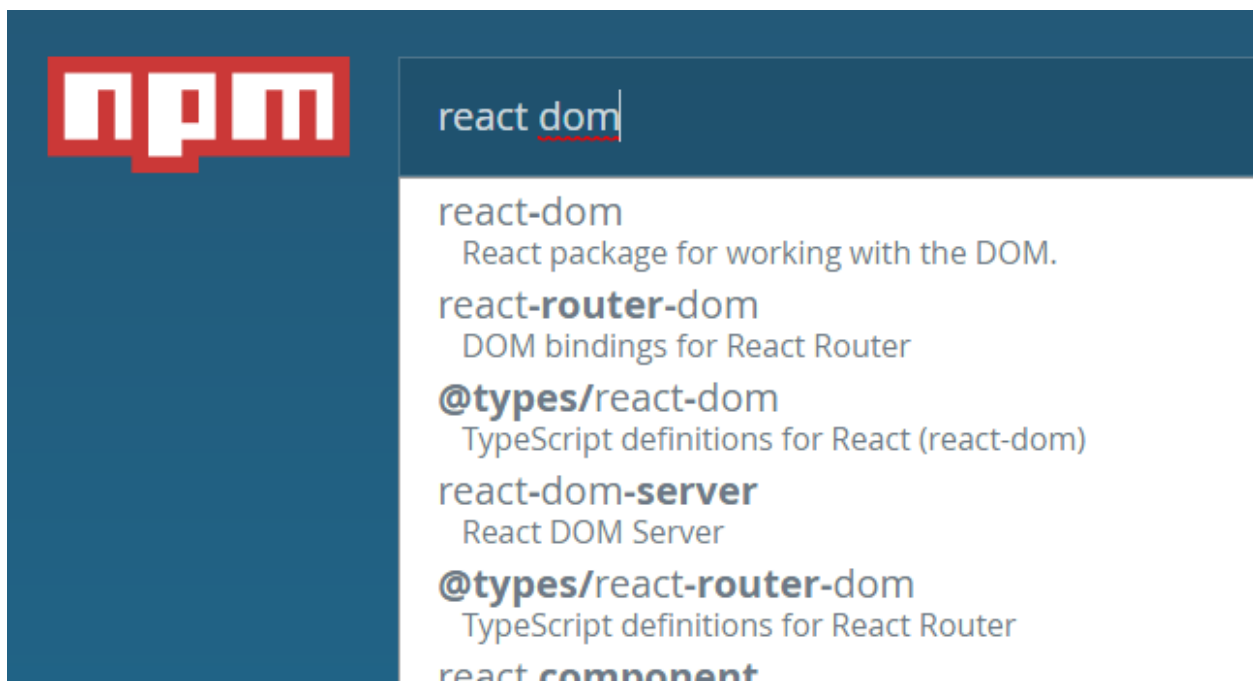
Sweet! We’ve installed our first package using npm. Now, it’s time to install React DOM.

React DOM is two words. So, what exactly do we do?

```
1 npm install --save reactdom or npm install --save react-dom
```

This brings up an important point. We can [find all the packages that are available using npm here](https://www.npmjs.com/)³⁴.

Let’s search with “react dom” as the keywords:



There it is right on top of the list! Go ahead and click on it.

We are brought to a page with some information and documentation:

³⁴<https://www.npmjs.com/>

The screenshot shows the npm website for the `react-dom` package. The header includes the npm logo and a search bar. The main content area displays the package name `react-dom` with a 'public' badge. Below this, a description states: 'This package serves as the entry point of the DOM-related rendering paths. It is intended to be paired with the isomorphic React, which will be shipped as `react` to npm.' The 'Installation' section shows the command `npm install react react-dom`. The 'Usage' section includes a code snippet for using React and ReactDOM in a browser. On the right side, there is a sidebar with a 'Unleash awesomeness' section, a download button for `npm install react-dom`, and a list of collaborators.

On the right-hand side, we can see the naming for the npm install command:

```
npm install react-dom
```

Note that we will edit this slightly:

```
npm install --save react-dom
```

Adding the save indicates that this is a package that will be *necessary* for our application to run.

Go ahead and run that command.

When it is finished, we will see the following:

```
~ -- react-dom@15.5.4
npm WARN react-landing-page@1.0.0 No description
npm WARN react-landing-page@1.0.0 No repository field.
```

This message in the command line is much shorter since it did not have any other packages that it was dependent on.

The React DOM package has now been added under `node_modules` and has been specified as a dependency in our `package.json`:

```
1  "dependencies": {  
2    "react": "^15.5.4",  
3    "react-dom": "^15.5.4"  
4  }
```

Pretty simple! We will shortly look at how to *use* these packages in our code. Before that, however, let's go ahead and set up our project directory.

Setting Up Our Project Directory

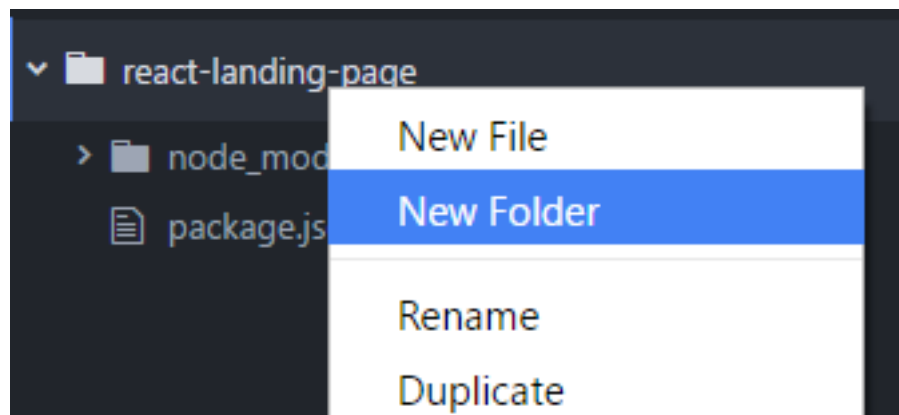
How we organize our project directory is ultimately gray area. However, this is how I prefer to approach it:

```
1  -- react-landing-page  
2  -- node_modules  
3  -- components  
4  -- villages  
5  -- neighborhoods  
6  -- blocks  
7  -- houses  
8  -- styles  
9  - index.js  
10 - index.html  
11 - package.json
```

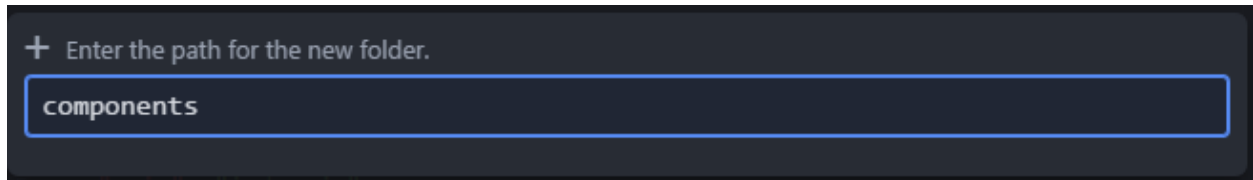
All of the above are folders with exception to package.json. Indentation is used to denote the hierarchy. For more complex projects, we can add more folders. Let's keep it simple like this for now.

Let's go through this together so you get the feel for it.

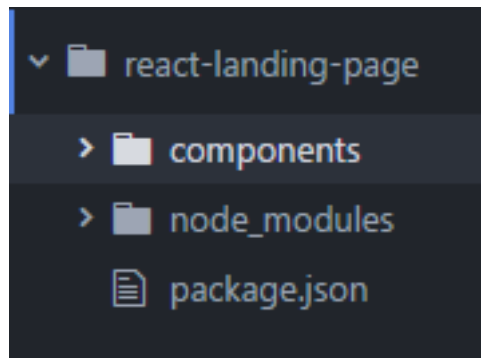
Right click **react-landing-page** and select **New Folder**:



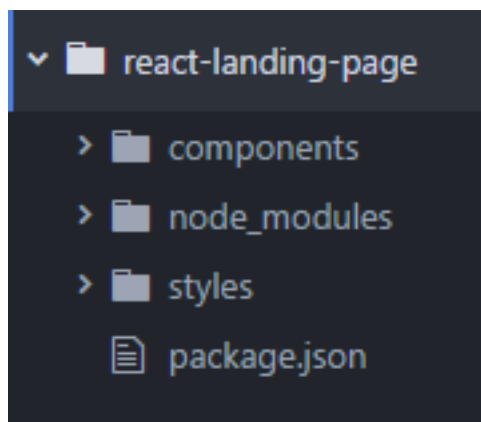
Name it **components** and hit enter:



This has created a folder called *components* that is right under *react-landing-page*:



Let's repeat that exact process so we can create a folder called **styles** right under *react-landing-page*.

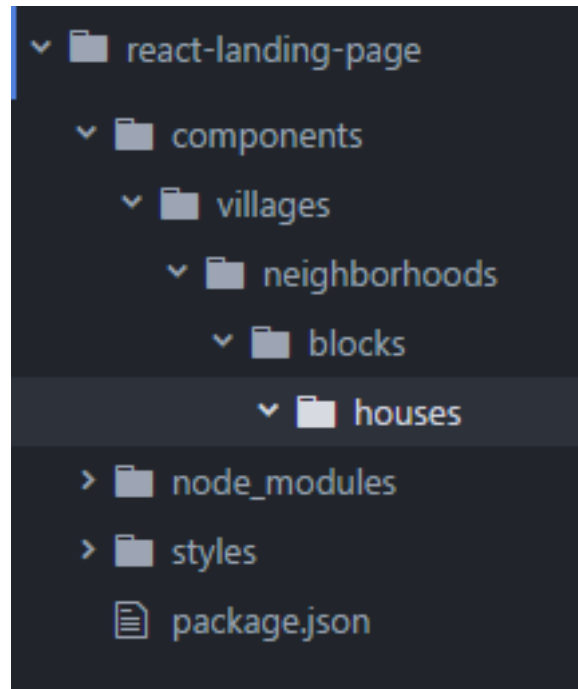


styles will contain all of our CSS files.

components will contain a folder for each level of the hierarchy in the village design (and their components in separate files).

Let's repeat the same process to create a folder for the top level of the component hierarchy which is **villages**. The only difference is that you have to right click the *components* folder and then select **New Folder**:

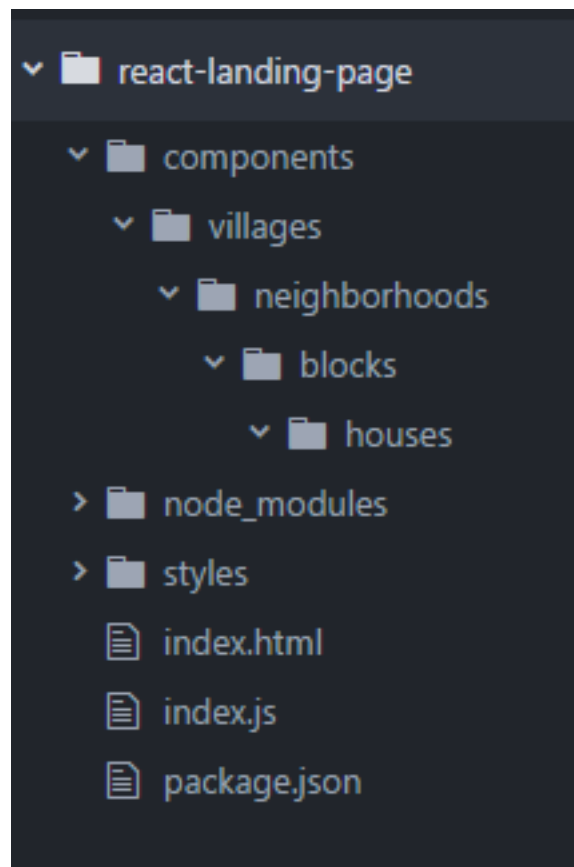
By now, you should have a hang of how to create folders in Atom. Go ahead and finish off the rest of the folders within components until we have the following:



Perfect! Let's go ahead and add our **index.js** file. To add this file, right click **react-landing-page**, select **New File**, name it, and hit enter.

Repeat the same process to add a new file called **index.html**.

Our project directory at this point should be:



Sweet! We are now ready to create our first file where we will learn how to *import* our packages and do some cool stuff with React.

Creating Our First File

Double click on *index.js* in Atom so we can add some code.

When we created our Koala in Codepen, the highest component in our hierarchy was a village component which we named Koala:

```
1  class Koala extends React.Component {
2    render() {
3      return (
4        <Box colors={this.props.colors}/>
5      )
6    }
7  }
```

As we have discussed a few times now, our *index.js* will contain a collection of multiple villages (views) that make up our single page application.

For that reason, let's create a new React class to define a component which we will call **App**:

```
1 class App extends React.Component {
2   render() {
3     return (
4       <div>Hello React via a project!</div>
5     )
6   }
7 }
```

Since we are just setting up our project in this chapter, let's just return a simple "Hello React via a project!" instead of nesting a village like we will eventually do.

Like before, we want the highest component in our hierarchy to be rendered using React DOM. Therefore, we also have to add:

```
1 ReactDOM.render(
2   <App/>,
3   document.getElementById('app')
4 );
```

In the code above, we specified the target for our app in the HTML to be a div element with an id of app.

Let's quickly add the HTML code in index.html:

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="utf-8">
5     <title>React Landing Page</title>
6   </head>
7   <body>
8     <div id="app">
9
10    </div>
11  </body>
12 </html>
```

We have written our React code. There is nothing new going on here.

Next, let's add the new code for *importing* the React and ReactDOM that has been installed and neatly added under node_modules.

The syntax for importing a module so it can be used in our file is:

```
1 import Name from 'insert name found in npm install ____';
```

Therefore, we do the following for React:

```
import React from 'react';
```

And React DOM...

```
import ReactDOM from 'react-dom';
```

Note that the capitalization/spelling for the variables do matter so it matches how we use it in the code:

```
1 class App extends React.Component {
2   render() {
3     return (
4       <div>Hello React via a project!</div>
5     )
6   }
7 }
8
9 ReactDOM.render(
10   <App/>,
11   document.getElementById('app')
12 );
```

Cool beans! Go ahead and make sure index.js and index.html is saved.

Configuring Webpack and Babel

Earlier, we mentioned that Babel is going to convert our ES6 JavaScript to ES5.

Webpack is the bundler that says: “Give me your entry file and anything you need for preprocessing. Oh, you need Babel? No problem! I’ll send you a final file that has everything converted for your browser to handle.”

To get these to work, there are quite a few things we have to install via npm. Fortunately, it’s all been documented well [here](https://github.com/babel/babel-loader)³⁵.

Run the following in command line:

```
1 npm install --save-dev babel-loader babel-core babel-preset-env babel-preset-react
2 webpack webpack-dev-server html-webpack-plugin
```

³⁵<https://github.com/babel/babel-loader>

Now, there's a slight difference in this npm install command. We have:

```
npm install --save-dev
```

This is used to specify *devDependencies* as distinguished from just *dependencies* (this is official npm terminology).

dependencies refer to packages required (imported) by our application.

devDependencies refer to packages for development and testing.

Think of it this way. React and ReactDOM were imported (via a require) and used to make our application. Therefore, they are *dependencies*.

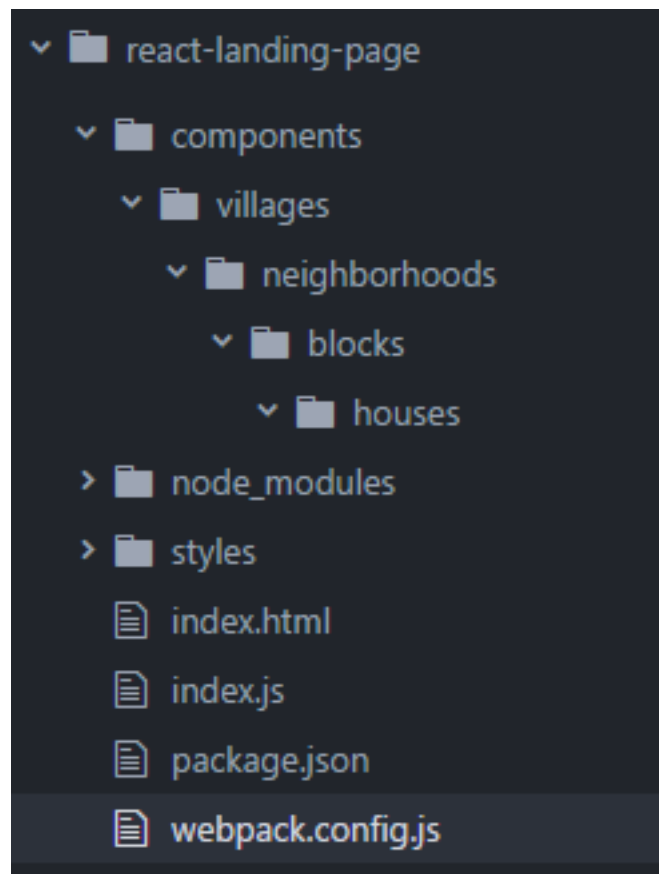
Babel and Webpack are just assisting in taking our application and making it compatible with browsers. They are not used to write the code to make our application. Therefore, they are *devDependencies*.

We can check our package.json to see these newly installed devDependencies:

```
1  "devDependencies": {  
2    "babel-core": "^6.24.1",  
3    "babel-loader": "^7.0.0",  
4    "babel-preset-env": "^1.5.1",  
5    "babel-preset-react": "^6.24.1",  
6    "html-webpack-plugin": "^2.28.0",  
7    "webpack": "^2.6.0",  
8    "webpack-dev-server": "^2.4.5"  
9  }
```

With these installed, we need to write the code so Webpack take our code, have Babel do it's thing, and spit out the final file we need.

Let's add another file right under react-landing-page called **webpack.config.js**.



Next, open the file and let's add some of the Webpack configuration code:

```
1  module.exports = {  
2    entry: './index.js',  
3    output: {  
4      filename: 'bundle.js'  
5    }  
6  }
```

In a lot of these chapters, I have gone into great detail to explain concepts and code that is introduced. With that being said, the Webpack code is something that I don't have memorized and I don't expect you to either. We are just going to cover discuss the main things that are going on. Once we have one working example, we will be good going forward.

In our code, we have an *entry* file (index.js) and the *output* file once Babel does the conversion (bundle.js).

Now, we need to add some rules that say: "Ok. You've got the entry file and output file configure. I want you to find every JavaScript file and apply Babel".

```
1  module.exports = {
2    entry: './index.js',
3    output: {
4      filename: 'bundle.js'
5    },
6    module: {
7      rules: [
8        {
9          test: /\.js$/,
10         exclude: /(node_modules)/,
11         use: {
12           loader: 'babel-loader',
13           options: {
14             presets: ['env', 'react']
15           }
16         }
17       ]
18     }
19   }
20 }
```

test specifies the type of files that need to have some preprocessing applied. In our case, we want our JavaScript files to be preprocessed using Babel since they will be using ES6.

exclude is specifying the exceptions to our test. We don't want Babel to touch any of the packages installed via npm so we exclude everything in the `node_modules` folder.

use specifies what is doing the preprocessing (loader) and the options (options) that we may want to specify.

Our *loader* is `babel_loader`. This just says use Babel for the preprocessing of our JavaScript files.

Under *options*, we specify two presets, *env* and *react*.

env is used to take generic JavaScript code using ES6 and make it compatible with browsers. *react* is used to take React code that uses ES6 (our component classes) and make them compatible with browsers.

In short, all the code we have so far is saying: "Hey Webpack! Take our JavaScript code and use Babel to make it compatible. Just a heads up, we need generic JavaScript code that uses ES6 code to work as well as React code that uses ES6."

The final step for completing our `webpack.config.js` file is to use a Webpack plugin that we installed via npm called *html-webpack-plugin*. This plugin will automatically populate our `index.html` file and inject a script tag into our HTML that contains the outputted JavaScript file.

Let's write the code to setup this plugin in our configuration file:

```
1  const HtmlWebpackPlugin = require('html-webpack-plugin');
2
3  const HtmlWebpackPluginConfig = new HtmlWebpackPlugin({
4    template: './index.html',
5    filename: 'index.html',
6    inject: 'body'
7  })
8
9  module.exports = {
10    entry: './index.js',
11    output: {
12      filename: 'bundle.js'
13    },
14    module: {
15      rules: [
16        {
17          test: /\.js$/,
18          exclude: /(node_modules)/,
19          use: {
20            loader: 'babel-loader',
21            options: {
22              presets: ['env', 'react']
23            }
24          }
25        }
26      ]
27    },
28    plugins: [HtmlWebpackPluginConfig]
29  }
```

In the added code, we import the plugin from `node_modules` and create a variable with all of our settings. We are simply saying: “Go to our `index.html` file and inject the script tag in the body.” We then specify the plugin settings at the end of the `module.exports{...}`.

The last piece of the puzzle is to add a script to our `package.json` file. We will add the following script:

```
1  "scripts": {
2    "start": "webpack-dev-server"
3  },
```

start is just the name of our script that we can run via `npm`. *webpack-dev-server* is what is fired on the start of our script. This just fires up a server that hosts our application on a local host address.

Now, we can run the script in command line:

```
npm start
```

We get the following:

```
Project is running at http://localhost:8080/
webpack output is served from /
Hash: 5e675ca7a92125bb99e0
Version: webpack 2.6.0
Time: 2097ms

   Asset      Size  Chunks             Chunk Names
bundle.js    1.05 MB      0 [emitted] [big]  main
index.html   233 bytes             [emitted]
chunk {0} bundle.js (main) 1.01 MB [entry] [rendered]
  [26] ./~/react/lib/React.js 3.32 kB {0} [built]
 [114] ./index.js 2.34 kB {0} [built]
 [115] (webpack)-dev-server/client?http://localhost:8080 5.68 kB {0} [built]
 [134] ./~/html-entities/index.js 231 bytes {0} [built]
 [140] ./~/punycode/punycode.js 14.7 kB {0} [built]
 [143] ./~/querystring-es3/index.js 127 bytes {0} [built]
 [144] ./~/react-dom/index.js 59 bytes {0} [built]
 [228] ./~/react/react.js 56 bytes {0} [built]
 [256] ./~/strip-ansi/index.js 161 bytes {0} [built]
 [258] ./~/url/url.js 23.3 kB {0} [built]
 [259] ./~/url/util.js 314 bytes {0} [built]
 [260] (webpack)-dev-server/client/overlay.js 3.73 kB {0} [built]
 [261] (webpack)-dev-server/client/socket.js 897 bytes {0} [built]
 [263] (webpack)/hot/emitter.js 77 bytes {0} [built]
 [264] multi (webpack)-dev-server/client?http://localhost:8080 ./index.js 40 bytes {0} [built]
      + 250 hidden modules
Child html-webpack-plugin for "index.html":
   chunk    {0} index.html 541 kB [entry] [rendered]
    [0] ./~/lodash/lodash.js 540 kB {0} [built]
    [1] ./~/html-webpack-plugin/lib/loader.js!./index.html 556 bytes {0} [built]
    [2] (webpack)/buildin/global.js 509 bytes {0} [built]
    [3] (webpack)/buildin/module.js 517 bytes {0} [built]
webpack: Compiled successfully.
```

You can then go to the localhost address where the application is running:

<http://localhost:8080/>³⁶

This will display the simple React component we created in index.js:

³⁶<http://localhost:8080/>



Finally!

Final Code:

Available on GitHub under Chapter 6³⁷

Concluding Thoughts

This definitely was a difficult chapter to write. It certainly is the hardest (and least visual) part about creating a React application.

Fortunately, it is a painful process that only really needs to be done once. Once you go through configuring a project for the first time, you will have a template for all other React projects going forward. Additionally, a lot of the code (especially Webpack) is something you only really need to get the gist of and at least understand the core concepts behind the code. It is not something that you will repeat over and over that you need to master everything like our React code.

This may have come across as a boring chapter. However, it is this topic that literally caused me to give up on learning React over a year ago. It is something that is so crucial to understand or else you will be caught in a never ending cycle of “fake it to make it”.

Great job. Pat yourself on the back. You *are* a developer, don't worry.

Now that we have our first project built from scratch, it's time to have some fun. Next up, we get to the fun part of making our first user interface via React.

Glossary

Codepen: CodePen is a playground for the front end side of the web. We can use it to write React code without dealing with the “behind the scenes” work.

DOM: The Document Object Model (DOM) is a programming interface for HTML and XML documents. It provides a structured representation of the document and it defines a way that the structure can be accessed from programs so that they can change the document structure, style and content. The DOM provides a representation of the document as a structured group of nodes

³⁷<https://github.com/michaelmang/React-for-the-Visual-Learner>

and objects that have properties and methods. Essentially, it connects web pages to scripts or programming languages.

React: A JavaScript library for building user interfaces.

React Village Design: Terminology to describe the hierarchy of React components.

ECMAScript: ECMAScript is a language standardized by ECMA International and overseen by the TC39 committee that is implemented in JavaScript.

ES5: The 5th edition of ECMAScript, standardized in 2009. This standard has been implemented fairly completely in all modern browsers.

ECMAScript 6 (ES6)/ ECMAScript 2015 (ES2015): The 6th edition of ECMAScript, standardized in 2015. This standard has been partially implemented in most modern browsers

Babel: Compiles ES6 code to ES5 so it can be processed by browsers. It can be applied using Webpack.

JSX: A JavaScript syntax extension used to specify what we want our component to render as.

React Core: The entire React library is divided into two separate libraries, React and ReactDOM. I find this to be poor naming convention so I refer to the separate core React library (as distinguished from ReactDOM) as React Core.

ReactDOM: The entire React library is divided into two separate libraries, React and ReactDOM. ReactDOM is the glue between React components and the DOM. It allows you to mount React components to HTML elements.

Package: A package is just a directory with one or more files in it, that also has a file called “package.json” with some metadata about this package.

[[src](https://docs.npmjs.com/getting-started/what-is-npm)]

Modules: JavaScript code that can be imported and exported for use throughout separate files.

Module System: ES6 code used to make our JavaScript code from a file exportable and import it from other files for use.

Webpack: A module bundler. In our project, we use it to take our JavaScript code (both generic and React) and apply Babel. It ultimately injects a new file in our HTML that is compatible with browsers.

Webpack Dev Server: Fires a server that will host our application. It is set to run in our project when we run “npm start”.

Node Package Manager (npm): Used to manage packages that can be used in our project. A full selection of available packages is available [here](https://www.npmjs.com/)³⁸.

Package.json: A JSON file that represents important data of our project. It is automatically created via npm init.

Dependencies: Refers to dependencies that are needed to write the code of our application. Example: React

³⁸<https://www.npmjs.com/>

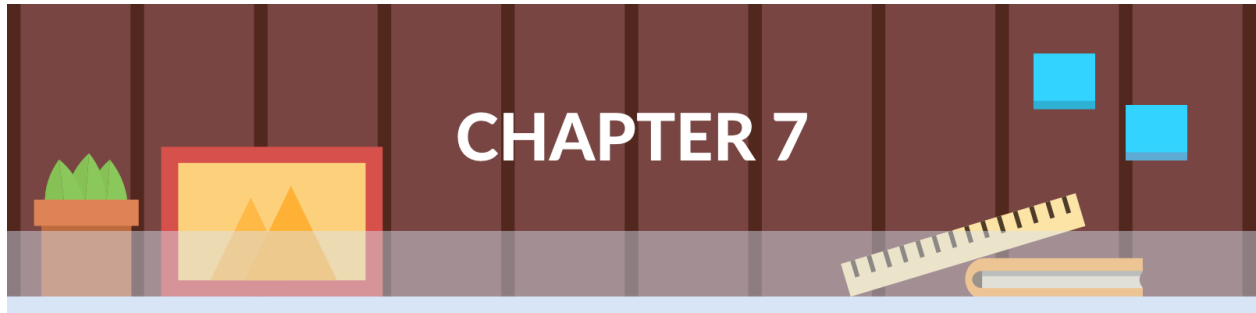
DevDependencies: Refers to dependencies needed for the build (development and testing). Example: Webpack

Preprocessing: In our project, this refers to the process of converting ES6 code to ES5 via Babel.

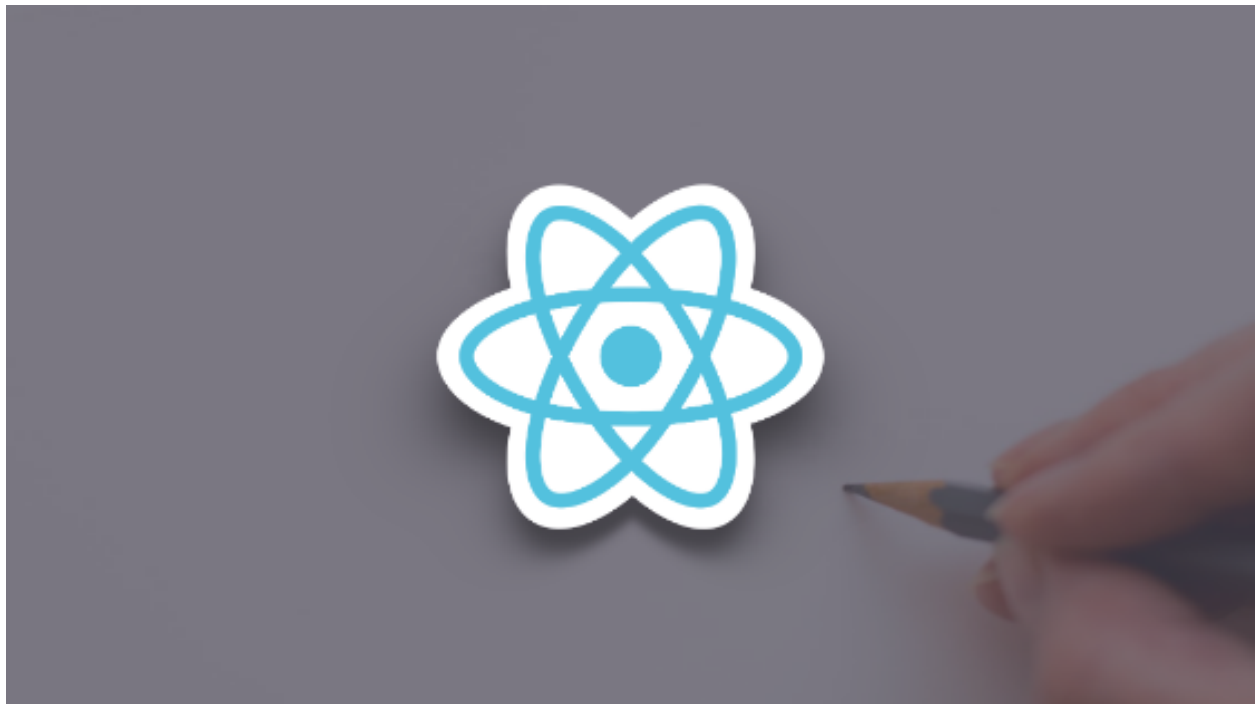
Atom: A text editor made by Github.

Bundling: Bundling refers to taking our index.js (the top of the hierarchy for our code) and specifying the preprocessing that we want to be done (Babel in our case). It then spits out a single file with the final code that can be processed in browsers.

Chapter 7 | Building Our First User Interface



Breaking Down the Mockup

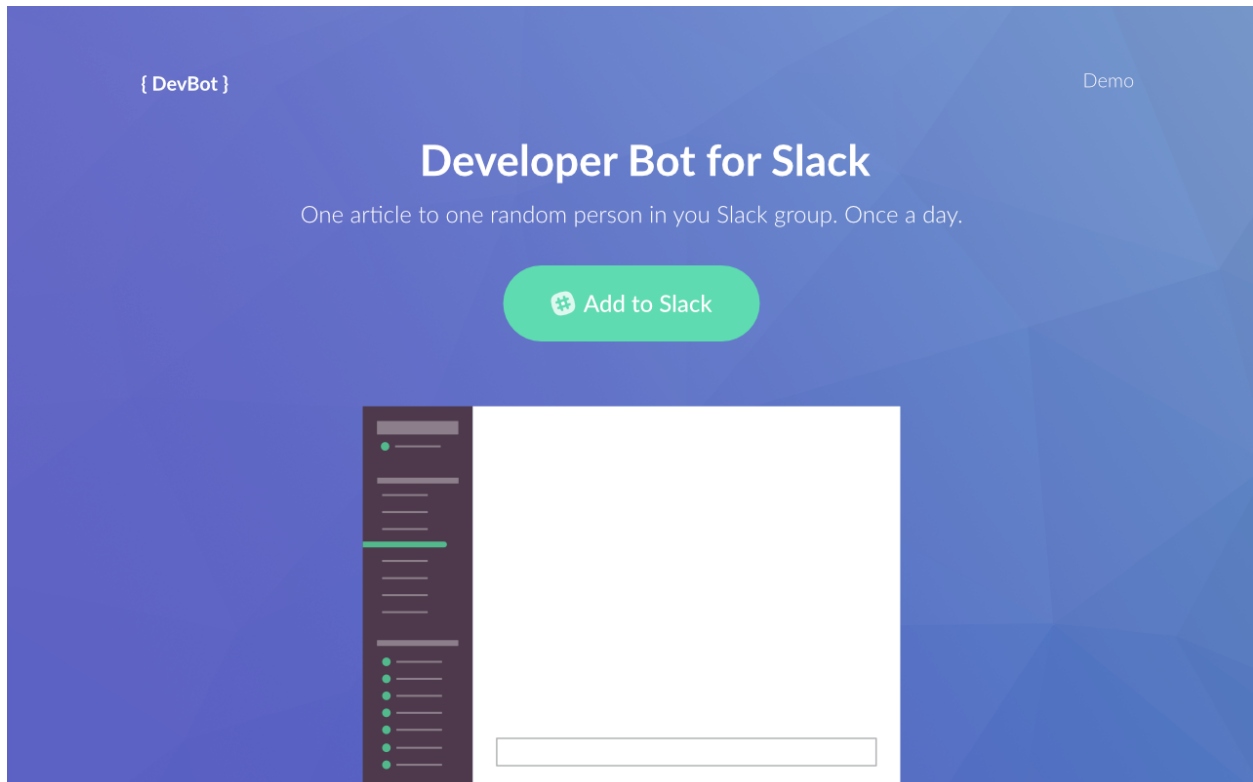


The previous chapter was a lot of setup and I know you are eager to dive into creating a user interface. So, let's jump right into it!

In this chapter, we are creating a landing page for our user interface. Personally, I am sort of a hybrid

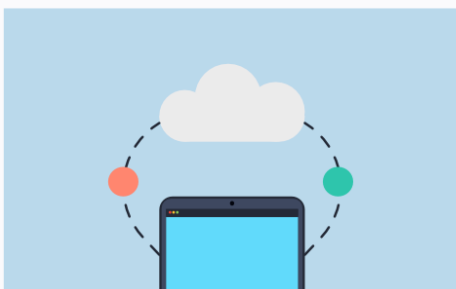
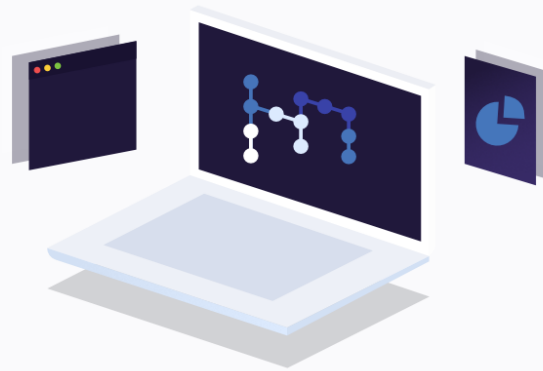
between a designer and developer which allows me to create complete mockups with vector graphics as well as the implementation of it with code.

Because of that, I would love to go through the entire process of creating a mockup for a landing page. However, for the sake of time and getting the hand of a more of a real-world application (since most jobs won't allow you to wear too many hats), we are just going to work from a preexisting mockup that I have created:



DevBot analyzes popular articles

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent volutpat, elit pulvinar aliquam convallis, libero mauris rhoncus erat, non porttitor lectus justo et ex. In hac habitasse platea dictumst.



DevBot sends the perfect article

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent volutpat, elit pulvinar aliquam convallis, libero mauris rhoncus erat, non porttitor lectus justo et ex. In hac habitasse platea dictumst.

Let's get in the mindset that this is a mockup created by a designer that we work with. They handed it to us and we were told to implement it with React.

What do we do first?

As we did with our Koala, we must start by breaking this landing page down into our component hierarchy as specified by The React Village Design.

Village

A village is a complete *view* of a user interface. View can be thought of as what you would normally associate as an entire *page* in more traditional web applications. Because we are making a single page application, we distinguish our terminology and call the current “page” that is toggled on as the view/village.

In our project, the entire landing page is the village. We can create a village component called *LandingPage*.

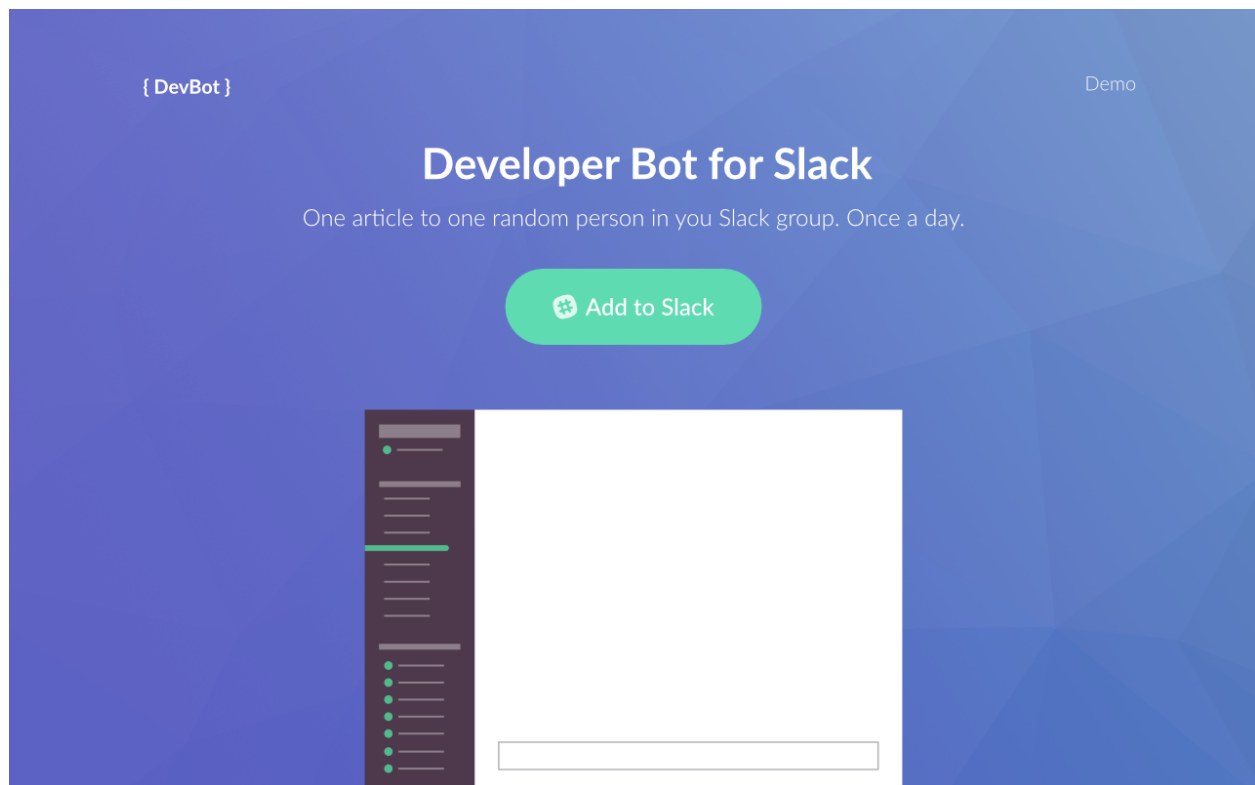
Neighborhoods

The next level down on the component hierarchy of the village design is neighborhoods.

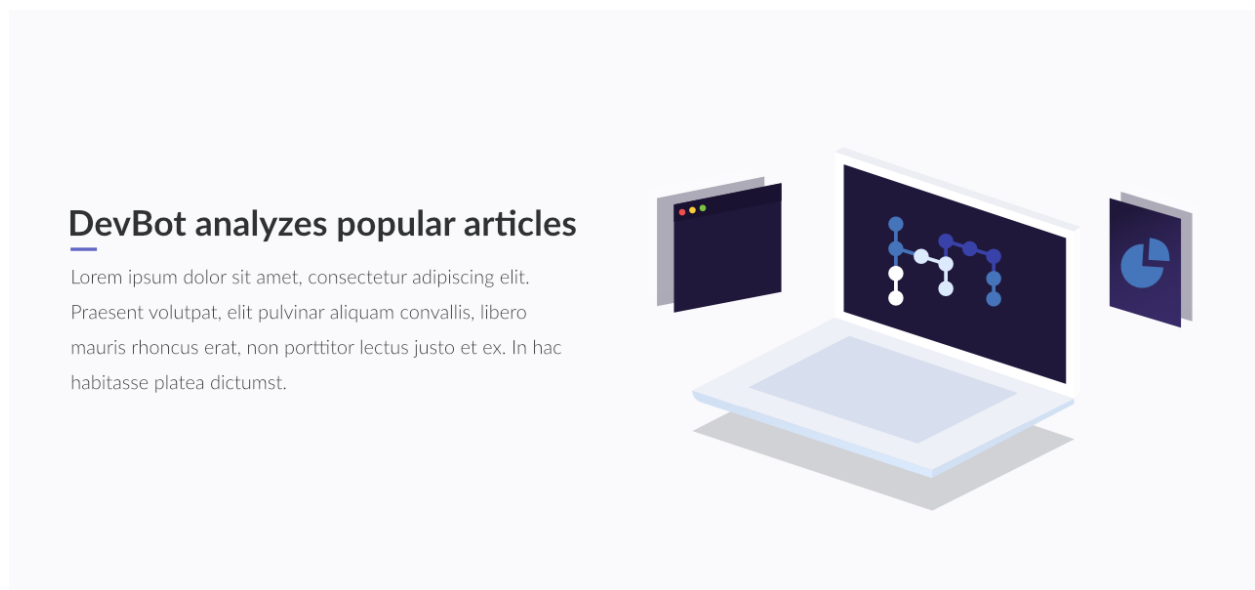
Neighborhoods in a user interface are collections of blocks that work together to form a complete *section* of the view.

In our project we have the following neighborhoods (sections):

Header



Section A



Section B



DevBot sends the perfect article

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent volutpat, elit pulvinar aliquam convallis, libero mauris rhoncus erat, non porttitor lectus justo et ex. In hac habitasse platea dictumst.

Section C

Developer Bot for Slack

One article to one random person in you Slack group. Once a day.

Add to Slack

Footer

{ DevBot }

Created with coffee by Mike Mangialardi

Demo

Blocks

Next, we go down one more level in the component hierarchy in terms of the village design, blocks.

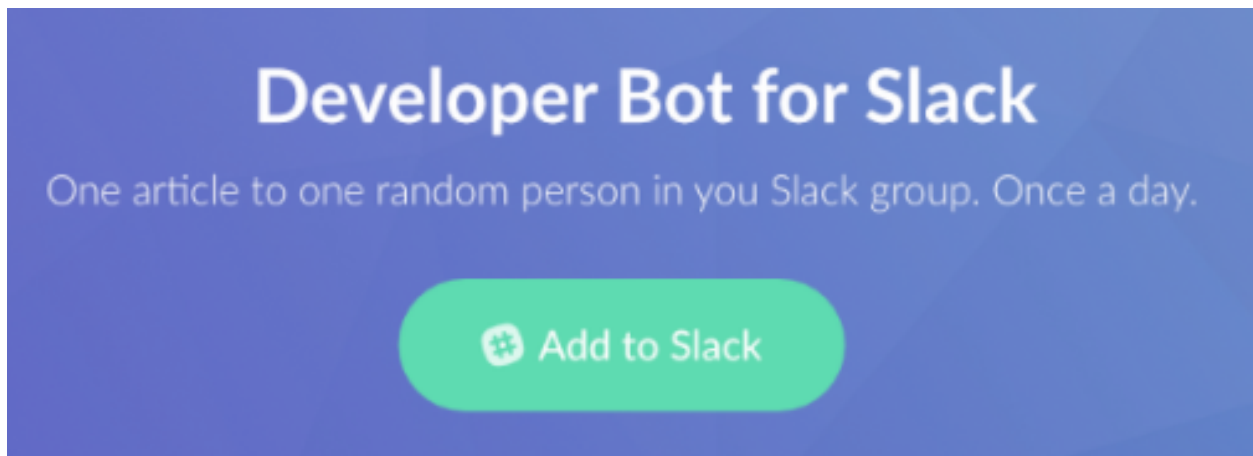
Blocks in a *user interface* are collections of houses that work together to form a specific functionality/purpose to the view.

In our project, we have the following blocks:

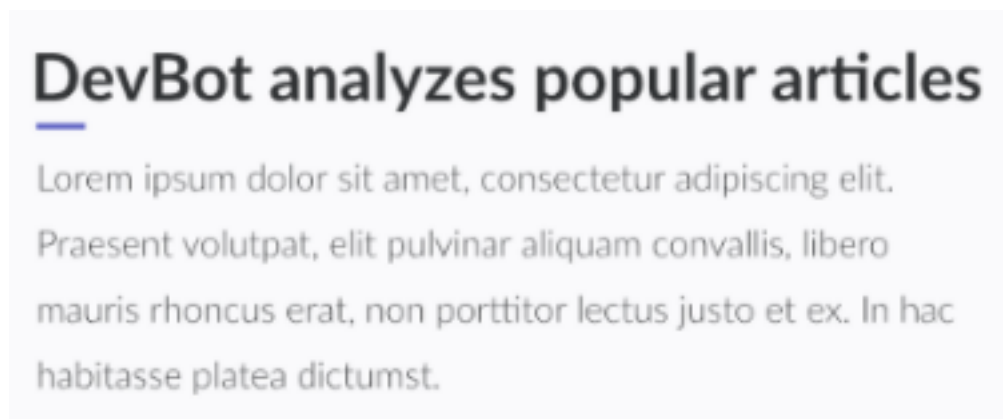
Navigation



Call to Action



Section Description



Houses

Finally, we have *houses*. Houses are the basic building blocks of a user interface. Some of our houses will be combined into a block (multiple components that form one functionality/purpose) and other houses will stand-alone (one component that is not working with other components to form one functionality/purpose). Let's look at all of our houses and distinguish between those that will be combined in blocks and those that will stand-alone:

Houses in Blocks

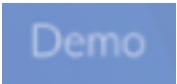
–

Navigation Block

Branding

A blue rectangular block containing the text "{ DevBot }" in white, representing a branding element.

Demo Link

A light blue rectangular button with the word "Demo" in white, representing a demo link.

Call to Action Block

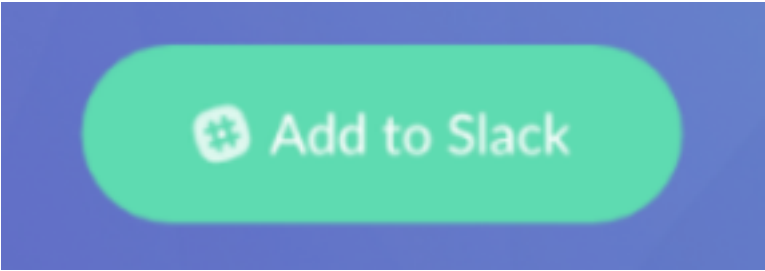
Title

A dark blue rectangular block containing the text "Developer Bot for Slack" in white, representing the title of the call to action.

Subtitle

A dark blue rectangular block containing the text "One article to one random person in you Slack group. Once a day." in white, representing the subtitle.

Button

A green rounded rectangular button with a white Slack logo icon and the text "Add to Slack" in white, representing the call to action button.

Section Description Block

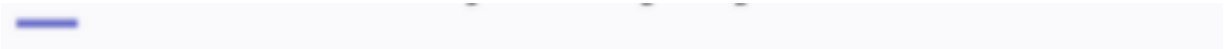
Heading

A light gray rectangular block containing the text "DevBot analyzes popular articles" in dark gray, representing the heading for the section description.

Paragraph

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent volutpat, elit pulvinar aliquam convallis, libero mauris rhoncus erat, non porttitor lectus justo et ex. In hac habitasse platea dictumst.

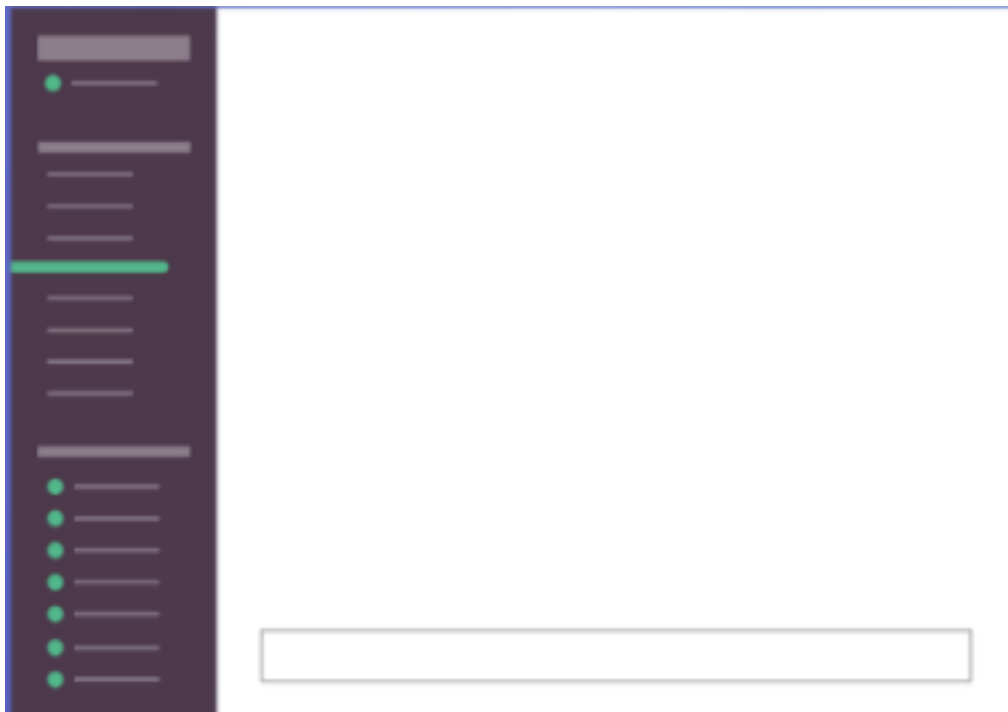
Divider



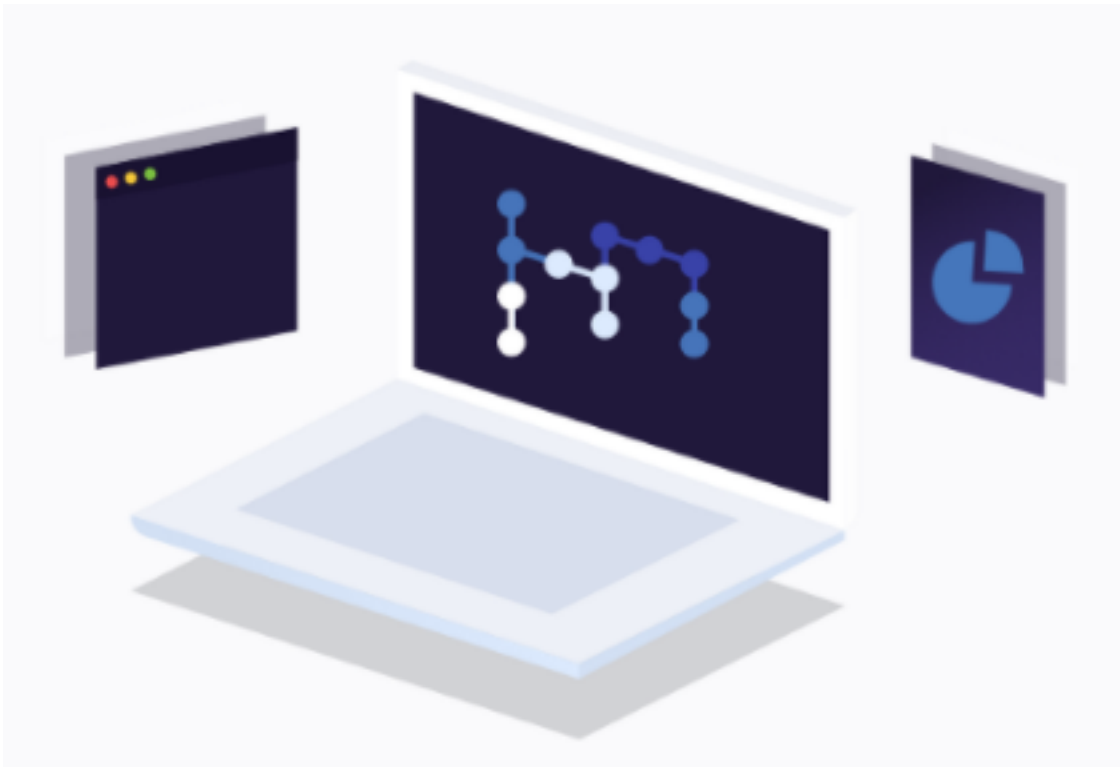
Stand-Alone Houses

–

Slack Channel



Laptop



Article Cover



These houses will stand-alone because they aren't being combined to form any blocks. Therefore, they can be nested right under the neighborhoods (sections) to which they belong.

Putting It All Together

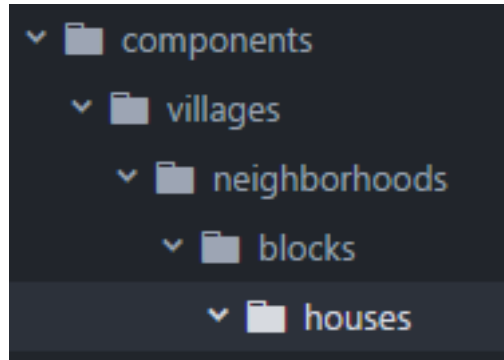
Cool! We've been able to break down our landing page into proper component hierarchy. Let's put all of these together so we understand how the nesting will work:

```
1      - App
2
3      - Landing Page (Village)
4
5          - Header (Neighborhood)
6              - Navigation (Block)
7                  - Branding (House)
8                  - Demo Link (House)
9              - Call To Action (Block)
10                  - Title (House)
11                  - Subtitle (House)
12                  - Button (House)
13                  - Slack Channel (House)
14
15          - Section A (Neighborhood)
16              - Section Description (Block)
17                  - Heading (House)
18                  - Divider (House)
19                  - Paragraph (House)
20              - Laptop (House)
21
22          - Section B(Neighborhood)
23              - Article Cover (House)
24              - Section Description (Block)
25                  - Heading (House)
26                  - Divider (House)
27                  - Paragraph (House)
28
29          - Section C (Neighborhood)
30              - Call To Action (Block)
31                  - Title (House)
32                  - Subtitle (House)
33                  - Button (House)
34
35          - Footer (Neighborhood)
36              - Navigation (Block)
37                  - Branding (House)
38                  - Demo Link (House)
```

Note I have put the components that will be used multiple times in bold.

Alright, sweet!

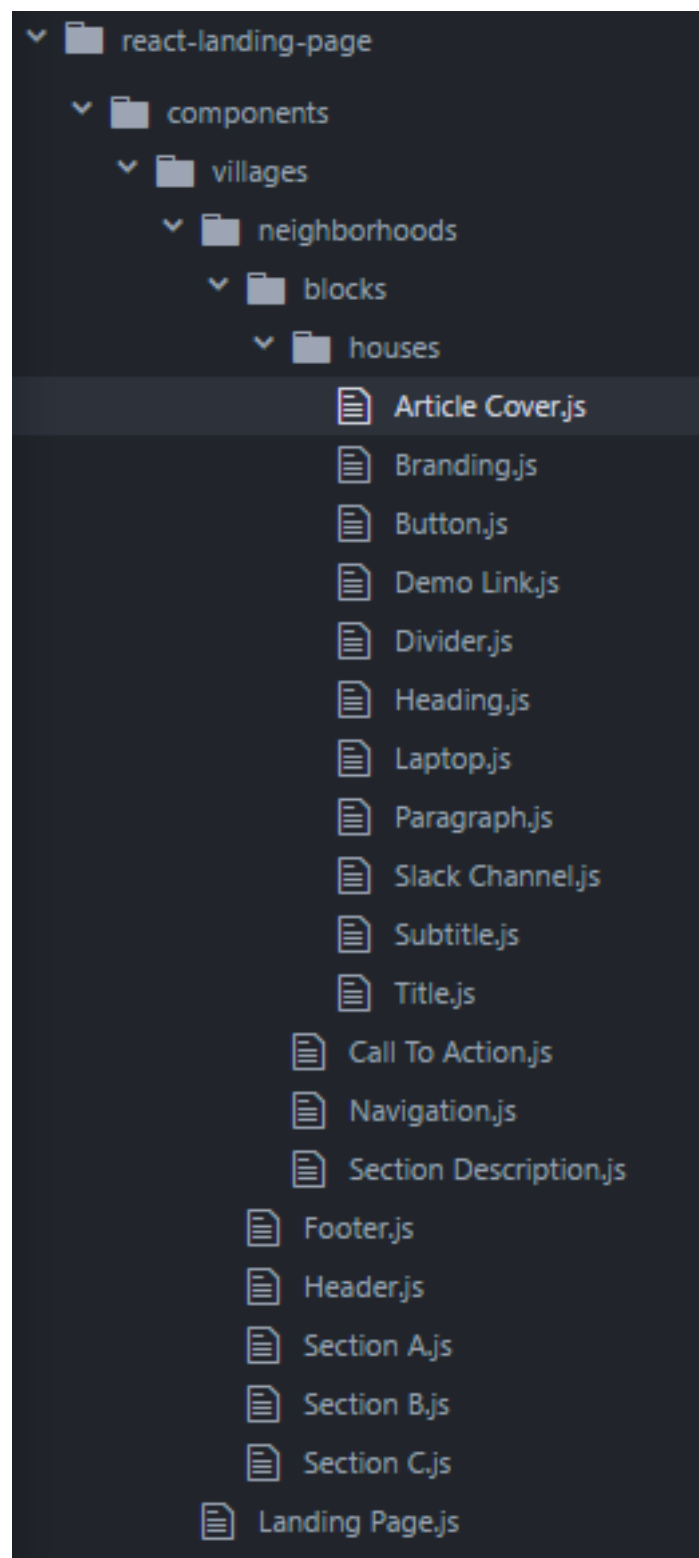
Next, let's open our project folder in Atom and take another look at the components folders we created:



Let's add files into the components folders to reflect the component hierarchy that we just put together. For practice, see if you can do this yourself and create files in our project according to the proper hierarchy. When you are done, you can check with mine to see if you have done it correctly and adjust as needed.

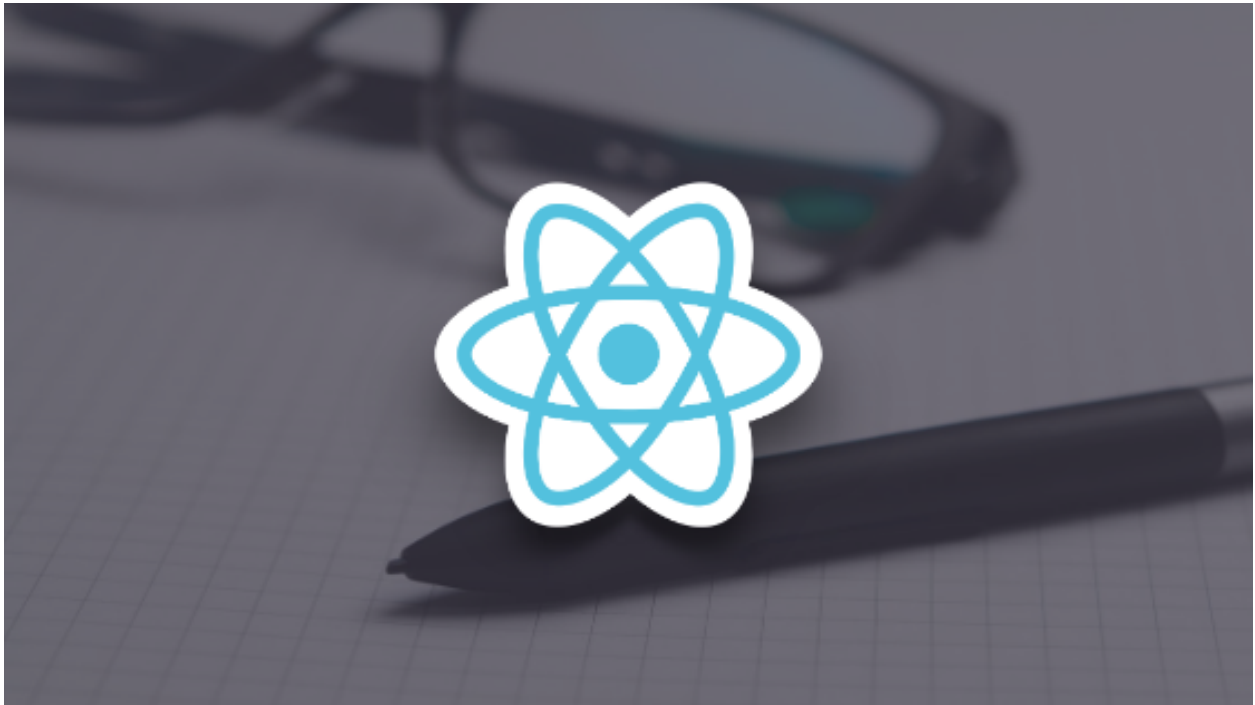
Note: Keep the naming conventions as I have them when we put everything together. Meaning, we should keep spaces and capitalization of every work. Also, all files should end with `.js` (i.e. Call To Action.js).

Here is what we should have when we finish:



Cool beans. Go ahead and close any tabs that are open in Atom.

Understanding the Layout



Before we start creating the components, we need to understand how we would create this landing page using just HTML. That way, we can break the HTML into each of our individual components.

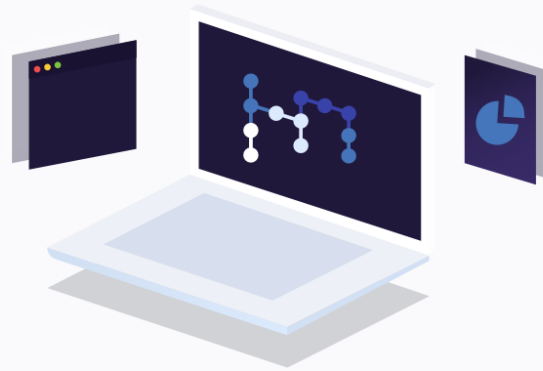
Rows and Columns

The main thing we need to understand is how can we achieve a proper layout for our user interface in HTML. When thinking about a proper layout, we need to be able to break down a user interface into *rows* and *columns*.

Let's take a look at one of our sections:

DevBot analyzes popular articles

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent volutpat, elit pulvinar aliquam convallis, libero mauris rhoncus erat, non porttitor lectus justo et ex. In hac habitasse platea dictumst.



How is this section laid out in terms of rows and columns?

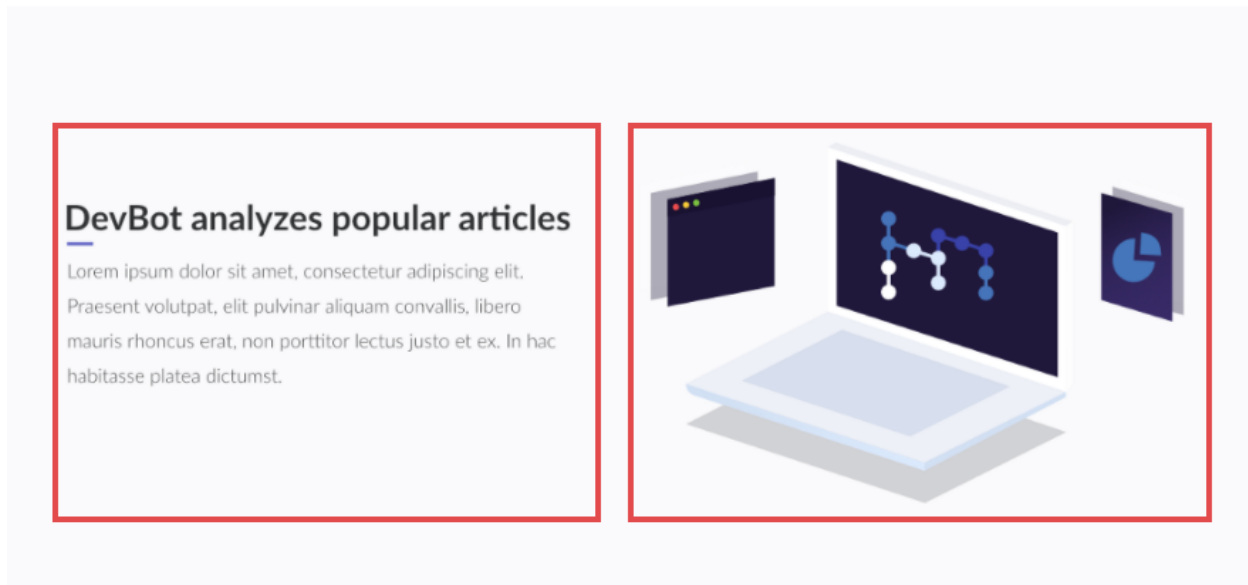
Well, we only have 1 *row* of content:

DevBot analyzes popular articles

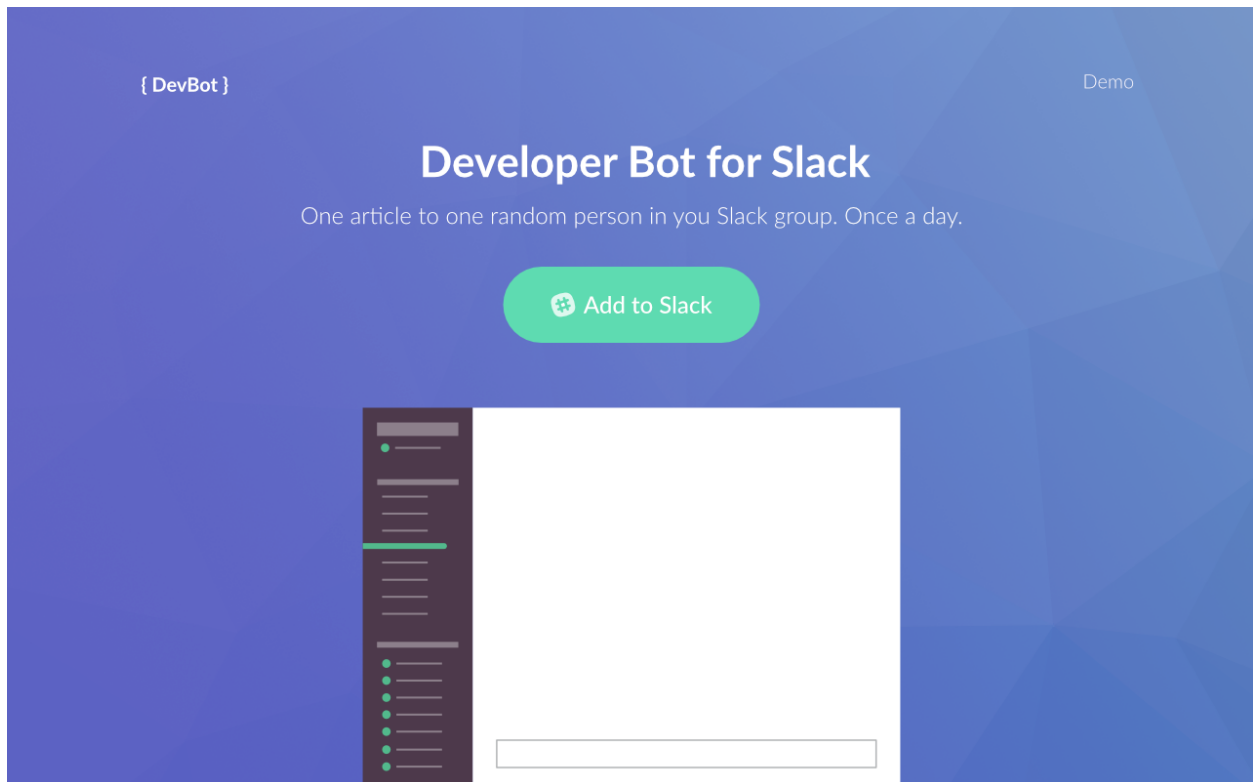
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent volutpat, elit pulvinar aliquam convallis, libero mauris rhoncus erat, non porttitor lectus justo et ex. In hac habitasse platea dictumst.



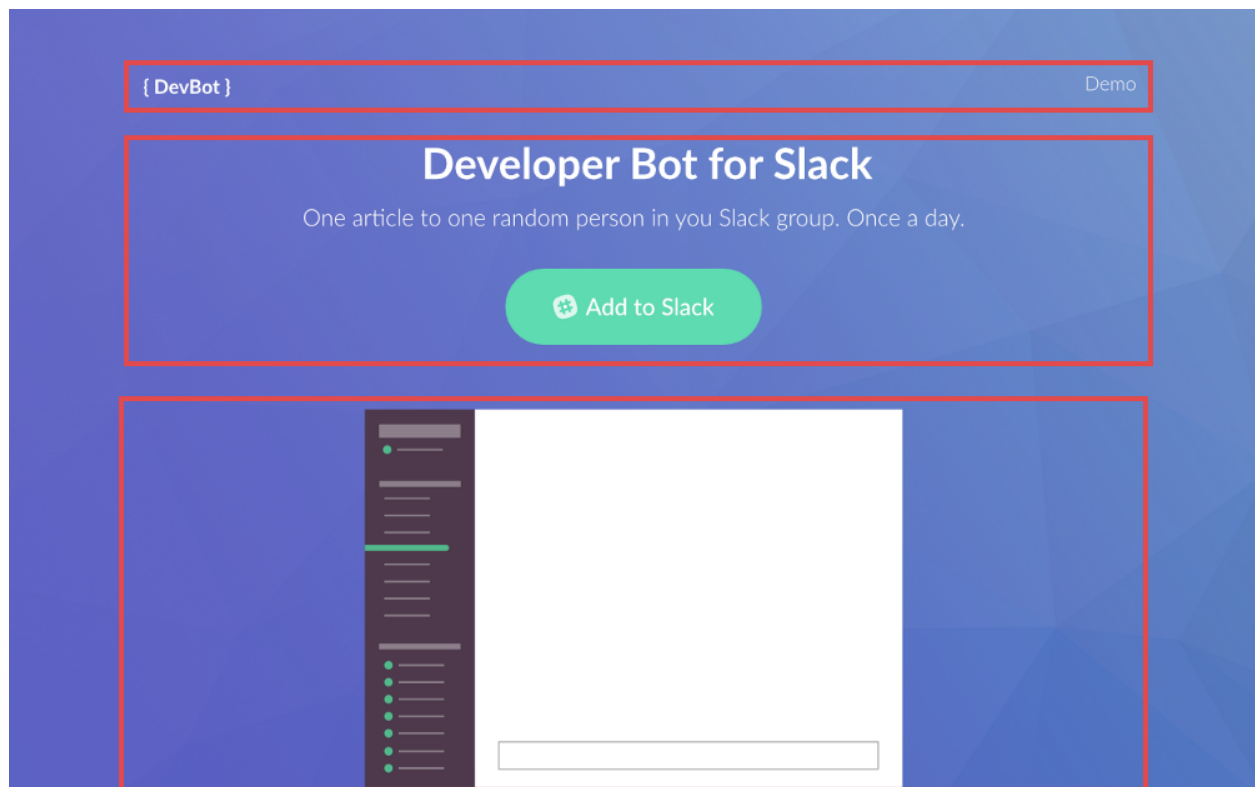
However, there are 2 columns within this row:



Now, let's compare this layout of rows and columns layout to our header:

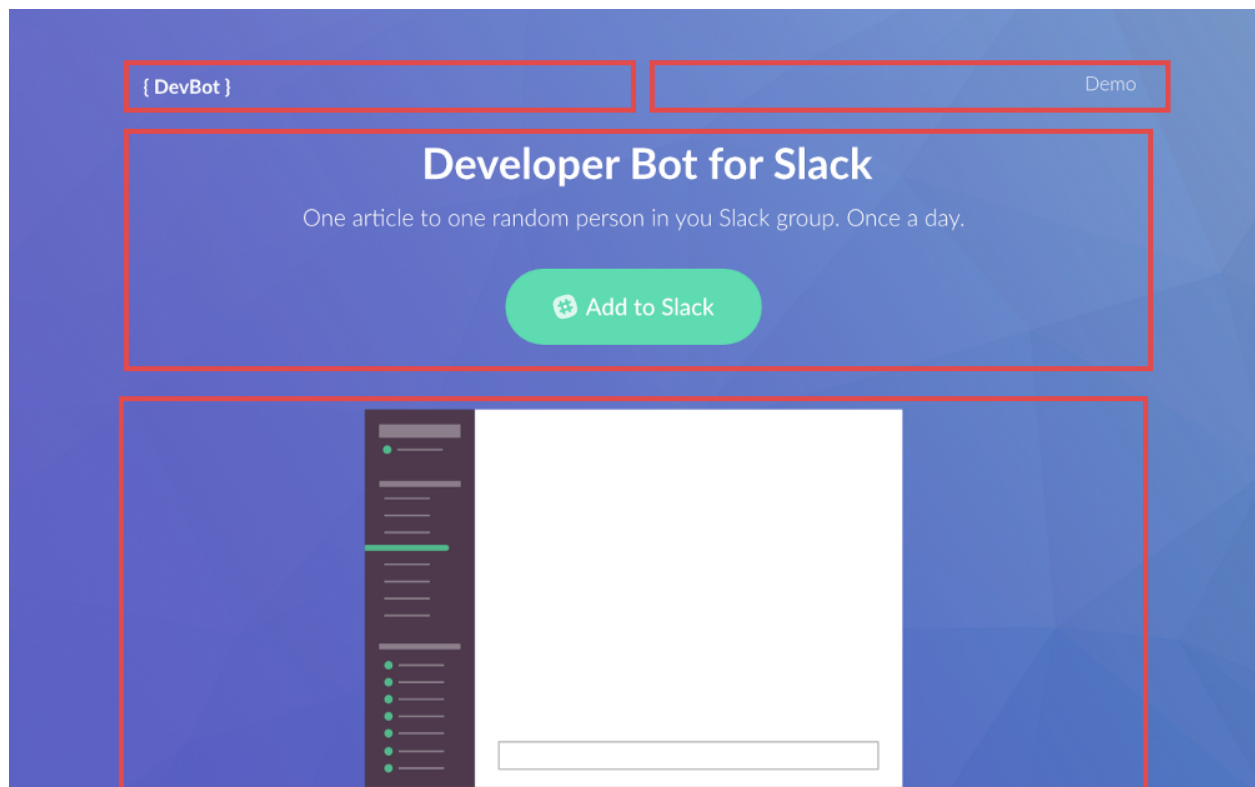


First, let's break this down in terms of *rows*:



We have 3 rows in total. Note that we keep the middle row as one row since those components work together as a block.

Next, let's look at it in terms of *columns*:



In the image above, we have 2 columns in the first row, 1 column in the middle row, and 1 column in the bottom row.

Once we have the layout expressed in rows and columns, we can fill it in with the stuff we need.

Understanding Bootstrap's Grid System

This begs the following question: “Can I define a layout in terms of rows and columns for our HTML?”

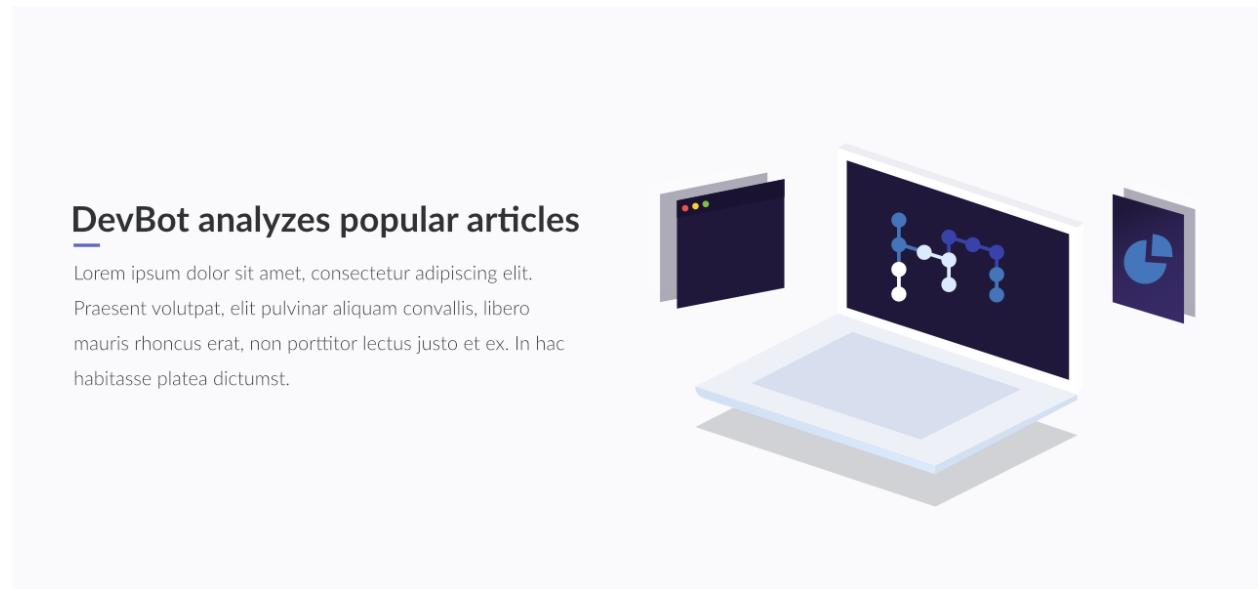
Yes, we can do this by making use of a grid system.

There's a few options but let's start with a popular option, *Bootstrap*.

Bootstrap is a framework for using predefined components (they do the CSS for us) in a UI and defining a grid system. I am not a fan of the former, but the latter is nice grid system for beginners.

Let's jump into the HTML code using Bootstrap so we can better understand how this works.

Let's look again at our section that we broke down into rows and columns:



Let's apply the Bootstrap grid system and write the HTML code to define the layout (note that I will write this in HTML and not JSX).

First, we place an outermost div with a class for the name of our section (neighborhood):

```
1 <div class="section-a">
2 </div>
```

Then, we have to add a *container* within this div:

```
1 <section class="section-a">
2   <div class="container">
3     </div>
4 </section>
```

Next, we create div elements with a class called *row* for each of our rows (just 1 in this example):

```
1 <section class="section-a">
2   <div class="container">
3     <div class="row">
4
5     </div>
6   </div>
7 </section>
```

To finish our layout, we add `div` elements for our columns. This will require a bit more explanation.

When it comes to columns within our layout, the *width* of the column is significant.

A column can have a *width* of at least $1/4$ (25%) of the *row* and at most, it can take up the entire row (100%). In between 25% width and 100% width, we can have a column with a width of 33% ($1/3$) or a width of 50% ($1/2$). Therefore, our width options are:

$1/4$

$1/3$

$1/2$

1

Because of that, these are all the possibilities for how we can construct columns within a row:



With Bootstrap, the width of columns is expressed by whole numbers. Specifically, an entire row is expressed by the number 12.

If I want to express the widths of columns by $1/4$, $1/3$, $1/2$, and 1, we do it by:

3 (3/12 or 1/4)

4 (4/12 or 1/3)

6 (6/12 or 1/2)

12 (12/12 or 1)

In Bootstrap, we specify a column and its width by the *class* like so:

```
<div class="col-lg-*insert whole number for width*"></div>
```

col is short for column.

lg has nothing to do with the width of the columns. Rather, it refers to how the grid system works across devices since it is a *responsive* grid system (responds to shrinking of the screen and display on mobile devices):

	Extra small devices Phones (<768px)	Small devices Tablets (≥768px)	Medium devices Desktops (≥992px)	Large devices Desktops (≥1200px)
Grid behavior	Horizontal at all times	Collapsed to start, horizontal above breakpoints		
Container width	None (auto)	750px	970px	1170px
Class prefix	.col-xs-	.col-sm-	.col-md-	.col-lg-
# of columns	12			
Column width	Auto	~62px	~81px	~97px
Gutter width	30px (15px on each side of a column)			
Nestable	Yes			
Offsets	Yes			
Column ordering	Yes			

Let's not worry about responsive design (for the sake of time) so I will use *col-lg-____*.

The last part of the syntax is to enter 3, 4, 6, or 12 to note the width of the column as was previously shown.

Our section has 2 columns that take up 1/2 of the row (col-lg-6):



Putting it all together, we can finish the code for our section like so:

```
1      <section class="section-a">
2        <div class="container">
3          <div class="row">
4            <div class="col-lg-6">
5              </div>
6            <div class="col-lg-6">
7              </div>
8          </div>
9        </div>
10     </section>
```

Just like that, we have been able to understand how to use Bootstrap's grid system to define our layout for our sections.

react-grid-system: A Better Grid System Alternative

While I like using the Bootstrap grid system to explain the core concepts of working with a grid system, we are going to use an alternative for our JSX.

Reason being, Bootstrap has a lot of JavaScript that it modifies and we want to avoid conflicts with React. Therefore, we will use [react-grid-system](https://github.com/JSxMachina/react-grid-system)³⁹.

react-grid-system works very similarly to Bootstrap except our rows and columns are defined using React *components* and not CSS.

Let's look at the *react-grid-system* implementation of the section we created with Bootstrap:

³⁹ <https://github.com/JSxMachina/react-grid-system>

```

1      import { Container, Row, Col } from 'react-grid-system';
2
3      //stuff goes here...
4
5      //JSX returned in render function
6
7      <section class="section-a">
8          <Container>
9              <Row>
10                 <Col lg={6}>
11                 </Col>
12                 <Col lg={6}>
13                 </Col>
14             </Row>
15         </Container>
16     </section>

```

In the code above, we import the components we need from react-grid-system which will be installed via npm. Then, we just those use components in place of the Bootstrap classes. The same exact concepts with numbering the width of the columns applies.

Let's go ahead and install this via npm:

```
npm install --save react-grid-system
```

Then, let's test it out in our *index.js*:

```

1      import React from 'react';
2      import ReactDOM from 'react-dom';
3      import { Container, Row, Col } from 'react-grid-system';
4      class App extends React.Component {
5          render() {
6              return (
7                  <div className="section-a">
8                      <Container>
9                          <Row>
10                             <Col lg={6}>
11                                 1/2 grid here!
12                             </Col>
13                             <Col lg={6}>
14                                 1/2 grid here!
15                             </Col>
16                         </Row>

```

```
17         </Container>
18     </div>
19 )
20 }
21 }
22 ReactDOM.render(
23     <App/>,
24     document.getElementById( 'app' )
25 );
```

Remember to save the file, open the root of this project in command line, and run:

```
npm start
```

Go to the local host address as noted in command line and we should see:



In our neighborhood components, we will have the JSX for the section and the container and rows. In our block components, we will have the columns and nested house components. In stand-alone house components, we will have the columns and whatever needs to go in between (no nested components).

Setting Up Our App

While we have index.js open, let's remove all the JSX code and nest our village component which we know will be called LandingPage:

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4
5   class App extends React.Component {
6     render() {
7       return (
8         <LandingPage/>
9       )
10    }
11  }
12  ReactDOM.render(
13    <App/>,
14    document.getElementById('app')
15  );
```

In the next chapter, we will add another village. For now, that is all we will have to do.

Setting Up Our Village



Landing Page

We have already created the file for our village called *LandingPage.js* and nested a component called *LandingPage* in our village.

Open that file and let's go ahead and create this LandingPage component.

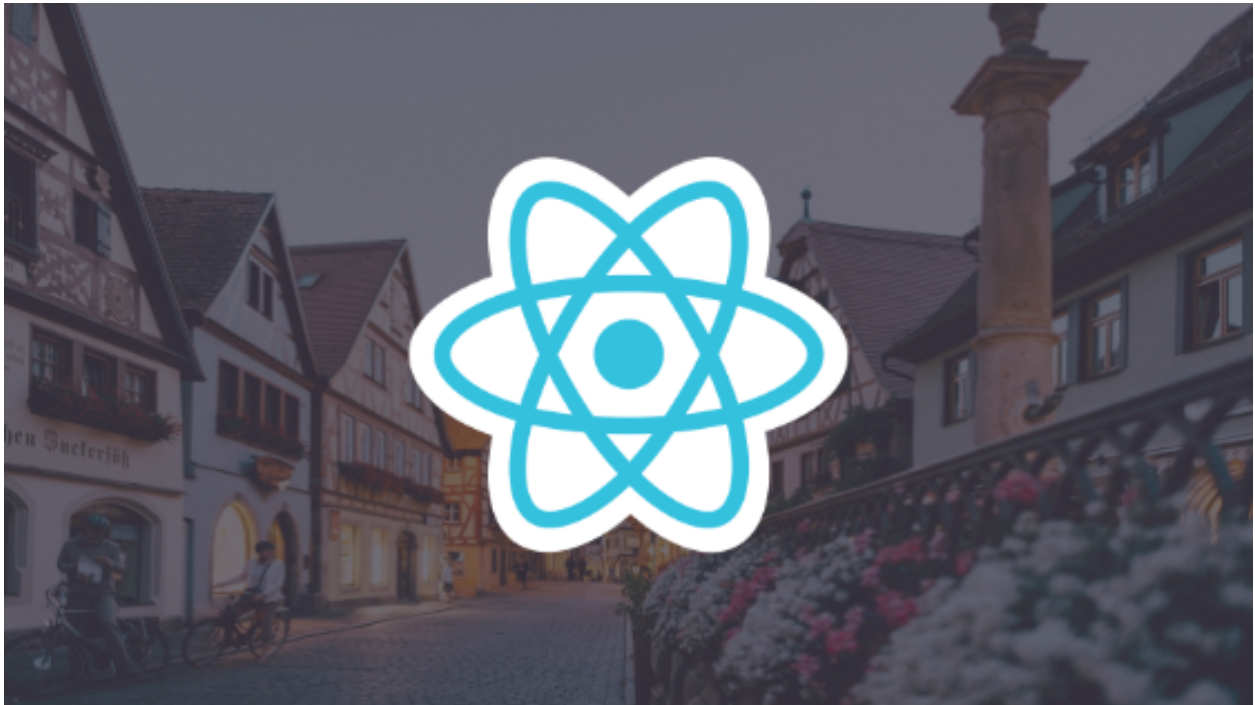
In this file, we need to import React, ReactDOM, and react-grid-system and have our JSX be all the nested neighborhoods. The neighborhoods will be Header, SectionA, SectionB, SectionC, and Footer.

Putting it together, we have:

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4
5   class LandingPage extends React.Component {
6     render() {
7       return (
8         <div className="landing-page">
9           <Header/>
10          <SectionA/>
11          <SectionB/>
12          <SectionC/>
13          <Footer/>
14        </div>
15      )
16    }
17  }
```

Note there is no ReactDOM code because that should only happen at the top of the hierarchy which was taken care of in our *index.js*.

Setting Up Our Neighborhoods



The neighborhood components are going to render as section elements of our landing page. Here, we will be using the react-grid-system code that we looked at earlier. Specifically, our neighborhood components will use the Container component while the blocks or stand-alone houses will contain the rows and columns.

Header

Open *Header.js* and let's create the shell of our Header component (no rows or columns):

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4
5   class Header extends React.Component {
6     render() {
7       return (
8         <section className="header">
9           <Container>
10
11             </Container>
12           </section>
```

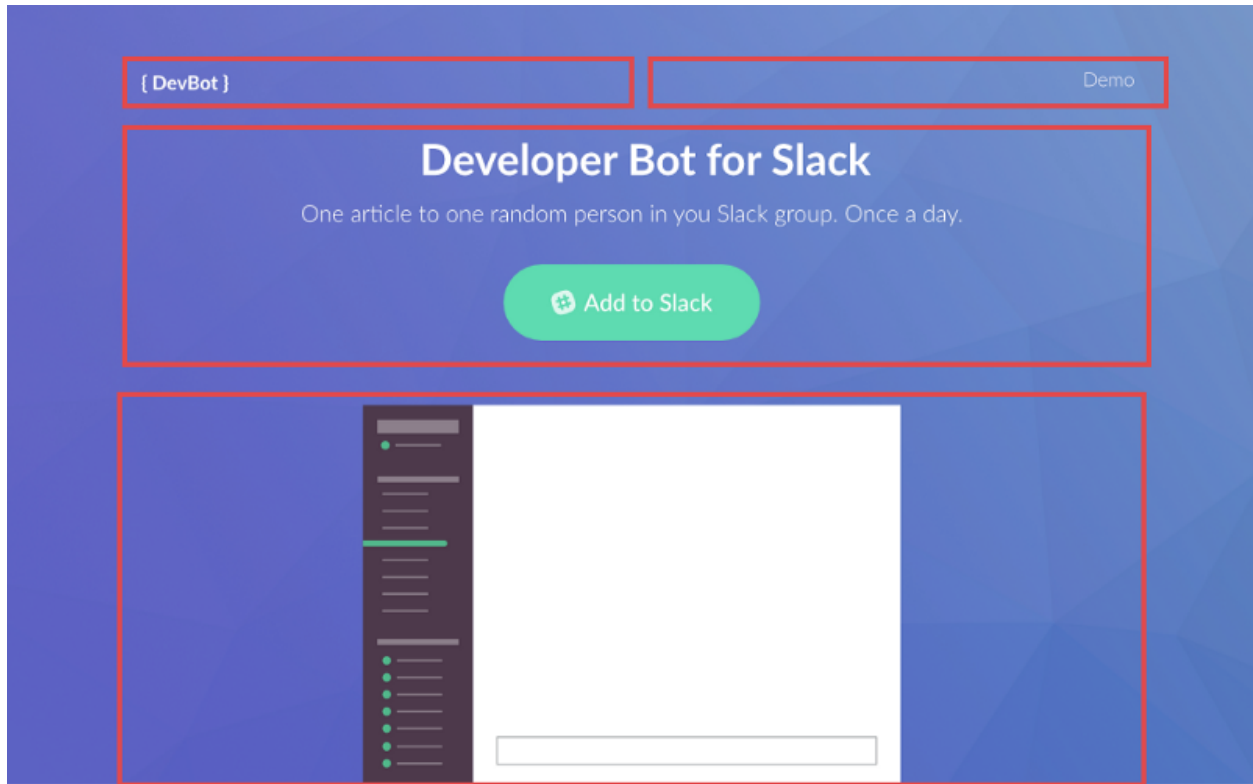
```

13     )
14   }
15 }

```

Now, we need to fill in our Header with the rows and nested components. We want to nest only our blocks and the houses that aren't under any blocks.

First off, we need to know how many rows are needed in our Header:



We will have 3 rows of content.

Next, let's check to see what we need to nest:

- 1 - Header (Neighborhood)
- 2 - Navigation (Block)
- 3 - Branding (House)
- 4 - Demo Link (House)
- 5 - Call To Action (Block)
- 6 - Title (House)
- 7 - Subtitle (House)
- 8 - Button (House)
- 9 - Slack Channel (House)

Knowing this, we can add the following:


```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4
5   class Header extends React.Component {
6     render() {
7       return (
8         <section className="header">
9           <Container>
10            <Row>
11              <Navigation/>
12            </Row>
13            <Row>
14              <CallToAction/>
15            </Row>
16            <Row>
17              <SlackChannel/>
18            </Row>
19          </Container>
20        </section>
21      )
22    }
23  }
```

Cool! We have added the 3 rows and the nested components which we will create later.

Section A

Open *Section A.js* so we can create the SectionA neighborhood component.

Again, we start by seeing how many rows of content there are:



We need 1 row of content.

We then just need to check what's to be nested:

- 1 - Section A (Neighborhood)
- 2 - Section Description (Block)
- 3 - Heading (House)
- 4 - Divider (House)
- 5 - Paragraph (House)
- 6 - Laptop (House)

Based off of this, we can do:

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4
5   class SectionA extends React.Component {
6     render() {
7       return (
8         <section className="section-a">
9           <Container>
10             <Row>
11               <SectionDescription/>
12               <Laptop/>
13             </Row>
14           </Container>

```

```

15         </section>
16     )
17 }
18 }

```

Section B

Let's move on to *Section B.js*.

Section B is just like the previous section in that there is only 1 row of content.

In this file, we need the following components nested:

```

1     - Section B (Neighborhood)
2       - Article Cover (House)
3       - Section Description (Block)
4         - Heading (House)
5         - Divider (House)
6         - Paragraph (House)

```

Therefore, we can do:

```

1     import React from 'react';
2     import ReactDOM from 'react-dom';
3     import { Container, Row, Col } from 'react-grid-system';
4
5     class SectionB extends React.Component {
6     render() {
7         return (
8             <section className="section-b">
9                 <Container>
10                     <Row>
11                         <ArticleCover/>
12                         <SectionDescription/>
13                     </Row>
14                 </Container>
15             </section>
16         )
17     }
18 }

```

We should be familiar with the process for our neighborhoods at this point. For the final two components, I will just paste the needed components and code.

Section C

Section C nested components:

- 1 - Section C (Neighborhood)
- 2 - Call To Action (Block)
- 3 - Title (House)
- 4 - Subtitle (House)
- 5 - Button (House)

Section C.js

```
1       import React from 'react';
2       import ReactDOM from 'react-dom';
3       import { Container, Row, Col } from 'react-grid-system';
4
5       class SectionC extends React.Component {
6       render() {
7         return (
8           <section className="section-c">
9             <Container>
10              <Row>
11                <CallToAction/>
12              </Row>
13             </Container>
14           </section>
15         )
16       }
17     }
```

Footer

Footer nested components:

- 1 - Footer (Neighborhood)
- 2 - Navigation (Block)
- 3 - Branding (House)
- 4 - Demo Link (House)

Footer.js

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4
5   class Footer extends React.Component {
6     render() {
7       return (
8         <section className="footer">
9           <Container>
10            <Row>
11              <Navigation/>
12            </Row>
13          </Container>
14        </section>
15      )
16    }
17  }
```

Save everything and let's move on to our blocks!

Setting Up Our Blocks



As we mentioned before, the block components will contain the react-grid-system code for the columns as well as the nested houses.

Let's look at the houses we already nested in each neighborhood in the previous section and knock them out.

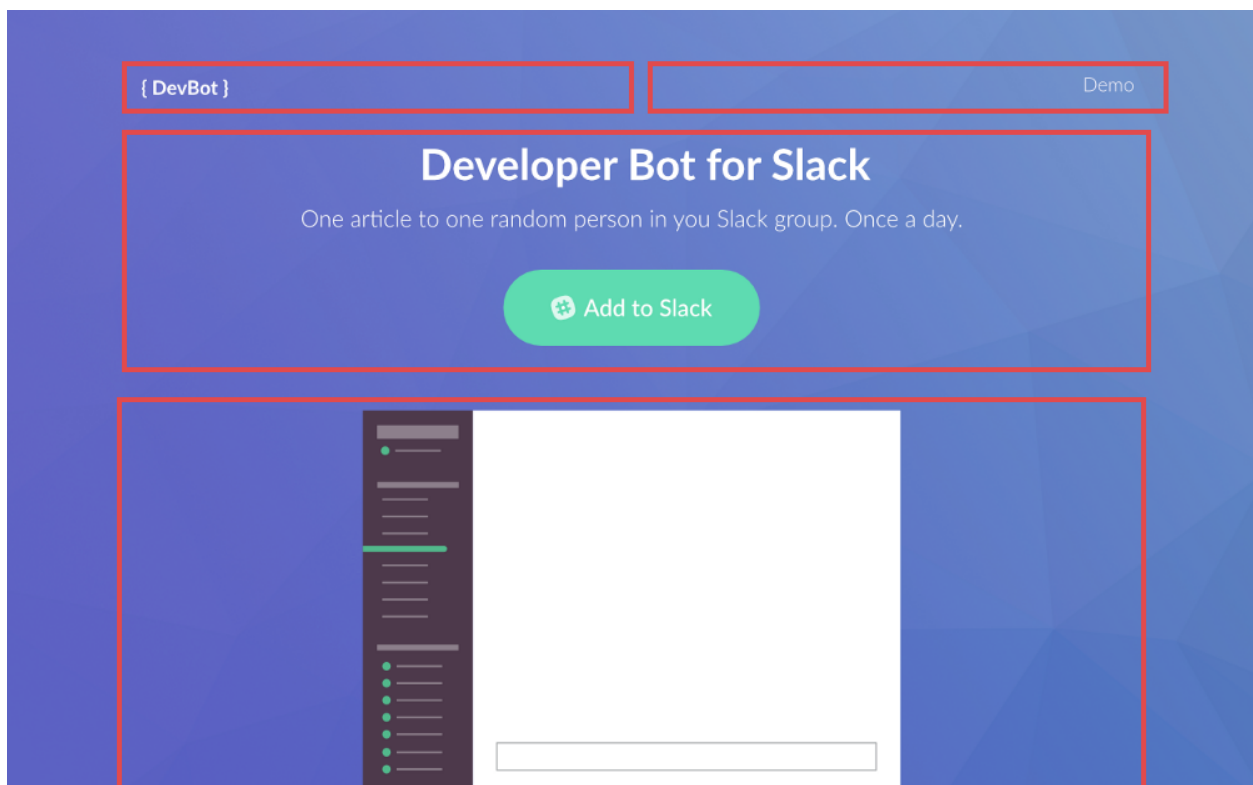
Header Blocks

Starting with our Header, we nested the following block components:

```
1    <Navigation/>
2    <CallToAction/>
```

The first thing to figure out is how these will be broken down into columns.

Let's take a look at our header mockup:



Our Call to Action block is a bit more clear. It will only be 1 column and everything will be centered with CSS. If you recall, a full-width column is represented by the number 12.

Open *Call To Action.js* and let's put in the following:

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4
5   class CallToAction extends React.Component {
6     render() {
7       return (
8         <Col lg={12}>
9           </Col>
10        )
11      }
12    }
```

Next, we just need to fill in our column with the nested houses:

```
1   - Call To Action (Block)
2     - Title (House)
3     - Subtitle (House)
4     - Button (House)
```

Putting it all together, we have:

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4
5   class CallToAction extends React.Component {
6     render() {
7       return (
8         <Col lg={12}>
9           <Title/>
10          <Subtitle/>
11          <Button/>
12        </Col>
13      )
14    }
15  }
```

Perfect.

Let's move on to *Navigation.js*. This block is a bit more tricky to describe.



Because we have two components hugging the edges, it might be hard to visualize the columns. However, we can break it down into 2 columns like we have shown before and use CSS to hug the edges:



Let's look at the components that will be nested in these columns:

- 1 - Navigation (Block)
- 2 - Branding (House)
- 3 - Demo Link (House)

Therefore, we can add the following code:

```

1      import React from 'react';
2      import ReactDOM from 'react-dom';
3      import { Container, Row, Col } from 'react-grid-system';
4
5      class Navigation extends React.Component {
6        render() {
7          return (
8            <Col lg={6}>
9              <Branding/>
10            </Col>
11            <Col lg={6}>
12              <DemoLink/>
13            </Col>
14          )
15        }
16      }

```

Section A Blocks

Let's see what blocks are in Section A:

```
<SectionDescription/>
```

Next, we look to see how our SectionDescription component can be broken down into rows and columns:



There will be 2 columns for our content.

Finally, let's look at the nested blocks within `SectionDescription` before we add the code:

- 1 - Section Description (Block)
- 2 - Heading (House)
- 3 - Divider (House)
- 4 - Paragraph (House)

Putting it all together, we get the following for *SectionDescription.js*:

```

1      import React from 'react';
2      import ReactDOM from 'react-dom';
3      import { Container, Row, Col } from 'react-grid-system';
4
5      class SectionDescription extends React.Component {
6        render() {
7          return (
8            <Col lg={6}>
9              <Heading/>
10             <Divider/>
11             <Paragraph/>
12           </Col>
13         )
14       }
15     }

```

Section B Blocks

The SectionB component has the same block as SectionA which we have already created the file for.

Section C Blocks

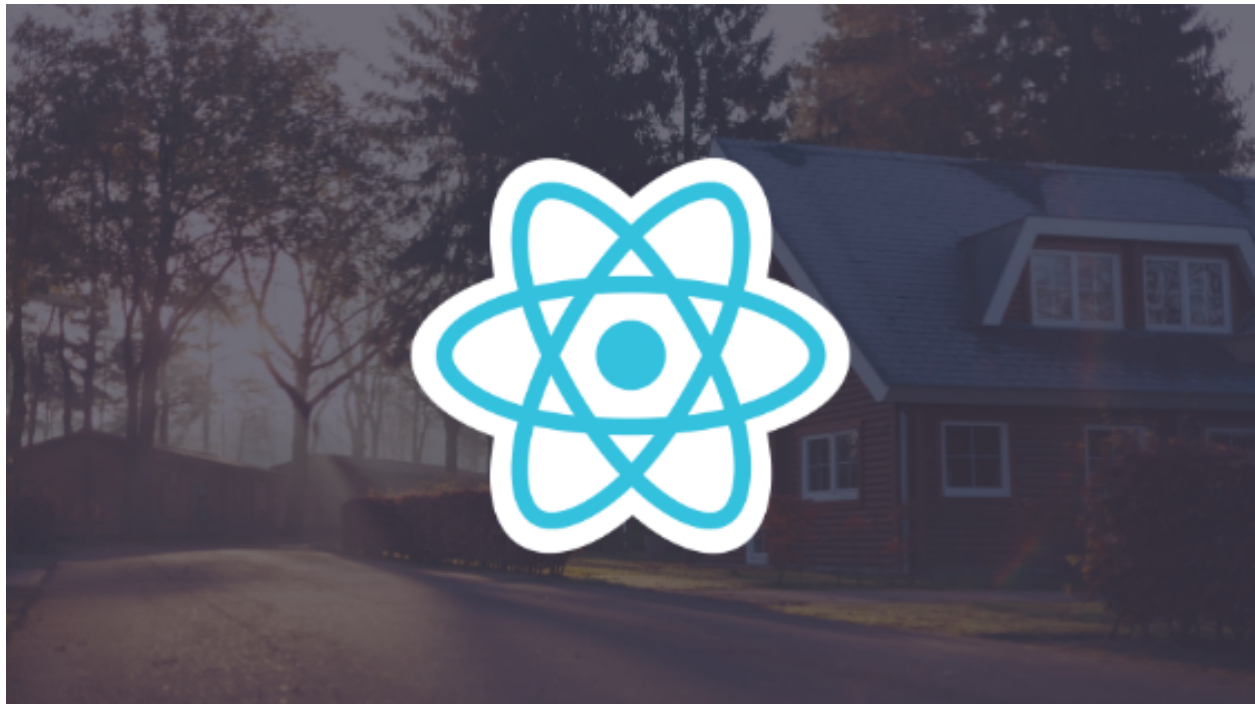
The SectionC component has the same block, CallToAction, as the Header component. Nothing else needs to be done.

Footer Blocks

The Footer component has the same block, Navigation, as was used in the Header component. Once again, nothing else needs to be done.

Cool beans! Off to our houses!

Setting Up Our Houses



Almost done setting up all of our components, hang in there!

As we have mentioned before, there are house components that will be nested in block components and houses components that will be nested in neighborhoods (stand-alone house components).

Our house components will not have anything nested. They are the building blocks of our user interface. For these reasons, the JSX will be more familiar to what is typically done with straight HTML.

The stand-alone components will require a little bit more code since we still have to add the react-grid-system rows and columns.

Since we just finished up our block components, let's do the house components that will go into our blocks.

Houses for Our Blocks

First, let's look at all that we have to knock out:

- 1 - Navigation (Block)
- 2 - Branding (House)
- 3 - Demo Link (House)
- 4
- 5 - Call To Action (Block)
- 6 - Title (House)
- 7 - Subtitle (House)
- 8 - Button (House)
- 9
- 10 - Section Description (Block)
- 11 - Heading (House)
- 12 - Divider (House)
- 13 - Paragraph (House)

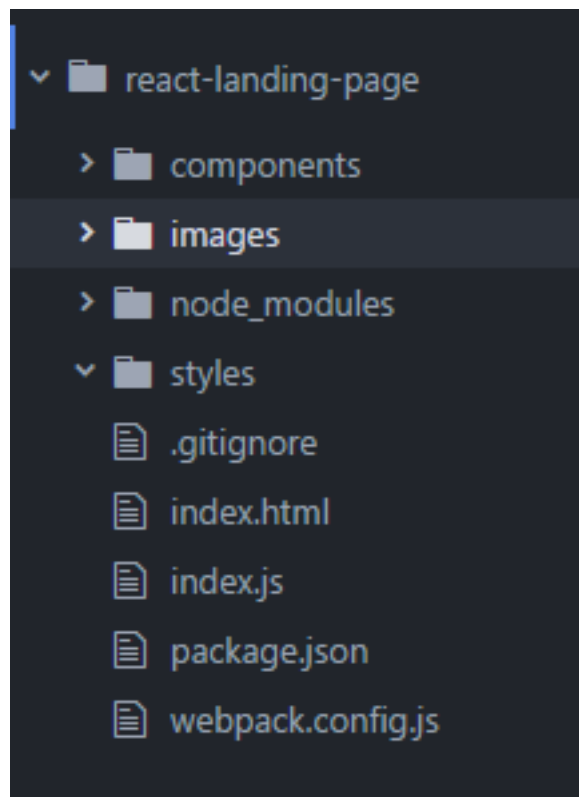
Starting with *Branding.js*, we just have to render the image of our logo which is an SVG file. [Download it here](https://www.evernote.com/shard/s736/sh/e43485a2-1d3d-47b8-b1ea-27a12aa7ab57/43a151d8bf2054686a21dbdb5223491d)⁴⁰:



Even though it is an SVG, we can render in an `img` tag just like you would with a PNG or JPG image.

Before we write the code, however, let's add a new folder in our project called **images**. This should go directly under *react-landing-page* like so:

⁴⁰<https://www.evernote.com/shard/s736/sh/e43485a2-1d3d-47b8-b1ea-27a12aa7ab57/43a151d8bf2054686a21dbdb5223491d>



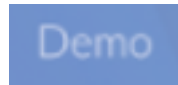
Next, make sure to place the downloaded *Branding.svg* file under the images folder.

After that, we can move on to our code:

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3
4   class Branding extends React.Component {
5     render() {
6       return (
7         <div>
8           
9         </div>
10      )
11    }
12  }
```

Remember that we always include an outermost div element in the JSX.

Next up, let's do the DemoLink component to finish up the Navigation block.



In this component, we want to simply return a link. We have not discussed what this will link to so the *href* should be #.

Open up *Demo Link.js* and add the following:

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3
4   class DemoLink extends React.Component {
5     render() {
6       return (
7         <div>
8           <a href="#">
9             <span className="demo-link">
10              Demo
11            </span>
12          </a>
13        </div>
14      )
15    }
16  }
```

The rest of the house components that will go into our block components are pretty easy to create. The process is to simply look at the mockup and translate it to JSX.

Try to give yourself a challenge and write the code without looking for the remaining house components:

```
1   - Call To Action (Block)
2   - Title (House)
3   - Subtitle (House)
4   - Button (House)
5
6   - Section Description (Block)
7   - Heading (House)
8   - Divider (House)
9   - Paragraph (House)
```

There are some things you won't be able to know.

One is what heading I will use for the title, subtitle, and heading. In that same order, let's h1, h3, and h2.

You can also keep all the classes the same name as the component just with all lowercase.

Additionally, you can use this for the paragraph text:

```
1 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent volutpat, elit\
2 pulvinar aliquam convallis, libero mauris rhoncus erat, non porttitor lectus ju\
3 sto et ex. In hac habitasse platea dictumst.
```

Finally, you can ignore writing the JSX for the Slack icon in the Button component.

Now give it a whirl!

Now that you have attempted this on your own, review the code for the remaining files:

Title.js

```
1 import React from 'react';
2 import ReactDOM from 'react-dom';
3
4 class Title extends React.Component {
5   render() {
6     return (
7       <div>
8         <h1 className="title">Developer Bot for Slack</h1>
9       </div>
10     )
11   }
12 }
```

Subtitle.js

```
1 import React from 'react';
2 import ReactDOM from 'react-dom';
3
4 class Subtitle extends React.Component {
5   render() {
6     return (
7       <div>
8         <h3 className="subtitle">One article to one random person in your Sl\
9 ack group. Once a day.</h3>
10       </div>
```

```
11     )
12   }
13 }
```

Button.js

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3
4   class Button extends React.Component {
5     render() {
6       return (
7         <div>
8           <button className="button">Add to Slack</button>
9         </div>
10      )
11    }
12  }
```

Heading.js

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3
4   class Heading extends React.Component {
5     render() {
6       return (
7         <div>
8           <h2 className="heading">
9             DevBot analyzes popular articles
10          </h2>
11        </div>
12      )
13    }
14  }
```

Paragraph.js

```

1     import React from 'react';
2     import ReactDOM from 'react-dom';
3
4     class Paragraph extends React.Component {
5       render() {
6         return (
7           <div>
8             <p className="paragraph">
9               Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent vo\
10 luptat, elit pulvinar aliquam convallis, libero mauris rhoncus erat, non porttiti\
11 or lectus justo et ex. In hac habitasse platea dictumst.
12           </p>
13         </div>
14       )
15     }
16   }

```

Divider.js

```

1     import React from 'react';
2     import ReactDOM from 'react-dom';
3
4     class Divider extends React.Component {
5       render() {
6         return (
7           <div>
8             <hr className="divider"/>
9           </div>
10         )
11       }
12     }

```

Once you are caught up, it's time to move on to our stand-alone house components.

Stand-Alone Houses

Our stand-alone houses will be very similar to the previous house components we just completed. There is only one extra step. We have to include the react-grid-system code for defining our columns since there are no blocks for these houses where that step would occur.

Let's look at what stand-alone houses we need:


```

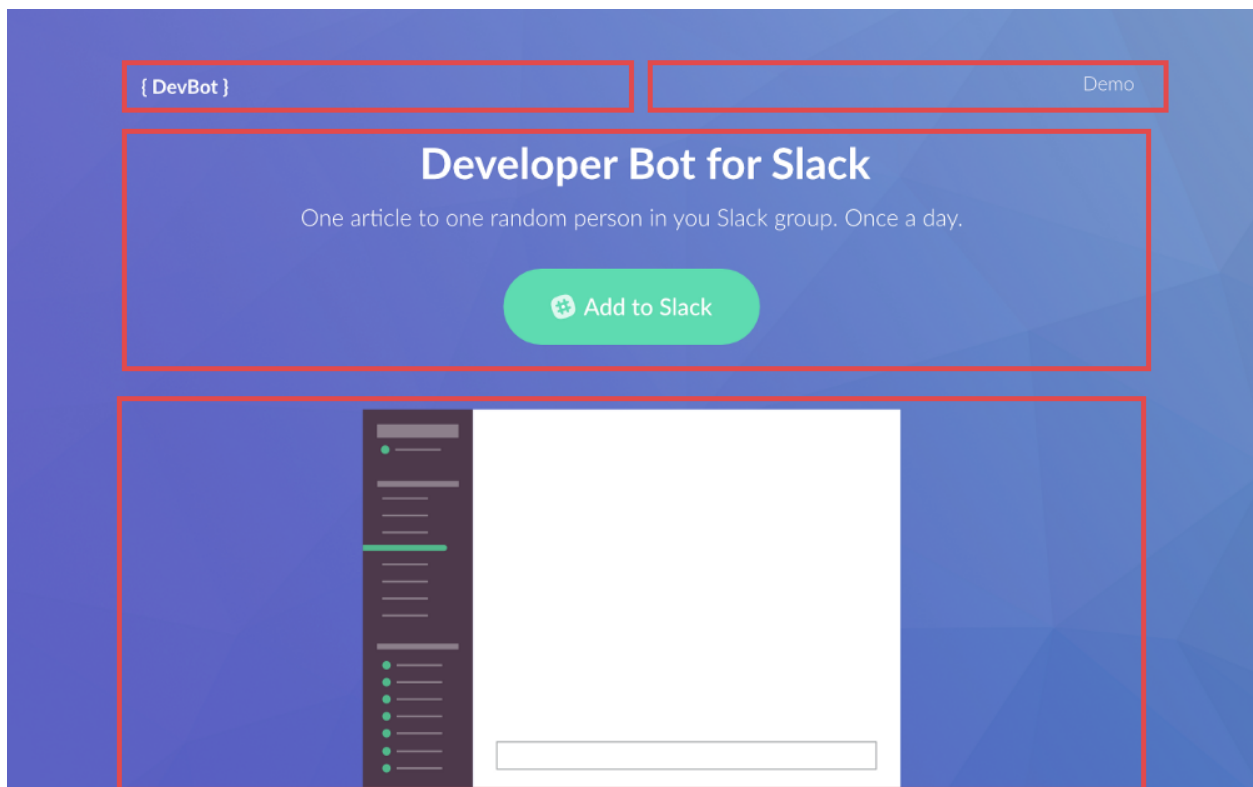
1      - App
2
3      - Landing Page (Village)
4
5      - Header (Neighborhood)
6          - Navigation (Block)
7          - Branding (House)
8          - Demo Link (House)
9          - Call To Action (Block)
10         - Title (House)
11         - Subtitle (House)
12         - Button (House)
13     **- Slack Channel (House)**
14
15     - Section A (Neighborhood)
16         - Section Description (Block)
17         - Heading (House)
18         - Divider (House)
19         - Paragraph (House)
20     **- Laptop (House)**
21
22     - Section B(Neighborhood)
23         **- Article Cover (House)**
24         - Section Description (Block)
25         - Heading (House)
26         - Divider (House)
27         - Paragraph (House)
28
29     - Section C (Neighborhood)
30         - Call To Action (Block)
31         - Title (House)
32         - Subtitle (House)
33         - Button (House)
34
35     - Footer (Neighborhood)
36         - Navigation (Block)
37         - Branding (House)
38         - Demo Link (House)

```

Let's knock these out to finish off our component setup!

Starting with our SlackChannel component. This component will just be an SVG image.

As we can see below, it will only be one column:



The SVG image can be grabbed [here](#) and then moved under the images folder.⁴¹

Therefore, we can do the following in our *Slack Channel.js*:

```

1  import React from 'react';
2  import ReactDOM from 'react-dom';
3  import { Container, Row, Col } from 'react-grid-system';
4
5  class SlackChannel extends React.Component {
6    render() {
7      return (
8        <Col lg={12}>
9          
10         </Col>
11       )
12     }
13   }

```

Our last two houses are very similar. Both of them will be an image in a section that is one column that takes up half of the row, therefore, the column can be expressed in the number 6.

⁴¹<https://www.evernote.com/shard/s736/sh/e43485a2-1d3d-47b8-b1ea-27a12aa7ab57/43a151d8bf2054686a21dbdb5223491d>

First, grab the Laptop.svg file⁴², move it to the images folder, and open Laptop.js:

```
1  import React from 'react';
2  import ReactDOM from 'react-dom';
3  import { Container, Row, Col } from 'react-grid-system';
4
5  class Laptop extends React.Component {
6    render() {
7      return (
8        <Col lg={6}>
9          
10        </Col>
11      )
12    }
13  }
```

Next, the ArticleCover component will just need a few tweaks from the previous code.

Make sure to grab the ArticleCover.svg file⁴³, move it under images, and open Article Cover.js:

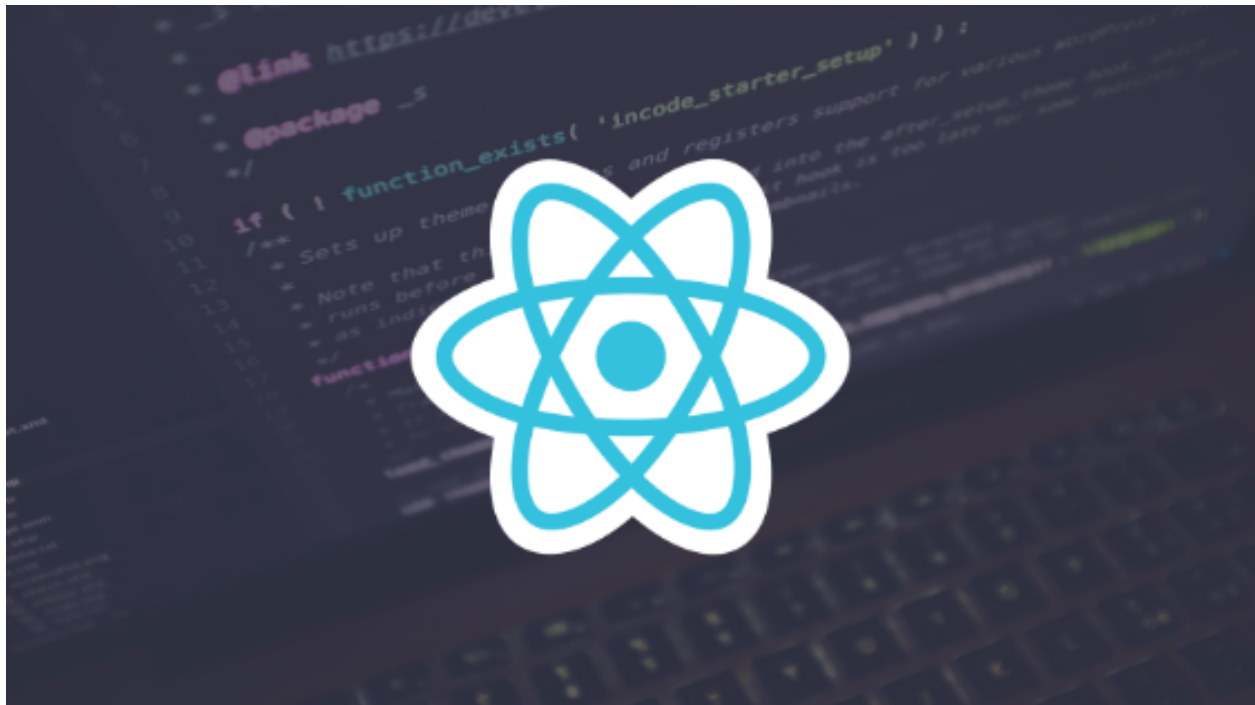
```
1  import React from 'react';
2  import ReactDOM from 'react-dom';
3  import { Container, Row, Col } from 'react-grid-system';
4
5  class ArticleCover extends React.Component {
6    render() {
7      return (
8        <Col lg={6}>
9          
10        </Col>
11      )
12    }
13  }
```

Finally! We have completed the setup of all our components. However, nothing will work until we add the code for the module system.

⁴²<https://www.evernote.com/shard/s736/sh/e43485a2-1d3d-47b8-b1ea-27a12aa7ab57/43a151d8bf2054686a21dbdb5223491d>

⁴³<https://www.evernote.com/shard/s736/sh/e43485a2-1d3d-47b8-b1ea-27a12aa7ab57/43a151d8bf2054686a21dbdb5223491d>

Implementing the Module System



In the previous chapter, we discussed a new problem that is created by breaking down our component hierarchy into separate files.

For example, we just created our ArticleCover component in its own file called Article Cover.js. It is used in our SectionB component like so:

```
1    class SectionB extends React.Component {
2      render() {
3        return (
4          <section className="section-b">
5            <Container>
6              <Row>
7                <ArticleCover/>
8              </Row>
9              <Row>
10               <SectionDescription/>
11             </Row>
12           </Container>
13         </section>
14       )
15     }
16   }
```

How does this file know what `<ArticleCover/>` is if it was defined in another file?

Currently, it doesn't. Therefore, we some code in *Article Cover.js* that will say: "Hey! I know we are making our ArticleCover component in this file, but I kinda want to use it somewhere else. I'm going to write some code so it can be imported in *Section B.js* where we will need to use it."

Writing code that makes our components exportable and importable is done by a *module system*. With ES6, there is a module system built in that we can make use of.

Since we brought up our ArticleCover component, let's test out this module system there.

Alright, you might be thinking: "Module system sounds pretty brutal concept! Please tell me this isn't going to be like all the Webpack nonsense."

Relax, it's actually incredibly simple.

We only need to add this one line of code at the end of the *Article Cover.js* file:

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4
5   class ArticleCover extends React.Component {
6     render() {
7       return (
8         <Col lg={6}>
9           
10          </Col>
11        )
12      }
13    }
14
15    module.exports = ArticleCover;
```

That's right. It's *that* simple.

Now, let's look at how we can do the *import* in *Section B.js*.

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4   import ArticleCover from './components/villages/neighborhoods/blocks/houses/\
5 Article Cover.js';
6
7   class SectionB extends React.Component {
8     render() {
9       return (
10        <section className="section-b">
11          <Container>
12            <Row>
13              <ArticleCover/>
14            </Row>
15            <Row>
16              <SectionDescription/>
17            </Row>
18          </Container>
19        </section>
20      )
21    }
22  }

```

Simple as that! We just have to import using the following syntax:

```
1 import *whatever was assigned to Module.exports* from *insert path*
```

Note the path relative to the file you are editing or creating.

We've had *a lot* of coding to do in this section and I bet you are dying to see some results. Let's test this out to make sure it is working correctly.

We will update *index.js* so it will import the ArticleCover component:

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4   import ArticleCover from './components/villages/neighborhoods/blocks/houses/\
5 Article Cover.js';
6
7   class App extends React.Component {
8     render() {
9       return (

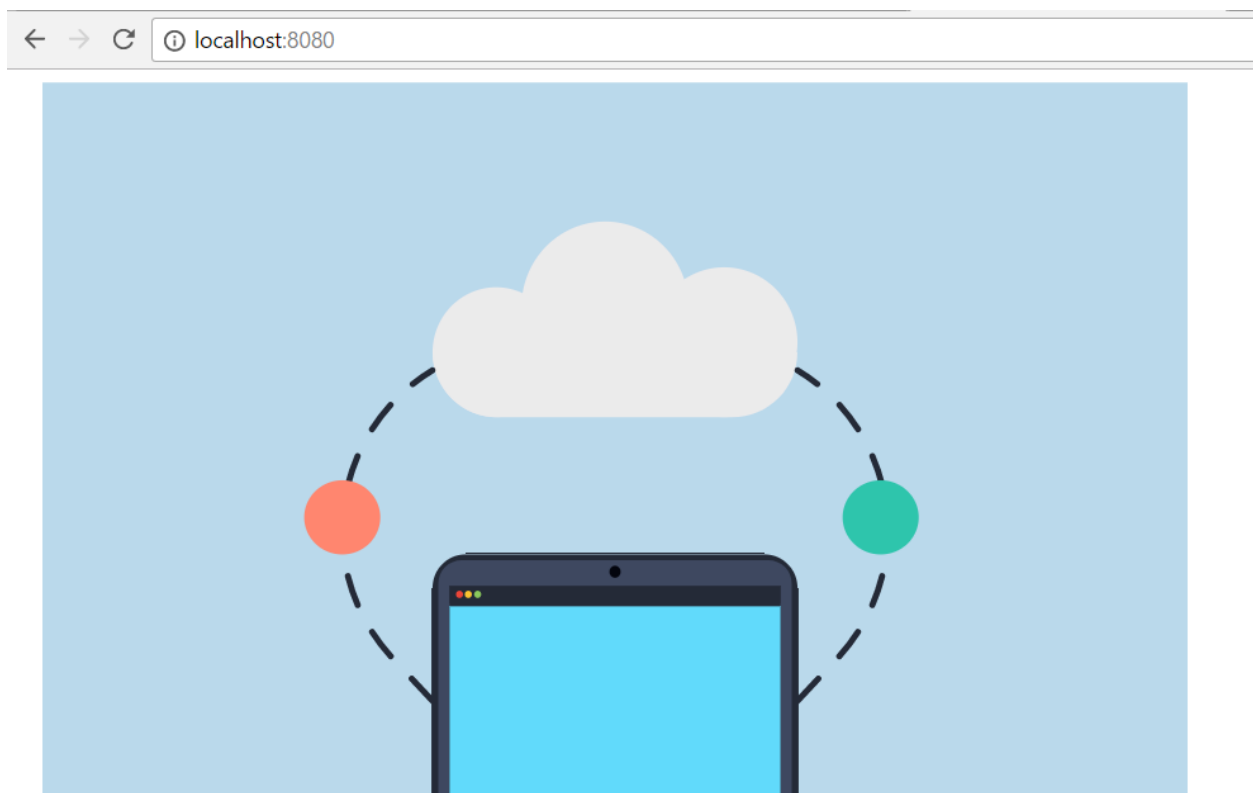
```

```
10         <ArticleCover/>
11     )
12 }
13 }
14
15 ReactDOM.render(
16   <App/>,
17   document.getElementById( 'app' )
18 );
```

Now, go to command line and run:

```
npm start
```

Sweet...it really works!!!!



Go ahead and undo those changes in *index.js*.

The next step would be to add the exports and imports to all of our files.

However, you can simply review the final code that I will provide shortly.

Sprinkling Some Style

While I would love to take the time to unpack how I do the CSS to put this user interface together, that's just way too much to fit into this chapter (plus this is about React).

Therefore, let's just add a new file under the *styles* folder called *landing-page.css* and copy the following code:

[landing-page.css via GitHub](#)⁴⁴

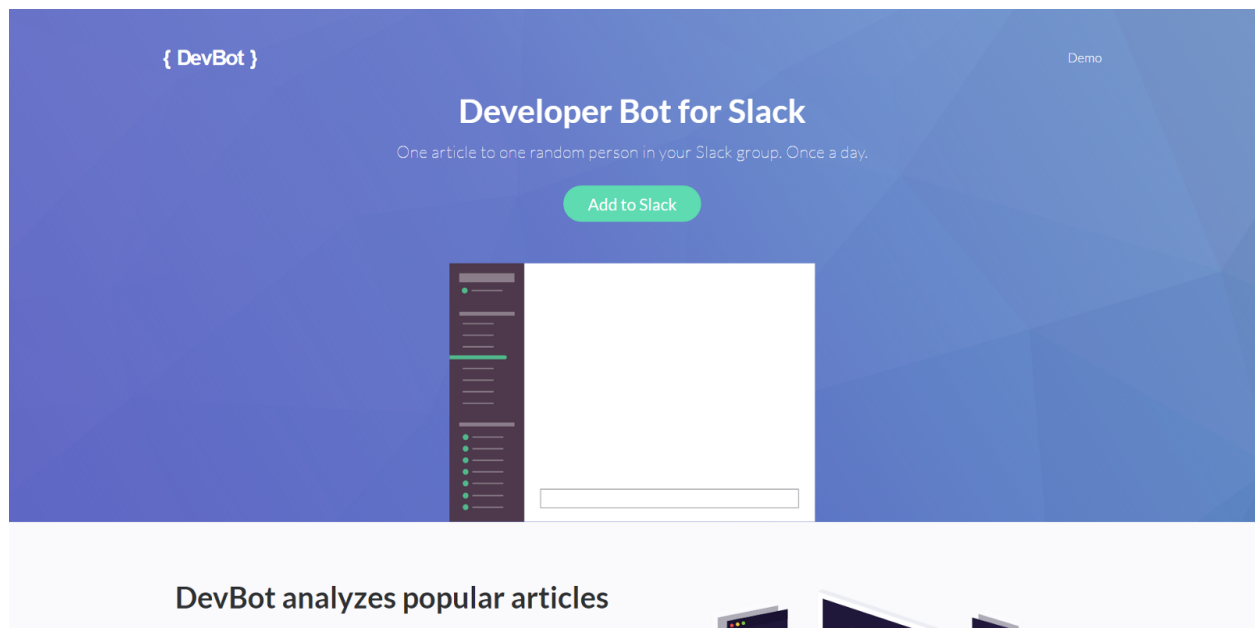
Lastly, we just add the link in our *index.html*:

```
1      <!DOCTYPE html>
2      <html>
3          <head>
4              <meta charset="utf-8">
5              <title>React Landing Page</title>
6              <link rel="stylesheet" href="styles/landing-page.css">
7          </head>
8          <body>
9              <div id="app"></div>
10         </body>
11     </html>
```

Feel free to explore the CSS and try to figure out how I went about all the styling.

Woohoo! We have successfully made our first user interface!

⁴⁴<https://gist.github.com/michaelmang/4d7e562880c7ea73f10c6b7290f3d271>



Final Code

Available here via Github⁴⁵

Note: I recommend trying to get all the import and exports to work after seeing a few examples of how I did it. Then, double check the final code.

When you have everything together, make sure to run:

```
npm start
```

Then, go to the local host to see the landing page.

Concluding Thoughts

I know that it has taken us seven chapters to finally get to make a user interface. However, I have no regrets about taking that much time to explain things. Too often, I have seen tutorials that start off setting the bar way too high. This leads to a lot of not so obvious terminologies being glanced over meanwhile fatigue increases for the learner. After all, if React teaches us the importance of breaking things down into smaller pieces, we should have the same approach to learning and teach it.

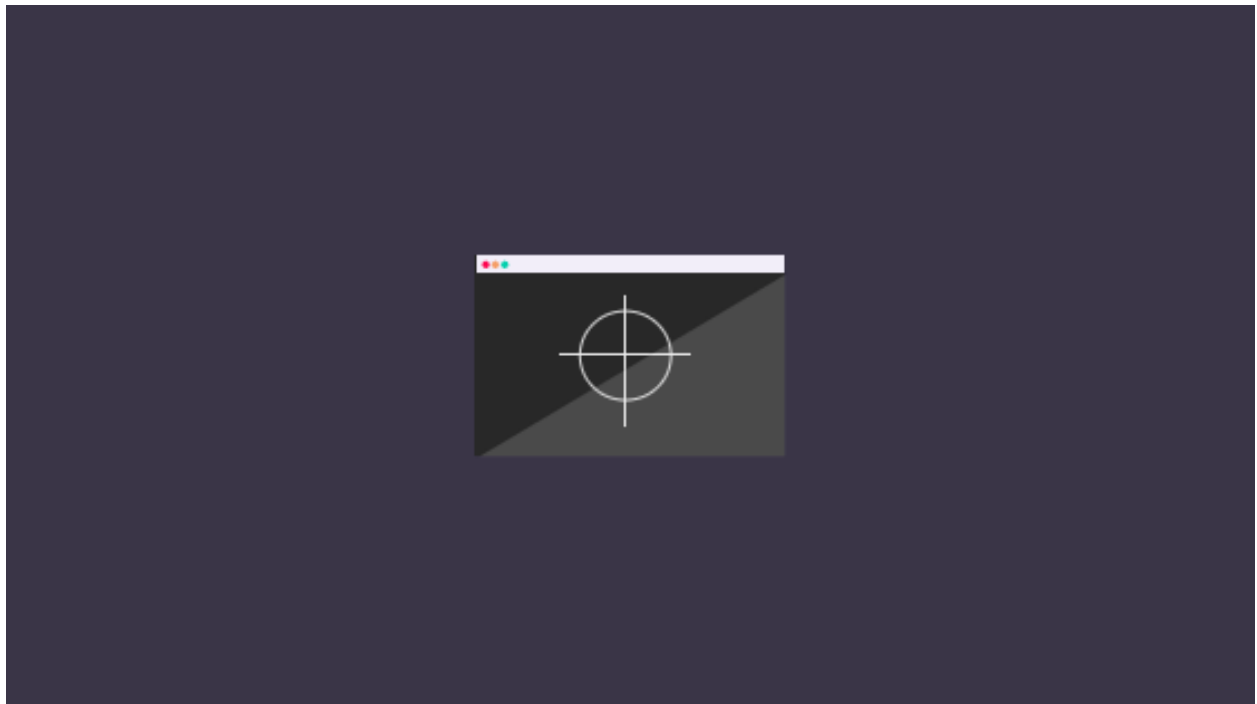
Anyways, give yourself a round of applause. Even though we need to make improvements and make things more *dynamic*, we have gained a lot of practical knowledge already.

⁴⁵<http://bit.ly/2qe6dyu>

Chapter 8 | Implementing React Router



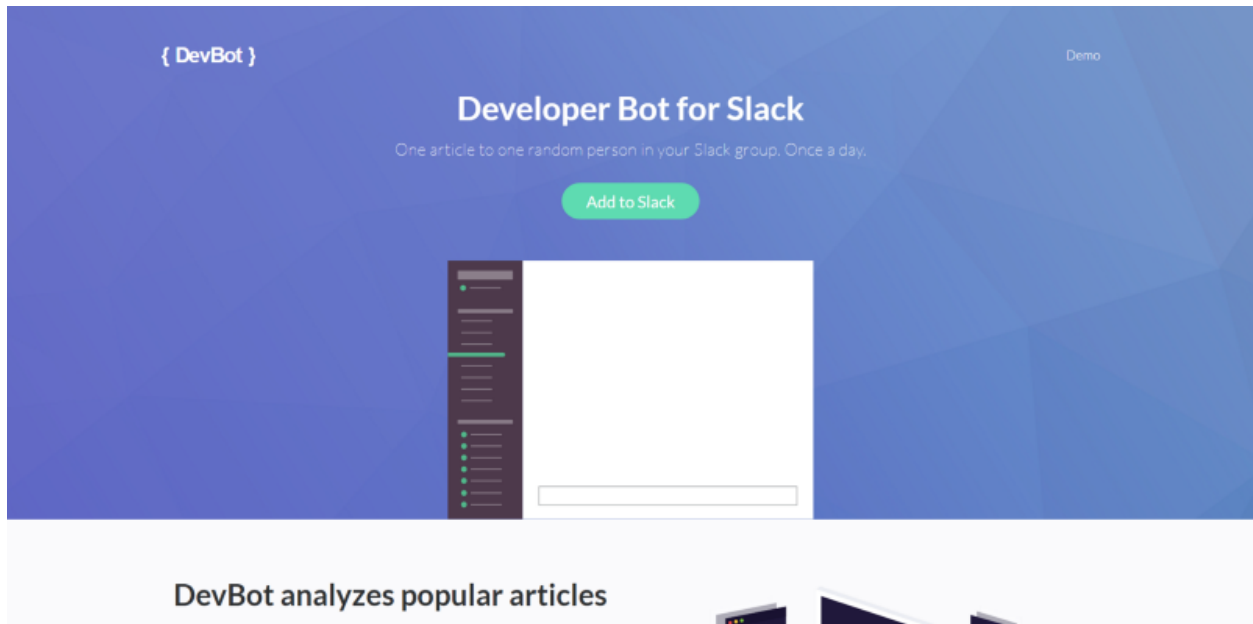
The Scope of This Chapter



I previously mentioned that the main focus of this chapter was going to be the creation of a chat bot demo. However, I thought I'd spend a chapter on some preliminary setup before we get to that.

Let me highlight what we will be doing.

In the landing page we just created, there is a link called Demo in the right-hand side of the navigation:



This link currently does nothing. However, we want to toggle off the current view (the landing page) and toggle on a new view (a chat bot demo) in our single page application. We will be going over the solution to this.

In this chapter, we are just going to focus on how to control the active view within a single page application using React Router. We will also go over how multiple views impacts the passing of props.

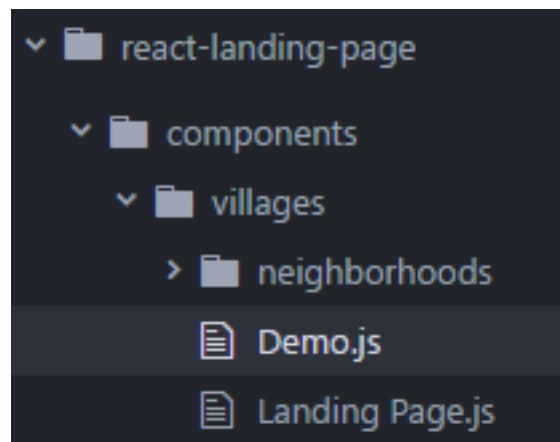
Fortunately, this chapter should be a bit shorter, and therefore, very refreshing.

Creating Our Demo Village



We will need to make a lot of changes to this but let's create a basic Demo village.

Create a new file called *Demo.js* underneath the villages folder:



This file is going to be just like our *Landing Page.js* except we only want to nest the Header and Footer components like so:

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3
4   import Header from './neighborhoods/Header.js';
5   import Footer from './neighborhoods/Footer.js';
6
7   class Demo extends React.Component {
8     render() {
9       return (
10        <div className="demo">
11          <Header/>
12          <Footer/>
13        </div>
14      )
15    }
16  }
17
18  module.exports = Demo;

```

Next, we can import and nest this within *index.js*:

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4   import LandingPage from './components/villages/Landing Page.js';
5   import Demo from './components/villages/Demo.js';
6
7   class App extends React.Component {
8     render() {
9       return (
10        <LandingPage/>
11        <Demo/>
12      )
13    }
14  }
15
16  ReactDOM.render(
17    <App/>,
18    document.getElementById('app')
19  );

```

Ok. We now have a LandingPage village and a Demo village. How do we control which one is active?

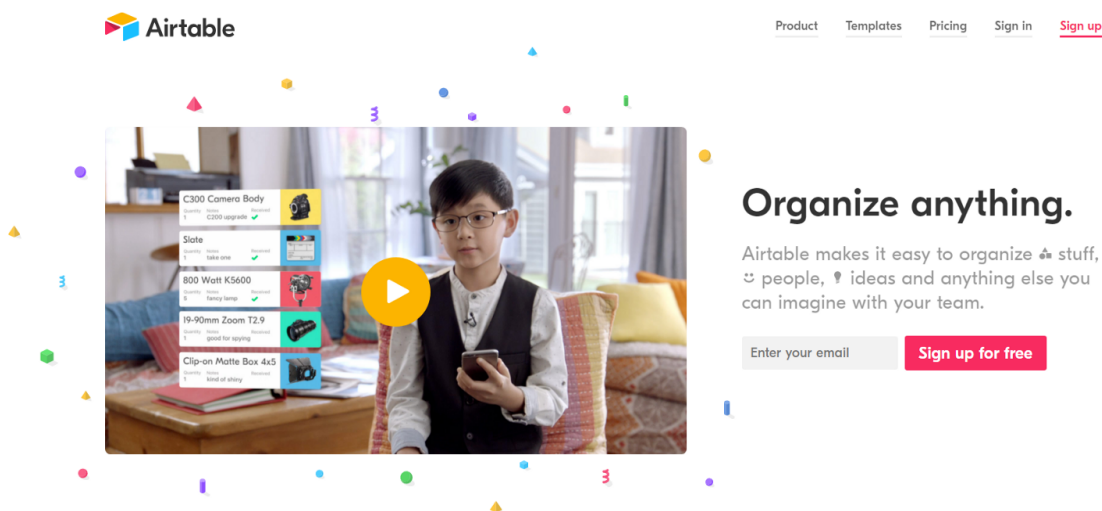
Implementing React Router



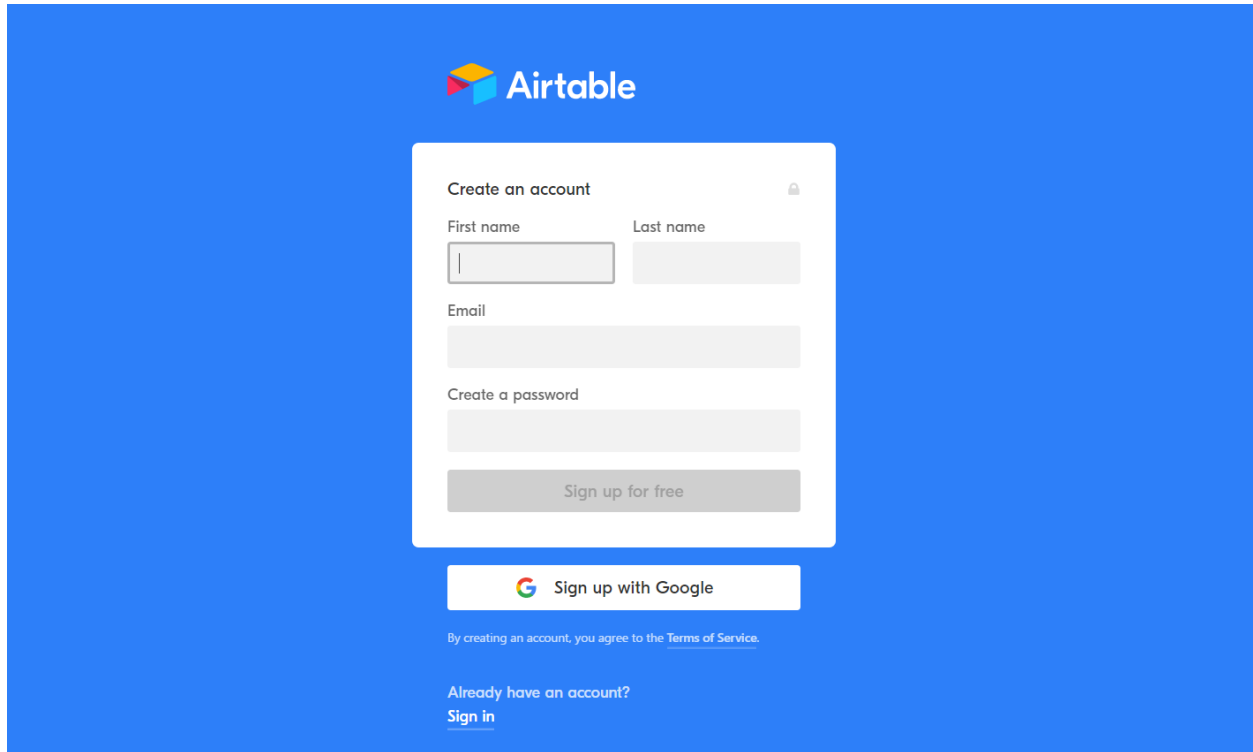
What is it and Why do we need it?

Let's say we made a single page application that had a landing page and a sign up page. Because it's a single page application, the landing page and sign up page are both two separate villages (views).

Let's say we displayed the following landing page:



This is the active village. Now, let's say we clicked the *Sign Up* link in the top right corner. We would want that to toggle off the current landing page village and toggle on the sign up village so the following is rendered:



How do we do this?

Well, we discussed conditional rendering back in Chapter 5.

Conditional rendering refers to being able to toggle on and toggle off what components and elements that render.

Here's an example of applying conditional rendering:

```
1  render() {  
2    const isLoggedIn = this.state.isLoggedIn;  
3    return (  
4      <div>  
5        {isLoggedIn ? (  
6          <LogoutButton onClick={this.handleLogoutClick} />  
7        ) : (  
8          <LoginButton onClick={this.handleLoginClick} />  
9        )}  
    )  
  }
```

```
10     </div>
11   );
12 }
```

In the example above, we can toggle a *LogoutButton* component or a *LoginButton* component depending if *isLoggedIn* is true or false.

In Chapter 5, we also discussed how to update data on some type of event. Specifically, we looked at changing data on a user's click.

Let's say the single page app without conditional rendering looked like this:

```
1  class App extends React.Component {
2    render() {
3      return (
4        <LandingPage/>
5        <SignUp/>
6      )
7    }
8  }
9
10 ReactDOM.render(
11   <App/>,
12   document.getElementById( 'app' )
13 );
```

In theory, we could control whether the *LandingPage* or *SignUp* village component is rendered based on some Boolean flag (data that is either true or false). If someone clicks the Sign Up link, we could change the flag via an event handler so the condition changes and we render the *SignUp* village in place of the *LandingPage* village.

There are a couple of issues with this.

It would get really complicated to have a bunch of flags and event handlers to control the conditional rendering for all of the links in the navigation menu that will change the view.

More importantly, we need to be able to update the URL depending on what view is active.

Continuing off of the second issue, we need to maintain the traditional change of the URL with the change of a page that is adopted for single page applications so that the URL changes on the change of a *view*.

Fortunately, there is a tool within the greater React ecosystem called *React Router*.

> React Router makes it easy for us to toggle on and off views and updating the URL accordingly.

Meaning, the LandingPage component would display a URL of:

<https://airtable.com/>⁴⁶

The click of the Sign Up link would toggle off the LandingPage component, toggle on the SignUp component, and update the URL to:

<https://airtable.com/signup>⁴⁷

All of this can be done neatly using React Router.

Installing via npm

The first step is for us to install React Router via npm.

For that, we can do the following:

```
npm install --save react-router-dom
```

Note: *-dom* is appended to distinguish between *-native*. The former is React Router for web application and the latter is React Router for mobile (native) applications.

Importing React Router

Now that we have installed React Router, we need to import it to our *index.js* since it is the highest level of our application where all the villages (views) are directly nested.

Like our *react-grid-system* import statements, we want to import specific properties.

With React Router, we can add the following import at the top of *index.js*:

```
1 import { BrowserRouter as Router, Route, Link } from 'react-router-dom'
```

We are basically saying: “Hey react-router-dom! I’m going to go ahead and grab BrowserRouter, Route, and Link. By the way, I think BrowserRouter is a bit of a tongue-twister. I’m going to go ahead and refer to BrowserRouter *as Router* in our code.

⁴⁶<https://airtable.com/signup>

⁴⁷<https://airtable.com/signup>

Configuring the Router With Routes

Router can be thought of as the encapsulation of all the possible URL paths and the components that are rendered on those paths.

Routes refer to the specific URL path (i.e. /login or /signup) and the view component that should be rendered on that path.

In *index.js*, let's add the shell of our Router in place of the two villages we have nested:

```

1      class App extends React.Component {
2          render() {
3              return (
4                  <Router>
5
6                  </Router>
7              )
8          }
9      }

```

Next, we need to add our Routes. Let's start by adding our LandingPage component as a route on the home :

```

1      class App extends React.Component {
2          render() {
3              return (
4                  <Router>
5                      <div>
6                          <Route exact path="/" component={LandingPage}/>
7                      </div>
8                  </Router>
9              )
10         }
11     }

```

The syntax for adding a Route can be seen to actually be quite simple:

```

1      <Route path="*insert relative path from home" component={*insert imported co\
2      mponent*} />

```

Note we add *exact* before *path* for our home page (path equal to "/").

Also, note that we have to have an outermost div in between our Router.

Next, let's add the Route for our Demo village component:

```

1    class App extends React.Component {
2      render() {
3        return (
4          <Router>
5            <div>
6              <Route exact path="/" component={LandingPage}/>
7              <Route path="/demo" component={Demo}/>
8            </div>
9          </Router>
10         )
11       }
12     }

```

Now, we will have a URL of

domain/demo //example: devbot.io/demo

when our Demo component is made active.

In total, our *index.js* should look like this:

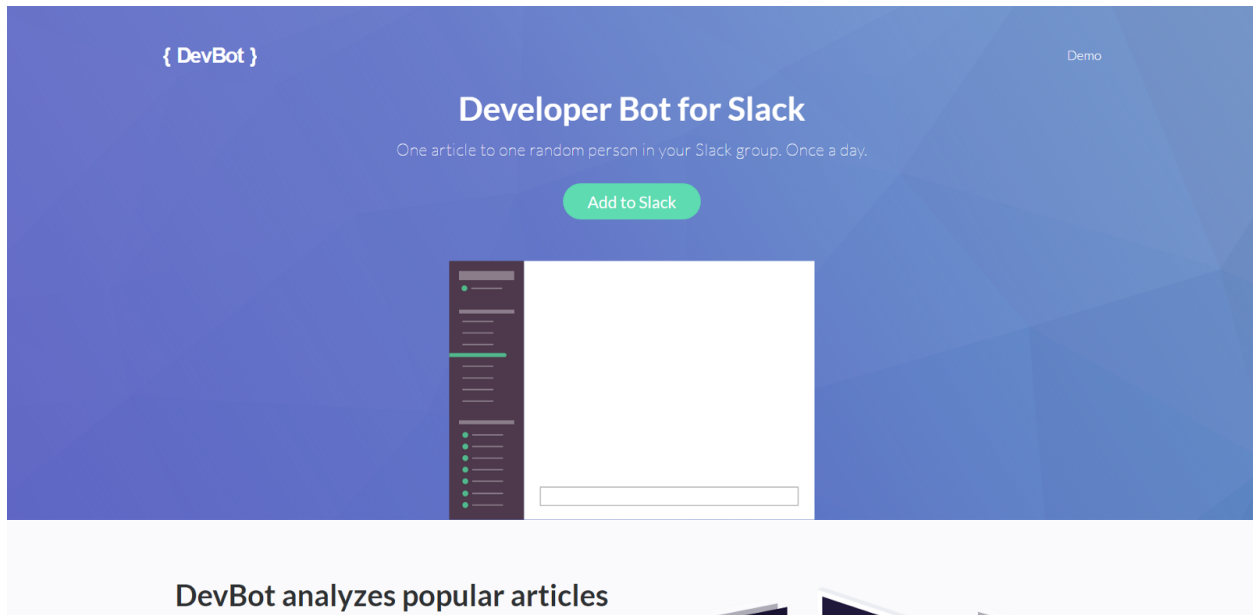
```

1    import React from 'react';
2    import ReactDOM from 'react-dom';
3    import { BrowserRouter as Router, Route, Link } from 'react-router-dom'
4    import { Container, Row, Col } from 'react-grid-system';
5    import LandingPage from './components/villages/Landing Page.js';
6    import Demo from './components/villages/Demo.js';
7
8    class App extends React.Component {
9      render() {
10        return (
11          <Router>
12            <div>
13              <Route exact path="/" component={LandingPage}/>
14              <Route path="/demo" component={Demo}/>
15            </div>
16          </Router>
17        )
18      }
19    }
20
21    ReactDOM.render(
22      <App/>,
23      document.getElementById('app')
24    );

```

Adding Links

cd into the project root, run `npm start`, and go to the local host address. You will see that our landing page is still rendering:



This means we haven't broken our code yet. However, our React Router configuration is incomplete. We need to have the Demo Route activated on the click of the link on the right-hand side of our navigation.

That link is configured in our DemoLink component so let's open *Demo Link.js*.

First off, we need to import the Link property from React Router like so:

```
import { Link } from 'react-router-dom'
```

The DemoLink component currently looks like this:

```
1  class DemoLink extends React.Component {
2    render() {
3      return (
4        <div>
5          <a href="#">
6            <span className="demo-link">
7              Demo
8            </span>
9          </a>
10         </div>
```

```

11     )
12   }
13 }

```

We can replace the `<a></a>` with `Link` which we just imported:

```

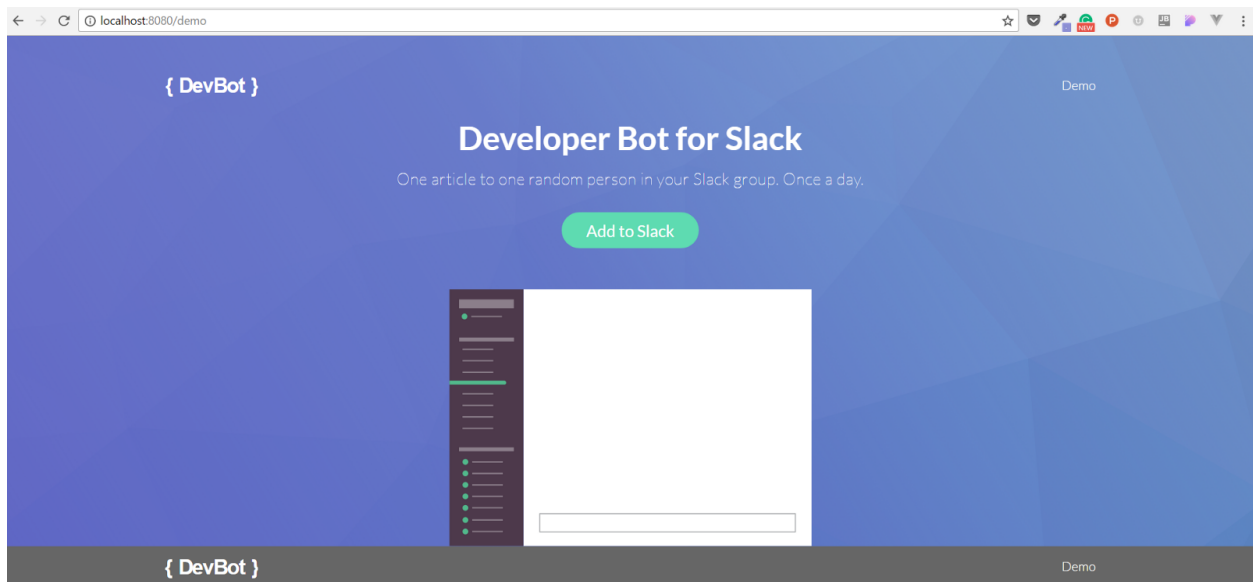
1  class DemoLink extends React.Component {
2    render() {
3      return (
4        <div>
5          <Link to="/demo">
6            <span className="demo-link">
7              Demo
8            </span>
9          </Link>
10         </div>
11       )
12     }
13   }

```

The syntax here is very straightforward.

Awesome! Let's test this out. (Note: make sure to update the CSS to match what's on [Github](https://github.com/michaelmang/React-for-the-Visual-Learner/blob/master/chapters/chapter%207/styles/landing-page.css)⁴⁸)

When we click on "Demo" in our navigation, it should now render our Demo component and update the URL:

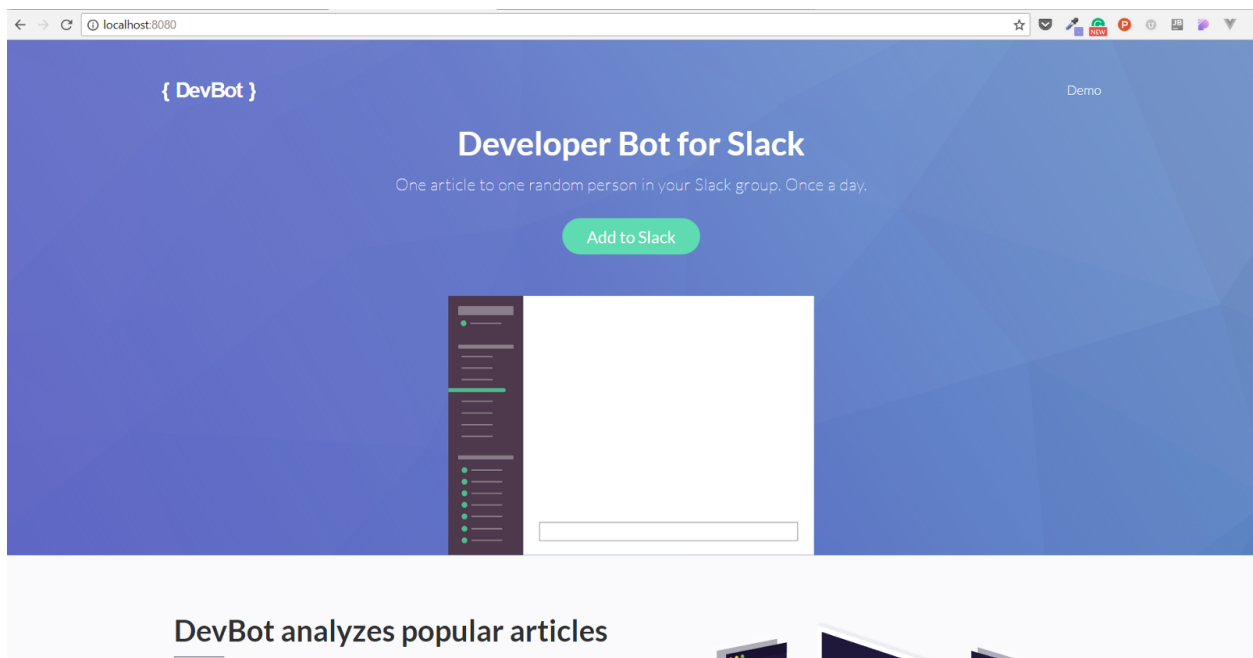


⁴⁸<https://github.com/michaelmang/React-for-the-Visual-Learner/blob/master/chapters/chapter%207/styles/landing-page.css>

Now, let's also add a link back to the landing page (home) on the click of our Branding component. Updated *Branding.js* with the following:

```
1  import React from 'react';
2  import ReactDOM from 'react-dom';
3  import { Link } from 'react-router-dom'
4
5  class Branding extends React.Component {
6    render() {
7      return (
8        <div>
9          <Link to="/">
10             
11          </Link>
12        </div>
13      )
14    }
15  }
16
17  module.exports = Branding;
```

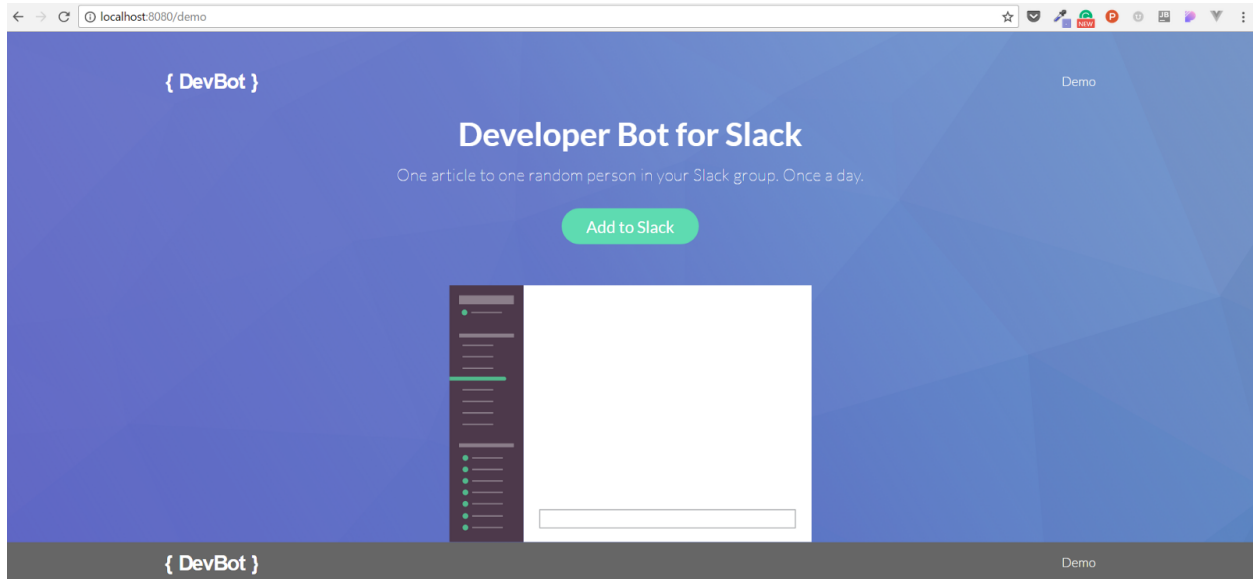
We can now go back to the LandingPage view from our Demo view via the logo:



Cool! We have successfully implemented React Router.

Adding Props

Let's take a look at what our basic Demo village:



We eventually will make a lot of changes to make this an interactive chat demo. However, we are going to keep the Navigation and CallToAction components.

The issue is that we want to have different text for the Title and Subtitle components within the CallToAction component depending on if we are on the LandingPage view or the Demo view. Create more components is not an option.

How do we resolve this? We can have the text of the Title and Subtitle be controlled by props. We can then pass different props for each view.

We will now see why we practiced passing down props previously in this book.

First things first, let's inject the text in Title and Subtitle that we will pass later.

Open up *Title.js* and *Subtitle.js*.

In each one, let's replace the text with an inject prop called *title* and *subtitle* respectively.

Title.js

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3
4   class Title extends React.Component {
5     render() {
6       return (
7         <div>
8           <h1 className="title">{this.props.title}</h1>
9         </div>
10      )
11    }
12  }
13
14  module.exports = Title;
```

Subtitle.js

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3
4   class Subtitle extends React.Component {
5     render() {
6       return (
7         <div>
8           <h3 className="subtitle">{this.props.subtitle}</h3>
9         </div>
10      )
11    }
12  }
13
14  module.exports = Subtitle;
```

The next step is to pass down these props starting at the Header component in *Demo.js* and *LandingPage.js*.

Let's do the following for *Demo.js*:


```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3
4   import Header from './neighborhoods/Header.js';
5   import Footer from './neighborhoods/Footer.js';
6
7   class Demo extends React.Component {
8     render() {
9       return (
10        <div className="demo">
11          <Header title="Chat Bot Demo" subtitle="Watch how it works."/>
12          <Footer/>
13        </div>
14      )
15    }
16  }
17
18  module.exports = Demo;
```

...and the following for *Landing Page.js*:

```
1   import React from 'react';
2   import ReactDOM from 'react-dom';
3
4   import Header from './neighborhoods/Header.js';
5   import SectionA from './neighborhoods/Section A.js';
6   import SectionB from './neighborhoods/Section B.js';
7   import SectionC from './neighborhoods/Section C.js';
8   import Footer from './neighborhoods/Footer.js';
9
10  class LandingPage extends React.Component {
11    render() {
12      return (
13        <div className="landing-page">
14          <Header title="Developer Bot for Slack" subtitle="One article to one\
15 random person in your Slack group. Once a day."/>
16          <SectionA/>
17          <SectionB/>
18          <SectionC/>
19          <Footer/>
20        </div>
21      )
22    }
23  }
```

```

22     }
23   }
24
25   module.exports = LandingPage;

```

Now, we have defined two different texts for Title and Subtitle for our different views. Next, we need to finish off by working our way down the component hierarchy and passing the props down.

Open *Header.js* which is the next level down.

This file now can access the props by doing `{this.props._____}`.

We then want to pass down title and subtitle as props to *CallToAction* which nests the Title and Subtitle components:

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4   import Navigation from './blocks/Navigation.js';
5   import CallToAction from './blocks/Call To Action.js';
6   import SlackChannel from './blocks/houses/Slack Channel.js';
7
8   class Header extends React.Component {
9     render() {
10      return (
11        <section className="header">
12          <Container>
13            <Row>
14              <Navigation/>
15            </Row>
16            <Row>
17              <CallToAction title={this.props.title} subtitle={this.props.subt\
18 title}/>
19            </Row>
20            <Row>
21              <SlackChannel/>
22            </Row>
23          </Container>
24        </section>
25      )
26    }
27  }
28
29  module.exports = Header;

```

Now, let's open *Call To Action.js*.

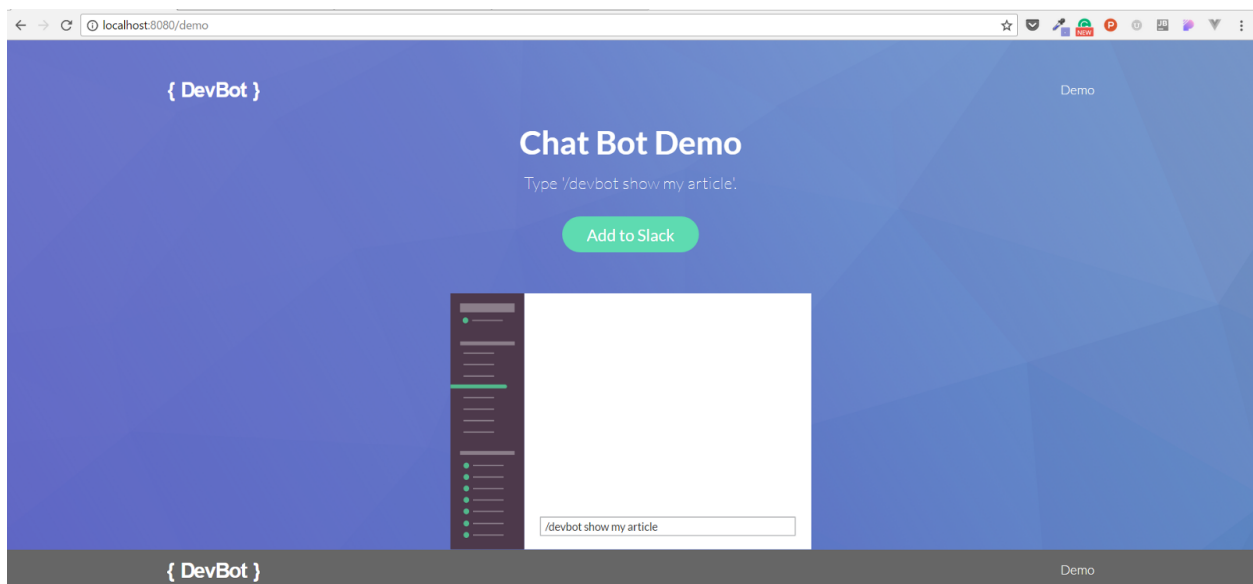
We want to pass the title prop in the Title component and the subtitle prop in the Subtitle component like so:

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4   import Title from './houses/Title.js';
5   import Subtitle from './houses/Subtitle.js';
6   import Button from './houses/Button.js';
7
8   class CallToAction extends React.Component {
9     render() {
10      return (
11        <Col lg={12}>
12          <Title title={this.props.title}/>
13          <Subtitle subtitle={this.props.subtitle}/>
14          <Button/>
15        </Col>
16      )
17    }
18  }
19
20  module.exports = CallToAction;

```

We already injected these in both our Title and Subtitle classes which means we are all set!



Cool beans! We have different text for each view using props.

Before we move on, we can't forget that we use the `CallToAction` component (which contains `Title` and `Subtitle`) in our `SectionC` component nested in *Landing Page.js*.

First, we need to define title and subtitle props in the `SectionC` component:

```
1  import React from 'react';
2  import ReactDOM from 'react-dom';
3
4  import Header from './neighborhoods/Header.js';
5  import SectionA from './neighborhoods/Section A.js';
6  import SectionB from './neighborhoods/Section B.js';
7  import SectionC from './neighborhoods/Section C.js';
8  import Footer from './neighborhoods/Footer.js';
9
10 class LandingPage extends React.Component {
11   render() {
12     return (
13       <div className="landing-page">
14         <Header title="Developer Bot for Slack" subtitle="One article to one\
15 random person in your Slack group. Once a day."/>
16         <SectionA/>
17         <SectionB/>
18         <SectionC title="Developer Bot for Slack" subtitle="One article to o\
19 ne random person in your Slack group. Once a day."/>
20         <Footer/>
21       </div>
22     )
23   }
24 }
25
26 module.exports = LandingPage;
```

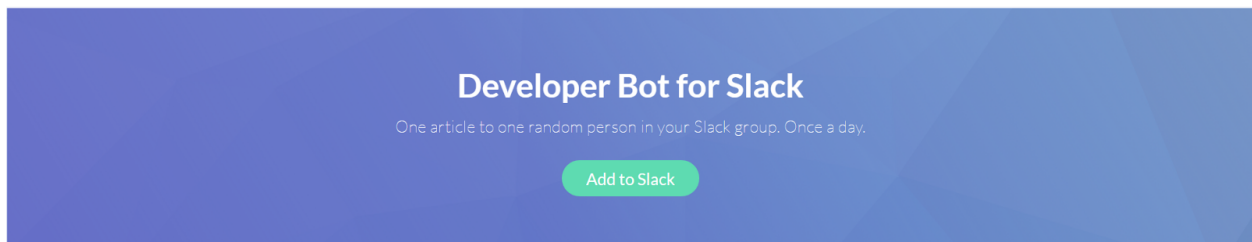
Then, let's update *Section C.js* so that it passes down the title and subtitle props that were defined initially in *Landing Page.js*:

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4   import CallToAction from './blocks/Call To Action.js';
5
6   class SectionC extends React.Component {
7     render() {
8       return (
9         <section className="section-c">
10           <Container>
11             <Row>
12               <CallToAction title={this.props.title} subtitle={this.props.subt\
13 title}/>
14             </Row>
15           </Container>
16         </section>
17       )
18     }
19   }
20
21   module.exports = SectionC;

```

Sweet! All better.



We now have a taste of how to pass down different props depending on the different villages (views). We also can see the practical use case where we would need to do so.

Final Code

Available here on Github.⁴⁹

Concluding Thoughts

The first time I got React Router working, I remember it was such a cool feeling. I get clicking links back and forth in amazement of the super fast switching between views.

⁴⁹<https://github.com/michaelmang/React-for-the-Visual-Learner/tree/master/chapters/chapter%208>

Hopefully, learning React Router was as cool for you as it was for me. In addition, this chapter was a nice change of pace to catch our breath from all the work we've been cramming into previous chapters.

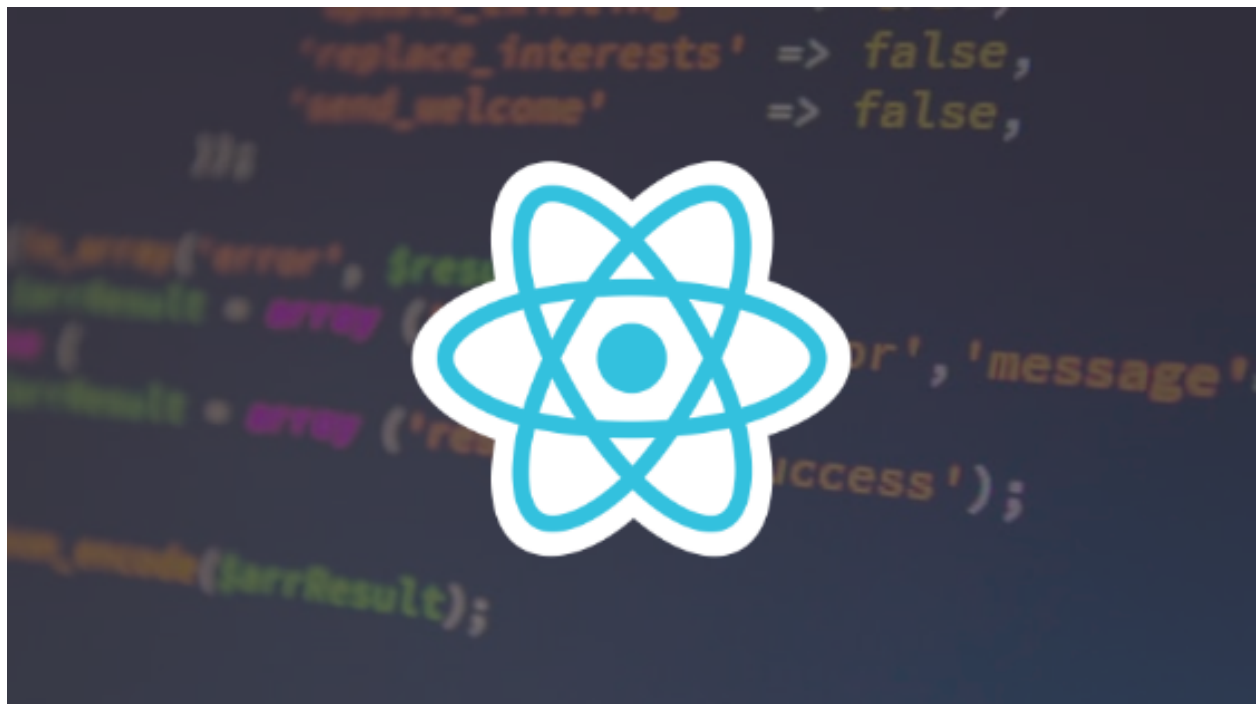
Take a breath and get excited because we've covered all the major fundamentals for making a React user interface.

The next chapter is going to tie all that we have learned together into a nice bow as we make a chat bot demo.

Chapter 9 | Chat Bot Demo Using Firebase



Defining “Chat Bot Demo”



I've already taken a few stabs at writing this chapter before I ended up scratching it and restarting. The reason for this is because I have been unsure about how complex I wanted to make it. On one hand, it could be very simple. Instead of actually making the chat bot, we could just do some

visual tricks to make it appear as if there was a chat bot demo in our Demo view. For example, we could have an image of a Slack channel with no message. After a few seconds, we change the image to a Slack channel with a message from a user to the chat bot. After a few more seconds, we could switch the image to a Slack channel with a response from the chat bot. We then could repeat this process. In total, it would create the visual appearance of a chat bot demo.

On the other hand, we could create a fully-functioning chat bot and embed it into the Demo view. This would require writing code to configure our own server and interact with a database to store information about the user and respond with a suggested article (matching the claims of the landing page text).

As I was pondering on how complex the latter option would be, I realized that thinking through it was actually a great way to explain the current state of our application.

The State of Our Application

By big picture, I mean where our current React app fits in the client/server architecture of web applications.

Earlier, we have discussed that React applications use data and instruct the DOM with what should be rendered. This all happens *client-side*.

On the *server-side*, there is a server that hosts our application and serves it to the browser. The server can fetch data from a database and send it to our React application where we can use it within components.

In our application, we have only written *client-side* code. We created our React application and used Webpack to take our app, apply Babel to do preprocessing, and spit out the final product. In *package.json*, we configured *webpack-dev-server* (which we installed via npm) to run on the following command:

```
npm start
```

webpack-dev-server hosts our final application and serves it to the browser. We then can enter the following address in our browser to see our application:

```
http://localhost:8080/
```

React Router controls which view is active and updates the URL accordingly.

All of this to say, we have not configured a server for ourselves to host our application and serve it to the browser. We also have not had to use data from a database.

The Functionality of Our Chat Bot Demo

With the state of our current application in mind, you may be able to see why I was unsure about whether to make a more complex chat bot demo that would require writing server-side code and

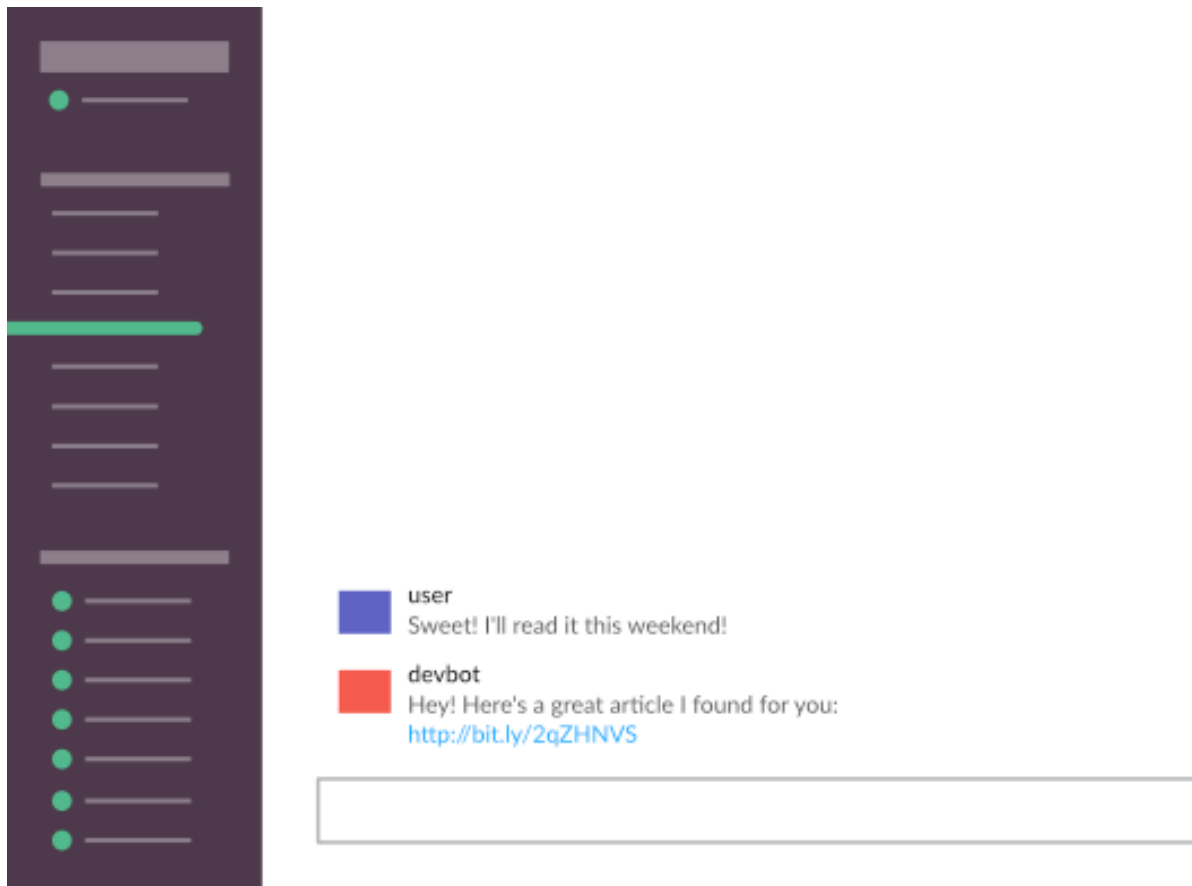
interact with a database. I am concerned that a complex chat bot would be crammed and rushed to fit into this chapter. On the other hand, mere visual tricks seem to be a lame way to wrap up this book.

So, what are we going to do about our chat bot demo?

I am going to land in-between a fully-functioning chat bot and just changing the images. Let's walk through the functionality of our application together.

First, we are going to add house components for a *user message* (*UserMessage*) and a *chat bot message* (*ChatBotMessage*). Both of these components will be nested within our *SlackChannel* component. Therefore, the *SlackChannel* component will be bumped up from a house component (as it currently is just an image) to a block component (as it will nest our new components).

A mockup for the *SlackChannel* component with the new *UserMessage* and *ChatBotMessage* components can be seen below:

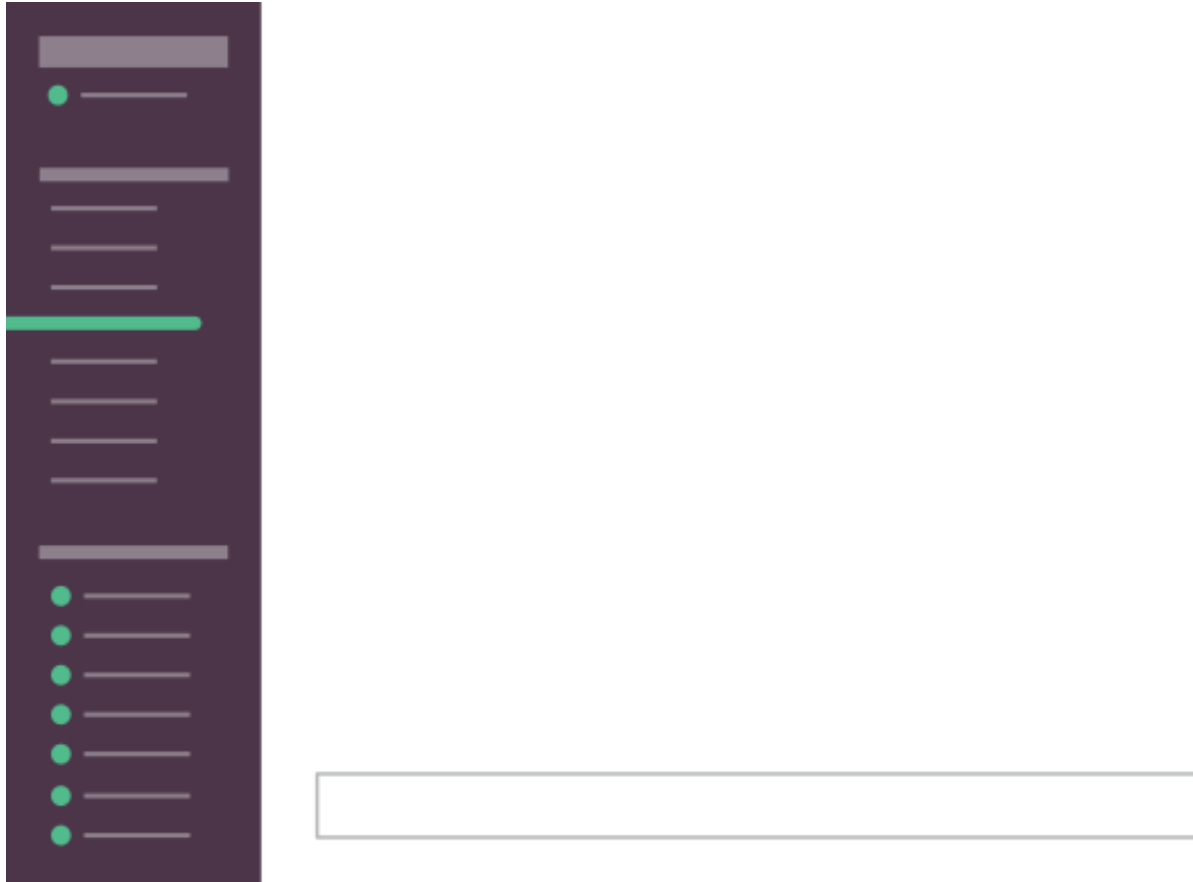


Because the *SlackChannel* component is used in the *LandingPage* village and the *Demo* village, we will use conditional rendering so the *UserMessage* and *ChatBotMessage* components only render for the *Demo* village.

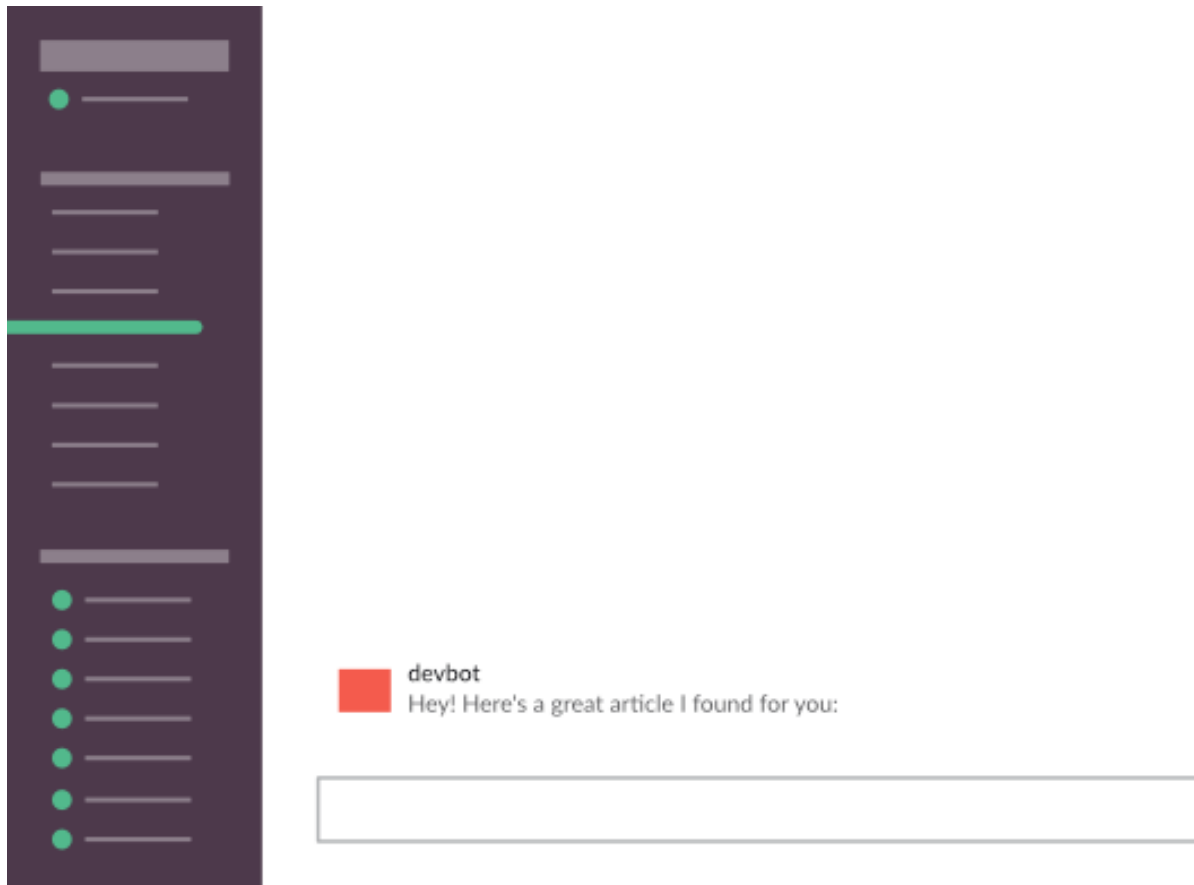
Although we want the *Demo* village to ultimately render *SlackChannel* block with the *UserMessage*

and ChatBotMessage houses as seen in the mockup above, we want to reveal them in a specific order.

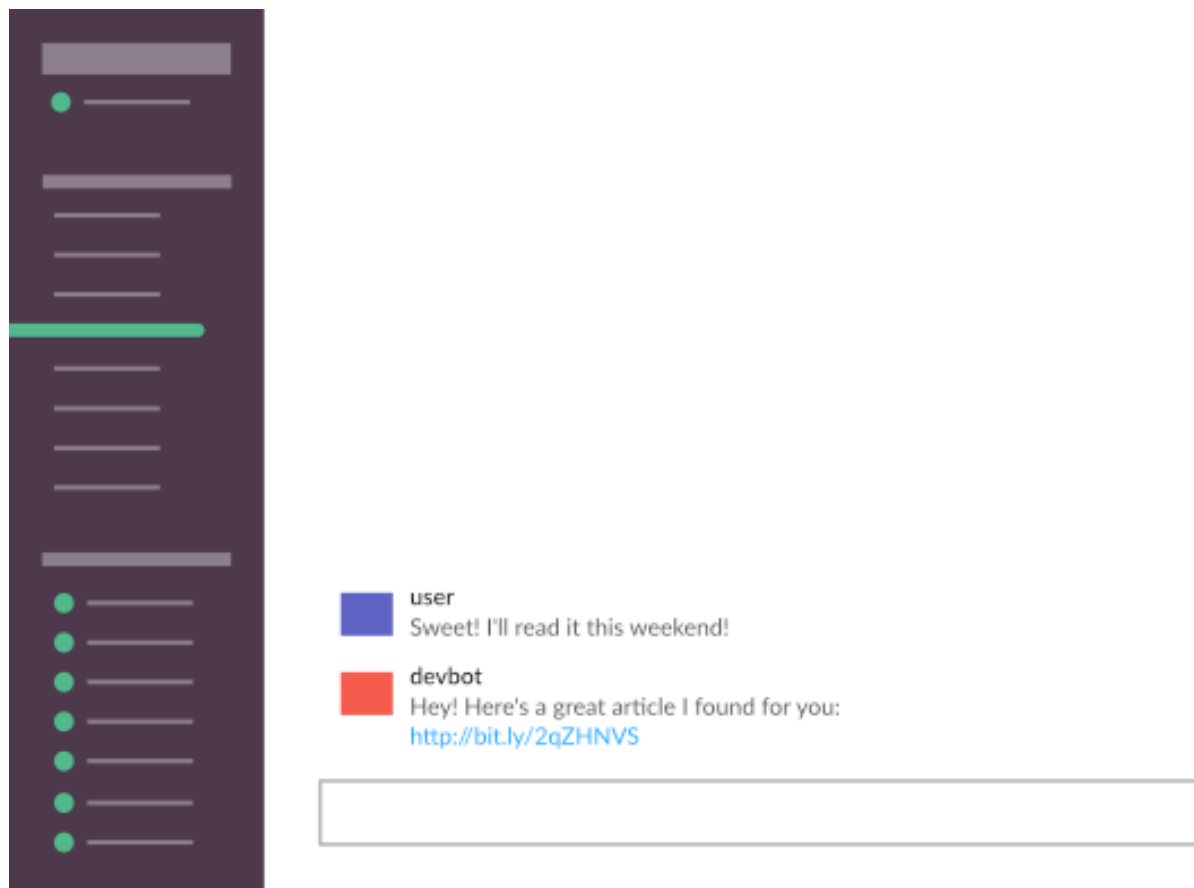
We will time the revealing through lifecycle hooks and control what is being rendered using more conditional rendering. When the SlackChannel component *mounts*, we will have just a blank Slack Channel image:



1 second after this mounts, we will render the ChatBotMessage component:



1 second after that, we will render the final `UserMessage` component:



While we won't be creating our own server, we will use a cloud-based database called Firebase. Firebase will store the information used to populate the `UserMessage` and `ChatBotMessage` components. Specifically, the usernames, avatar colors, message texts, and the article link will be stored and retrieved from Firebase to populate the components instead of us manually passing down props.

If this seems a bit fuzzy, don't fret. We are going to explain this in plenty of detail when we begin the coding. You only need to understand the basic gist.

Additionally, we are going to spend a lot of time understanding Firebase before any of the coding for the chat bot demo.

Understanding Firebase



Firebase is a backend as a service. In short, it takes care of a lot of backend development for you (authorization, databases, etc.). In our use case, it can provide a cloud-based database for storing and syncing data without having to write our own backend code. It will do a lot of the traditional setup and coding for a database behind the scenes. This will allow us to get the feel of using a data from a database in a React application without being overwhelming.

With that being said, it's not as if Firebase is just a temporary thing that we will use for learning purposes that you wouldn't use in the real-world. Firebase is a valid alternative to traditional databases and can be used for full-scale, dynamic web applications.

Let's get started with learning Firebase. To learn Firebase, we will create a Tile Background Changer Mini-App via Codepen.

Setup for Tile Background Changer Mini-App

First things first, go ahead and [fork my template](https://codepen.io/mikemang/pen/5239a204de421a0f2a8ff40c33454da9)⁵⁰ which contains all the HTML, CSS, and proper React settings.

Then, rename it **Tile Background Changer**

Firebase Demo using React.

⁵⁰<https://codepen.io/mikemang/pen/5239a204de421a0f2a8ff40c33454da9>

Functionality of the Tile Background Changer Mini-App



Let's go over what we want this mini-app to do.

We are going to one component called Tile nested 3 times within an App component. Each time to Tile component is nested, we will pass the class down as a prop. This allows us to reuse 1 component 3 times yet having a different class for each for styling purposes.

Each tile's background color will be controlled using inline JSX styling. We will initialize the state and have a property called color (which will be changing hence why we are using state) that will have an initial value of the hex code for white (#FFFFFF).

The Tile component's JSX will render three elements. One element will be the tile with the color defined by state. Another element will be a text box that will contain the current color's hex code. The final element will be the actual text for the hex code.

Here is the visual of the initial tiles (before any events):



On the click of each tile, we are going to have an event handler that will *read* a certain hex code that we will have stored in a Firebase database. The value of the hex code from the database will be set as the new *color* in the state. This will cause the background color of the tile to change and the hex code text underneath the tile like so:



Here's the final app which we will make together⁵¹

React Components Setup

First, we need a component for our app that will nest the tile components:

```
1    //App
2    class App extends React.Component {
3      render() {
4        return (
5          <div className="container">
6            </div>
7          )
8        }
9      }
```

Next, let's add the ReactDOM portion:

```
1    // "inject" to DOM
2    ReactDOM.render(
3      <App/>,
4      document.getElementById('app')
5    );
```

Piece of cake!

Now, let's define the shell of our Tile component:

```
1    class Tile extends React.Component {
2
3      render() {
4        return (
5          <div className={this.props.class}>
6            <div className="color-text-box">
7              <div className="color-text">
8                </div>
9              </div>
10             </div>
11           )
12         }
13       }
```

⁵¹<https://codepen.io/mikemang/pen/9868a5661de1ff52700b77ac11194396?editors=0010>

As previously mentioned, we want this component to render the tile (tile-1, tile-2, or tile-3 depending on the class prop we will pass down), a box to contain the text of the current color (color-text-box), and the actual text of the current color (color-text). The elements are nested for styling purposes.

Next, let's nest this component in our App component and define a prop called *initialColor* with a hex code for the color white:

```

1      //App
2      class App extends React.Component {
3          render() {
4              return (
5                  <div className="container">
6                      <Tile class={"tile-1"}/>
7                  </div>
8              )
9          }
10     }

```

We have 3 more steps remaining to complete our first tile component:

Initialize a state with a property called *color* and assign it the value of the #FFFFFF

Use inline JSX styling so the background of the tile-1 element is set to the color property defined in the previous step

Inject the text of the color property within the color-text element

For the first step, we can add the following:

```

1      class Tile extends React.Component {
2
3          **state = {color: #FFFFFF};**
4
5          render() {
6              return (
7                  <div className="{this.props.class}">
8                      <div className="color-text-box">
9                          <div className="color-text">
10                             </div>
11                         </div>
12                     </div>
13                 )
14             }
15         }

```

Because the property *color* is assigned the value of the *initialColor* prop, we did *color: this.props.initialColor*.

To complete the next step, we add the following code for the inline JSX styling:

```

1      class Tile extends React.Component {
2
3          state = {color: #FFFFFF};
4
5      render() {
6          return (
7              <div style={{background: this.state.color}}** className={this.props.\
8 class}>
9                  <div className="color-text-box">
10                     <div className="color-text">
11                         </div>
12                     </div>
13                 </div>
14             )
15         }
16     }

```

Last but not least, we can inject the color property like so:

```

1      class Tile extends React.Component {
2
3          state = {color: #FFFFFF};
4
5      render() {
6          return (
7              <div style={{background: this.state.color}} className="{this.props.cla\
8 ss}">
9                  <div className="color-text-box">
10                     <div className="color-text">
11                         **{this.state.color}**
12                     </div>
13                 </div>
14             </div>
15         )
16     }
17 }

```

We should now see the following:



To add two more tiles, we will reuse our Tile component with a different class prop:

```
1    //App
2    class App extends React.Component {
3      render() {
4        return (
5          <div className="container">
6            <Tile class={"tile-1"}/>
7            <Tile class={"tile-2"}/>
8            <Tile class={"tile-3"}/>
9          </div>
10         )
11       }
12     }
```



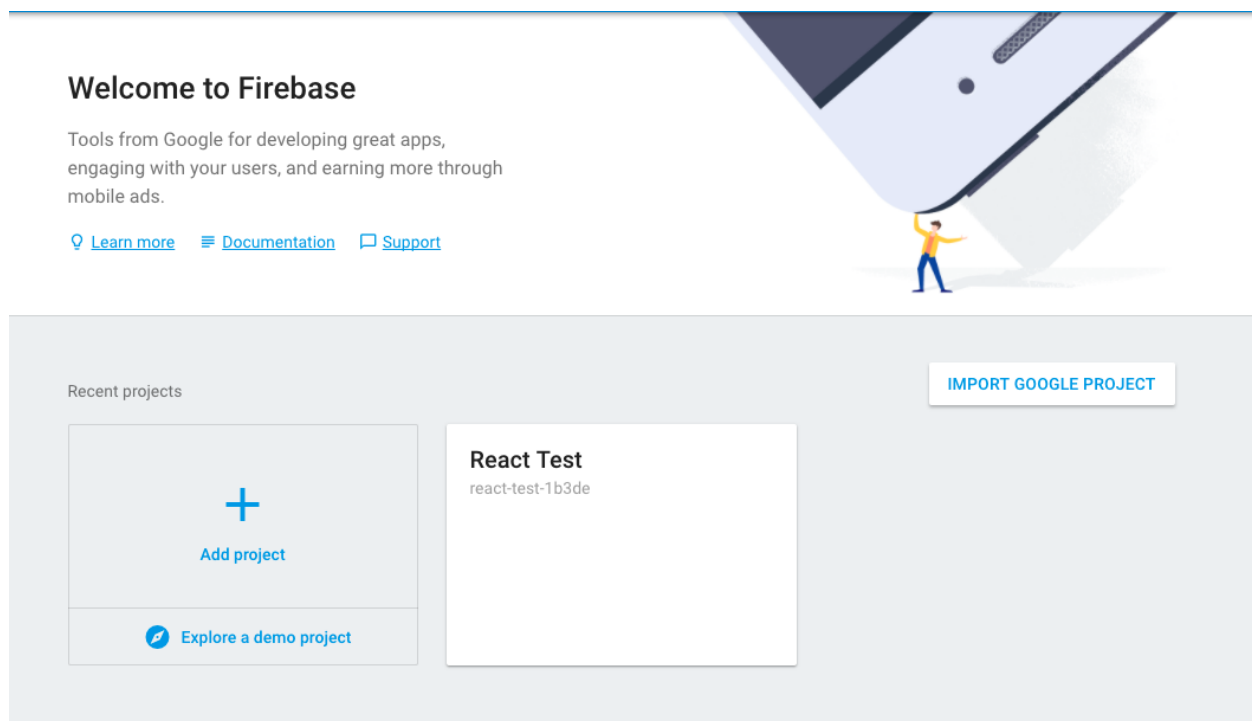
Awesome! Before we start adding event handlers, we need to setup our database in Firebase.

Creating a New Firebase Project

Head over to [Firebase](https://firebase.google.com/)⁵² and select **Get Started**.

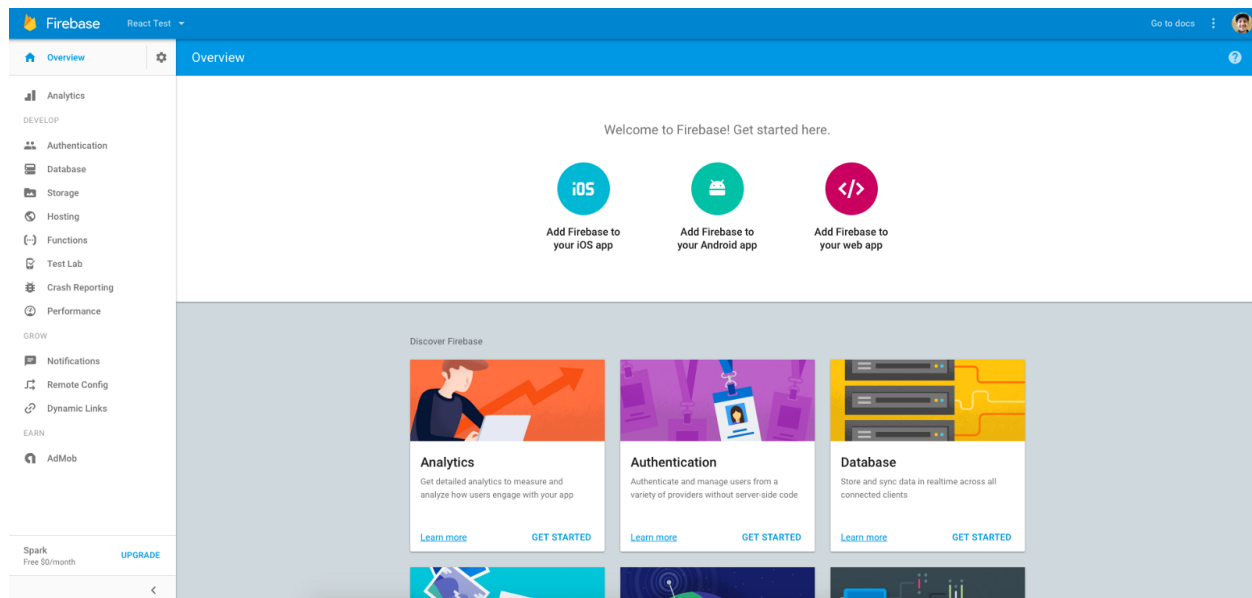
You'll be brought to the following dashboard:

⁵²<https://firebase.google.com/>



Click **Add project** and name it **React Test** as you can see I have already done.

It will take you to a page with a lot going on:



Resist the urge to go off and explore and select **Database** on the left.

Before we doing anything further, we need to understand how structuring of data works.

The Structure of Data in Firebase

If you are familiar with SQL databases, you might expect to see our data structured in tables and rows. However, Firebase stores its data as JSON objects. Data in Firebase is then structured like a JSON tree.

An easy place to go to explain this is the `package.json` in our `react-landing-page` project:

```
1  {
2    "name": "react-landing-page",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "start": "webpack-dev-server"
8    },
9    "author": "",
10   "license": "ISC",
11   "dependencies": {
12     "react": "^15.5.4",
13     "react-dom": "^15.5.4",
14     "react-grid-system": "^2.9.1",
15     "react-router-dom": "^4.1.1"
16   },
17   "devDependencies": {
18     "babel-core": "^6.24.1",
19     "babel-loader": "^7.0.0",
20     "babel-preset-env": "^1.5.1",
21     "babel-preset-react": "^6.24.1",
22     "html-webpack-plugin": "^2.28.0",
23     "node-sass": "^4.5.3",
24     "sass-loader": "^6.0.5",
25     "webpack": "^2.6.0",
26     "webpack-dev-server": "^2.4.5"
27   }
28 }
```

Here, we have data about our project structured in the format of *name* and *value*. One example being:

```
1      "name": "react-landing-page"
2
3      //name: value
```

Sometimes, data needs to be nested under a *node* like with scripts, dependencies, and devDependencies in our package.json. One example being:

```
1      "scripts": {
2          "start": "webpack-dev-server"
3      }
4
5      //node
6      //name: value
```

The node is simply a name that can be used to access a collection of names and values.

Here's another example of a JSON tree to represent some data that might be needed for an application:

```
1      {
2          "users": {
3              "alovelace": {
4                  "name": "Ada Lovelace",
5                  "role": "Developer",
6              },
7              "ghopper": { ... },
8              "eclarke": { ... }
9          }
10     }
```

Not too bad, huh?

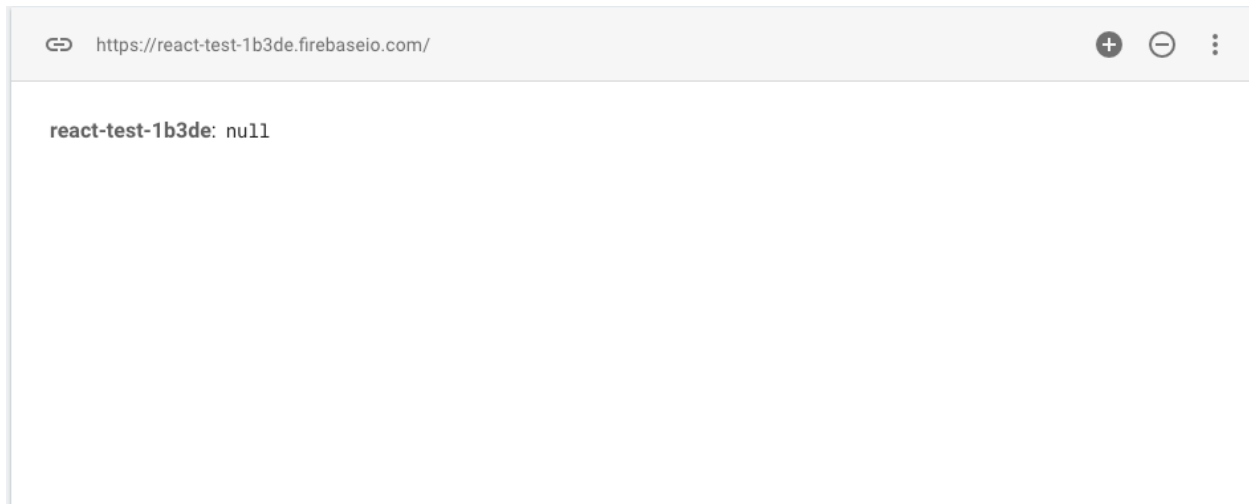
The only other important information to provide is that we can have 32 levels of nesting nodes. However, you want to keep the structured data as flat (least amount of nesting) as you possibly can. Reason being, if you request access one node, you also get access to all the children. Therefore, accessing a node with a lot of children can hurt performance.

This is something to keep in mind for large applications and it's always good to have good practice regardless. But to be totally honest, this won't be a major concern for our mini-app or chat bot demo.

Constructing Our Database

Now that we get how data is structured in Firebase, let's go ahead and add some data.

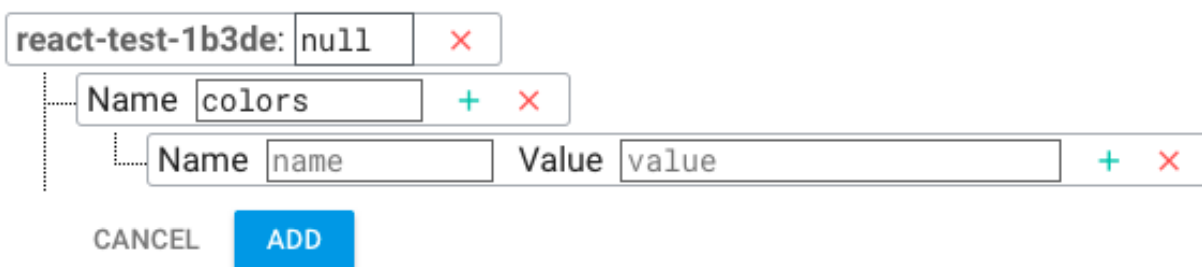
Go to Firebase and we should be in the Database view:



Here, we can create our JSON tree through this nice user interface.

Hover over the react-test and click +.

We first need to add a node called **colors**. Just type this in the name and then hit the + so that we now see:



Now, we can add name/value pairs for our colors node. The name will be the name of the color and the value will be the hex code.

We will use the following name and pairs. Note that you should enter them as they are and quotes should not be added:


```
1    mint: #B4EBCA
2    pink: #FFB7C3
3    purple: #4C2C69
```

We should have the following after entering them in:

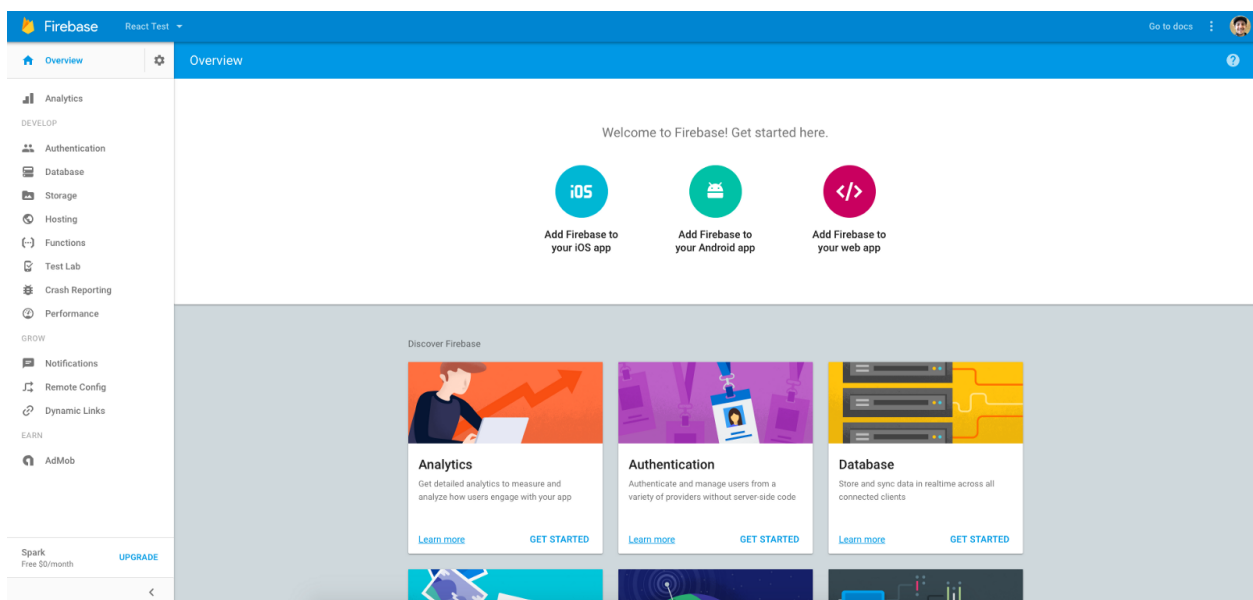
```
react-test-1b3de
└─┬─ colors
   │
   │ mint: "#B4EBCA"
   │ pink: "#FFB7C3"
   │ purple: "#4C2C69"
```

Cool!

Configuring Our Database

This sounds like a painful step, however, it's very simple.

Click on **Overview** on the left-hand menu so that you see the following view:



Click **Add firebase to your web app**.

This brings up the following popup:

Add Firebase to your web app

Copy and paste the snippet below at the bottom of your HTML, before other `script` tags.

```
<script src="https://www.gstatic.com/firebasejs/4.1.0/firebase.js"></script>
<script>
  // Initialize Firebase
  var config = {
    apiKey: "AIzaSyBmaoA1nVF3xqZ-_gtINZR86ub_hEN-_5s",
    authDomain: "react-test-1b3de.firebaseio.com",
    databaseURL: "https://react-test-1b3de.firebaseio.com",
    projectId: "react-test-1b3de",
    storageBucket: "react-test-1b3de.appspot.com",
    messagingSenderId: "568077550139"
  };
  firebase.initializeApp(config);
</script>
```

COPY

Check these resources to learn more about Firebase for web apps:

- [Get Started with Firebase for Web Apps](#)
- [Firebase Web SDK API Reference](#)
- [Firebase Web Samples](#)

We can use this info to add the configuration in our Tile Background Changer pen.

First, copy just the URL between the first script tag:

```
https://www.gstatic.com/firebasejs/4.1.0/firebase.js
```

Add this as an external resource in your pen like so:

Pen Settings

HTML CSS **JavaScript** Behavior Screenshot

JavaScript Preprocessor

Babel 

Add External JavaScript

These scripts will run in this order and before the code in the JavaScript editor. You can also link to another Pen here, and it will run the JavaScript from it. Also try typing the name of any popular library.



<https://cdnjs.cloudflare.com/ajax/libs/react/15.4.2/react.min.js>



<https://cdnjs.cloudflare.com/ajax/libs/react/15.4.2/react-dom.min.js>



<https://www.gstatic.com/firebasejs/4.1.0/firebase.js>



Quick-add:



[+ add another resource](#)

Save and close.

Next, add the following code to the top of your JS column:

```
// Initialize Firebase
var config = {
  apiKey: "AIzaSyBmaoA1nVF3xqZ-_gtINZR86ub_hEN-_5s",
  authDomain: "react-test-1b3de.firebaseio.com",
  databaseURL: "https://react-test-1b3de.firebaseio.com",
  projectId: "react-test-1b3de",
  storageBucket: "react-test-1b3de.appspot.com",
  messagingSenderId: "568077550139"
};
firebase.initializeApp(config);
```

I made a slight change to changed *var* to *const* since it's good practice to make this an immutable variable (we don't want to mutate anything in this variable).



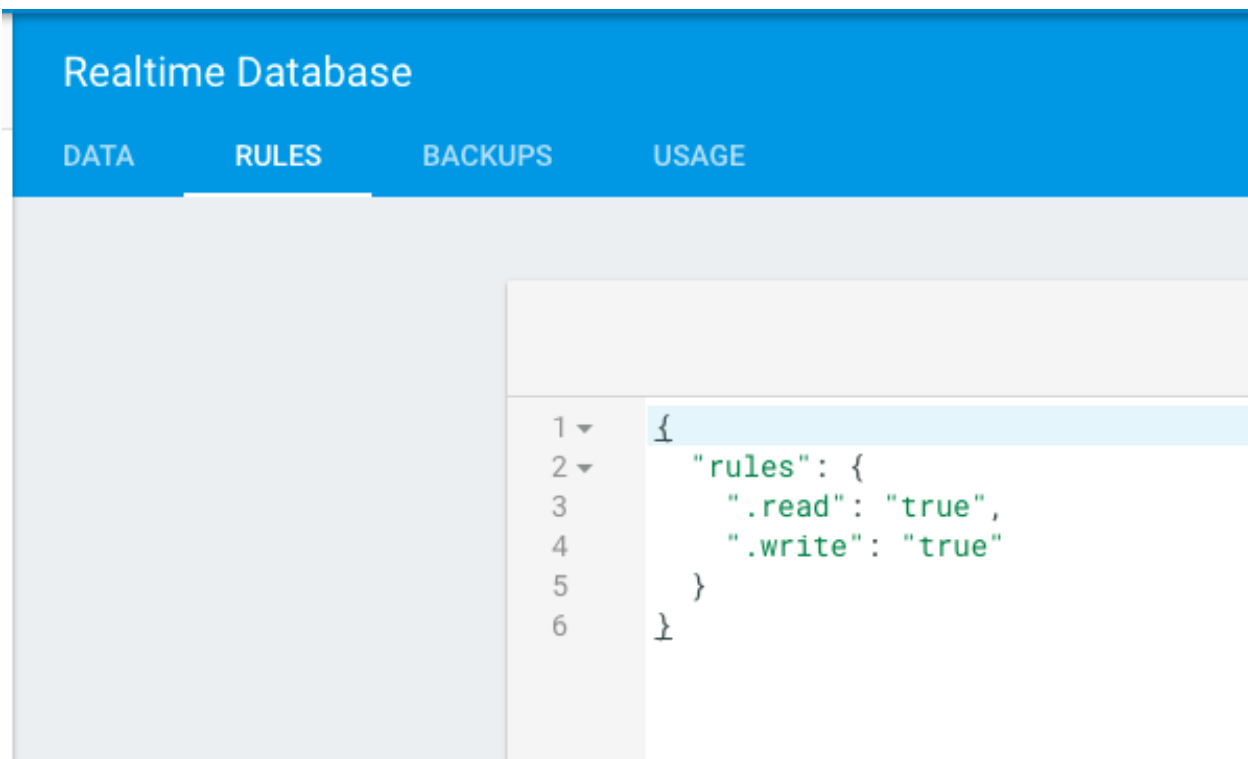
```
JS (Babel)
1 const config = {
2   apiKey: "AIzaSyBmaoA1nVF3xqZ-_gtINZR86ub_hEN-_5s",
3   authDomain: "react-test-1b3de.firebaseio.com",
4   databaseURL: "https://react-test-1b3de.firebaseio.com",
5   projectId: "react-test-1b3de",
6   storageBucket: "react-test-1b3de.appspot.com",
7   messagingSenderId: "568077550139"
8 };
9 firebase.initializeApp(config);
10
```

We have now successfully configured our pen with our Firebase project.

Reading Data

By default, reading and writing data is blocked unless a user is authenticated. We are not worried about authentication so let's change that.

Click **Database** on the left-hand menu and then select the **Rules** tab. Here, let's make read and write available without authentication:



With that out of the way, we now need to establish a *reference* to our database like so:

```
1    //config here
2
3    const database = firebase.database();
4
5    //App component here
```

Specifically, we want to reference the *colors* node from our database. Therefore, let's add another variable:

```
1    //config here
2
3    const database = firebase.database();
4    const colors = database.ref('colors');
5
6    //App component here
```

Now, we have the variables set to reference what we need from the database.

Let's test to see if we can access the hex code values defined within our colors. To test this, we will try to log the data to the console when the App component mounts.

First, we can add a lifecycle hook to do something when the App component mounts:

```

1      //App
2      class App extends React.Component {
3
4          componentDidMount () {
5
6          }
7
8          render() {
9              return (
10                 <div className="container">
11                     <Tile class={"tile-1"}/>
12                     <Tile class={"tile-2"}/>
13                     <Tile class={"tile-3"}/>
14                 </div>
15             )
16         }
17     }

```

Next, we will add some code that is saying: “I’m going to try out this database reference to the colors node. If there’s a value there, then I want to take a snapshot of the the node and display it to the console.”

```

1      componentDidMount () {
2          colors.on('value', function(snapshot) {
3              console.log(snapshot.val())
4          });
5      }

```

If you go into your developer tools and then the console tab, we should see that the data object for the colors node is being logged:

► **Object {mint: "#B4EBCA", pink: "#FFB7C3", purple: "#4C2C69"}**

Note that the snapshot parameter is a predefined thing when using Firebase to make it easy for us to access the value of the node being reference.

Cool! Now, let’s try to print out the hex code for each color in this object like so:

```
1    componentDidMount () {  
2        colors.on('value', function(snapshot) {  
3            console.log(snapshot.val().mint)  
4            console.log(snapshot.val().pink)  
5            console.log(snapshot.val().purple)  
6        });  
7    }
```

We should now see the following in the console:



```
#B4EBCA  
#FFB7C3  
#4C2C69
```

Perfect! We don't need this lifecycle hook so you can go ahead and remove it.

Adding Event Handler

Now that we understand how to access our Firebase data, we will do the following in each tile:

Add an event handler to be called on the click of the tile element

Have the event handler get a random color (any of the 3 hex codes in our database)

Have the event handler then update the color property in the state of the tile component with the random color

First, we add the shell of the event handler:

```

1    class Tile extends React.Component {
2
3    state = {color: this.props.initialColor};
4
5    updateTile() {
6
7    }
8
9    render() {
10     return (
11       <div style={{background: this.state.color}} className="{this.props.cla\
12 ss}">
13         <div className="color-text-box">
14           <div className="color-text">
15             {this.state.color}
16           </div>
17         </div>
18       </div>
19     )
20   }
21 }

```

Then, let's call this on the click of the div element with the className of tile-1:

```

1    class Tile extends React.Component {
2
3    state = {color: this.props.initialColor};
4
5    updateTile() {
6
7    }
8
9    render() {
10     return (
11       <div
12         onClick={this.updateTile.bind(this)}**
13         style={{background: this.state.color}}
14         className="tile-1">
15         <div className="color-text-box">
16           <div className="color-text">
17             {this.state.color}
18           </div>

```



```

19         </div>
20     </div>
21 )
22 }
23 }

```

After that, let's add code that will have the event handler choose a random color from our database:

```

1  updateTile() {
2      //get random number between 1 and 3
3      const random = Math.floor((Math.random() * 3) + 1);
4
5      //empty variable for selectedColor
6      let selectedColor;
7
8      //retrieve color depending on random number
9      colors.on('value', function(snapshot) {
10         if(random === 1) {
11             selectedColor = snapshot.val().mint;
12         } else if (random === 2) {
13             selectedColor = snapshot.val().pink;
14         } else {
15             selectedColor = snapshot.val().purple;
16         }
17     });
18 }

```

In the code above, we get a random number between 1 and 3. If it is 1, then we select mint from Firebase. If it is 2, we select pink. Lastly, if it is 3, then we select purple.

The final piece is to use `setState` to update the color property that is controlling the background of the tile to be the value of the `selectedColor` variable:

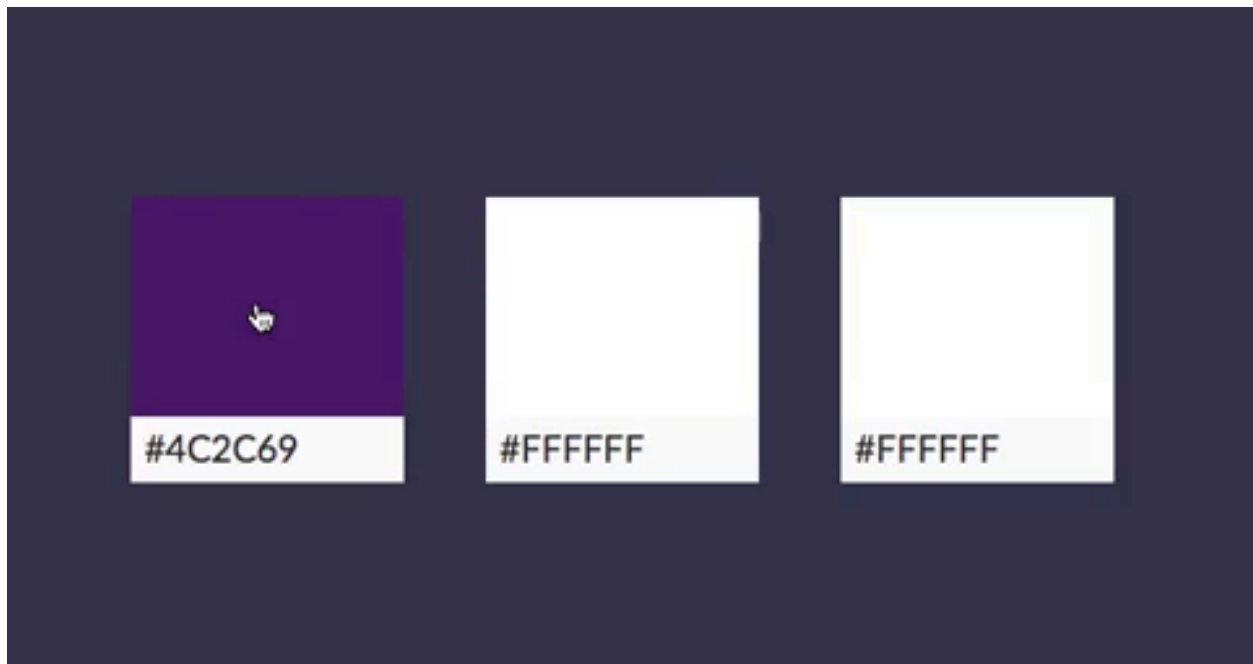
```

1  updateTile() {
2      //get random number between 1 and 3
3      const random = Math.floor((Math.random() * 3) + 1);
4
5      //empty variable for selectedColor
6      let selectedColor;
7
8      //retrieve color depending on random number
9      colors.on('value', function(snapshot) {

```

```
10         if(random === 1) {
11             selectedColor = snapshot.val().mint;
12         } else if (random === 2) {
13             selectedColor = snapshot.val().pink;
14         } else {
15             selectedColor = snapshot.val().purple;
16         }
17     });
18
19     //setState of color property to selected color
20     this.setState({color: selectedColor});
21 }
```

We can now click on the first tile and see the following occur:



So cool! However, you have noticed that the tile was blank on the first click.

What happened?

Well, the following code has to wait until Firebase returns the requested value:

```

1      //retrieve color depending on random number
2      colors.on('value', function(snapshot) {
3          if(random === 1) {
4              selectedColor = snapshot.val().mint;
5          } else if (random === 2) {
6              selectedColor = snapshot.val().pink;
7          } else {
8              selectedColor = snapshot.val().purple;
9          }
10         });

```

Therefore, *selectedColor* most likely has no value by the time this line arrives:

```
this.setState({color: selectedColor});
```

Because of this, the newly set state assigns a color of blank.

How do we fix this?

We can use a promise. Let's update our code then I will explain:

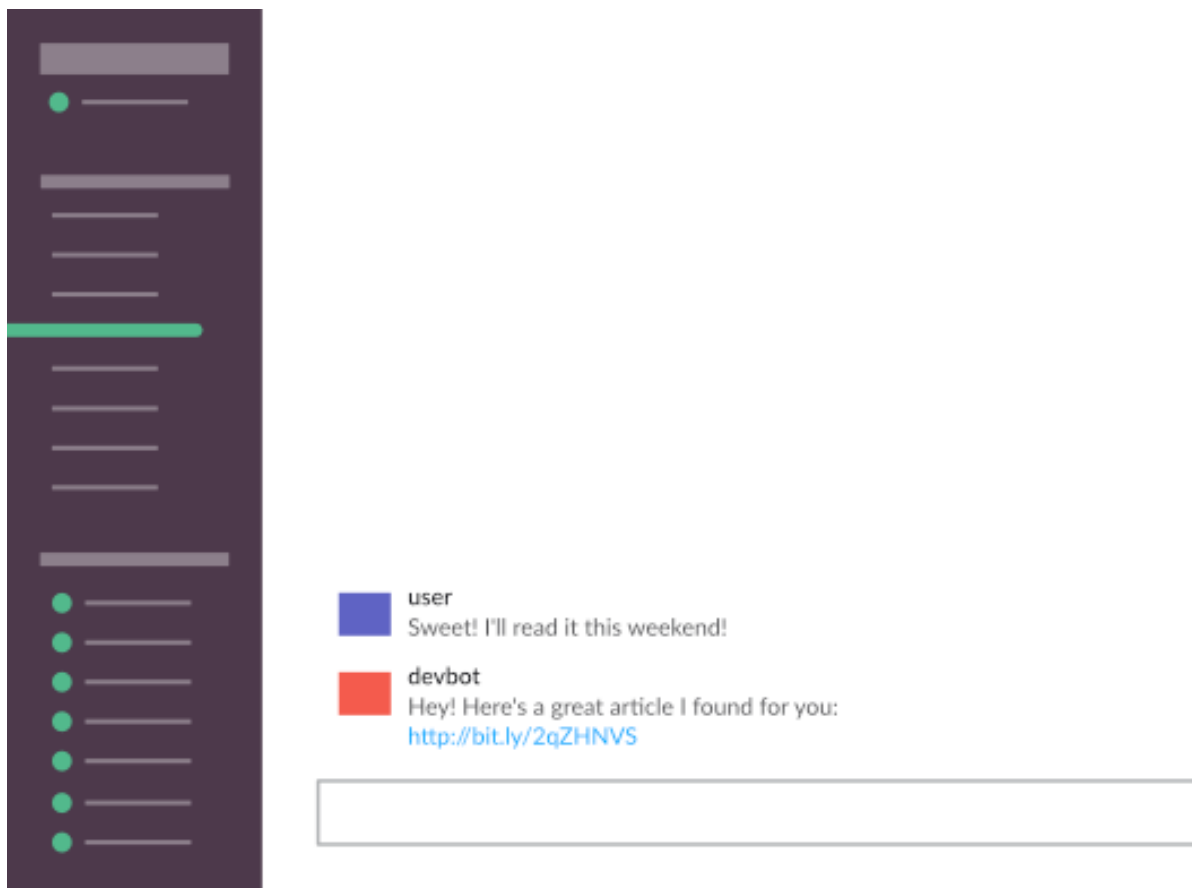
```

1      let promise = new Promise((resolve, reject) => {
2          colors.on('value', function(snapshot) {
3              if(random === 1) {
4                  selectedColor = snapshot.val().mint;
5              } else if (random === 2) {
6                  selectedColor = snapshot.val().pink;
7              } else {
8                  selectedColor = snapshot.val().purple;
9              }
10             resolve(selectedColor);
11         });
12     });
13
14     promise.then((resolvedColor) => {
15         this.setState({color: resolvedColor});
16     });

```

Now, we are essentially saying: “Okay this.setState I know you want to update the color with the value we fetch from Firebase. However, it's going to take time. I *promise* that once we can *resolve* this *then* you can go ahead and update the color with the fetched value.”

A promise is used when a line of code is waiting on something to resolve. We can make the line of code that is waiting run *after* the code that it is waiting to complete is resolved.



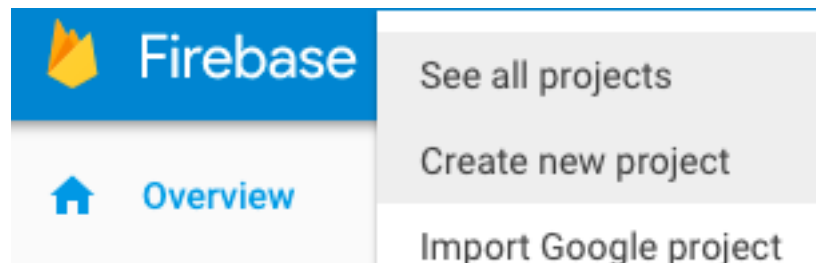
We want the message components to be revealed one after another. We also only want the message components only to be rendered in the Demo village and not the Landing Page village.

Let's get to work!

Constructing the Chat Bot Demo Database



First, we need to create a new Firebase project. Go to **Overview** in Firebase, click the down arrow next to the name of the current project, and select **Create new project**:



Name it **Chat Bot Demo** and then select **Create project**:

Create a project ×

Project name

Country/region ?

United States ▼

By default, your Analytics data will enhance other Firebase features and Google products. You can control how your analytics data is shared in your settings at anytime. [Learn more](#)

CANCELCREATE PROJECT

When that is finished, click **Database** on the left-hand side and then click the **Rules** tab. Once again, we want to allow reading and writing without requiring authentication:



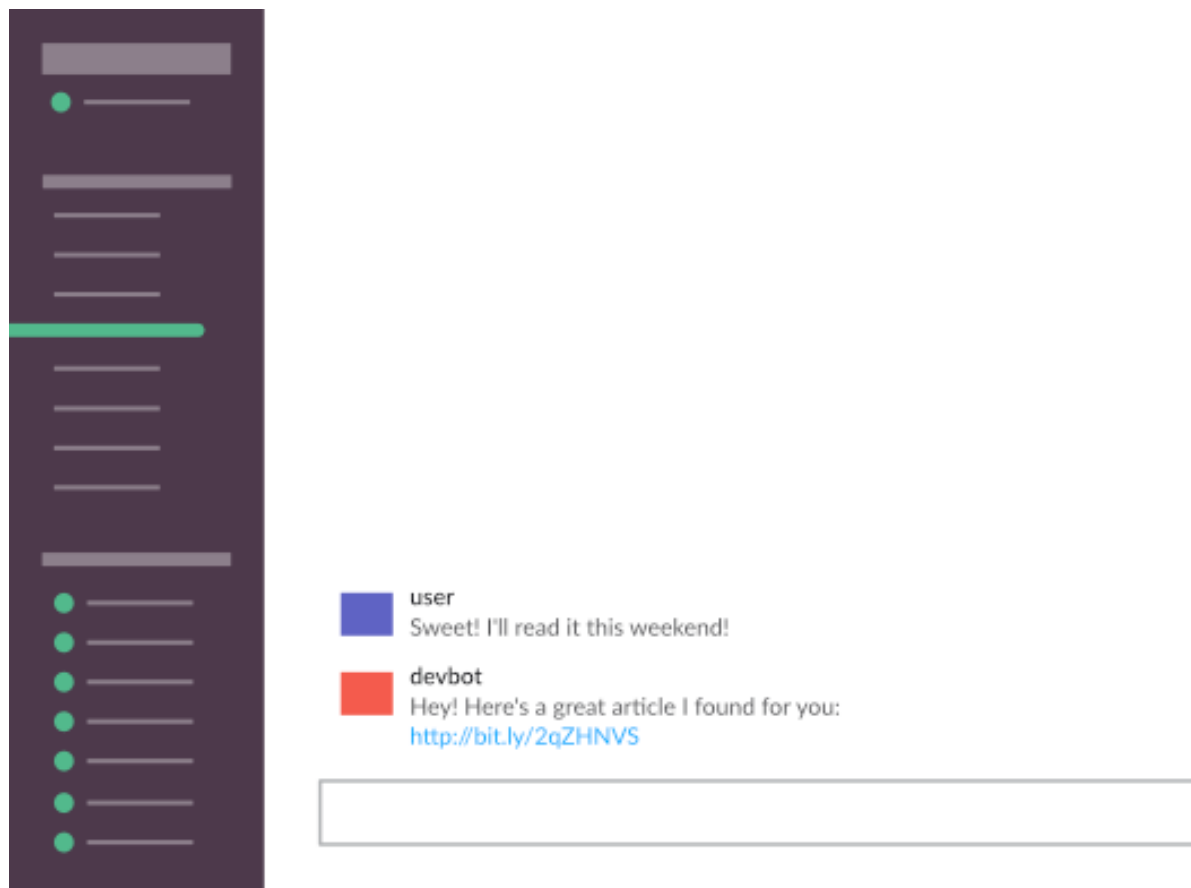
Hit **Publish** when finished updating.

Click on the **Data** tab and try making the following data tree yourself:

chat-bot-demo



In both nodes, there is a color that will be used to style the avatar square, a message, and username. The bot node has an additional article link. Again, we will use this data to populate the components so they render as the following:



Installing Firebase via npm

Now that we have constructed the database for our chat bot demo, let's open Atom and pick up where we left off in our react-landing-page project.

Also, cd into the root of the project in command line.

When you're ready, let's install firebase as a dependency:

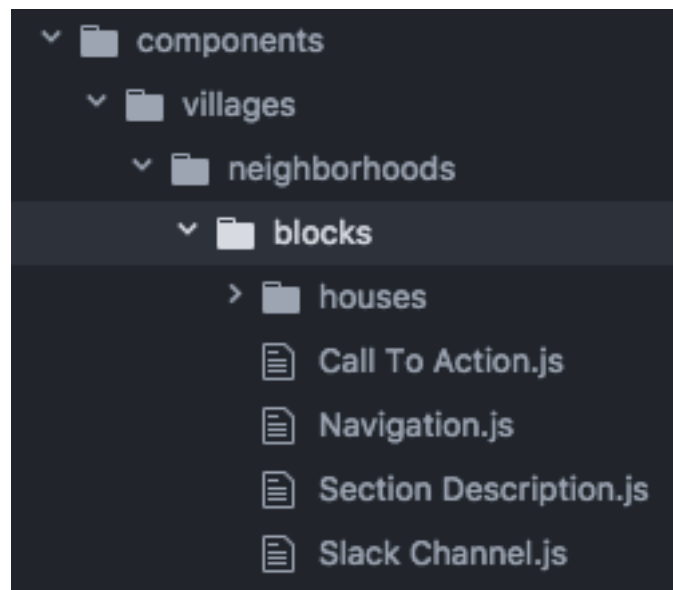
```
npm install --save firebase
```

Cool beans.

Setting Up Our Message Components

Let's create 2 new component files that will go under our houses folder, *User Message.js* and *Chat Bot Message.js*.

Before we add some code to these, go ahead and drag and drop the *Slack Channel.js* file under the Blocks folder since it will now be a block component.



To start off, let's add the basic setup to *User Message.js*:

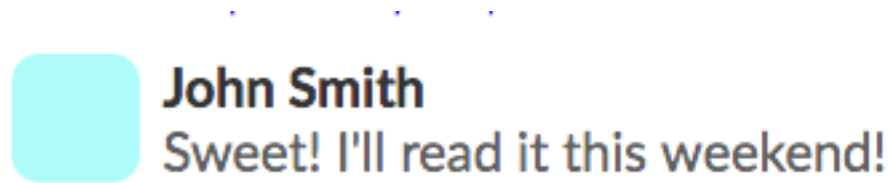
```

1    import React from 'react';
2    import ReactDOM from 'react-dom';
3
4    class UserMessage extends React.Component {
5
6      constructor(){
7        super()
8        this.state = {avatar: "", username: "", message: ""}
9      }
10
11    }
12
13    render() {
14      return (
15        <div className="user-message">
16          <div style={{background: this.state.avatar}} className="user-avatar"\
17 ></div>
18          <div className="username">{this.state.username}</div>
19          <div className="message">{this.state.message}</div>
20        </div>
21      )
22    }
23  }
24
25  module.exports = UserMessage;

```

In the code above, we have a state that has the initial values of the avatar, username, and message properties set as nothing. We will retrieve the data for these properties from Firebase and update the state with that data.

The injecting of the state has been handled. The avatar data is a hex code that will control the background color of the avatar box. The username data will be used to display the username. The message data will be used to display a message. All of this forms our user-message element which will ultimately render as:



An important note is that I've made a slight change on how the state is initialized. I've done this for best practice purposes.

```

1      constructor(){
2          super()
3          this.state = {
4              avatar: "",
5              username: "",
6              message: ""
7          }
8      }

```

The constructor method will handle initialization of our component class. Since our component class is a *child* of the `React.Component` class:

```
class UserMessage extends React.Component
```

we need to call the constructor of that parent class using:

```

1      constructor(){
2          super()
3          //...
4      }

```

Now, we can initialize the state so that it can be accessed by doing `this.state.____` so we do:

```

1      constructor(){
2          super()
3          this.state = {avatar: "", username: "", message: ""}
4      }

```

We will be making a lot of updates to this file, but for now that's enough for the basic setup.

Let's move on to the *Chat Bot Message.js*:

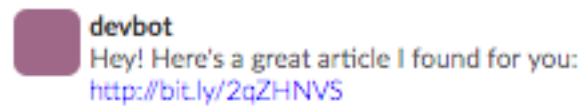
```

1      import React from 'react';
2      import ReactDOM from 'react-dom';
3
4      class ChatBotMessage extends React.Component {
5
6      constructor(){
7          super()
8          this.state = {avatar: "", username: "", message: "", article: ""}
9      }
10
11      }
12
13      render() {
14          return (
15              <div className="bot-message">
16                  <div style={{background: this.state.avatar}} className="bot-avatar">\
17      </div>
18                  <div className="username">{this.state.username}</div>
19                  <div className="message">{this.state.message}</div>
20                  <a href={this.state.article}>
21                      <span className="article">{this.state.article}</span>
22                  </a>
23              </div>
24          )
25      }
26  }
27
28  module.exports = ChatBotMessage;

```

Most of this code is just like the UserMessage component except that we have an additional piece of data to fetch from Firebase called article which needs to also be used to render an article link. In addition, there's a few different class names for styling purposes.

We want this to eventually render as:

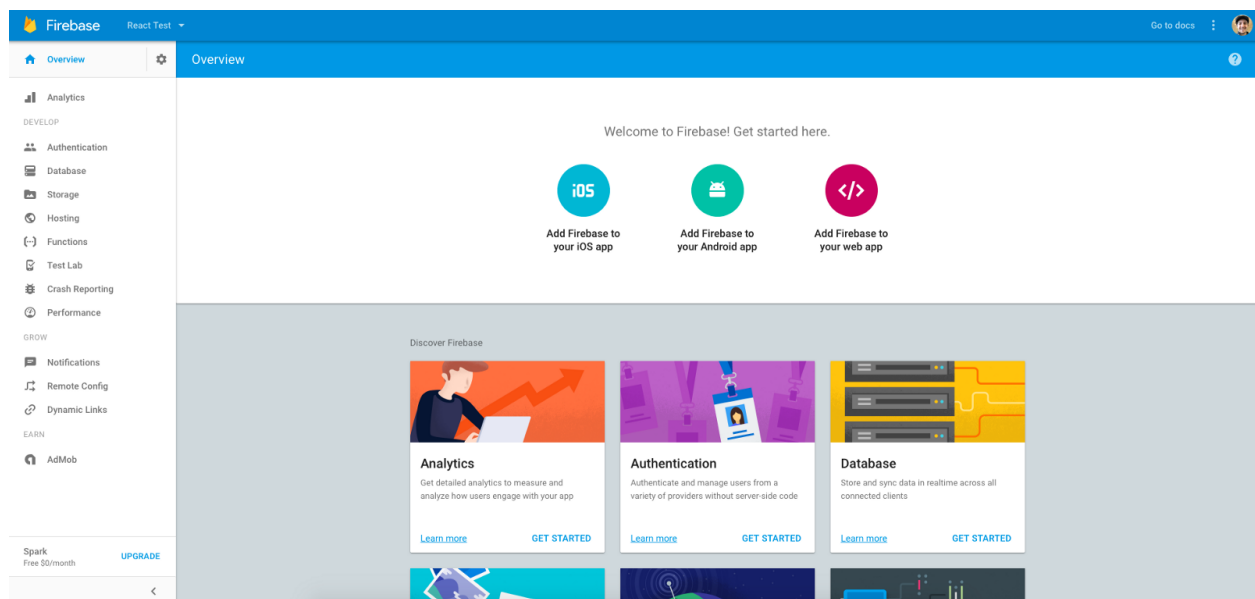


Again, there's much to add but that is good enough for the initial setup.

Configuring Firebase

Now that we have created the database, installed Firebase via npm, and setup our house components, we now need to get the configuration code.

Again, we go to the **Overview** view in the Firebase console.



Then, select **Add Firebase to your web app** and copy the configuration code which will be something similar to this:

```
1  var config = {
2    apiKey: "AIzaSyA1JBmMSIg7X_lSEHr3vh7sEc_p83hB4g4",
3    authDomain: "chat-bot-demo.firebaseio.com",
4    databaseURL: "https://chat-bot-demo.firebaseio.com",
5    projectId: "chat-bot-demo",
6    storageBucket: "chat-bot-demo.appspot.com",
7    messagingSenderId: "740047634033"
8  };
```

In this file, let's paste that code at the top of *User Message.js*, change the variable to *const*, and change the variable name to *FirebaseConfig*:

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3
4   const config = {
5     apiKey: "AIzaSyA1JBmMSIg7X_lSEHr3vh7sEc_p83hB4g4",
6     authDomain: "chat-bot-demo.firebaseio.com",
7     databaseURL: "[https://chat-bot-demo.firebaseio.com](https://chat-bot-de\
8 mo.firebaseio.com)",
9     projectId: "chat-bot-demo",
10    storageBucket: "chat-bot-demo.appspot.com",
11    messagingSenderId: "740047634033"
12  };
13
14  firebase.initializeApp(config);
15
16  //UserMessage component stuff

```

We also have to add the import for the firebase which was installed via npm:

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import * as firebase from "firebase";

```

Note: We only need the configuration code to appear once in our application. Therefore, we will just keep it in the *User Message.js* but there's a few ways we could have done this.

Retrieving Data from Firebase

Now that we have our new message components and Firebase initialized, let's write the code to fetch the Firebase data to update the states of these components.

Let's continue in *User Message.js*.

First, let's add the reference to our Firebase database. Specifically, we want to refer to the user node which contains avatar, username, and message:

```
1 //imports and Firebase config here
2
3 const database = firebase.database();
4 const user = database.ref('user');
5
6 //component here
```

Next, let's add a lifecycle where we will fetch the Firebase data:

```
1 class UserMessage extends React.Component {
2   constructor(){
3     super()
4     this.state = {avatar: "", username: "", message: ""}
5   }
6
7   componentDidMount() {
8     let firebaseAvatar;
9     let firebaseUsername;
10    let firebaseMessage;
11  }
12
13    //more stuff
14
15  }
```

In this lifecycle (when the component has mounted), we have defined three variables that will store the fetched data.

In our Tile Background Changer pen, we needed to use a *promise* when retrieving our Firebase data:

```
1 //fetch data
2 let promise = new Promise((resolve, reject) => {
3   colors.on('value', function(snapshot) {
4     if(random === 1) {
5       selectedColor = snapshot.val().mint;
6     } else if (random === 2) {
7       selectedColor = snapshot.val().pink;
8     } else {
9       selectedColor = snapshot.val().purple;
10    }
11    resolve(selectedColor);
12  });
```

```

13     });
14
15     //then do something with it (update the state)
16
17     promise.then((resolvedColor) => {
18         this.setState({color: resolvedColor});
19     });

```

We did this since we want to wait for the state to update until we have *resolved* fetching the Firebase data.

Therefore, let's add the following promise to our file:

```

1     class UserMessage extends React.Component {
2         constructor(){
3             super()
4             this.state = {avatar: "", username: "", message: ""}
5         }
6
7         componentDidMount() {
8             let firebaseAvatar;
9             let firebaseUsername;
10            let firebaseMessage;
11
12            let promise1 = new Promise((resolve, reject) => {
13                user.on('value', function(snapshot) {
14                    firebaseAvatar = snapshot.val().avatar;
15                    resolve(firebaseAvatar);
16                });
17            });
18        }
19
20        //more stuff
21
22    }

```

In this promise, we are going to get a snapshot of the Firebase database at the reference we specified and get the avatar value. The value will be stored in `firebaseAvatar`.

Unlike our pen, we have multiple values to fetch. Therefore, we need a separate promise for each value that needed to be fetched (hence why it is named `promise1`).

We will add the other promises before we do the *promise.then* stuff since there's a way to do it for all our promises at once.

Here's the remaining promises:


```
1    //promise 1 here
2
3    let promise2 = new Promise((resolve, reject) => {
4        user.on('value', function(snapshot) {
5            firebaseUsername = snapshot.val().username;
6            resolve(firebaseUsername);
7        });
8    });
9
10   let promise3 = new Promise((resolve, reject) => {
11       user.on('value', function(snapshot) {
12           firebaseMessage = snapshot.val().message;
13           resolve(firebaseMessage);
14       });
15   });
```

Perfect! Next, we want to take all of the resolved data and update our state. To do that, we can do the following:

```
1    Promise.all([promise1, promise2, promise3]).then(values => {
2        this.setState({
3            avatar: values[0],
4            username: values[1],
5            message: values[2]
6        })
7    });
```

In the code above, we are saying: “Take all of our promises and store the resolved values into an array. Use those values in the array to update our state properties.”

To see how this works, log *values* to the console:

```
1    Promise.all([promise1, promise2, promise3]).then(values => {
2        this.setState({
3            avatar: values[0],
4            username: values[1],
5            message: values[2]
6        })
7        console.log(values);
8    });
```

We will now see the values array and understand how we used them to update our state:

```
▼ Array(3) ⓘ
  0: "#ADFCF9"
  1: "John Smith"
  2: "Sweet! I'll read it this weekend!"
```

Awesome!

Let's move on to *Chat Bot Message.js*.

We will start by adding the firebase import and the database references:

```
1  import * as firebase from "firebase";
2
3  const database = firebase.database();
4  const bot = database.ref('bot');
```

We can then add the lifecycle and promises code just like the previous file, except we have the additional data to fetch for the article:

```
1  componentDidMount() {
2    let firebaseAvatar;
3    let firebaseUsername;
4    let firebaseMessage;
5    let firebaseArticle;
6
7    let promise1 = new Promise((resolve, reject) => {
8      bot.on('value', function(snapshot) {
9        firebaseAvatar = snapshot.val().avatar;
10       resolve(firebaseAvatar);
11     });
12   });
13
14   let promise2 = new Promise((resolve, reject) => {
15     bot.on('value', function(snapshot) {
16       firebaseUsername = snapshot.val().username;
17       resolve(firebaseUsername);
18     });
19   });
20 }
```

```

21     let promise3 = new Promise((resolve, reject) => {
22         bot.on('value', function(snapshot) {
23             firebaseMessage = snapshot.val().message;
24             resolve(firebaseMessage);
25         });
26     });
27
28     let promise4 = new Promise((resolve, reject) => {
29         bot.on('value', function(snapshot) {
30             firebaseArticle = snapshot.val().article;
31             resolve(firebaseArticle);
32         });
33     });
34
35     Promise.all([promise1, promise2, promise3, promise4]).then(values => {
36         this.setState({avatar: values[0], username: values[1], message: values\
37 [2], article: values[3]})
38     });
39     }

```

All the differences between the previous file can be seen in bold.

We can also log the resolved values to make sure it's working:

```

1     Promise.all([promise1, promise2, promise3, promise4]).then(values => {
2         this.setState({
3             avatar: values[0],
4             username: values[1],
5             message: values[2],
6             article: values[3]
7         })
8         console.log(values);
9     });

```

▼ Array(4) 

0: "#9D6A89"

1: "devbot"

2: "Hey! Here's a great article I found for you:"

3: "http://bit.ly/2qZHNVS"

Perfect!

Updating Our SlackChannel Component

To test our code, first, we need to update the *Slack Channel.js* file.

We need to nest these new message components like so:

```
1  import React from 'react';
2  import ReactDOM from 'react-dom';
3  import { Container, Row, Col } from 'react-grid-system';
4  import UserMessage from './houses/User Message.js';
5  import ChatBotMessage from './houses/Chat Bot Message.js';
6
7  class SlackChannel extends React.Component {
8    render() {
9      return (
10        <Col lg={12}>
11          
12          <UserMessage/>
13          <ChatBotMessage/>
14        </Col>
15      )
16    }
17  }
18
19  module.exports = SlackChannel;
```

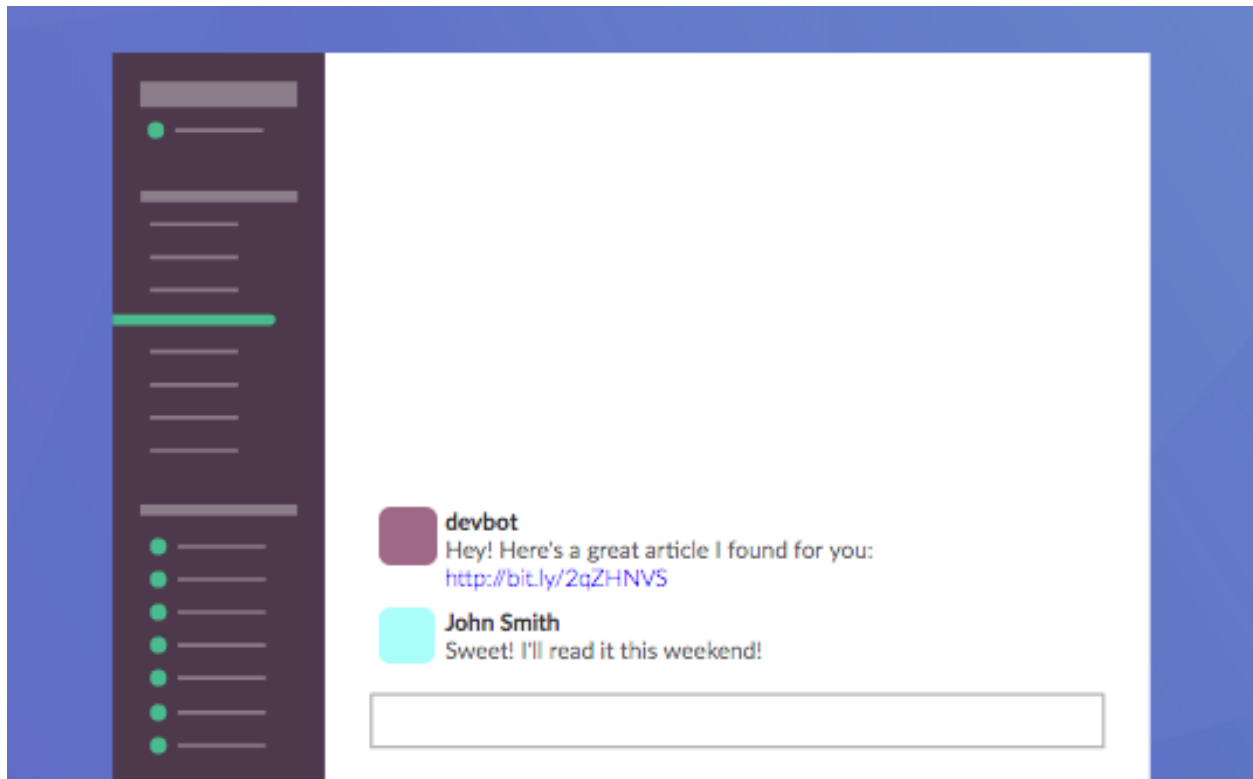
Then, let's update our CSS with the code that you can find [here](https://gist.github.com/michaelmang/125410e125f5212de89a5cef861a30d4)⁵⁵.

If you haven't already, *cd* into the project directory and run *npm start*.

Then, go to the local host.

We should now see the following:

⁵⁵<https://gist.github.com/michaelmang/125410e125f5212de89a5cef861a30d4>



Excellent!

A couple issues to note.

One issue is that we only want these messages to appear on the Demo view but they are appearing on the LandingPage view as well. In addition, we mentioned that we wanted to have these messages be revealed in order.

For all these reasons, it's time to add some conditional rendering.

Adding Conditional Rendering

Slack Channel.js currently contains:

```

1  import React from 'react';
2  import ReactDOM from 'react-dom';
3  import { Container, Row, Col } from 'react-grid-system';
4  import UserMessage from './houses/User Message.js';
5  import ChatBotMessage from './houses/Chat Bot Message.js';
6
7  class SlackChannel extends React.Component {
8    render() {
9      return (
```

```

10      <Col lg={12}>
11        
12        <UserMessage/>
13        <ChatBotMessage/>
14      </Col>
15    )
16  }
17 }
18
19 module.exports = SlackChannel;

```

We want to use conditional rendering to control whether `UserMessage` and `ChatBotMessage` render or not. We only want them to render on the Demo village, however, both the `LandingPage` village and the `Demo` village make use of the `Header` containing the `SlackChannel`.

In order to control this, we will pass a prop (called *messages*) down to the `Header` component nested in both *Landing Page.js* and *Demo.js*:

Landing Page.js

```

1  import React from 'react';
2  import ReactDOM from 'react-dom';
3
4  import Header from './neighborhoods/Header.js';
5  import SectionA from './neighborhoods/Section A.js';
6  import SectionB from './neighborhoods/Section B.js';
7  import SectionC from './neighborhoods/Section C.js';
8  import Footer from './neighborhoods/Footer.js';
9
10 class LandingPage extends React.Component {
11   render() {
12     return (
13       <div className="landing-page">
14         <Header messages={false} title={"Developer Bot for Slack"} subtitle=\
15 {"One article to one random person in your Slack group. Once a day."}/>
16         <SectionA/>
17         <SectionB/>
18         <SectionC title={"Developer Bot for Slack"} subtitle={"One article t\
19 o one random person in your Slack group. Once a day."}/>
20         <Footer/>
21       </div>
22     )
23   }

```

```
24     }
25
26     module.exports = LandingPage;
```

Demo.js

```
1     import React from 'react';
2     import ReactDOM from 'react-dom';
3
4     import Header from './neighborhoods/Header.js';
5     import Footer from './neighborhoods/Footer.js';
6
7     class Demo extends React.Component {
8       render() {
9         return (
10           <div className="demo">
11             <Header messages={true} title={"Chat Bot Demo"} subtitle={"Watch how\
12 it works."}/>
13             <Footer/>
14           </div>
15         )
16       }
17     }
18
19     module.exports = Demo;
```

If the *messages* prop is true, the message components will render. That is why we pass down a value of false in *Landing Page.js* and a value of true in *Demo.js*.

Next, we need to pass the messages prop down to the SlackChannel component so we can use it for conditional rendering.

The next level below the LandingPage and Demo components is our Header.

In *Header.js*, let's pass down the messages prop down to the SlackChannel component like so:

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4   import Navigation from './blocks/Navigation.js';
5   import CallToAction from './blocks/Call To Action.js';
6   import SlackChannel from './blocks/Slack Channel.js';
7
8   class Header extends React.Component {
9     render() {
10      return (
11        <section className="header">
12          <Container>
13            <Row>
14              <Navigation/>
15            </Row>
16            <Row>
17              <CallToAction title={this.props.title} subtitle={this.props.subt\
18 title}/>
19            </Row>
20            <Row>
21              <SlackChannel messages={this.props.messages}/>
22            </Row>
23          </Container>
24        </section>
25      )
26    }
27  }
28
29  module.exports = Header;

```

Now, we can apply conditional rendering in *Slack Channel.js* like so:

```

1   import React from 'react';
2   import ReactDOM from 'react-dom';
3   import { Container, Row, Col } from 'react-grid-system';
4   import UserMessage from './houses/User Message.js';
5   import ChatBotMessage from './houses/Chat Bot Message.js';
6
7   class SlackChannel extends React.Component {
8     render() {
9       //Boolean flag passed down for deciding what to render
10      const messages = this.props.messages;

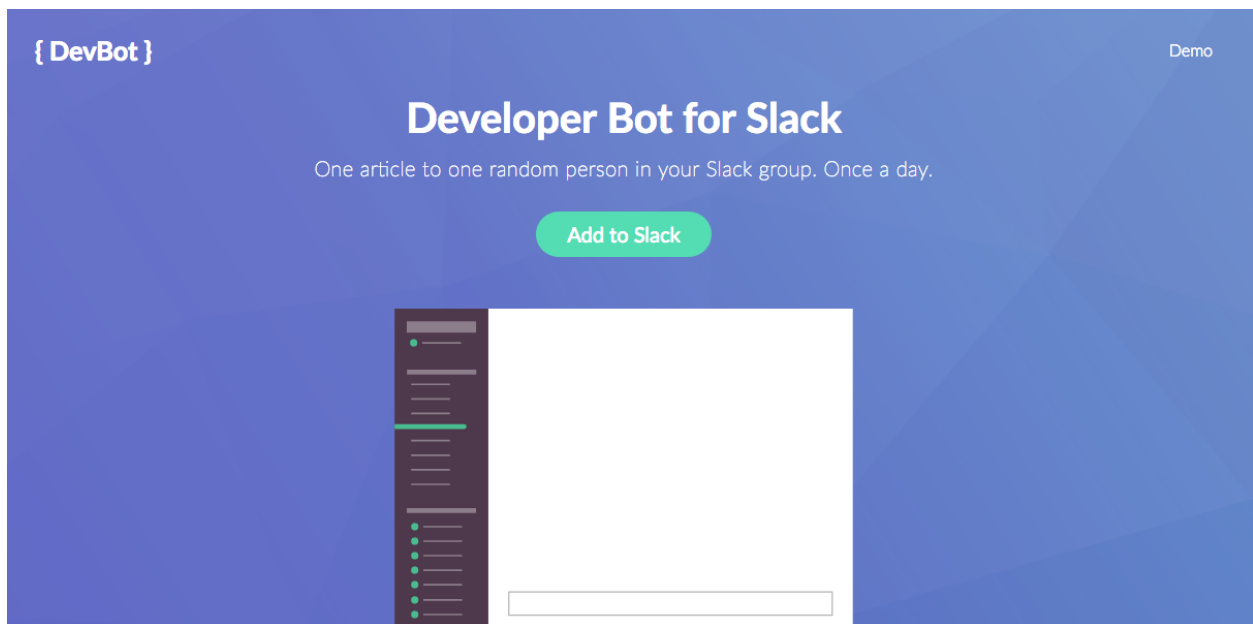
```



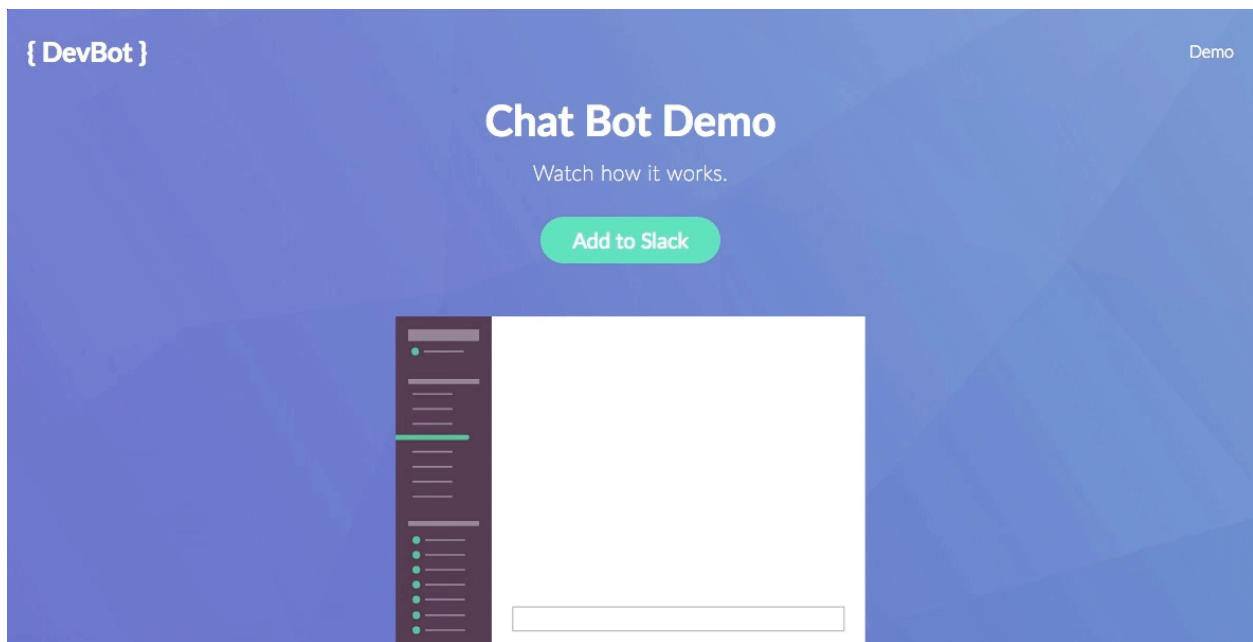
```
11     return (
12       <Col lg={12}>
13         
14         {messages ? (
15           //render messages is true (demo)
16           <div>
17             <UserMessage/>
18             <ChatBotMessage/>
19           </div>
20         ) : (
21           //render nothing messages is false (landing page)
22           <div></div>
23         )}
24       </Col>
25     )
26   }
27 }
28
29 module.exports = SlackChannel;
```

The highlighted code which we have gone over before can be read as: “If the inherited messages prop is true, then the Demo component will render these message components. Else, we render an empty div tag so only the Slack channel image appears.”

If we refresh our local host, we should see that the Landing Page view does not render these message components.



We can finally wrap up this chat bot demo by having the message components reveal in order:



Let's start adding the code in *Chat Bot Message.js* to make this work.

First, we will add another property to our local state called *showComponent*:

```

1      constructor(){
2          super()
3          this.state = {avatar: "", username: "", message: "", article: "", showCo\
4 mponent: false}
5      }

```

This property will be our flag to control whether the message renders or not. To be clear, we are using conditional rendering with a flag in our local state and not as a passed prop in this case because:

It will be changed from false to true.

There is no higher level component that will determine how we apply conditional rendering.

The second step is to update our render function so that we can display the message depending on the value (true or false) of showComponent:

```

1      render() {
2          const showComponent = this.state.showComponent;
3          return (
4              <div>
5                  {showComponent ? (
6                      //render component
7                      <div className="bot-message">
8                          <div style={{background: this.state.avatar}} className="bot-avat\
9 ar"></div>
10                     <div className="username">{this.state.username}</div>
11                     <div className="message">{this.state.message}</div>
12                     <a href={this.state.article}>
13                         <span className="article">{this.state.article}</span>
14                     </a>
15                 </div>
16             ) : (
17                 //render nothing
18                 <div></div>
19             )}
20         </div>
21     )
22 }

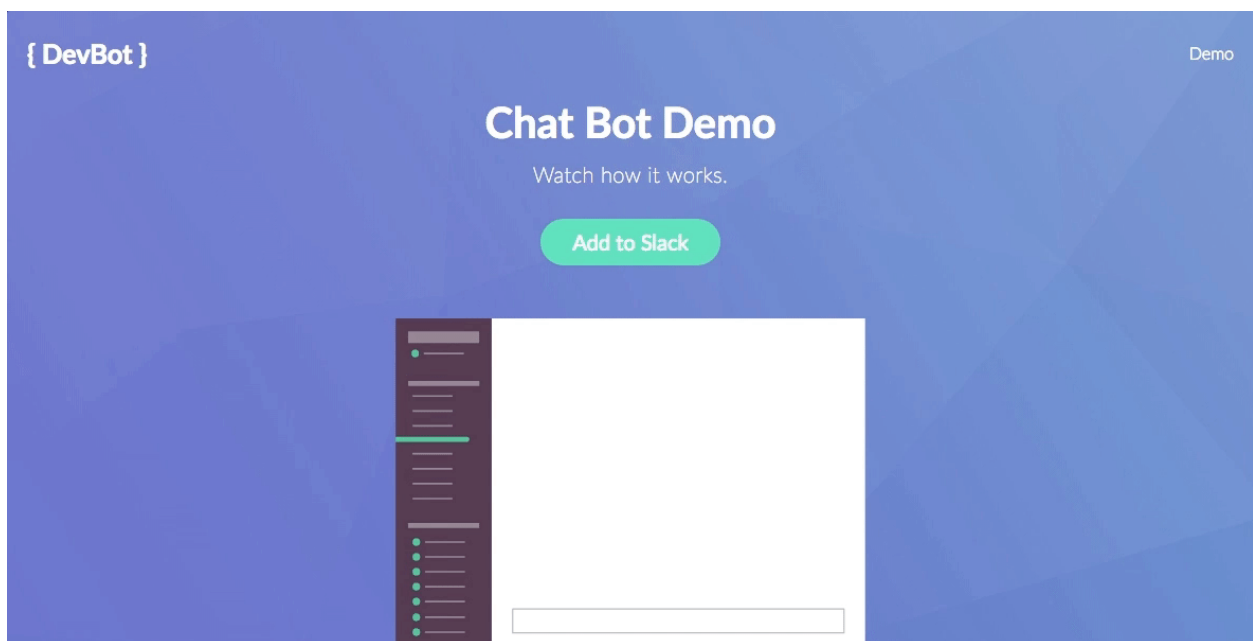
```

We've already discussed this type of logic so let's go to our final step.

We are going to have our update state in the componentDidMount lifecycle hook after 1.5 seconds. We will use a *setTimeout* to control the timing:

```
1    componentDidMount() {  
2        let firebaseAvatar;  
3        let firebaseUsername;  
4        let firebaseMessage;  
5        let firebaseArticle;  
6  
7        //promise stuff here  
8  
9        setTimeout(() => {  
10           this.setState({showComponent: true});  
11        }, 1500)  
12    }
```

After 1.5 seconds (1500), we will change `showComponent` to `true`. This will cause the bot message to render.

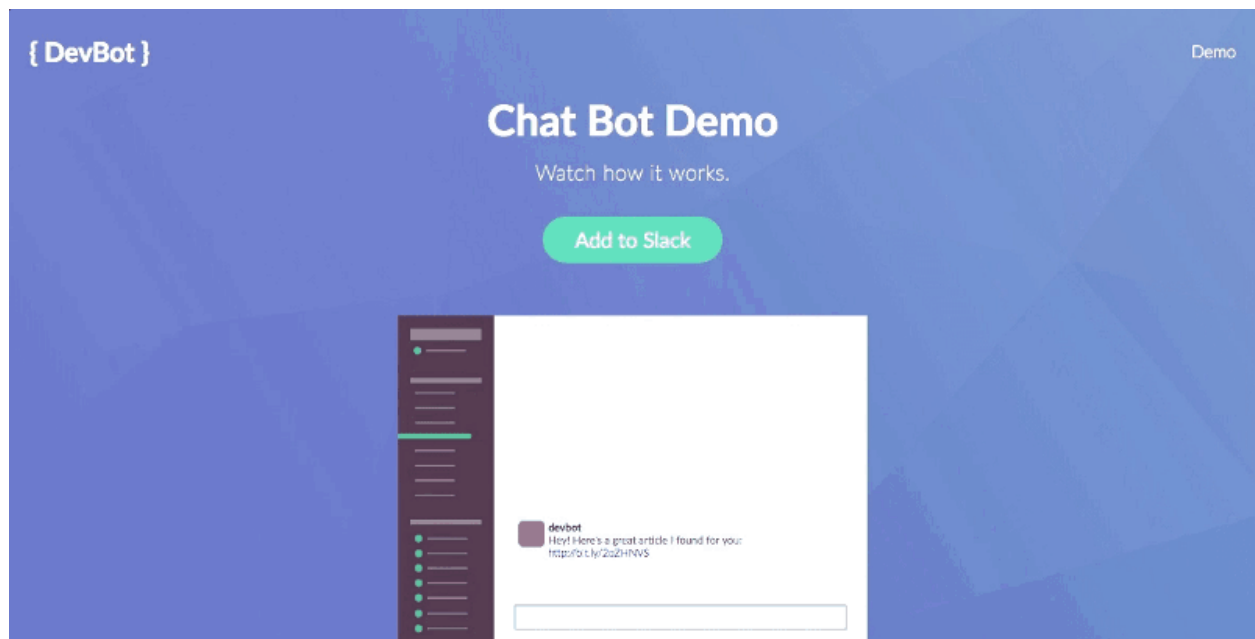


The *User Message.js* file will have the same exact updates with one minor difference. The *setTimeout* duration will be specified to be 3000. Meaning, it will wait 3 seconds after the component mounts:

```
1    setTimeout(() => {  
2        this.setState({showComponent: true});  
3    }, 3000)
```

This will cause it to be rendered after the `ChatBotMessage`.

Reload the local host and you should now see that we have a functioning chat bot.



Finally! We have finished all of the major coding for this book!

Give yourself a pat on the back. You did it!

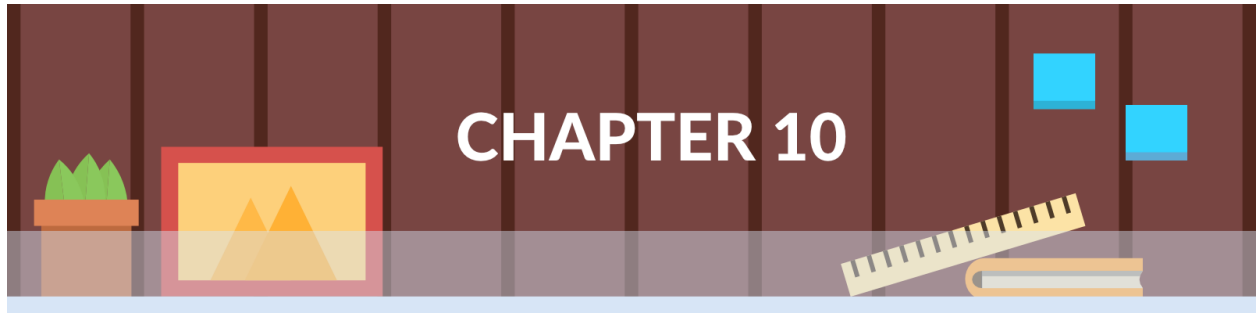
Since this has been a long and I will dedicate an entire chapter to concluding thoughts. Let's end it here.

Final Code

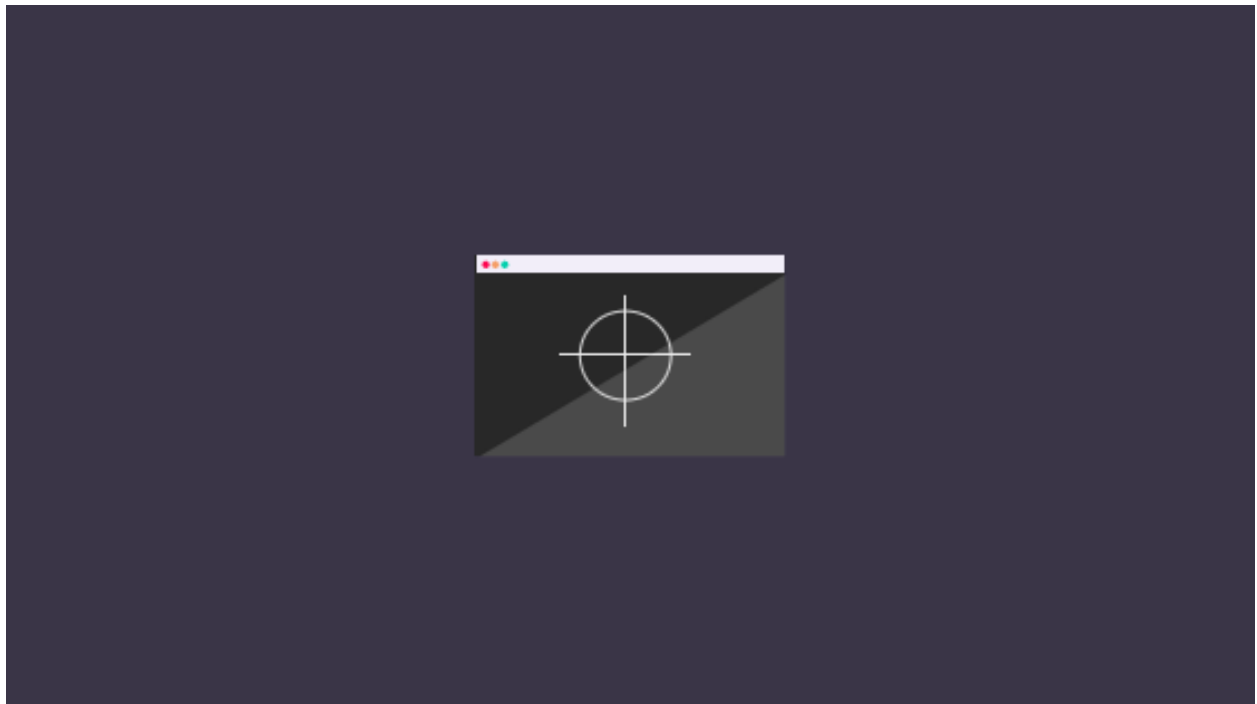
Chapter 9 final code is available on Github⁵⁶.

⁵⁶<https://github.com/michaelmang/React-for-the-Visual-Learner/tree/master/chapters/chapter%209>

Chapter 10 | Reacting to What We've Learned



The Scope of This Chapter



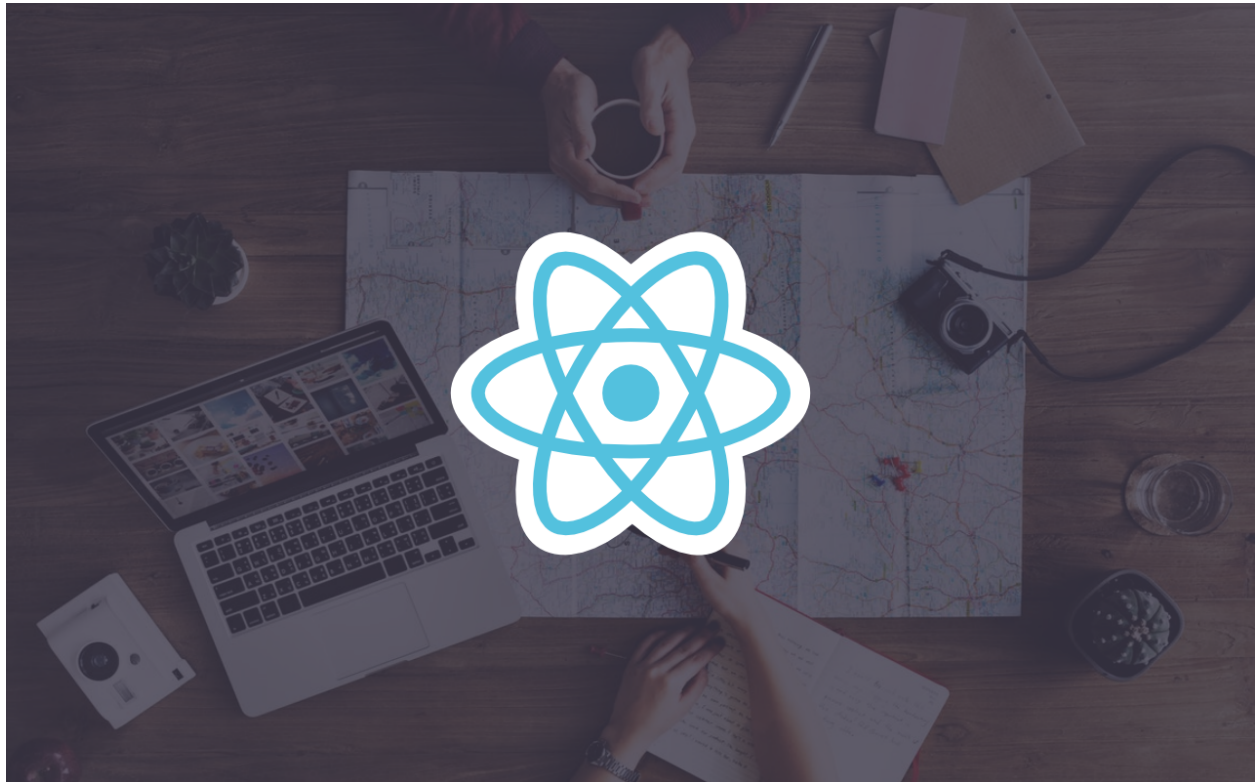
To wrap up this book, I want to discuss a few different things.

First, I want to provide a plan for being able to take these core skills of React that we have learned and how to build off of them to progress.

Second, I will be describing the process of using GitHub for a React application as this will be helpful when using React in the real world. It also will give you an opportunity to showcase your work to others.

Lastly, I want to reflect on how this project went from the perspective of the being the author.

Map for Going Forward



The top highlight in this book so far has been the following quote:

The best thing about modern development is that there is a tremendous amount of available resources. The worst thing about modern development is that there is a tremendous amount of available resources.

I feel like this point needs to be re-emphasized at the end of this book as well.

Reason being, I have been able to walk you through the process of learning React as an absolute beginner. We've learned the core skills, but there's a lot more to learn and a lot more cool things that people are doing nowadays with React.

It's tempting to want to explore Medium at this point and try to do some crazy thing with React because you've been taught the basics in this book.

It's not easy...I get that.

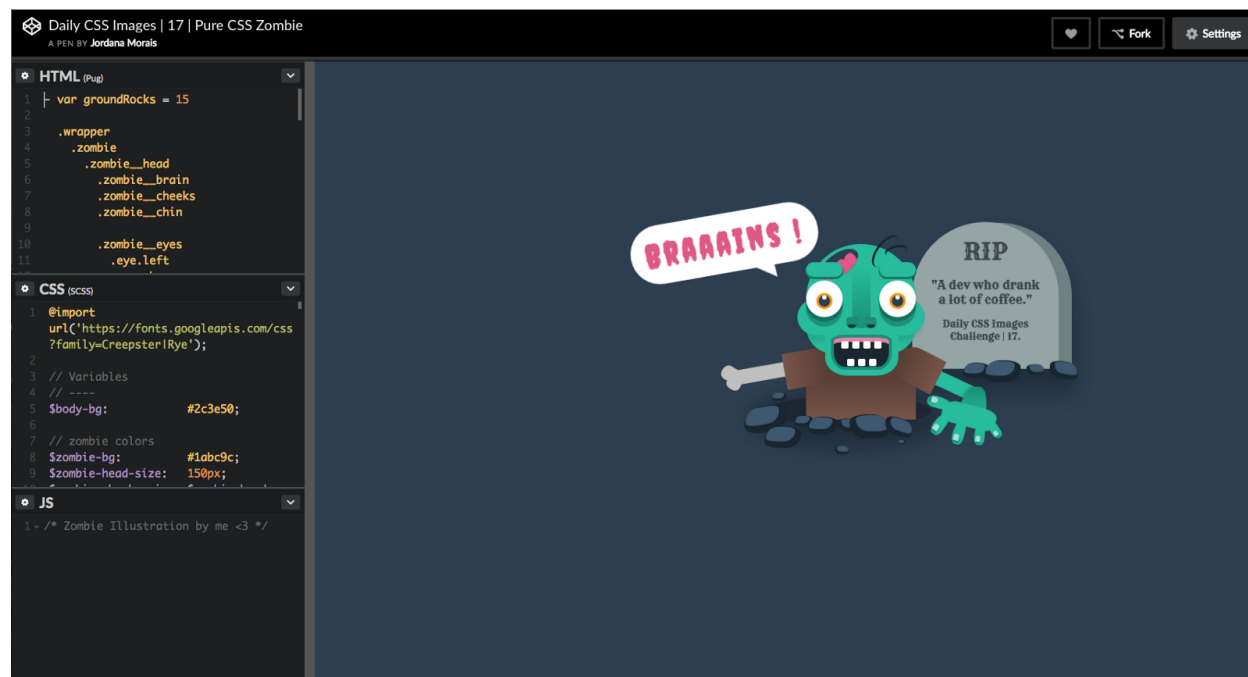
I struggle with not jumping around on things to learn too. However, at the end of the day, practice makes perfect. Shortly, I will be providing a map for ways to get better with React that may be at a slower pace than you want to go. Nevertheless, I think the wise saying fits well here: “Slow and steady wins the race.”

Step 1: Craft Beautiful Components

Although we have a nice template at this point for making our own user interface, I suggest that you start off by just making individual components that make up a user interface.

Take a look around [Codepen](https://codepen.io/)⁵⁷ for user interface components that grab your attention. Try to bring up sharp design skills and a hefty dose of CSS.

If design or CSS is not your thing, I highly recommend taking my [Daily CSS Images](http://challenges.codingartist.io/daily-css-images/)⁵⁸ challenge.



In this challenge, you will get prompts to create pure CSS images over a period of 50 days. This is a great way to stretch your creative, learn design principles, and pick up on core CSS skills. It's already proven to help hundreds of developers in this area.

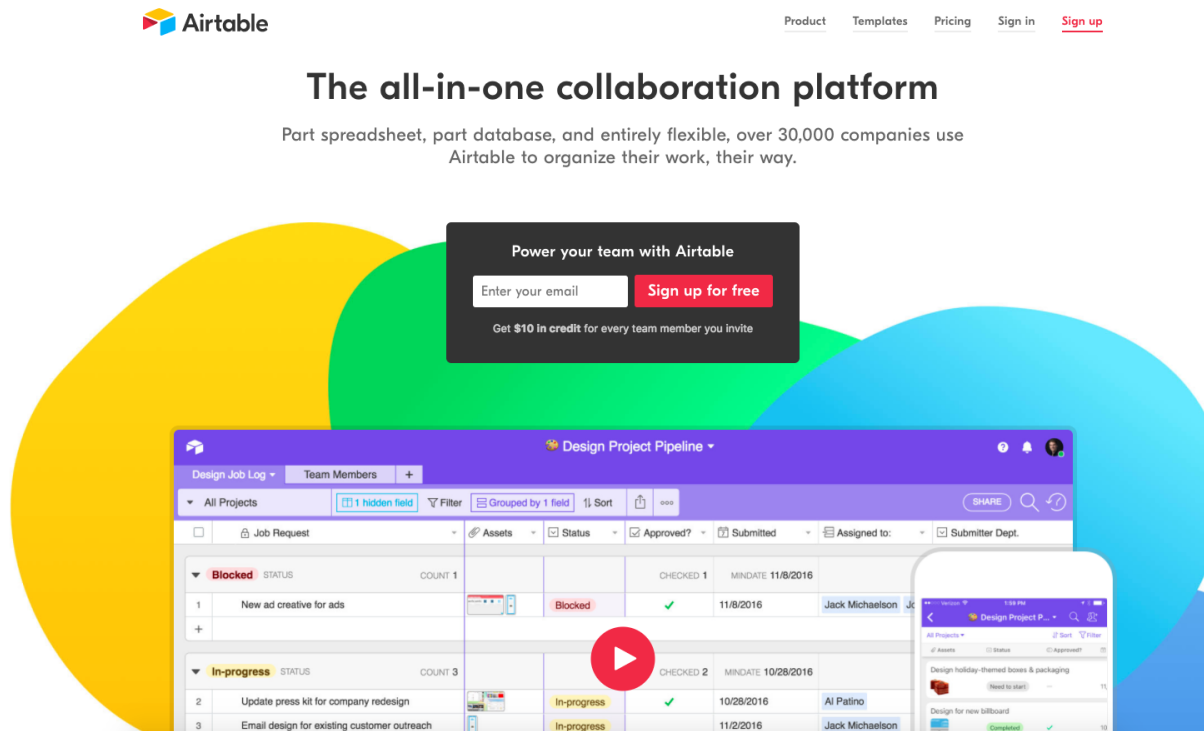
Once you have created all the major components of a user interface with React in Codepen, you'll be ready to move to the next step.

⁵⁷ <https://codepen.io/>

⁵⁸ <http://challenges.codingartist.io/daily-css-images/>

Step 2: Create Static User Interfaces

In this step, you can take the skills from making simple components and the nice template from the project in this book to create static user interfaces.



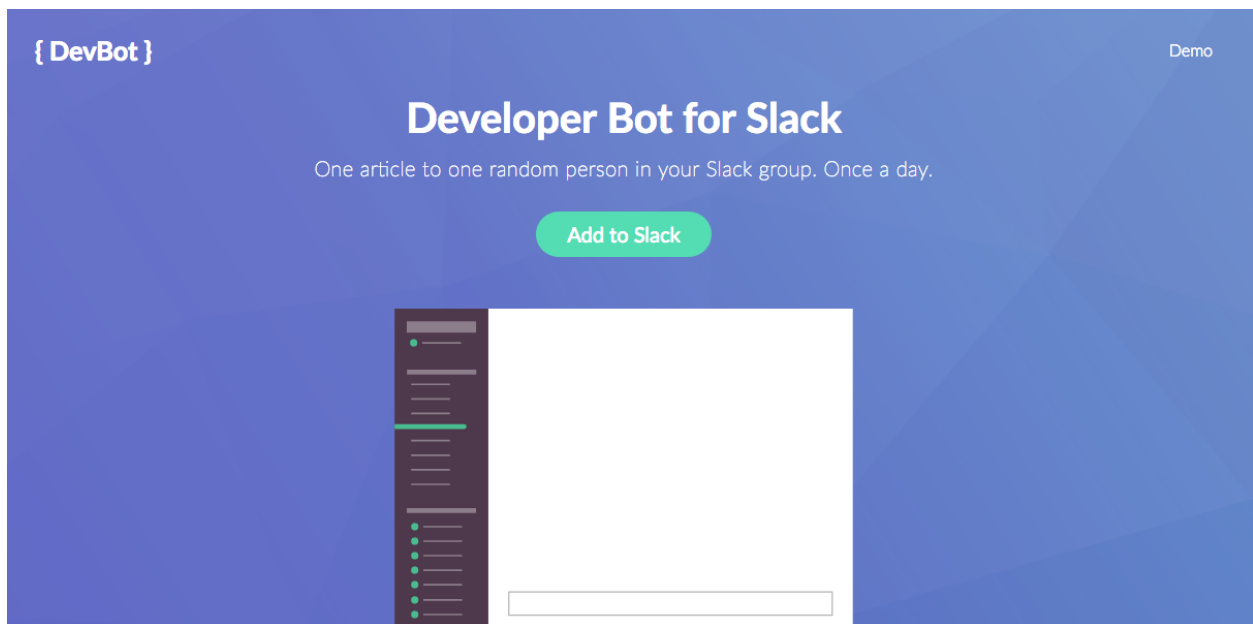
You'll need to learn how to not only style good individual components but how to style when all of it is put together.

Remember to think of everything in terms of village design and check out [CSS Tricks](https://css-tricks.com/)⁵⁹ for all your CSS question.

Step 3: Add Data, Event Handlers, Conditional Rendering, & Lifecycle Hooks

This step is what we did to conclude our book. We made use of props, state, event handling, conditional rendering, and lifecycle hooks.

⁵⁹<https://css-tricks.com/>



I recommend practicing different types of events, making data more complex, and using all your creative juices to figure out how to improve your user interface.

If you need help getting ideas for what to practice, look around at websites on [Product Hunt](#)⁶⁰ and try to emulate things that they are doing to make the user interface a dynamic experience.

Step 4: Working With APIs

API is probably the biggest buzz word you hear in web development. It's treated like something that you are supposed to know like the back of your hand. However, there's often a lack of clarity in teaching this concept.

API stands for **Application Programming Interface**.

You can think of it as the portion of the server that receives and requests and sends responses.

Practically speaking, APIs are often used by developers to tap into big web services like Google Maps, Facebook, Twitter, etc. Developers can mix and match existing web services to create their own web application.

For example, Yelp uses the Google Maps API to tap in and get data so they can plot maps for businesses on Google Maps instead of creating their own mapping service. In short, it's a way to get data from other web services without reinventing the wheel and losing all your time and money.

There are two good resources that I recommend for practicing with APIs (*and understand what in the world they are*):

- [What is an API? In English, please](#)⁶¹.

⁶⁰<http://producthunt.com/>

⁶¹<https://medium.freecodecamp.com/what-is-an-api-in-english-please-b880a3214a82>

- [Use the Twitch.tv JSON API via FreeCodeCamp](#)⁶²

Step 5: Learn How to Deploy Your Application

Deploying a React application that is using Webpack is not as straightforward as you might hope. I don't think this is worth attempting until you've learned a lot about React and won't become overwhelmed with fatigue figuring it out.

[Here's a recommended resource for learning how to deploy a React app that uses Webpack.](#)⁶³

Step 6: Studying Best Practice

Before you dive into the next step, creating a dynamic web application, it's best to learn the best practices for React development.

What do I mean by studying best practices?

Understanding different ways to organize your application

Looking into modular approaches (being able to break down your code into manageable pieces and avoiding repetition)

Exploring all of ES6's features

Consistent coding style/conventions

There's a lot to cover here but here are several recommendations to get started:

[Presentational and Container Components](#)⁶⁴

[How Flexbox works](#)⁶⁵

[Practical Modern JavaScript](#)⁶⁶

[How to Better Organize Your React Applications?](#)⁶⁷

- [ESLint](#)⁶⁸

⁶²<https://www.freecodecamp.com/challenges/use-the-twitchtv-json-api>

⁶³<https://online.reacttraining.com/courses/50507/lectures/2466854>

⁶⁴https://medium.com/@dan_abramov/smart-and-dumb-components-7ca2f9a7c7d0

⁶⁵<https://medium.freecodecamp.com/an-animated-guide-to-flexbox-d280cf6afc35>

⁶⁶<https://ponyfoo.com/books/practical-modern-javascript/chapters>

⁶⁷<https://medium.com/@alexmgmn/how-to-better-organize-your-react-applications-2fd3ea1920f1>

⁶⁸<http://eslint.org/blog/2014/11/es6-jsx-support>

Step 7: Learning Redux



Redux is a predictable state container for JavaScript apps.

(If you're looking for a WordPress framework, check out [Redux Framework](#).)

It helps you write applications that behave consistently, run in different environments (client, server, and native), and are easy to test. On top of that, it provides a great developer experience, such as [live code editing combined with a time traveling debugger](#).

You can use Redux together with [React](#), or with any other view library.

It is tiny (2kB, including dependencies).

build error npm v3.6.0 downloads 2M/month discord #redux @ reactiflux irc #rackt @ freenode changelog #187

Learn Redux from its creator:

[Part 1: Getting Started with Redux](#) (30 free videos)

[Part 2: Building React Applications with Idiomatic Redux](#) (27 free videos)

[Redux](#)⁶⁹ is another key component in the React ecosystem. It allows for an improved approach to handling state management in a React application. There's a very good chance that larger React applications will be using this method. Therefore, it's a good skill to learn for your career.

Here's a good course of Redux via [Egghead.io](#).⁷⁰

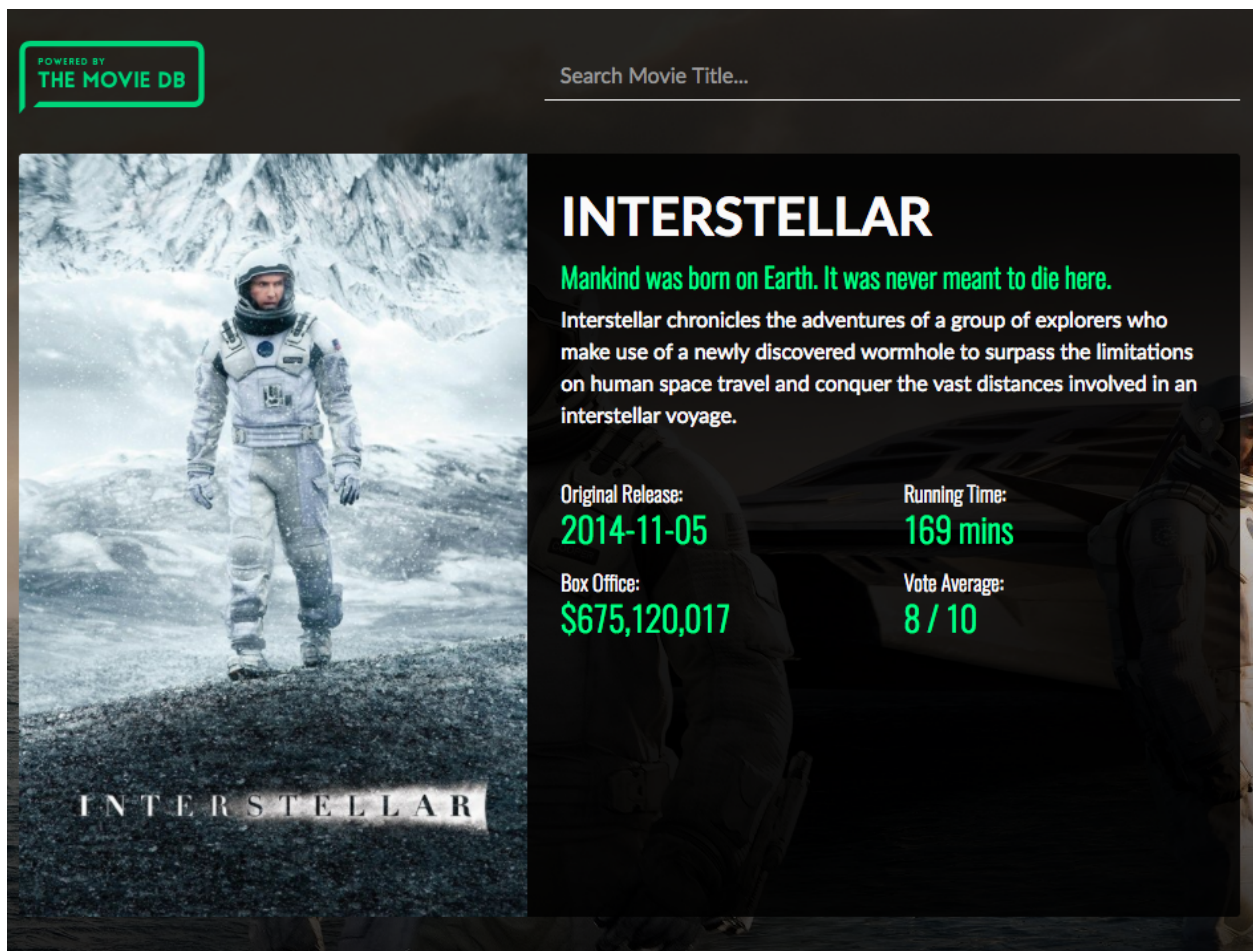
Step 8: Create a Dynamic Web App using Firebase

We only dabbled slightly into Firebase. There's a lot more that can be done with Firebase to create a complete, dynamic web application. This type of web application should have user authentication (sign up and sign in) as well as well-structured data.

A good way to practice would be to create a movie web application. I user can sign up to be able to search through a database of movies, add to a wishlist, and leave ratings and reviews.

⁶⁹<http://redux.js.org/>

⁷⁰<https://egghead.io/courses/getting-started-with-redux>



TMDb Movie Search App

*TMDb Movie Search App*⁷¹

Here are a few existing projects to get you started.⁷²

Step 9: Creating Your Own Backend

In this step, you can replace Firebase with your own Node.js backend code if you prefer.

Here's a good video course that teaches these concepts by Wes Bos.⁷³

Alternative Routes

There's a ton of things that you can do with React and a lot of alternative skills that you can focus on.

Here are some examples:

⁷¹<https://skempin.github.io/reactjs-tmdb-app/>

⁷²<https://react.rocks/?q=movie>

⁷³<http://wesbos.com/learn-node/>

React Native⁷⁴: Learn once, write anywhere: Build mobile apps with React

React VR⁷⁵: Build VR websites and interactive 360 experiences with React

Electron⁷⁶: Build React desktop applications

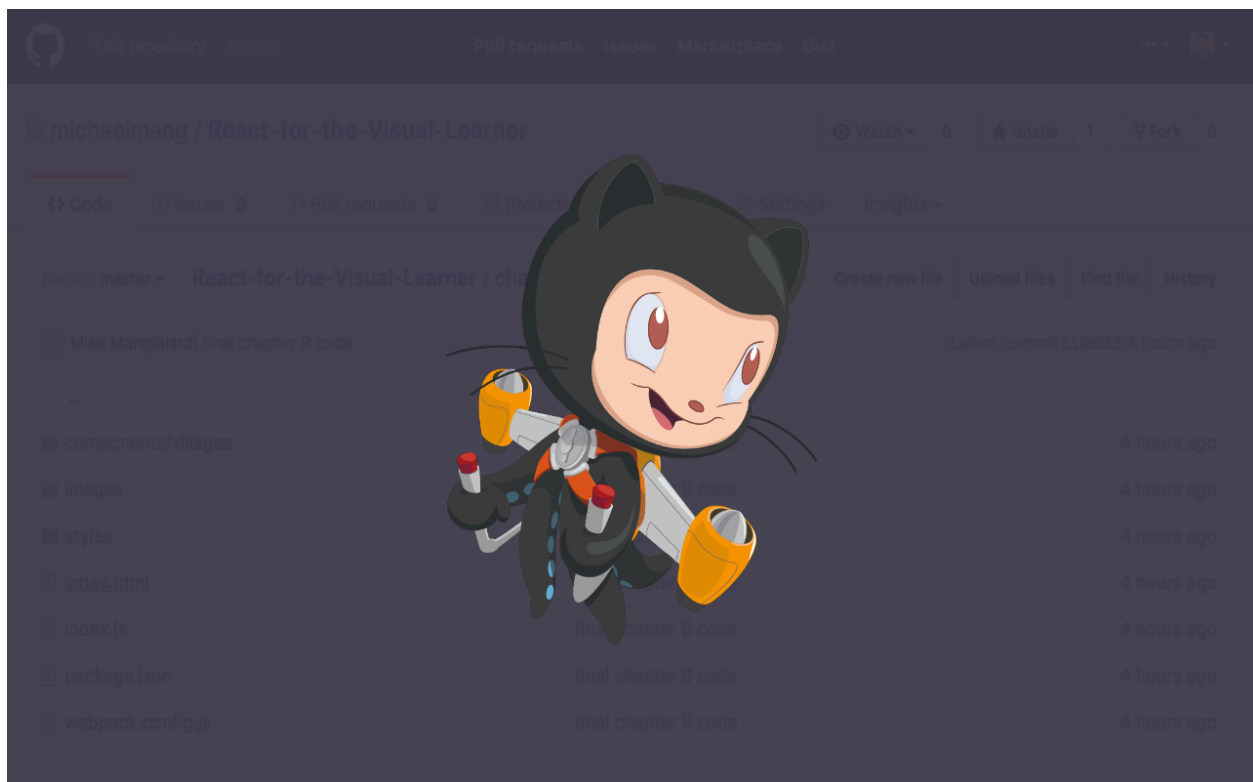
Conclusion

First off, this is a long list that takes a lot of time and experience. You don't need to master all of these things to call yourself a real developer. You don't need to cram all of this in a month or two before you can start applying for jobs.

Build off the core skills we learned in this course, see if any particular scope in React development peaks your interest, and get better piece by piece.

You have learned React. You are a developer. You can do this. Now, it's time to learn more and have fun!

Creating a Repository on Github



⁷⁴<https://facebook.github.io/react-native/>

⁷⁵<https://facebook.github.io/react-vr/>

⁷⁶<https://electron.atom.io/>

Now that we've covered a map for going forward with React, I wanted to demonstrate how to quickly deploy an application.

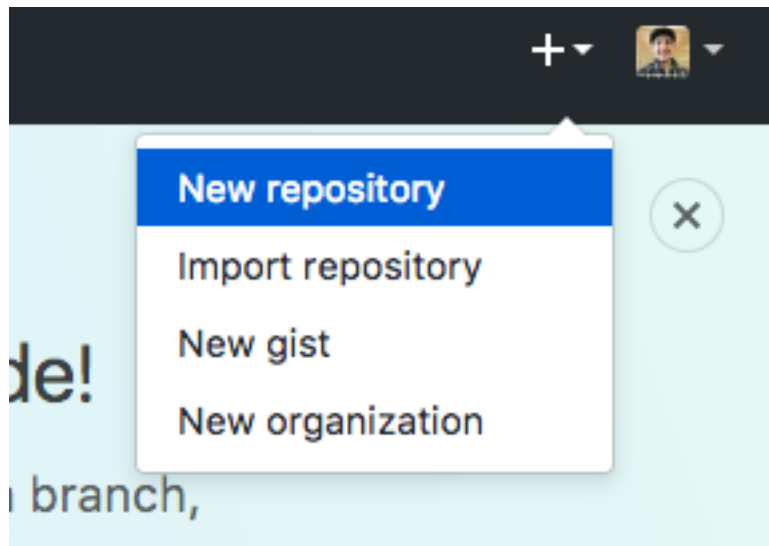
The first step to that process is to create a GitHub repository.

Most people have heard of Github and I'm sure that this section might be trivial for more experienced developers.

With that being said, I thought it would be useful to actually walk through creating a GitHub repository and uploading our code to it.

First things first, you'll need to configure [Git and Github](#)⁷⁷. The official documentation should be straightforward enough to complete the installation and configuration step.

Once that is complete, we will login to GitHub and create a new repository.



Create the repository with the following settings (don't forget the .gitignore option):

⁷⁷ <https://help.github.com/articles/set-up-git/>

Create a new repository

A repository contains all the files for your project, including the revision history.

Owner

 michaelmang ▾

Repository name

/ react-landing-page ✓

Great repository names are short and memorable. Need inspiration? How about **cuddly-disco**.

Description (optional)



Public

Anyone can see this repository. You choose who can commit.



Private

You choose who can see and commit to this repository.



Initialize this repository with a README

This will let you immediately clone the repository to your computer. Skip this step if you're importing an existing repository.

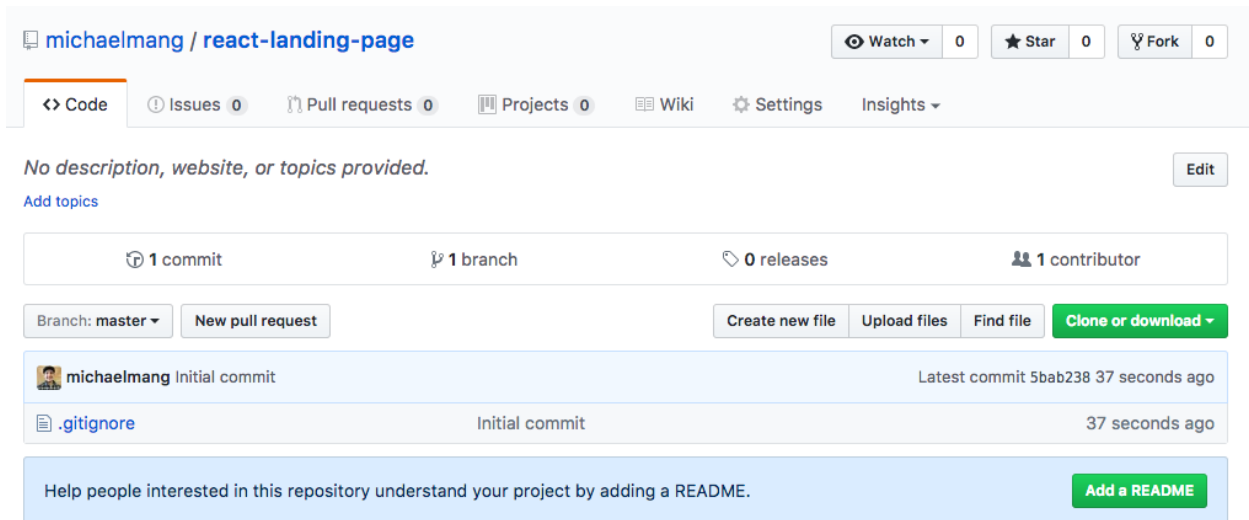
Add .gitignore: **Node** ▾

Add a license: **None** ▾



Create repository

Our project will look like this:



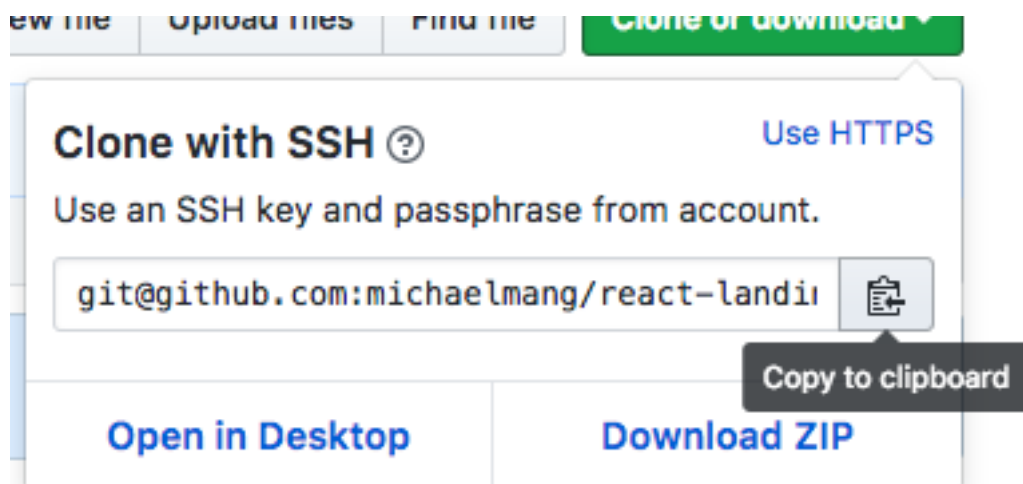
The only file is the `.gitignore` which was populated for us. This file just tells GitHub not to upload unnecessary files such as our big `node_modules` folder.

```
# Dependency directories
node_modules/
```

As long as someone had our `package.json` file, they could run `npm install` to create their own `node_modules` folder locally.

Alright. Now, let's click **Clone or download**.

We want to clone with ssh. Go ahead and click the copy icon:



Next, open up command line and `cd` wherever you want to clone (download) the code from this

repo.

Once you have done that, we can do:

```
git clone *paste what we just copied here*
```

This will clone the repository to the location where our command line was open:

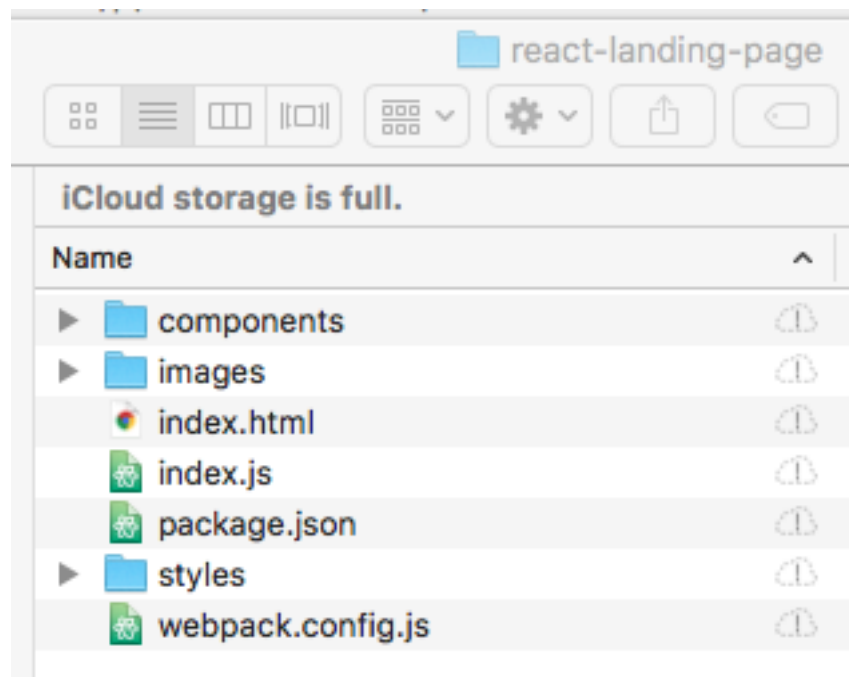
```
Cloning into 'react-landing-page'...
remote: Counting objects: 3, done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (3/3), done.
```

Go to your file explorer where command line was open.

You'll see the project folder for the GitHub repo.



In this folder, copy the code from the react-landing-page project we have been working in like so:



Going forward, you would make edits from within this folder when using Atom. For future reference, you want to create the repository in GitHub before you started development.

...

From command line, let's run the following command:

```
git add .
```

This prepares to add any changes to our code from the current repo on GitHub.

Next, we can run

```
git status
```

to see the result of the previous command:

```
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   components/villages/Demo.js
    new file:   components/villages/Landing Page.js
    new file:   components/villages/neighborhoods/Footer.js
    new file:   components/villages/neighborhoods/Header.js
    new file:   components/villages/neighborhoods/Section A.js
    new file:   components/villages/neighborhoods/Section B.js
    new file:   components/villages/neighborhoods/Section C.js
    new file:   components/villages/neighborhoods/blocks/Call To Action.js
    new file:   components/villages/neighborhoods/blocks/Navigation.js
    new file:   components/villages/neighborhoods/blocks/Section Description.js
    new file:   components/villages/neighborhoods/blocks/Slack Channel.js
    new file:   components/villages/neighborhoods/blocks/houses/Article Cover.js
    new file:   components/villages/neighborhoods/blocks/houses/Branding.js
    new file:   components/villages/neighborhoods/blocks/houses/Button.js
    new file:   components/villages/neighborhoods/blocks/houses/Chat Bot Message.js
    new file:   components/villages/neighborhoods/blocks/houses/Demo Link.js
    new file:   components/villages/neighborhoods/blocks/houses/Divider.js
    new file:   components/villages/neighborhoods/blocks/houses/Heading.js
    new file:   components/villages/neighborhoods/blocks/houses/Laptop.js
    new file:   components/villages/neighborhoods/blocks/houses/Paragraph.js
    new file:   components/villages/neighborhoods/blocks/houses/Subtitle.js
    new file:   components/villages/neighborhoods/blocks/houses/Title.js
    new file:   components/villages/neighborhoods/blocks/houses/User Message.js
    new file:   images/Article Cover.svg
    new file:   images/Branding.svg
    new file:   images/Laptop.svg
    new file:   images/Slack Channel.svg
```

Then, we can commit our code which will record changes to our repo:

```
git commit -m "Insert a message saying what you changed"
```

Finally, we need to push (upload) our code so it appears on the official GitHub repo:

```
git push origin master
```

We can now see our code on GitHub:

The screenshot shows a GitHub repository page for 'michaelmang / react-landing-page'. At the top, there are buttons for 'Watch', 'Star', and 'Fork', each with a count of 0. Below these are tabs for 'Code', 'Issues', 'Pull requests', 'Projects', 'Wiki', 'Settings', and 'Insights'. The 'Code' tab is selected. Below the tabs, there is a message: 'No description, website, or topics provided.' with an 'Edit' button. Below this, there are statistics: '2 commits', '1 branch', '0 releases', and '1 contributor'. Below the statistics, there are buttons for 'Branch: master', 'New pull request', 'Create new file', 'Upload files', 'Find file', and 'Clone or download'. Below these buttons, there is a table of commits. The table has three columns: 'Commit', 'Commit message', and 'Time'. The first commit is by 'Mike Mangialardi' with the message 'Commit test' and the time '11 seconds ago'. Below the table, there is a message: 'Help people interested in this repository understand your project by adding a README.' with an 'Add a README' button.

michaelmang / react-landing-page

Watch 0 Star 0 Fork 0

<> Code Issues 0 Pull requests 0 Projects 0 Wiki Settings Insights

No description, website, or topics provided. Edit

Add topics

2 commits 1 branch 0 releases 1 contributor

Branch: master New pull request Create new file Upload files Find file Clone or download

Commit	Commit message	Time
Mike Mangialardi	Commit test	11 seconds ago
	components/villages	11 seconds ago
	images	11 seconds ago
	styles	11 seconds ago
	.gitignore	14 minutes ago
	index.html	11 seconds ago
	index.js	11 seconds ago
	package.json	11 seconds ago
	webpack.config.js	11 seconds ago

Help people interested in this repository understand your project by adding a README. Add a README

Awesome! Now you can show off what you make with other people. You can also easily work with multiple computers and look back at the history of your changes in case you want to revert pieces of your code.

Editing Code From a Repository

The screenshot shows the GitHub interface for the repository 'michaelmang / react-landing-page'. At the top, there are buttons for 'Watch' (0), 'Star' (0), and 'Fork' (0). Below this is a navigation bar with links for 'Code', 'Issues' (0), 'Pull requests' (0), 'Projects' (0), 'Wiki', 'Settings', and 'Insights'. A message states 'No description, website, or topics provided.' with an 'Edit' button and a link to 'Add topics'. Below this, statistics show '2 commits', '1 branch', '0 releases', and '1 contributor'. A bar contains 'Branch: master', 'New pull request', and buttons for 'Create new file', 'Upload files', 'Find file', and 'Clone or download'. The main content area shows a commit history table:

Mike Mangialardi Commit test		Latest commit de6ff88 11 seconds ago
components/villages	Commit test	11 seconds ago
images	Commit test	11 seconds ago
styles	Commit test	11 seconds ago
.gitignore	Initial commit	14 minutes ago
index.html	Commit test	11 seconds ago
index.js	Commit test	11 seconds ago
package.json	Commit test	11 seconds ago
webpack.config.js	Commit test	11 seconds ago

At the bottom, a blue box prompts the user to 'Add a README' to help people understand the project.

Let's say I wanted to edit code from a repo in GitHub. How do I do that?

cd into the root folder of the cloned project.

Run the following command:

```
git pull origin master
```

This will *pull* the code on GitHub so we know that are editing the latest version of our code.

You would then open the project folder in Atom to make some changes.

If there's no *node_modules* folder in the project, then you run:

```
npm install
```

This will create the *node_modules* folder containing all the packages as specified in the *package.json* file.

After making changes, you want to update the repo with the new changes.

Again, we do the following commands in order:

```
git add .
```

```
git status
```

```
git commit -m "build for production"
```

```
git push origin master
```

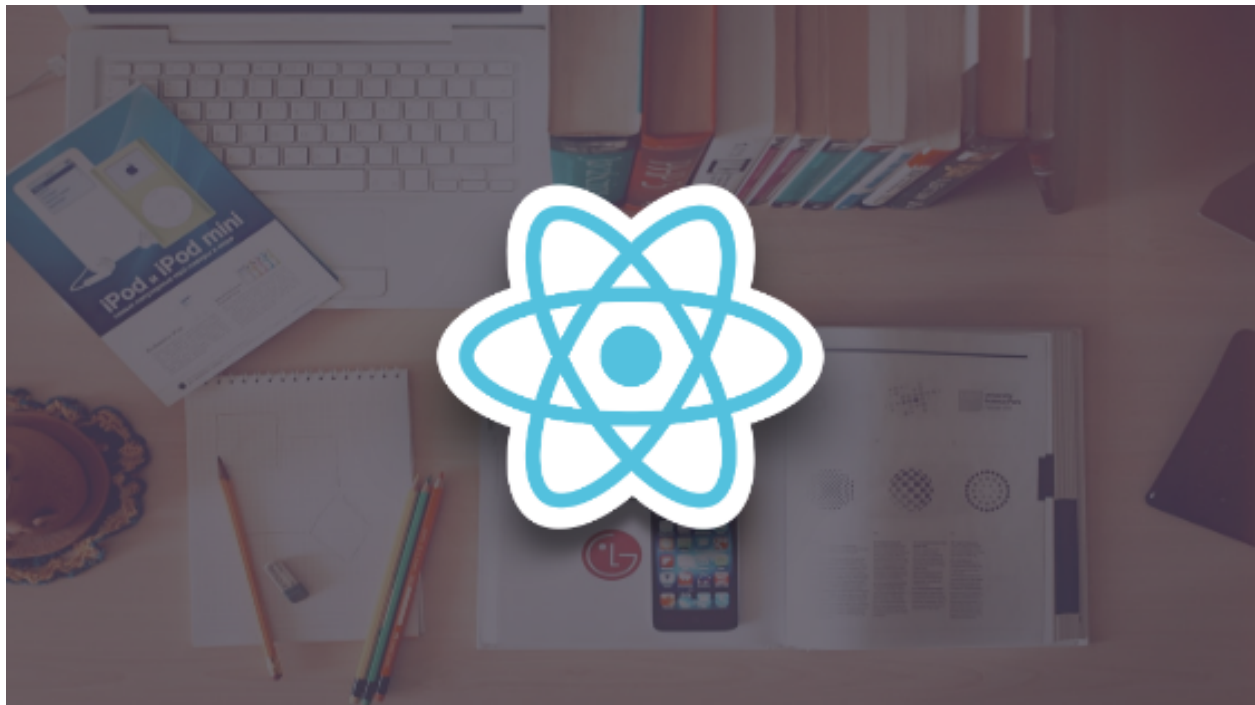
You can then go to GitHub and check your changes:

The screenshot shows the GitHub interface for the repository 'michaelmang / react-landing-page'. At the top, there are buttons for 'Watch' (0), 'Star' (0), and 'Fork' (0). Below these are tabs for 'Code', 'Issues' (0), 'Pull requests' (0), 'Projects' (0), 'Wiki', 'Settings', and 'Insights'. The main content area displays the repository description: 'Final landing page made in React.js for the Visual Learner book' with a link to 'http://bit.ly/2qZHNVS'. Below this, it shows '6 commits', '1 branch', '0 releases', and '1 contributor'. A yellow progress bar is visible. The 'Branch: master' dropdown is set to 'master', and there is a 'New pull request' button. To the right are buttons for 'Create new file', 'Upload files', 'Find file', and a green 'Clone or download' button. The commit history table lists the following files and their commit details:

File	Commit Message	Time
components/villages	Commit test	2 hours ago
images	Commit test	2 hours ago
styles	Commit test	2 hours ago
.gitignore	Initial commit	2 hours ago
index.html	build for production	an hour ago
index.js	Commit test	2 hours ago
package.json	build for production	an hour ago
webpack.config.js	Update webpack.config.js	an hour ago

Sweet! We now we know how to work with GitHub.

Reacting to What I've Taught



At this point, I think I've written enough to achieve what I had aimed to do. That is, to create a book teaching the absolute basics of React to an absolute beginner. A means of that ultimate goal was to try to be visual-friendly when possible. While I think I could have been a bit more organized with emphasizing the visual-friendly part, I'm pleased with the outcome.

One of the cool parts about this book was being able to learn as I was writing the book. I had previously never been able to successfully make my own React project with Webpack and Babel from scratch, never touched Firebase, and never organized my components according to the village design. That might explain some quirks in the flow of this book, however, I think it brings a sense of transparency which is refreshing when it comes to technical guides.

I hope this book was a breath of fresh air from existing material by being practical, engaging, and didactic. Have fun on your React journey.